

Microsoft Cloud Storage Partner Program

Integrate Microsoft 365 for the web into your experiences to empower users to view and edit Word, Excel, and PowerPoint documents stored directly on the cloud.



OVERVIEW

[Integrate with Microsoft 365 for the web](#)



GET STARTED

[Apply for the Cloud Partner Storage Program](#)



HOW-TO GUIDE

[Create new files with Microsoft 365 for the web](#)



QUICKSTART

[Frequently asked questions](#)

Microsoft Cloud Storage Partner Program documentation



CSPP Overview

Use the WOPI protocol to integrate with Microsoft 365 for the web

[Integrate with Microsoft 365 for the web](#)



Get started

[Apply for the Cloud Storage Partner Program](#)

[WOPI REST API Key concepts](#)



Onboard

[Set up your environment](#)

[Set up a host page](#)



Reference

[Microsoft 365 for the web UI Guidelines](#)



Build and customize

[Create new files with Microsoft 365 for the web](#)



Test and launch

[Test Microsoft 365 for the web integration](#)

[WOPI discovery](#)

[Microsoft-configured settings](#)

[Microsoft 365 for the web environments](#)

[More >](#)

[Edit binary document formats](#)

[Use PostMessage to interact with Microsoft 365 for the web](#)

[Co-author with Microsoft 365 for the web](#)

[More >](#)

[WOPI Validator](#)

[Use proof keys to verify requests originate from Microsoft 365 for the web](#)

[Launch your Microsoft 365 for the web integration](#)

[Load time performance](#)

[More >](#)



Get support

[Troubleshoot interactions with Microsoft 365 for the web](#)

[Common implementation issues](#)

[Frequently asked questions](#)

[Known issues](#)

[Program Terms](#)



Integrate with Microsoft 365 for mobile

[Integrate with Microsoft 365 for mobile](#)

[Onboarding information for mobile integration](#)

[WOPI implementation requirements for mobile integration](#)

[Browse, open, and save files from Microsoft 365 for mobile](#)

[Open files from your app in Microsoft 365 for mobile](#)

[More >](#)



Integrate with Cloud Storage Partner Program Plus (CSPP Plus)

[Cloud Storage Partner Program Plus Overview](#)

[WOPI extensions for coauthoring - Overview](#)

[WOPI extensions for coauthoring - Additional CheckFileInfo properties](#)

[WOPI extensions for coauthoring - GetNewAccessToken](#)

[Locks - Overview](#)

[More >](#)

Use the WOPI protocol to integrate with Microsoft 365 for the web

Article • 01/29/2025

You can use the Web Application Open Platform Interface (WOPI) protocol to integrate Microsoft 365 for the web with your application. The WOPI protocol lets Microsoft 365 for the web access and change files that are stored in your service.

To integrate your application with Microsoft 365 for the web, do the following:

1. Be a member of the Microsoft 365 - Cloud Storage Partner Program. Currently, integration with the Microsoft 365 for the web cloud service is available to cloud storage partners. You can learn more about the program, as well as how to apply, at <https://developer.microsoft.com/office/cloud-storage-partner-program> .

Important

The Microsoft 365 - Cloud Storage Partner Program is for independent software vendors whose business is cloud storage. It's not open to Microsoft 365 customers directly.

2. Set up the WOPI protocol - a set of REST endpoints that expose information about the documents that you want to view or edit in Microsoft 365 for the web. The set of WOPI operations that must be supported is described in the section titled [WOPI implementation requirements on Microsoft 365 for the web integration](#).
3. Read some XML from an Microsoft 365 for the web URL that provides information about the capabilities that Microsoft 365 for the web applications expose, and information on how to invoke them. This process is called [WOPI discovery](#).
4. Provide an HTML page (or pages) that host the Microsoft 365 for the web iframe. This is called the [host page](#) and is the page your users visit when they open or edit Microsoft 365 documents in Microsoft 365 for the web.
5. Optionally integrate your own UI elements with Microsoft 365 for the web. For example, when users choose **Share** in Microsoft 365 for the web, you can show your own sharing UI. These interaction points are described in the section titled [Using PostMessage to interact with the Microsoft 365 for the web application iframe](#).

How to read this documentation

This documentation contains complete information on how to integrate with Microsoft 365 for the web, including:

- How to implement the WOPI protocol
- How Microsoft 365 for the web uses the protocol
- How to test your integration
- The [process for shipping your integration](#), and much more.

It can be difficult to know where to begin. The following guidelines can direct you to the specific sections of the most help to you.

If you want to know why Microsoft 365 integration might be useful to you, and what capabilities it provides, read the following sections:

- [Integrate with Microsoft 365 for the web](#) - A high level overview of the scenarios enabled by Microsoft 365 for the web integration, as well as a brief description of some of the key technical elements in a successful integration.
- [Use the WOPI protocol to integrate with Microsoft 365 for the web](#) - A brief description of the technical pieces that you must implement to integrate with Microsoft 365 for the web.

If you are an engineer about to begin implementing a WOPI host, read the [Key concepts](#) section first. When designing your WOPI implementation, you must keep in mind the expectations around [file IDs](#), [access tokens](#), and [locks](#). These concepts are critical to a successful integration with Microsoft 365 for the web.

And also be sure to read the following sections:

- [WOPI Validator](#)
- [Troubleshooting interactions with Microsoft 365 for the web](#)
- [Microsoft 365 for the web environments](#)

If you are a back-end engineer, begin with the following sections in addition to the [Key concepts](#) section and other general sections listed above:

- [WOPI discovery](#)
- [WOPI REST endpoints](#)
- [CheckFileInfo](#)
- [Lock](#)

Once you have read these sections, any of the other core WOPI operations are useful to read through, such as [GetFile](#), [PutFile](#), [PutRelativeFile](#), [UnlockAndRelock](#), and so on.

If you are a front-end engineer, begin with the following sections in addition to the [Key concepts](#) section and other general sections listed above:

- [Building a host page](#)
- [Using PostMessage to interact with the Office for the web application iframe](#)
- [WOPI discovery](#), specifically the [WOPI actions](#) section

Finally, if you are looking for more details about the process for shipping your integration, see the [Shipping your Microsoft 365 for the web integration](#) section.

Integrate with Microsoft 365 for the web

Article • 01/29/2025

You can integrate with Microsoft 365 for the web to enable your users to view and edit Excel, PowerPoint, and Word files directly in the browser.

Important

The Microsoft 365 - Cloud Storage Partner Program is for independent software vendors whose business is cloud storage. It's not open to Microsoft 365 customers directly.

If you deliver a web-based experience that lets your users store Microsoft 365 files or include Microsoft 365 files as a key part of your solution, you can integrate Microsoft 365 for the web into your experience. This integration works directly on files stored by you. Your users won't need a separate storage solution to view and edit Microsoft 365 files.

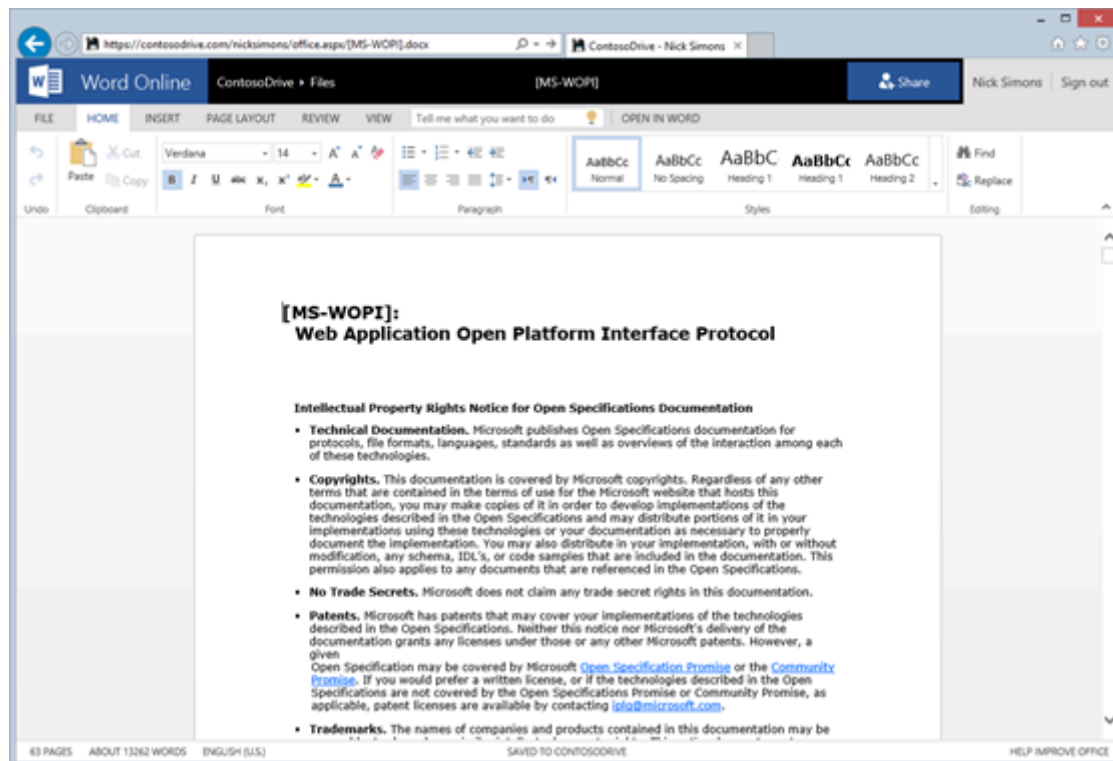


Figure 1 - Word file open for editing in Microsoft 365 for the web

View Microsoft 365 files

Important

Participants in the Microsoft 365 - Cloud Storage Partner Program must support document editing and multi-user co-authoring.

You can make viewing available in two ways:

- By using the high-fidelity previews in Microsoft 365 for the web as an integrated part of your experience. For example, you can use these previews in a light box view of a Word document.
- By offering to show Microsoft 365 files in a full-page interactive preview. Depending on your solution, this might be useful for file browsing or showing read-only files or in cases where users don't have a license to edit files in Microsoft 365 for the web.

Edit Microsoft 365 files

Editing is a core part of Microsoft 365 for the web integration. When you integrate with Microsoft 365 for the web, your users can edit Excel, PowerPoint, and Word files directly in the browser. Also, they can [edit documents collaboratively with other users using Microsoft 365](#). To edit documents, users require an Microsoft 365 license. For more information, see [Co-authoring using Microsoft 365 for the web](#).

Integration process

Integrating with Microsoft 365 for the web includes some HTML and JavaScript work and setting up a few simple REST endpoints. If you're familiar with existing Microsoft 365 protocols, note that you don't have to implement the [MS-FSSHTTP]: File Synchronization via SOAP over HTTP Protocol (Cobalt). At a high level, to integrate with Microsoft 365 for the web, you:

- Read XML from Microsoft 365 for the web that describes the capabilities of Microsoft 365 for the web. This is called [WOPI discovery](#).
- Implement [REST endpoints](#) that Microsoft 365 for the web uses to learn about, fetch, and update files. To do this, you implement the server side of the WOPI protocol.
- Provide an HTML page (or pages) that wrap Microsoft 365 for the web. This page is called the [host page](#).

The following figure shows the WOPI protocol workflow.

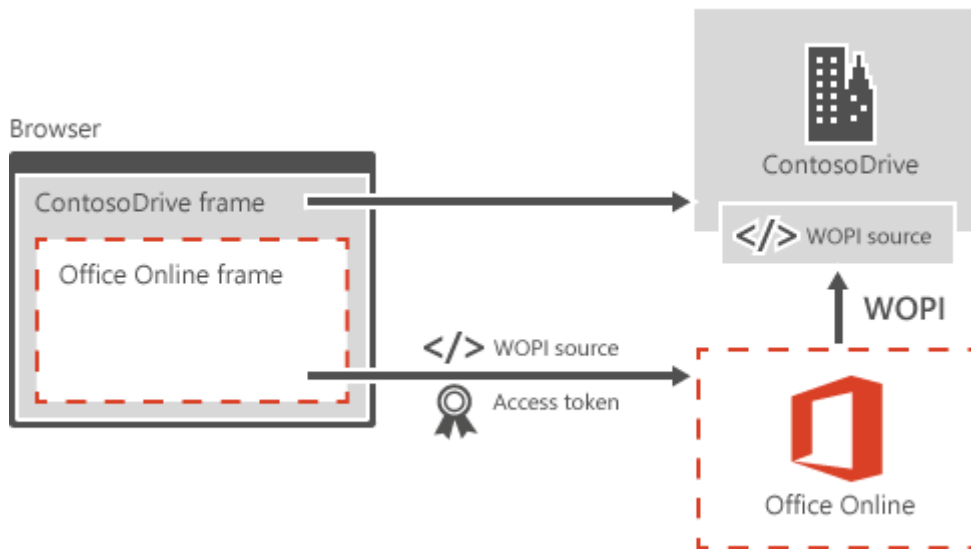


Figure 2 - WOPI protocol workflow

Your solution must meet the following basic requirements.

Authentication

Authentication is handled by passing Microsoft 365 for the web an access token that you generate. Assign this token a reasonable expiration date. Also, we recommend that tokens are only valid for a single user against a single file, to help mitigate the risk of token leaks. For more information, see [Access token](#).

Conflict resolution

Microsoft 365 for the web supports multiuser authoring scenarios if all users are using Microsoft 365 for the web. However, you're responsible for managing conflicts that might come from applications other than Microsoft 365 for the web. You'd want to set up some form of file locking or use another type of conflict resolution.

File IDs

Ensure that files are represented by a persistent ID. This ID must be URL-safe because it might be passed as part of the URL at different times. Also, the ID can't change when the file is renamed, moved, or edited. This ensures an uninterrupted editing experience for your users. For more information, see [File ID](#).

File versions

You should have a mechanism for users to clearly identify file versions through the REST APIs. Because files are cached to improve viewing performance, file versions are extremely helpful. Without them, users can't easily determine whether they have the latest version of the file.

Consider security

Microsoft 365 for the web is designed to work for enterprises that have strict security requirements. To make your integration as secure as possible, be sure that:

- All traffic is SSL encrypted.
- Initial requests to Microsoft 365 for the web are made by using POST, where the access token is in the body of the POST request.

Set up Microsoft 365 for the web identity by using a public [proof key](#) to decrypt part of the WOPI requests. Also, the file cache indexes store file contents by using a SHA256 hash as the cache key. You can pass the hash value to Microsoft 365 for the web by using the [SHA256](#) property in the [CheckFileInfo](#) response. If not provided, Microsoft 365 for the web generates a cache key from the file ID and version. To ensure that users don't force a cache collision and view the wrong file, none of the user-provided information should generate the cache key.

Manage Microsoft 365 subscriptions

Important

The Microsoft 365 - Cloud Storage Partner Program doesn't support Government Licenses

Business users require an Microsoft 365 subscription to edit files in Microsoft 365 for the web. The best way to support this is to have users sign in with a Microsoft account or other valid identity. This establishes that they have the correct subscription. To limit the number of times a user needs to sign in, Microsoft 365 for the web first checks for a cookie.

To provide a better experience for users with Microsoft 365 subscriptions, hosts can optionally implement the [PutUserInfo](#) WOPI operation. For more information, see [Tracking users' subscription status](#).

Interested?

If you're interested in integrating your solution with Microsoft 365 for the web, take a moment to register at [Microsoft 365 Cloud Storage Partner Program](#).

Apply for CSPP to integrate with Microsoft 365 for the web

Article • 01/29/2025

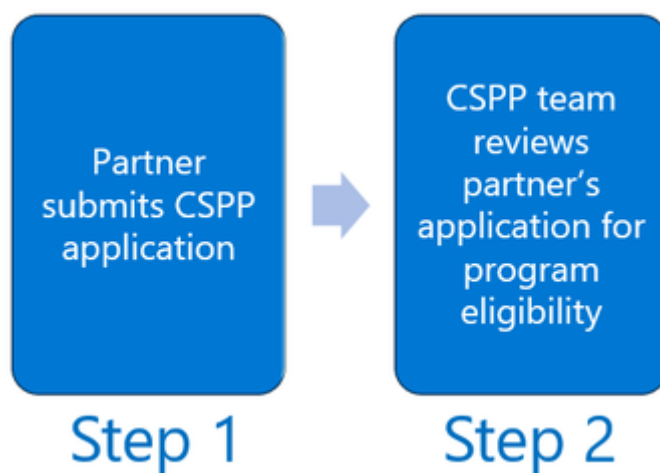
You can integrate with Microsoft 365 for the web to enable your users to view and edit Word, Excel, and PowerPoint documents directly in the browser using the Web Application Open Platform Interface Protocol (WOPI).

Partner companies must join the Microsoft Cloud Solution Partner Program to use WOPI in their solution.

CSPP engagement cycle

The CSPP engagement cycle includes the following steps:

Application



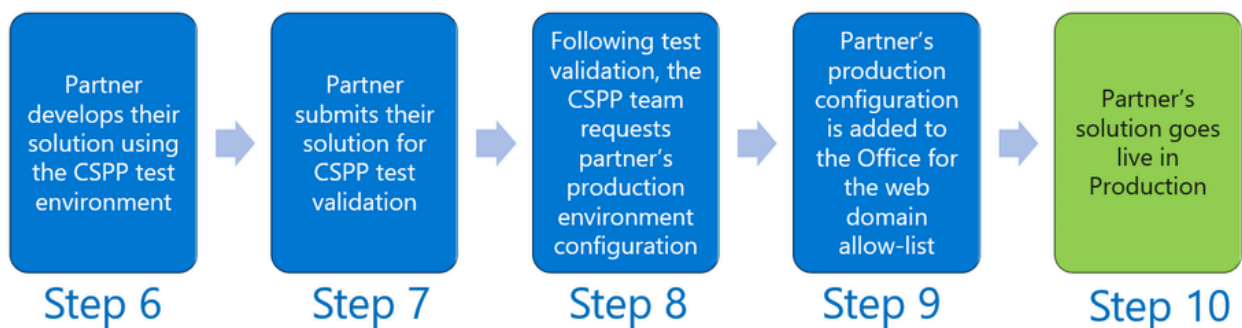
1. Start by [applying to the CSPP program](#) 
2. CSPP program managers review your CSPP application for program eligibility

Onboarding



3. If approved for participation in the CSPP program, you must agree with the [Program Terms](#)
4. Once you've agreed to the Program Terms, you will be sent a "Welcome" email asking for your test environment configuration
5. The CSPP team will then provision your test environment for access to Microsoft 365 for the web

Development and release



6. You are now able to develop your solution using the CSPP test environment integrated fully with Microsoft 365 for the web
7. After you complete your development, you will need to submit your solution for validation by the CSPP team. This will ensure your solution works well and meets program requirements such as Microsoft branding guidelines
8. After your solution is validated by the CSPP team, you will be asked to provide Microsoft your production environment configuration
9. Your solution production configuration is added to the Microsoft 365 for the web production domain allow-list
10. Your solution goes live in Production for our joint end-users

📌 **Note**

The propagation of the environment configuration changes needed in Step 5 and Step 9 above, typically take 4-5 weeks *each* and is not something that can be expedited.

Engage with the CSPP team

If you think the CSPP program can help you provide Microsoft 365 for the web to your users [start the CSPP application process](#). [↗](#).

Next steps

- [Build your solution](#)
- [WOPI discovery](#)
- [Build a hostpage](#)

Key concepts to understand for WOPI integration

Article • 02/10/2025

The Web Application Open Platform Interface (WOPI) protocol is used by Microsoft 365 applications, such as Microsoft 365 for the web or Microsoft 365 for Mobile, to view and edit files that are stored in a cloud service. The following concepts are referred to throughout the Cloud Storage Partner Program documentation. Understanding these concepts is vital to understanding the requirements for integrating with WOPI clients.

File ID

A File ID is a string that represents a file or folder being operated on by way of WOPI operations. A host must issue a unique ID for any file used by a WOPI client. The client will then include the file ID when making requests to the WOPI host. So, a host must be able to use the file ID to locate a particular file.

A file ID must:

- Represent a single file.
- Be a URL-safe string, because IDs are passed in URLs, principally via the [WOPISrc](#) parameter.
- Remain the same when the file is edited.
- Remain the same when the file is moved or renamed.
- Remain the same when any ancestor container, including the parent container, is renamed.
- In the case of shared files, the ID for a given file must be the same for every user that accesses the file.

ⓘ Note

The file ID isn't provided to a WOPI client directly. It's passed as part of the [WOPISrc](#) value.

For more information on action URLs, see [Action URLs](#). For more information on how the file ID should be passed to Microsoft 365 for the web, see [WOPI_SOURCE](#).

Access token

An access token is a string used by the host to determine the identity and permissions of the issuer of a WOPI request.

The WOPI host that stores the file has the information about user permissions, not the WOPI client. For this reason, the WOPI host must provide an access token that the client then passes back to it on subsequent WOPI requests. When the WOPI host receives the token, it either validates it, or responds with an appropriate HTTP status code if the token is invalid or unauthorized.

Tip

In Microsoft 365 for the web, an access token generates by the host and passes to the client using the `access_token` parameter before the first WOPI request. See [Action URLs](#) and for more information on how access tokens should be passed to Microsoft 365 for the web, see [Passing access tokens securely](#).

A WOPI client requires no understanding of the format or content of the token. The WOPI client just includes the token in WOPI requests and expects the host to validate it. But WOPI access tokens must adhere to the following guidelines:

- Access tokens must be scoped to a single user and resource combination. A WOPI client never assumes that an access token issued for a particular user and resource combination is valid for a different user and resource combination.
- Access tokens must be valid for the [user permissions](#) that are provided by the host in the [CheckFileInfo](#) response. For example, if the `view` action is invoked, and the [UserCanWrite](#) property is set to `true` in the [CheckFileInfo](#) response, the client might re-use that token when transitioning to edit mode. So, a WOPI client expects that any access token is valid for operations that the user has permissions to perform. If a host wishes to issue access tokens that are more narrowly scoped, the [user permissions properties](#) in the [CheckFileInfo](#) response must reflect the permissions that the token provides.
- Access tokens should expire (become invalid) automatically after a period of time. Hosts can use the [access_token_ttl](#) property to specify when an access token expires.

Important

WOPI clients expect that an access token remains valid until it expires (as indicated by the [access_token_ttl](#) value). Hosts shouldn't revoke access tokens as a standard part of their operations. Tokens should only be revoked if a user's permissions have changed or been revoked.

If hosts revoke access tokens regularly, users might experience strange behavior depending on the WOPI client. For example, in some cases, Microsoft 365 for the web will time out a user's session and present them with a dialog asking if they'd like to refresh their session. If the access token isn't yet expired, based on the [access_token_ttl](#), Microsoft 365 for the web will refresh the session using the existing access token, assuming it's still valid.

There are a number of cases like this, and WOPI clients regularly make such assumptions. For this reason, access tokens should remain valid until their expiry.

Important

WOPI clients aren't required to pass the [access_token](#) in the [Authorization](#)  header, but they must send it as a URL parameter in all WOPI operations. So, for maximum compatibility, WOPI hosts should either use the URL parameter in all cases, or fall back to it if the [Authorization](#)  header isn't included in the request.

The `access_token_ttl` property

The `access_token_ttl` property tells a WOPI client when an access token expires, represented as the number of milliseconds since January 1, 1970 UTC (the date epoch in JavaScript). Despite its misleading name, it doesn't represent a time duration for which the access token is valid. The `access_token_ttl` is never used by itself; it's always attached to a specific access token.

Tip

To prevent data loss for users, Microsoft 365 for the web prompts users to save and refresh their sessions if the access token for their session is close to expiring. In order to do this, Microsoft 365 for the web needs to know when the access token expires, which it determines based on the `access_token_ttl` value.

We recommend using an `access_token_ttl` value that makes the access token valid for 10 hours. Hosts can also set the `access_token_ttl` value to `0`. This effectively tells the client that the token expiry is either infinite or unknown. In this case, clients might disable any

UI prompting users to refresh their sessions. This can lead to unexpected data loss due to access token expiry. So, we strongly recommend specifying a value for `access_token_ttl`.

ⓘ Note

Future updates to the WOPI protocol might include renaming this parameter so its name is less confusing.

Lock

A lock is a conceptual construct that's associated with a file. Locks serve two purposes in WOPI:

- First, a lock prevents anyone that doesn't have a valid lock ID from making changes to a file. A WOPI client locks files prior to editing them to prevent other entities from changing the file while the client is also editing them.
- Two, a lock is also used to store a small bit of temporary data associated with a file. This metadata is called the lock ID and is a string with a maximum length of 1024 ASCII characters. For more information, see [note on lock ID lengths](#)). WOPI clients can use this metadata for a variety of purposes, but hosts don't need any knowledge or understanding of the contents of the lock ID. Hosts must treat it as an opaque string.

Therefore, WOPI locks must:

- Be associated with a single file.
- Contain a lock ID of maximum length 1024 ASCII characters.
- Prevent all changes to that file unless a proper lock ID is provided.
- Expire after 30 minutes unless refreshed. For more information, see [RefreshLock](#).
- Not be associated with a particular user.

All WOPI operations that modify files, such as [PutFile](#), include a lock ID as a parameter in their request. Usually the expected lock ID is passed in the **X-WOPI-Lock** request header (but not always. [UnlockAndRelock](#) is an exception that uses **X-WOPI-OldLock** instead).

WOPI requires that hosts compare the lock ID passed in a WOPI request with the lock ID currently on a file and respond appropriately when the lock IDs don't match. In

particular, WOPI clients expect that when a lock ID does *not* match the current lock, the host sends back the current lock ID in the **X-WOPI-Lock** response header. This behavior is critical, because WOPI clients use the current lock ID in order to determine what further WOPI calls to make to the host.

Lock length

It's important to understand that WOPI locks aren't user-owned. In other words, a WOPI client can execute lock-related operations using multiple access tokens. And hosts are expected to run those operations as long as they are valid, as described in this documentation. For example, a WOPI host might receive a [Lock](#) call with an access token that belongs to User A. The host might later receive an [Unlock](#) call with an access token that belongs to User B. As long as User B has rights to edit the file, and the **X-WOPI-Lock** request header matches the lock ID, the [Unlock](#) request should be honored.

WOPI locks must automatically expire after 30 minutes if not renewed by the WOPI client. This ensures that files don't stay locked indefinitely in error cases. Since locks are client-controlled from a protocol perspective (that is, the WOPI client sets and manages the lock) and clients can be unreliable, the WOPI host must expire the locks in such cases.

Tip

WOPI defines a [GetLock](#) operation. However, Microsoft 365 for the web doesn't use it in all cases, even if the host indicates support for the operation using the [SupportsGetLock](#) property in [CheckFileInfo](#). Instead, Microsoft 365 for the web sometimes runs lock-related operations on files with missing or known incorrect lock IDs and expects the host to provide the current lock ID in its WOPI response. Typically, the [Unlock](#) and [RefreshLock](#) operations serve this purpose, but other lock-related operations can be used.

The specific conditions for each response are covered in the documentation for each of the following lock-related WOPI operations:

- [Lock](#)
- [RefreshLock](#)
- [Unlock](#)
- [UnlockAndRelock](#)

- [PutFile](#)

ⓘ Note

Lock ID lengths are currently less than 256 ASCII characters. But we anticipate needing longer lock IDs to support future WOPI integration scenarios. So, we've increased the limit to 1024 ASCII characters. Hosts must indicate that they support lock IDs of this length using the [SupportsExtendedLockLength](#) property in [CheckFileInfo](#).

Share URL

A Share URL is a URL to a webpage that's suitable for viewing a shared WOPI file or container. The URL should be appropriate for launching in a web browser, but the experience is defined by the host. For example, the host might choose to have the URL navigate to the host's browse experience or to a preview of the file using Microsoft 365 for the web or another file previewer.

A host might support different types of Share URLs that can be used for different purposes. For example, a particular Share URL type might not allow users to edit the file by using the Share URL. The list of possible types are defined under the [SupportedShareUrlTypes](#) property.

WOPISrc

The WOPISrc (*WOPI Source*) is the URL that runs WOPI operations on a file. It's a combination of the Files endpoint URL for the host, along with a particular [file ID](#). The WOPISrc doesn't include an [access token](#).

For example, a WopISrc might look like this:

```
https://wopi.contoso.com/wopi/files/abcdef0123456789
```

The WOPISrc is needed beyond just a file ID. It tells the WOPI client what URL to call back to when running WOPI operations on a file. In practice, the WOPISrc and a [file ID](#) are synonymous, since the WOPI client typically works with the WOPISrc itself, not the raw [file ID](#).

For more details on how the WOPISrc is passed to Microsoft 365 for the web, see [WOPI_SOURCE](#).

Set up your Cloud Storage Partner Program environment

Article • 02/24/2023

If your company is accepted into CSPP, you will need to give us technical and legal information about yourself, your company, and your plans for WOPI so the CSPP team can provision your environment at Microsoft. At a minimum you will be asked to provide the following:

Technical information

- Test Domain(s)
- Production Domain(s)
- Homepage URL(s)
- Other configuration information as appropriate

Legal information

- Legal business name
- Full business address (including the number, street, city, state or province, postal or zip code, and country/region)
- Phone and fax numbers
- The name(s) and email address(es) of any third parties working on your behalf
- Your WOPI solution's name

Note

Before you can start using WOPI, you must be a member of the Microsoft Cloud Storage Partner Program (CSPP).

Do not apply for the program if you are developing a solution for another party to sell to their customers and/or use internally within their organization. If you are developing a solution for a third party, have that third party apply for CSPP membership and include the appropriate people in your organization as additional contacts.

For information about applying to become a CSPP member, see [Apply for the CSPP program](#).

Next steps

- [Build your solution](#)
- [WOPI discovery](#)
- [Build a hostpage](#)
- [Customize Office for the web](#)

Related resources

- [Use WOPI protocol to integrate with Office for the web](#)
- [Integrate with Office for the web](#)

Building a host page

Article • 01/29/2025

To instantiate Microsoft 365 for the web applications, a host must create an HTML page that contains an `iframe` element within it that's pointing to a particular [WOPI action URL](#). A host page provides benefits, including:

1. Since the Microsoft 365 for the web application is hosted within a host page, the host page can communicate with the frame easily using [PostMessage](#). This supports a richer integration between the host and Microsoft 365 for the web.
2. URLs displayed in the user's browser are host URLs. So, from a user's perspective, they're interacting with the host, not Microsoft 365 for the web. This also ensures that URLs copied and pasted by users are host URLs, not Microsoft 365 for the web URLs.

The host page is typically simple and must only meet the following requirements:

- It must use a `form` element and [POST](#) the [access token](#) and [access_token_ttl](#) values to the Microsoft 365 for the web iframe [for security purposes](#). Also, it's recommended to post `user_id` and `owner_id` values to the iframe. It's a requirement to post `user_id` and `owner_id` values for CSPP Plus scenarios.
- It must include any JavaScript needed to interact with the Microsoft 365 for the web iframe using [PostMessage](#).
- It must manage [wd* query string parameters](#).
- It must apply some specific CSS styles to the `body` element and Microsoft 365 for the web iframe to avoid visual bugs.
- It must declare a `viewport` meta tag to avoid visual and functional problems in mobile browsers.

Host page example

The [GitHub repository](#) contains a [minimal example host page](#).

ⓘ Note

The example host page doesn't illustrate managing [wd query string parameters](#) or [using PostMessage to interact with the Microsoft 365 for the web application iframe](#). The following sections refer to these considerations in more detail.

Passing access tokens securely

For security purposes, it's important that access tokens not be passed to the Microsoft 365 for the web iframe as a query string parameter. Doing so would greatly increase the likelihood of token leakage. To avoid this problem, hosts must pass the [access token](#) and [access_token_ttl](#) values to the Microsoft 365 for the web iframe using a form [POST](#) [↗](#). This technique is illustrated, along with [dynamic iframe creation](#), in code sample one.

Work around browser iframe behaviors

Some browsers exhibit unexpected handling of iframes when using bookmarks or the browser forward and back buttons.

In some cases, the Microsoft 365 for the web iframe is loaded twice in a single navigation, which can cause *file locked* or *access token expired* errors for users.

Also, occasionally the iframe isn't recreated at all, which causes the Microsoft 365 for the web application to load with the previous session's state. This might cause a session to fail for a variety of reasons, including an expired token.

In order to avoid this result, hosts must dynamically create the Microsoft 365 for the web iframe using JavaScript, then dynamically submit it. This technique is shown in the sample host page:

Code sample 1 - Markup from [SampleHostPage.html](#) [↗](#) illustrating how to dynamically create the Microsoft 365 for the web iframe and pass access tokens securely

HTML

```
<body>

    <form id="office_form" name="office_form" target="office_frame" action="
<OFFICE_ACTION_URL>" method="post">
        <input name="access_token" value="<ACCESS_TOKEN_VALUE>"
type="hidden" />
        <input name="access_token_ttl" value="<ACCESS_TOKEN_TTL_VALUE>"
type="hidden" />
        <input name="user_id" value="<USER_ID_VALUE>" type="hidden" />
        <input name="owner_id" value="<OWNER_ID_VALUE>" type="hidden" />
    </form>

    <span id="frameholder"></span>

    <script type="text/javascript">
        var frameholder = document.getElementById('frameholder');
        var office_frame = document.createElement('iframe');
        office_frame.name = 'office_frame';
```

```

office_frame.id = 'office_frame';

// The title should be set for accessibility
office_frame.title = 'Office Frame';

// This attribute allows true fullscreen mode in slideshow view
// when using PowerPoint's 'view' action.
office_frame.setAttribute('allowfullscreen', 'true');

// The sandbox attribute is needed to allow automatic redirection to
the 0365 sign-in page in the business user flow
office_frame.setAttribute('sandbox', 'allow-scripts allow-same-
origin allow-forms allow-popups allow-top-navigation allow-popups-to-escape-
sandbox');
frameholder.appendChild(office_frame);

```

In an actual implementation, the `<OFFICE_ONLINE_ACTION_URL>`, `<ACCESS_TOKEN_VALUE>`, and `<ACCESS_TOKEN_TTL_VALUE>` strings must be replaced with appropriate [action URL](#), [access token](#), and [access_token_ttl](#) values.

Pass additional query string parameters

Microsoft 365 for the web sometimes passes additional query string parameters to your host page. These query string parameters are of the form `wd*`. When you receive these query string parameters on your host page URLs, you must pass them, unchanged, to the Microsoft 365 for the web iframe.

In addition, if the [replaceState](#) method from the HTML5 History API is available in the user's browser, you should remove the following parameters from your host page URL after passing them to the Office for the web iframe:

- `wdPreviousSession`
- `wdPreviousCorrelation`

Other `wd*` parameters must not be removed from the host page URL.

Host page headers

If the host page headers are not correctly set, some browsers might cache the response, which can result in the host page not properly reloading when the user navigates to it. This can result in errors if the user reloads a cached page after the [access token](#), or [access_token_ttl](#) have expired. One way this can happen is by reloading the page using the forward and back buttons. For more information about cache management, see [RFC 7234](#).

To prevent this, at the very least, set the following headers on the host page.

- [Cache-Control](#): no-cache, no-store
- [Expires](#): -1
- [Pragma](#): no-cache

Other headers, such as [Date](#) and [Vary](#) are useful as well.

Apply appropriate CSS styles

To ensure that the Microsoft 365 for the web applications behave appropriately when displayed through the host page, the host page must do the following:

- Apply specific CSS styles to the Microsoft 365 for the web iframe (lines 22-33) and the `body` element (lines 15-20).
- Set a `viewport` meta tag for mobile browsers (lines 11-12).
- Set an appropriate favicon for the page using the [favicon URL provided in WOPI discovery](#).

These requirements are shown in the [sample host page](#):

Code sample 2 - Markup from [SampleHostPage.html](#) illustrating appropriate styles and favicons in a host page

HTML

```
<head>
  <meta charset="utf-8">

  <!-- Enable IE Standards mode -->
  <meta http-equiv="x-ua-compatible" content="ie=edge">

  <title></title>
  <meta name="description" content="">
  <meta name="viewport"
    content="width=device-width, initial-scale=1, maximum-scale=1,
    minimum-scale=1, user-scalable=no">

  <link rel="shortcut icon" href="<OFFICE APPLICATION FAVICON URL>" />

  <style type="text/css">
    body {
      margin: 0;
      padding: 0;
      overflow: hidden;
      -ms-content-zooming: none;
    }
  </style>
```

```
#office_frame {  
  width: 100%;  
  height: 100%;  
  position: absolute;  
  top: 0;  
  left: 0;  
  right: 0;  
  bottom: 0;  
  margin: 0;  
  border: none;  
  display: block;  
}  
</style>  
</head>
```

Create new files using Microsoft 365 for the web

Article • 02/27/2025

Hosts can create new Microsoft 365 files using blank document templates from Microsoft 365 for the web. In order to support this, hosts use the `editnew` WOPI action as follows:

1. Create a zero-byte file with the appropriate file extension. Use `docx` for Word documents, `pptx` for PowerPoint presentations, and `xlsx` for Excel workbooks.
2. Run the `editnew` action on the newly created file. Microsoft 365 for the web detects that the file is zero bytes based on the [Size](#) property in the [CheckFileInfo](#) response.

Once the `editnew` action runs, Microsoft 365 for the web runs a [PutFile](#) operation on the file, overwriting it with template content appropriate to the file type.

This [PutFile](#) operation is done on an unlocked file. Hosts must ensure that [PutFile](#) operations on the unlocked, zero bytes files succeed. For more information, see the [PutFile](#) documentation.

Important

Because creating new files is done in two steps, it's possible for the [PutFile](#) operation to fail, leaving the zero-byte file. Since such a file isn't valid, hosts should expect and handle this case. The ultimate solution is left to the host. Options for handling this situation include:

- Hide zero-byte files from the user.
- Prevent users from opening zero-byte files in Microsoft 365 for the web.
- If a file is zero bytes, when opening Microsoft 365 for the web against the file, always use the `editnew` action. (This is only an option if the user has edit permissions to the document.)

This isn't an exhaustive list of options; ultimately, the host must decide the best approach to this issue.

Edit legacy binary document formats

Article • 02/27/2025

Microsoft 365 for the web doesn't support editing files in binary formats such as `doc`, `ppt`, and `xls`, directly. However, Microsoft 365 for the web can convert documents in the legacy formats to modern formats like `docx`, `pptx`, and `xlsx`, to let users edit the content in Microsoft 365 for the web.

Important

Conversion is almost always a lossless process, and there are typically very few, if any, user-visible changes to the layout and formatting of the document during conversion. However, this isn't always the case. Hosts should be aware that users might wish to revert to the previous binary version of the document after it's been converted.

The conversion capability is exposed in WOPI discovery as the `convert` action. To support editing binary file formats, hosts must support the `PutRelativeFile` WOPI operation. The conversion process is started by running the `convert` action on the binary file. Following is the high-level process:

1. The host runs the `convert` action on a binary file.
2. Microsoft 365 for the web retrieves the file from the host and converts it.
3. Microsoft 365 for the web sends the converted document back to the host by issuing a `PutRelativeFile` request against the original file ID.
4. Hosts use the `X-WOPI-FileConversion` header on the `PutRelativeFile` request to determine that the request is being made in the context of a file conversion. Hosts can then treat these requests differently than other `PutRelativeFile` requests.
5. After the document is successfully converted, Microsoft 365 for the web redirects the user to the `HostEditUrl` that's returned in the `PutRelativeFile` response.
Microsoft 365 for the web always redirects the topmost window (`window.top`).

Enabling convert and edit from within the Microsoft 365 for the web viewer

In the basic conversion flow described earlier, a user runs the `convert` action using some UI on the host. But hosts might wish to open a document first in the Microsoft 365

for the web viewer and use the Microsoft 365 for the web in-application `Edit` button to convert and edit the document, as is done with documents in editable formats.

Hosts can support this process in the same way that view -> edit transitions are typically supported. Hosts must do the following when opening binary documents using the `view` action:

1. Set the `UserCanWrite` property to `true`.
2. Set the `UserCanNotWriteRelative` property to `false` (or simply omit it).
3. Set the `HostEditUrl` property to a host URL that runs the `convert` action when loaded.

Following these steps ensures that the in-application `Edit` button displays. When clicked, this button navigates the user to the `HostEditUrl` provided for the binary file, which starts the conversion process and eventually redirects the user to the `HostEditUrl` for the newly converted document.

Hosts might optionally handle the in-application `Edit` button themselves by setting the `EditModePostMessage` property to `true` and handling the `UI_Edit` PostMessage.

Customizing the conversion process

In the basic conversion process, Microsoft 365 for the web creates a new file each time a user attempts to edit a file in a binary file format. For example, consider this scenario:

1. A user opens a binary file named `File.doc` in the Microsoft 365 for the web viewer.
2. The user clicks the `Edit` button in the Microsoft 365 for the web viewer.
3. The conversion process starts, and Microsoft 365 for the web calls `PutRelativeFile` on the host, creating a newly converted file, `File.docx`.
4. The user edits the newly converted document, then ends the editing session.
5. Later, the user returns and opens the original binary file, `File.doc`, in the Microsoft 365 for the web viewer.

At this point, the user might be confused as to why the changes made earlier aren't in the document. If the user attempts to edit the file again, Microsoft 365 for the web again converts it and creates a *second* converted file, for example `File1.docx`.

This can be confusing for users depending on how the host UI user experience is designed. Thus, it's important to consider how to manage user confusion around converted documents. There are three basic customization options that hosts can employ to help manage this.

First, the host can choose to display some UI to the user prior to beginning the conversion process. Because hosts ultimately control when the `convert` action is invoked, a host could choose to display a notification message when a user attempts to edit a binary document, informing them that the document will be converted. This can also apply to the in-application `Edit` button by setting the `EditModePostMessage` property to `true` and handling the `UI_Edit` `PostMessage`.

Second, the host can choose to handle converted documents in a unique way, by handling the `PutRelativeFile` operation differently when called from the conversion flow. The **X-WOPI-FileConversion** header tells hosts when the operation is being called from the conversion flow, so the host can choose how best to handle those requests.

Finally, the host can control where the user is navigated after conversion is complete. Microsoft 365 for the web navigates to the `HostEditUrl` that's returned in the `PutRelativeFile` response, which the host controls. So, hosts can customize where the user lands after the conversion is finished. This let hosts opt not to send the user directly to the Microsoft 365 for the web editor, but to any URL they wish. For example, a host might redirect the user to an interstitial page that informs them their document has been converted.

The following are some examples illustrating how these options can be used by hosts to change the user experience around file conversion. These examples are not meant to be exhaustive. Hosts can opt to customize the conversion process and flow in ways not described here.

Example 1

In the following example, the host helps the user understand the conversion process by naming the converted file so it's clear that the file was converted from a binary file.

1. A user selects a binary file in the host UI and chooses to edit it using Microsoft 365 for the web.
2. The conversion process starts, and Microsoft 365 for the web calls `PutRelativeFile` with the converted document content.
3. The host creates a new file as part of the `PutRelativeFile` request and appends `(Editable)` to the name of the file.
4. The user is navigated to a page that lets them edit the newly converted file in Microsoft 365 for the web.

Example 2

In the following example, the host wishes to hide the conversion process from the user to provide the most frictionless experience possible.

1. A user selects a binary file in the host UI and chooses to edit it using Microsoft 365 for the web.
2. The conversion process starts, and Microsoft 365 for the web calls [PutRelativeFile](#) with the converted document content.
3. Rather than create a new file, the host chooses to add the converted file as a new version to the existing binary file.
4. The user is navigated to a page that lets them edit the newly converted file in Microsoft 365 for the web.
5. The user can restore the binary version of the file by using the version history features within the host.

ⓘ Note

This approach might not be feasible for all hosts, depending on how file metadata and versions are handled within their system. However, it does offer the following benefits:

- The user only ever sees a single document both before and after the document is converted.
- Since there is always only a single document, the user always finds the *right* document. That is, if the user edited the file - which is likely since they invoked the conversion process by attempting to edit a binary document - then when they open the file a second time, their previous edits are there, just as they expect.

Example 3

In the following example, the host deems it important to inform users explicitly about the conversion process and its possible side effects.

1. A user selects a binary file in the host UI and chooses to edit it using Microsoft 365 for the web.
2. The host displays a notification message with the following text:

In order to edit **File.doc**, it must be converted to a modern file format. If the document doesn't look the same after it's converted, don't worry - you can always get the original file back if you need to.

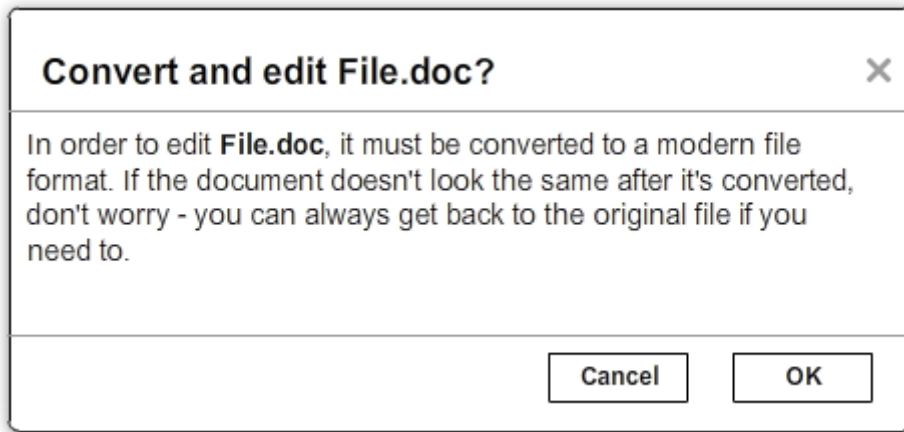


Figure 16 - Example conversion notification message

The user can cancel the conversion operation or choose to continue with it.

3. If the user chooses to continue, the host navigates them to a page that runs the `convert` action on the file.
4. The conversion process starts, and Microsoft 365 for the web calls `PutRelativeFile` with the converted document content.
5. The host returns a special URL in the `HostEditUrl` property in the `PutRelativeFile` response. Microsoft 365 for the web navigates the user to that URL once the conversion completes.
6. The user lands on the URL specified by the host, and sees the following message:

Your file, **File.doc**, has been converted to a new file, **File.docx**. The new file is in a modern file format, and the file extension has changed. If you don't need the original file any more, you can delete it.

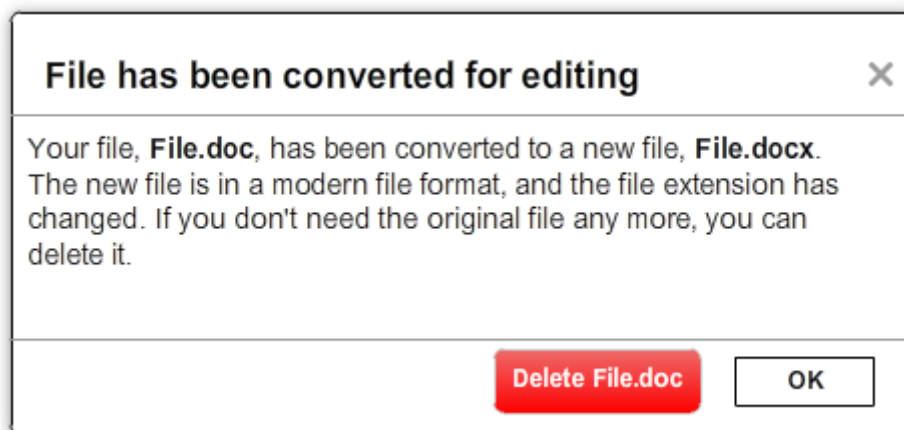


Figure 17 - Example conversion completed message

The message includes a button that the user can use to delete the original file immediately if they wish.

7. Once the user clicks `OK`, they're navigated to a page that invokes the `edit` action on the converted file.

Variant 3.1: Display post-conversion message in the Microsoft 365 for the web UI

In steps 5 and 6, rather than navigating the user to an interstitial page, the host can choose to append some parameters to the standard `HostEditUrl`. Then, when that `HostEditUrl` is navigated to, the host page uses the parameters that were added to the URL to determine that the dialog described in step 6 should be displayed. The host can display that notification above the Microsoft 365 for the web editor frame. This is similar to what hosts do when handling the `UI_Sharing` `PostMessage`.

Tip

Hosts must ensure that they properly use the `Blur_Focus` and `Grab_Focus` messages when drawing UI over the Microsoft 365 for the web frame.

Using PostMessage to interact with the Microsoft 365 for the web application iframe

Article • 02/27/2025

You can integrate your own UI into Microsoft 365 for the web applications. You can then use your UI for actions on Microsoft 365 documents, such as sharing.

To integrate with Microsoft 365 for the web in this way, set up the [HTML5 Web Messaging protocol](#). The Web Messaging protocol, also known as PostMessage, lets Microsoft 365 for the web frame communicate with its parent [host page](#), and vice-versa. The following example shows the general syntax for PostMessage. In this example, `otherWindow` is a reference to another window that `msg` posts to.

`otherWindow.postMessage(msg, targetOrigin)`

Arguments

- `msg` (*string*) – A string (or JSON object) that contains the message data.
- `targetOrigin` (*string*) – Specifies what the origin of `otherWindow` is for the event to be dispatched. This value is set to the [PostMessageOrigin](#) property provided in [CheckFileInfo](#). The literal string `*`, while supported in the PostMessage protocol, isn't allowed by Microsoft 365 for the web.

Message format

All messages posted to and from the Microsoft 365 for the web application frame are posted using the `postMessage()` function. Each message (the `msg` parameter in the `postMessage()` function) is a JSON-formatted object of the form:

message

`MessageId` (*string*)

The name of the message being posted.

SendTime (*long*)

The time the message was sent, expressed as milliseconds since midnight, January 1, 1970, UTC.

Tip

You can get this value in most modern browsers using the `Date.now()` method in JavaScript.

Values (*JSON-formatted object*)

The data associated with the message. This varies per message.

The following example shows the msg parameter for the `Host_PerfTiming` message.

JSON

```
{
  "MessageId": "Host_PerfTiming",
  "SendTime": 1329014075000,
  "Values": {
    "Click": 1329014074800,
    "Iframe": 1329014074950,
    "HostFrameFetchStart": 1329014074970,
    "RedirectCount": 1
  }
}
```

Sending messages to the Microsoft 365 for the web iframe

To send messages to the Microsoft 365 for the web iframe, set the [PostMessageOrigin](#) property in your WOPI [CheckFileInfo](#) response to the URL of your host page. If you don't do this, Microsoft 365 for the web ignores any messages you send to its iframe.

You can send the following messages; all others are ignored:

- `App_PopState`
- `Blur_Focus`
- `CanEmbed`
- `Grab_Focus`

- `Host_PerfTiming`
- `Host_PostmessageReady`

App_PopState

OneNote for the web

ⓘ Note

This message is only used by OneNote for the web and isn't required to integrate with Microsoft 365 for the web or Microsoft 365 for mobile. It's included for completeness but doesn't need to be implemented.

OneNote for the web integration isn't included in the Microsoft 365 - Cloud Storage Partner Program.

The App_PopState message signals the Microsoft 365 for the web application that state has been popped from the HTML5 History API to which the application should navigate using the URL. This message should be triggered from an `onpopstate` listener in the host page.

Values

Url (*string*)

The URL associated with the popped history state.

State (*JSON-formatted object*)

The data associated with the state.

Example message

JSON

```
{
  "MessageId": "App_PopState",
  "SendTime": 1329014075000,
  "Values": {
    "Url": "https://www.contoso.com/abc123/contents?wdtarget=pagexyz",
    "State": {
```



```
    "Value": 0
  }
}
```

Blur_Focus

The `Blur_Focus` message signals the Microsoft 365 for the web application to stop aggressively grabbing focus. Hosts should send this message whenever the host application UI is drawn over the Microsoft 365 for the web frame, so that the Microsoft 365 application doesn't interfere with the UI behavior of the host.

This message only affects Microsoft 365 for the web edit modes. It doesn't affect view modes.

Tip

When the host application displays UI over Microsoft 365 for the web, it should put a full-screen dimming effect over the Microsoft 365 for the web UI, so that it's clear that the Microsoft 365 application isn't interactive.

Values

Empty.

Example Message

JSON

```
{
  "MessageId": "Blur_Focus",
  "SendTime": 1329014075000,
  "Values": { }
}
```

CanEmbed

The `CanEmbed` message is sent by the host in response to a request to create a [HostEmbeddedViewUrl](#) using the `UI_FileEmbed` message.

Values

HostEmbeddedViewUrl (*string*)

A URI to a web page that provides access to a viewing experience for the file that can be embedded in another HTML page. This string is equivalent to the [HostEmbeddedViewUrl](#) in [CheckFileInfo](#).

Example message

JSON

```
{
  "MessageId": "CanEmbed",
  "SendTime": 1329014075000,
  "Values": {
    "HostEmbeddedViewUrl":
      "https://www.contosodrive.com/documents/1234/embedded/"
  }
}
```

Grab_Focus

The `Grab_Focus` message signals the Microsoft 365 for the web application to resume aggressively grabbing focus. Hosts should send this message whenever the host application UI that's drawn over the Microsoft 365 for the web frame is closing. This lets the Microsoft 365 application resume functioning.

This message only affects Microsoft 365 for the web edit modes. It doesn't affect view modes.

Values

Empty.

Example message

JSON

```
{
  "MessageId": "Grab_Focus",
  "SendTime": 1329014075000,
```

```
"Values": { }  
}
```

Host_IsFrameTrusted

The `Host_IsFrameTrusted` message is sent by the host in response to `App_IsFrameTrusted` message.

Values

`isTopFrameTrusted` (*boolean*)

A boolean value indicating whether the top frame is trusted.

Example message

JSON

```
{  
  "MessageId": "Host_IsFrameTrusted",  
  "SendTime": 1329014075000,  
  "Values": {  
    "isTopFrameTrusted": true  
  }  
}
```

Host_PerfTiming

Provides performance related timestamps from the host page. Hosts should send this message when the Microsoft 365 for the web frame is created so load performance can be more accurately tracked.

Values

`Click` (*integer*)

The timestamp, in ticks, when the user selected a link that launched the Microsoft 365 for the web application. For example, if the host exposed a link in its UI that launches an Microsoft 365 for the web application, this timestamp is the time the user originally selected that link.

Iframe (*integer*)

The timestamp, in ticks, when the host created the Microsoft 365 for the web iframe at the time the user selected the link.

HostFrameFetchStart (*integer*)

The result of the [PerformanceTiming.fetchStart](#) attribute, if the browser supports the [W3C NavigationTiming API](#). If the NavigationTiming API isn't supported by the browser, this value must be 0.

RedirectCount (*integer*)

The result of the [PerformanceNavigation.redirectCount](#) attribute, if the browser supports the [W3C NavigationTiming API](#). If the NavigationTiming API isn't supported by the browser, this value must be 0.

Example message

JSON

```
{
  "MessageId": "Host_PerfTiming",
  "SendTime": 1329014075000,
  "Values": {
    "Click": 1329014074800,
    "Iframe": 1329014074950,
    "HostFrameFetchStart": 1329014074970,
    "RedirectCount": 1
  }
}
```

Host_PostmessageReady

Microsoft 365 for the web delay-loads much of its JavaScript code, including most of its PostMessage senders and listeners. You might choose to follow this pattern in your WOPI host page. So, your outer host page and the Microsoft 365 for the web iframe must coordinate to ensure that each is ready to receive and respond to messages.

To enable this coordination, Microsoft 365 for the web sends the `App>LoadingStatus` message only after all its message senders and listeners are available. In addition,

Microsoft 365 for the web listens for the `Host_PostmessageReady` message from the outer frame. Until it receives this message, some UI, such as the `Share` button, is disabled.

Until your host page receives the `App>LoadingStatus` message, the Microsoft 365 for the web frame cannot respond to any incoming messages except `Host_PostmessageReady`. Microsoft 365 for the web does not delay-load its `Host_PostmessageReady` listener; it is available almost immediately upon iframe load.

If you're delay-loading your `PostMessage` code, ensure that your `App>LoadingStatus` listener is not delay-loaded. This ensures that you can receive the `App>LoadingStatus` message even if your other `PostMessage` code hasn't yet loaded.

Following is the typical flow:

1. Host page begins loading.
2. Microsoft 365 for the web frame begins loading. Some UI elements are disabled, because `Host_PostmessageReady` hasn't yet been sent by the host page.
3. Host page finishes loading and sends `Host_PostmessageReady`. No other messages are sent because the host page hasn't received the `App>LoadingStatus` message from the Microsoft 365 for the web frame.
4. Microsoft 365 for the web frame receives `Host_PostmessageReady`.
5. Microsoft 365 for the web frame finishes loading and sends `App>LoadingStatus` to host page.
6. Host page and Microsoft 365 for the web communicate by using other `PostMessage` messages.

Values

Empty.

Example message

JSON

```
{
  "MessageId": "Host_PostmessageReady",
  "SendTime": 1329014075000,
```

```
"Values": { }  
}
```

Listening to messages from the Microsoft 365 for the web iframe

The Microsoft 365 for the web iframe sends messages to the host page. On the receiving end, the host page receives a `MessageEvent`. The `origin` property of the `MessageEvent` is the origin of the message, and the `data` property is the message being sent. The following code example shows how you might consume a message.

JavaScript

```
function handlePostMessage(e) {  
    // The actual message is contained in the data property of the event.  
    var msg = JSON.parse(e.data);  
  
    // The message ID is now a property of the message object.  
    var msgId = msg.MessageId;  
  
    // The message parameters themselves are in the Values  
    // parameter on the message object.  
    var msgData = msg.Values;  
  
    // Do something with the message here.  
}  
window.addEventListener('message', handlePostMessage, false);
```

The host page receives the following messages, all others are ignored:

- App_LoadingStatus
- App_PushState
- Edit_Notification
- File_Rename
- UI_Close
- UI_Edit
- UI_FileEmbed
- UI_FileVersions
- UI_Sharing
- UI_Workflow

Common values

In addition to message-specific values passed with each message, Microsoft 365 for the web sends the following common values with every outgoing PostMessage:

ui-language (*string*)

The LCID of the language Microsoft 365 for the web was loaded in. This value won't match the value provided using the `UI_LLCC` placeholder. Instead, this value is the numeric LCID value (as a string) that corresponds to the language used. For more information, see [What languages does Microsoft 365 for the web support?](#).

This value might be needed in the event that Microsoft 365 for the web renders using a language different than the one requested by the host. This might occur if Microsoft 365 for the web isn't localized in the language requested. In that case, the host might choose to draw its own UI in the same language that Microsoft 365 for the web used.

wdUserSession (*string*)

The ID of the Microsoft 365 for the web session. This value is logged by the host and used when [troubleshooting](#) issues with Microsoft 365 for the web. For more information on this value, see [Session IDs](#).

App_IsFrameTrusted

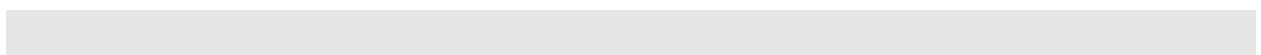
The `App_IsFrameTrusted` message posts by the Microsoft 365 for the web application frame to initiate handshake flow. The host is expected to check whether the top frame is trusted and post `Host_IsFrameTrusted` message back to Microsoft 365 for the web application. The host first needs to include a query string: `&sftc=1` to url of POST request sent to Microsoft 365 for the web application, to indicate to Microsoft 365 for the web application that the host supports frame trust post Message.

`sftc` stands for *SupportsFrameTrustedPostMessage*. Only when `&sftc` is included in the query string, will the `App_IsFrameTrusted` message be posted by Microsoft 365 for the web application frame to initiate handshake flow.

Values

[Common values](#) only.

Example Message



JSON

```
{
  "MessageId": "App_IsFrameTrusted",
  "SendTime": 1329014075000,
  "Values": {
    "wdUserSession": "3692f636-2add-4b64-8180-42e9411c4984",
    "ui-language": "1033"
  }
}
```

App_LoadingStatus

The `App_LoadingStatus` message posts after the Microsoft 365 for the web application frame has loaded. Until the host receives this message, it must assume that the Microsoft 365 for the web frame can't react to any incoming messages except `Host_PostmessageReady`.

Values

DocumentLoadedTime (*long*)

The time that the frame was loaded.

Example message

JSON

```
{
  "MessageId": "App_LoadingStatus",
  "SendTime": 1329014075000,
  "Values": {
    "DocumentLoadedTime": 1329014074983,
    "wdUserSession": "3692f636-2add-4b64-8180-42e9411c4984",
    "ui-language": "1033"
  }
}
```

App_PushState

OneNote for the web

ⓘ Note

This message is only used by OneNote for the web and isn't required to integrate with Microsoft 365 for the web or Microsoft 365 for mobile. It's included for completeness but doesn't need to be implemented.

OneNote for the web integration isn't included in the Microsoft 365 - Cloud Storage Partner Program.

The `App_PushState` message posts when the user changes the state of Microsoft 365 for the web application in such a way as to return to it later, requesting to capture it in the HTML 5 History API. In receiving this message, the Host page should use `history.pushState` to capture the state for a potential later state pop.

To send this message, the `AppStateHistoryPostMessage` property in the `CheckFileInfo` response from the host must be set to `true`. Otherwise, Microsoft 365 for the web doesn't send the message.

Values

Url (*string*)

The URL associated with the message.

State (*JSON-formatted object*)

The data associated with the state.

Example message

JSON

```
{
  "MessageId": "App_PushState",
  "SendTime": 1329014075000,
  "Values": {
    "Url": "https://www.contoso.com/abc123/contents?wdtarget=pagexyz",
    "State": {
      "Value": 0
    },
    "wdUserSession": "3692f636-2add-4b64-8180-42e9411c4984",
    "ui-language": "1033"
  }
}
```

```
}  
}
```

Edit_Notification

The `Edit_Notification` message posts when the user first makes an edit to a document, and every five minutes thereafter, if the user has made edits in the previous five minutes. Hosts can use this message to gauge whether users are interacting with Microsoft 365 for the web. In coauthoring sessions, hosts can't use the WOPI calls for this purpose.

To send this message, the [EditNotificationPostMessage](#) property in the [CheckFileInfo](#) response from the host must be set to `true`. Otherwise, Microsoft 365 for the web won't send this message.

Values

[Common values](#) only.

Example message

JSON

```
{  
  "MessageId": "Edit_Notification",  
  "SendTime": 1329014075000,  
  "Values": {  
    "wdUserSession": "3692f636-2add-4b64-8180-42e9411c4984",  
    "ui-language": "1033"  
  }  
}
```

File_Rename

The `File_Rename` message posts when the user renames the current file in Microsoft 365 for the web. The host can use this message to optionally update the UI, such as the title of the page.

ⓘ Note

If the host doesn't return the [SupportsRename](#) parameter in their [CheckFileInfo](#) response, the rename UI won't be available in Microsoft 365 for the web.

Values

NewName (*string*)

The new name of the file.

Example message

JSON

```
{
  "MessageId": "File_Rename",
  "SendTime": 1329014075000,
  "Values": {
    "NewName": "Renamed Document",
    "wdUserSession": "3692f636-2add-4b64-8180-42e9411c4984",
    "ui-language": "1033"
  }
}
```

UI_Close

The `UI_Close` message posts when the Microsoft 365 for the web application is closing, either due to an error or a user action. Typically, the URL specified in the [CloseUrl](#) property in the [CheckFileInfo](#) response is displayed. However, hosts can intercept this message instead and navigate in an appropriate way.

To send this message, the [ClosePostMessage](#) property in the [CheckFileInfo](#) response from the host must be set to `true`. Otherwise, Microsoft 365 for the web won't send this message.

Values

[Common values](#) only.

Example message

JSON

```
{
  "MessageId": "UI_Close",
  "SendTime": 1329014075000,
  "Values": {
```

```
    "wdUserSession": "3692f636-2add-4b64-8180-42e9411c4984",  
    "ui-language": "1033"  
  }  
}
```

UI_Edit

The `UI_Edit` message posts when the user activates the *Edit* UI in Microsoft 365 for the web. This UI is only visible when using the `view` action.

To send this message, the `EditModePostMessage` property in the `CheckFileInfo` response from the host must be set to `true`. Otherwise, Microsoft 365 for the web won't send this message and redirects the inner iframe to an edit action URL instead.

Hosts can choose to use this message in cases where they want more control over the user's transition to edit mode. For example, a host might wish to prompt the user for some additional host-specific information before navigating.

Values

[Common values](#) only.

Example message

JSON

```
{  
  "MessageId": "UI_Edit",  
  "SendTime": 1329014075000,  
  "Values": {  
    "wdUserSession": "3692f636-2add-4b64-8180-42e9411c4984",  
    "ui-language": "1033"  
  }  
}
```

UI_FileEmbed

The `UI_FileEmbed` message posts when the user activates the *Embed* UI in Microsoft 365 for the web. The host should use this message to trigger the creation of a [HostEmbeddedViewUrl](#), which the host then passes back to the WOPI client using the `CanEmbed` message.

To send this message, the [FileEmbedCommandPostMessage](#) property in the [CheckFileInfo](#) response from the host must be set to `true`, and [FileEmbedCommandUrl](#) must be provided. Otherwise, Microsoft 365 for the web won't send this message.

Also, Microsoft 365 for the web won't send the message if a [HostEmbeddedViewUrl](#) is provided in the [CheckFileInfo](#) response. In this case, since a [HostEmbeddedViewUrl](#) is already provided, there's no need to retrieve it from the host via PostMessage.

Values

[Common values](#) only.

Example message

JSON

```
{
  "MessageId": "UI_FileEmbed",
  "SendTime": 1329014075000,
  "Values": {
    "wdUserSession": "3692f636-2add-4b64-8180-42e9411c4984",
    "ui-language": "en-us"
  }
}
```

UI_FileVersions

The `UI_FileVersions` message posts when the user activates the *Previous Versions* UI in Microsoft 365 for the web. The host should use this message to trigger any custom file version history UI.

To send this message, the [FileVersionPostMessage](#) property in the [CheckFileInfo](#) response from the host must be set to `true`. Otherwise, Microsoft 365 for the web won't send this message.

Values

[Common values](#) only.

Example message

JSON

```
{
  "MessageId": "UI_FileVersions",
  "SendTime": 1329014075000,
  "Values": {
    "wdUserSession": "3692f636-2add-4b64-8180-42e9411c4984",
    "ui-language": "1033"
  }
}
```

UI_Sharing

The `UI_Sharing` message posts when the user activates the *Share* UI in Microsoft 365 for the web. The host should use this message to trigger any custom sharing UI.

To send this message, the `FileSharingPostMessage` property in the `CheckFileInfo` response from the host must be set to `true`. Otherwise, Microsoft 365 for the web won't send this message.

Values

[Common values](#) only.

Example message

JSON

```
{
  "MessageId": "UI_Sharing",
  "SendTime": 1329014075000,
  "Values": {
    "wdUserSession": "3692f636-2add-4b64-8180-42e9411c4984",
    "ui-language": "1033"
  }
}
```

UI_Workflow

The `UI_Workflow` message posts when the user activates the `Workflow` UI in Microsoft 365 for the web. The host should use this message to trigger any custom workflow UI.

To send this message, the `WorkflowPostMessage` property in the `CheckFileInfo` response from the host must be set to `true`. Otherwise, Microsoft 365 for the web won't send this message.

Values

WorkflowType (*string*)

The [WorkflowType](#) associated with the message. This matches one of the values provided by the host in the [WorkflowType](#) property in [CheckFileInfo](#).

Example message

JSON

```
{
  "MessageId": "UI_Workflow",
  "SendTime": 1329014075000,
  "Values": {
    "WorkflowType": "Submit",
    "wdUserSession": "3692f636-2add-4b64-8180-42e9411c4984",
    "ui-language": "1033"
  }
}
```

Co-authoring using Microsoft 365 for the web

Article • 02/27/2025

Important

Participants in the Microsoft 365 - Cloud Storage Partner Program must support document editing and multi-user co-authoring.

Microsoft 365 for the web supports multiple users editing a document at the same time. Co-authoring in Microsoft 365 for the web includes real-time content updates between all users editing the document, as well as presence information and real-time cursor tracking for each user.

There are no unique WOPI host requirements beyond those described in this documentation in order to support co-authoring. But the guidelines around file IDs and locks, as described in the [Key concepts](#) section, are important for enabling co-authoring.

Benefits from co-authoring support

Editing documents requires that Microsoft 365 for the web take a [lock](#) on the file to ensure that only Microsoft 365 for the web is writing to the file. In cases where co-authoring isn't supported, this causes two problems.

First, users are unable to make changes to the file while someone else is editing it. You avoid this problem when you support co-authoring. With co-authoring in Microsoft 365 for the web, users are never locked out of editing a document.

Second, while Microsoft 365 for the web makes every attempt to unlock files after users have finished editing documents, there are circumstances where this might not happen. If [Unlock](#) isn't called successfully, the file stays locked until the lock times out. This means that a user can be locked out of editing a file even if they were the one who originally locked it. When co-authoring is supported, the fact that the file is locked isn't a problem. Microsoft 365 for the web allows a user to edit the document as if they're co-authoring with themselves.

So, in addition to the obvious benefits for multi-user editing that come with real-time co-authoring, co-authoring in Microsoft 365 for the web provides benefits for single-user editing as well.

How co-authoring works in Microsoft 365 for the web

When multiple users edit a single document using Microsoft 365 for the web, the Microsoft 365 for the web service manages the document changes and any necessary merges internally.

When a user attempts to edit a document, Microsoft 365 for the web takes a lock using the [Lock](#) operation and the access token of the current user. When additional users then attempt to edit the same document, Microsoft 365 for the web will verify those users have permission to edit the document by calling [CheckFileInfo](#) using each user's access token. If the `CheckFileInfo` call succeeds and the user has permissions to edit, they'll join the editing session already in progress.

Users who are collaborating then see the document content update in real-time as other users edit. But Microsoft 365 for the web only calls [PutFile](#) periodically with the updated document contents. There are three critical questions to consider in co-authoring sessions:

1. **Auto-save frequency:** How frequently does the application call [PutFile](#)?
2. **PutFile access token:** Which user's access token is used when [PutFile](#) is called?
3. **Permissions check frequency:** How often does the application verify that a user still has appropriate permissions to edit the document?

Question 3 is important because [PutFile](#) is called using a single user's access token, so a WOPI host is only able to check permissions of the user whose access token being used. Hosts rely on Microsoft 365 for the web to periodically verify that users still have edit permissions.

The answers to these questions differ between the Microsoft 365 for the web applications. The following table summarizes the behavior for each application, and the following sections describe the behavior in more detail.

Table 1 - Summary of co-authoring behavior for Microsoft 365 applications

 Expand table

Application	Auto-save frequency	PutFile access token	Permissions check frequency
Word	Every 30 seconds if document is updated.	The access token of the user who made the most recent change to the document.	At least every five minutes.

Application	Auto-save frequency	PutFile access token	Permissions check frequency
Excel	Every two minutes.	The access token of the user who joined the editing session most recently.	At least every 15 minutes.
PowerPoint	Every 60 seconds if document is updated.	The access token of the user who made the most recent change to the document.	At least every five minutes.

Word for the web co-authoring behavior

Auto-save frequency: Every 30 seconds if document is updated. In other words, if users are actively editing a document, [PutFile](#) is called every 30 seconds. But if users stop editing for a period of time, Word for the web doesn't call [PutFile](#) until document changes are made again.

PutFile access token: For each auto-save interval, Word for the web uses the access token of the user who made the most recent change to the document. In other words, if User A and B both make changes to the document within the same auto-save interval, but User B made the last change, Word for the web uses User B's access token when calling [PutFile](#). The file has both users' changes, but the PutFile request uses User B's access token.

But if User A made a change in one auto-save interval, and User B made a change in another auto-save interval, Word for the web makes two PutFile requests, each using the access token of the user who made the change.

Permissions check frequency: Word for the web verifies that a user has permissions by calling `CheckFileInfo` at least every five minutes while the user is in an active session.

Excel for the web co-authoring behavior

Auto-save frequency: Every two minutes.

PutFile access token: Excel for the web always uses the access token of the user who joined the editing session most recently. This user is called the *principal user*. If the principal user leaves the session, the last user who joined the session becomes the principal user. In other words, if User A starts editing, User A is the principal user. If User B then joins the session, User B becomes the principal user, and Excel for the web uses User B's access token when calling [PutFile](#). The file has both users' changes, but the

PutFile request uses User B's access token. If User C then joins the session, User C becomes the principal user.

If User C then leaves the session, User B becomes the principal user, and User B's access token is used when calling PutFile.

Permissions check frequency: Excel for the web verifies that a user has permissions by calling **RefreshLock** at least every 15 minutes while the user is in an active session.

PowerPoint for the web co-authoring behavior

Auto-save frequency: Every 60 seconds if a document is updated. In other words, if users are actively editing a document, **PutFile** is called every 60 seconds. But if users stop editing for a period of time, PowerPoint for the web won't call **PutFile** until document changes are made again.

ⓘ Note

During a single-user editing session, PowerPoint for the web only calls **PutFile** every three minutes. During an active co-authoring session, that frequency is increased to every 60 seconds.

PutFile access token: For each auto-save interval, PowerPoint for the web uses the access token of the user who made the most recent change to the document. In other words, if User A and B both make changes to the document within the same auto-save interval, but User B made the last change, PowerPoint for the web uses User B's access token when calling **PutFile**. The file has both users' changes, but the **PutFile** request uses User B's access token.

But if User A made a change in one auto-save interval, and User B made a change in another auto-save interval, PowerPoint for the web makes two **PutFile** requests, each using the access token of the user who made the change.

Permissions check frequency: PowerPoint for the web verifies that a user has permissions by calling **CheckFileInfo** at least every five minutes while the user is in an active session.

Scenarios

The following scenarios illustrate the behavior WOPI hosts can expect for each Microsoft 365 for the web application when users are co-authoring.

All scenarios described here assume the following baseline flow.

 Note

The pattern of WOPI calls described in the following section isn't meant to be absolutely accurate. Microsoft 365 for the web might make additional WOPI calls beyond those described here. These scenarios are meant only to illustrate the key behavioral aspects of the Microsoft 365 for the web applications. These scenarios aren't an absolute transcript of WOPI traffic between Microsoft 365 for the web and a WOPI host.

Scenario baseline

1. User A begins editing a document.
2. Microsoft 365 for the web calls [CheckFileInfo](#) using User A's access token to verify the user has edit permissions.
3. Microsoft 365 for the web calls [Lock](#) using User A's access token.
4. User B tries to edit the same document.
5. Microsoft 365 for the web calls [CheckFileInfo](#) using User B's access token to verify the user has edit permissions.

Key points

- Microsoft 365 for the web always verifies that each user has appropriate edit permissions by calling [CheckFileInfo](#) using that user's access token before letting them join the edit session.
- [Lock](#) is always called using the access token of the first user to start editing the document.
- If users leave the editing session while others are still editing, Microsoft 365 for the web calls other lock-related operations, such as [Unlock](#) or [RefreshLock](#), using the access tokens of other users that are still editing.

Scenario 1

1. User A continues editing the document.
2. User B makes no changes.

Table 2 - Co-authoring scenario 1

Application	PutFile access token used	Additional notes
Word for the web	User A, since User B isn't editing.	
Excel for the web	User B	
PowerPoint for the web	User A, since User B isn't editing.	

Scenario 2

1. User A continues editing the document.
2. User B also edits the document.

Table 3 - Co-authoring scenario 2

 Expand table

Application	PutFile access token used	Additional notes
Word for the web	Either access token can be used, depending on which user changed the document most recently during the auto-save interval.	
Excel for the web	User B	
PowerPoint for the web	Either access token can be used, depending on which user changed the document most recently during the auto-save interval.	

Scenario 3

1. User A leaves the editing session by closing the Microsoft 365 for the web application or navigating away.
2. User B continues editing the document.
3. User C tries to edit the same document.
4. Microsoft 365 for the web calls [CheckFileInfo](#) using User C's access token to verify the user has edit permissions.

Table 4 - Co-authoring scenario 3

 Expand table

Application	PutFile access token used	Additional notes
Word for the web	Any access token can be used, depending on which user changed the document most recently during the auto-save interval.	
Excel for the web	User C	
PowerPoint for the web	Any access token can be used, depending on which user changed the document most recently during the auto-save interval.	

Scenario 4

1. User A continues editing the document.
2. User B is in the session but isn't editing the document.
3. While the editing session is in progress, User B's permissions to edit the document are removed.

Table 5 - Co-authoring scenario 4

 Expand table

Application	PutFile access token used	Additional notes
Word for the web	User A, since User B isn't editing.	After no more than five minutes, Word for the web calls <code>CheckFileInfo</code> with User B's access token. That call indicates that User B no longer has edit permissions, so User B is removed from the editing session. User A can continue editing the document.
Excel for the web	User B	Once User B's edit permission is removed, all <code>PutFile</code> requests for the session fail. In no more than three minutes after the first <code>PutFile</code> failure, all users, including User A, are removed from the editing session.
PowerPoint for the web	User A, since User B isn't editing.	After no more than five minutes, PowerPoint for the web calls <code>CheckFileInfo</code> with User B's access token. That call indicates that User B no longer has edit permissions, so User B is removed from the editing session. User A can continue editing the document.

Scenario 5

1. User A continues editing the document.

2. User B also continues editing the document.
3. While the editing session is in progress, User B's permissions to edit the document are removed.

Table 6 - Co-authoring scenario 5

[Expand table](#)

Application	PutFile access token used	Additional notes
Word for the web	Either access token can be used, depending on which user changed the document most recently during the auto-save interval.	If a <code>PutFile</code> request is made using User B's access token, it fails, and all users, including User A, are removed from the editing session. If <code>PutFile</code> is never called with User B's access token, after no more than five minutes, Word for the web calls <code>CheckFileInfo</code> with User B's access token. That call indicates that User B no longer has edit permissions, so User B is removed from the editing session. User A can continue editing the document.
Excel for the web	User B	Same as scenario 4.
PowerPoint for the web	Either access token can be used, depending on which user changed the document most recently during the auto-save interval.	If a <code>PutFile</code> request is made using User B's access token, it fails, and all users, including User A, are removed from the editing session. If <code>PutFile</code> is never called with User B's access token, after no more than five minutes, PowerPoint for the web calls <code>CheckFileInfo</code> with User B's access token. That call indicates that User B no longer has edit permissions, so User B is removed from the editing session. User A can continue editing the document.

Scenario 6

1. User A continues editing the document.
2. User B also continues editing the document.
3. While the editing session is in progress, User A's permissions to edit the document are removed.

Table 7 - Co-authoring scenario 6

[Expand table](#)

Application	PutFile access token used	Additional notes
Word for the web	Either access token can be used, depending on which user changed the document most recently during the auto-save interval.	If a <code>PutFile</code> request is made using User A's access token, it fails, and all users, including User B, are removed from the editing session. If <code>PutFile</code> is never called with User A's access token, after no more than five minutes, Word for the web calls <code>CheckFileInfo</code> with User A's access token. That call indicates that User A no longer has edit permissions, so User A is removed from the editing session. User B can continue editing the document.
Excel for the web	User B	After no more than 15 minutes, Excel for the web calls <code>RefreshLock</code> with User A's access token. That call fails since User A no longer has edit permissions. So User A is removed from the editing session. User B can continue editing the document.
PowerPoint for the web	Either access token can be used, depending on which user changed the document most recently during the auto-save interval.	If a <code>PutFile</code> request is made using User A's access token, it fails, and all users, including User B, are removed from the editing session. If <code>PutFile</code> is never called with User A's access token, after no more than five minutes, PowerPoint for the web calls <code>CheckFileInfo</code> with User A's access token. That call indicates that User A no longer has edit permissions, so User A is removed from the editing session. User B can continue editing the document.

Supporting document editing for business users

Article • 02/27/2025

Business users require an Microsoft 365 subscription to edit files in Microsoft 365 for the web. To support this scenario, Microsoft 365 for the web requires that hosts specify a user as a business user when using any actions that include the [BUSINESS_USER](#) placeholder value, such as `edit`, `editnew`, and `view`.

When business users open documents for editing, Microsoft 365 for the web may validate that the user has a valid Microsoft 365 subscription that includes Microsoft 365 applications.

Important

You must implement the Business User Flow to support document editing for end-users with a Microsoft 365 business license.

Indicate that a user is a business user

The first requirement to support document editing for business users is to indicate a user as a business user. Do the following:

1. Set the [BUSINESS_USER](#) placeholder value on the Microsoft 365 for the web action URL. **This parameter must always be on the action URL for business users.**

Important

Hosts must properly set the [BUSINESS_USER](#) placeholder value on the Microsoft 365 for the web action URL for all actions that include the placeholder, including the `view` action.

2. Include the [LicenseCheckForEditIsEnabled](#) property in the [CheckFileInfo](#) response. This property must always be set to `true` for business users.
3. Include the [HostEditUrl](#) in the [CheckFileInfo](#) response. This property must be included in the [CheckFileInfo](#) response for business users. The [HostEditUrl](#) redirects the user back to the host edit page after the subscription check completes.

4. *(Optional)* To include a direct download link to a user's file, you can also include the [DownloadUrl](#) in the [CheckFileInfo](#) response.

ⓘ Important

For security purposes, both of these URLs must be served from domains that are on the [Redirect domain allow list](#).

Tracking users' subscription status

To provide a better experience for users with Microsoft 365 subscriptions, hosts can implement the [PutUserInfo](#) WOPI operation. Microsoft 365 for the web uses this operation to pass back the user information, including subscription status, to the host. The host can, in turn, pass the `UserInfo` string back to Microsoft 365 for the web on subsequent [CheckFileInfo](#) responses for that user. Microsoft 365 for the web may use the data in the `UserInfo` string to determine whether a subscription check is needed. Hosts must treat the `UserInfo` string as an opaque string.

Customize Microsoft 365 for the web with CheckFileInfo properties

Article • 01/29/2025

You can customize the user interface and experience of Microsoft 365 for the web by using [CheckFileInfo](#) properties and setting up the [PostMessage API](#).

CheckFileInfo properties

The `CheckFileInfo` properties follow.

CloseUrl

When the `close` UI is activated, Office for the web navigates the outer page (`window.top.location`) to the URI provided.

Hosts can also use the [ClosePostMessage](#) property to indicate a PostMessage should be sent when the `close` UI is activated rather than navigate to a URL. Or set the [CloseButtonClosesWindow](#) property to indicate that the `close` UI should close the browser tab or window (`window.top.close`).

If the [CloseUrl](#), [ClosePostMessage](#), and [CloseButtonClosesWindow](#) properties are omitted, the `close` UI is hidden in Microsoft 365 for the web.

ⓘ Note

The `close` UI never displays when using the `embedview` action.

For more information, see [\[CloseUrl\]\(~/rest/files/CheckFileInfo/CheckFileInfo-Response.md#closeurl\)](#) in the WOPI REST documentation.

DownloadUrl

If a `DownloadUrl` isn't provided, Microsoft 365 for the web hides all UI to download the file.

If provided, Word and PowerPoint for the web display UI to download the file. When a user attempts to download the file, Word and PowerPoint ensure that the latest

document content is saved back to the WOPI host before navigating the user to the `DownloadUrl` for downloading the file.

Excel for the web

Excel for the web doesn't use the `DownloadUrl` when users click the **Download a Copy** button. Excel for the web always downloads the file directly from the Office for the web server. This has the following effects:

- Any content that Excel for the web doesn't currently support, such as diagrams, are stripped from the downloaded file.
- Excel for the web doesn't guarantee that the latest document content is saved back to the WOPI host before downloading the file.
- **Download a Copy** contains the most recent document edits, even when the `DownloadUrl` is implemented incorrectly and doesn't point to the latest version of the document.

For more information, see [DownloadUrl](#) in the WOPI REST documentation.

FileSharingUrl

If provided, when the `Share` UI is activated, Office for the web opens a new browser window to the URI provided.

Hosts can also use the [FileSharingPostMessage](#) property to indicate a `PostMessage` should be sent when the *Share* UI is activated rather than navigate to a URL.

If neither the [FileSharingUrl](#) nor the [FileSharingPostMessage](#) properties are set, the *Share* UI is hidden in Microsoft 365 for the web.

For more information, see [FileSharingUrl](#) in the WOPI REST documentation.

HostEditUrl

This URL is used by Microsoft 365 for the web to navigate between view and edit mode.

For more information, see [FileSharingUrl](#) in the WOPI REST documentation.

HostViewUrl

This URL is used by Microsoft 365 for the web to navigate between view and edit mode.

For more information, see [HostViewUrl](#) in the WOPI REST documentation.

SignoutUrl

If you don't provide this property, no sign out UI is shown in Microsoft 365 for the web.

For more information, see [SignoutUrl](#) in the WOPI REST documentation.

CloseButtonClosesWindow

If set to `true`, Microsoft 365 for the web closes the browser window or tab (`window.top.close`) when the *Close* UI in Microsoft 365 for the web is activated.

If Microsoft 365 for the web displays an error dialog when booting, dismissing the dialog is treated as a close button activation with respect to this property.

Hosts can also use the [CloseUrl](#) property to indicate that the outer frame should be navigated (`window.top.location`) when the `Close` UI is activated rather than closing the browser tab or window. Or set the [ClosePostMessage](#) property to indicate a `PostMessage` should be sent when the `Close` UI is activated.

If the [CloseUrl](#), [ClosePostMessage](#), and [CloseButtonClosesWindow](#) properties are omitted, the *Close* UI is hidden in Office for the web.

ⓘ Note

The `Close` UI never displays when using the `embedview` action.

For more information, see [CloseButtonClosesWindow](#) in the WOPI REST documentation.

Breadcrumb properties

Microsoft 365 for the web displays the [Breadcrumb properties](#) if they're provided.

PostMessage properties

The `PostMessage` properties control the behavior of Microsoft 365 for the web with respect to incoming `PostMessages`. If you're using the `PostMessage` extensibility features, you must set the [PostMessageOrigin](#) property to ensure that Microsoft 365 for the web accepts messages from your outer frame. For more information about `PostMessage`

integration, see [Using PostMessage to interact with the Microsoft 365 for the web application iframe](#).

In cases where a PostMessage is triggered by the user activating some Microsoft 365 for the web UI, such as [FileSharingPostMessage](#) or [EditModePostMessage](#), Microsoft 365 for the web does nothing when the relevant UI is activated except send the appropriate PostMessage. Thus, hosts must accept and handle the relevant messages when the Microsoft 365 for the web UI is triggered. Otherwise, the Microsoft 365 for the web UI appears to do nothing when activated.

If the PostMessage API isn't supported (for example, the user's browser doesn't support it, or the browser security settings prohibit it, and so on), Microsoft 365 for the web UI that triggers a PostMessage is hidden.

AppStateHistoryPostMessage

A **Boolean** value that, when set to `true`, indicates the host outer frame supports the use of [HTML5 Session History](#). The outer frame should then expect to receive `App_PushState` PostMessages and propagate `onpopstate` events to Microsoft 365 for the web through the `App_PopState` PostMessage.

OneNote Online

ⓘ Note

This message is only used by OneNote for the web and isn't required to integrate with Microsoft 365 for the web or Microsoft 365 for mobile. It's included for completeness but doesn't need to be implemented.

OneNote for the web integration isn't included in the Microsoft 365 - Cloud Storage Partner Program.

ClosePostMessage

A **Boolean** value that, when set to `true`, indicates the host expects to receive the `UI_Close` PostMessage when the `Close` UI in Microsoft 365 for the web is activated.

Hosts should use the [CloseUrl](#) property to indicate that the outer frame should be navigated (`window.top.location`) when the `Close` UI is activated rather than sending a

PostMessage. Or to set the [CloseButtonClosesWindow](#) property to indicate that the `Close` UI should close the browser tab or window (`window.top.close`).

If the [CloseUrl](#), [ClosePostMessage](#), and [CloseButtonClosesWindow](#) properties are omitted, the `Close` UI is hidden in Office for the web.

Important

The [CloseUrl](#) must always be provided in order for the `Close` UI to appear in Microsoft 365 for the web, even if [ClosePostMessage](#) is `true`.

Most PostMessage-related properties don't require that the corresponding URL property be provided in order to enable the relevant UI in Microsoft 365 for the web. [CloseUrl](#) is an exception to this.

See also

[Best practices when using PostMessage properties](#)

Note

The `Close` UI never displays when using the `embedview` action.

EditModePostMessage

A **Boolean** value that, when set to `true`, indicates the host expects to receive the `UI_EditPostMessage` when the `Edit` UI in Microsoft 365 for the web is activated.

If this property is not set to `true`, Microsoft 365 for the web navigates the inner iframe URL to an edit action URL when the `Edit` UI is activated.

EditNotificationPostMessage

A **Boolean** value that, when set to `true`, indicates the host expects to receive the `Edit_Notification` PostMessage.

FileEmbedCommandPostMessage

A **Boolean** value that, when set to `true`, indicates the host expects to receive the `UI_FileEmbed` PostMessage when the Embed UI in Microsoft 365 for the web is

activated.

Hosts can also use the [FileEmbedCommandUrl](#) property to indicate that a new browser window should be opened when the Embed UI is activated rather than sending a `PostMessage`. The [FileEmbedCommandUrl](#) property is ignored completely if the `FileEmbedCommandPostMessage` property is set to `true`.

If neither the [FileEmbedCommandUrl](#) nor the [FileSharingPostMessage](#) properties are set, the Embed UI is hidden in Microsoft 365 for the web unless a [HostEmbeddedViewUrl](#) is provided in [CheckFileInfo](#).

FileSharingPostMessage

A **Boolean** value that, when set to `true`, indicates the host expects to receive the `UI_Sharing` `PostMessage` when the `Share` UI in Microsoft 365 for the web is activated.

Hosts can also use the [FileSharingUrl](#) property to indicate that a new browser window should be opened when the `Share` UI is activated rather than sending a `PostMessage`. The [FileSharingUrl](#) property is ignored completely if the `FileSharingPostMessage` property is set to `true`.

If neither the [FileSharingUrl](#) nor the [FileSharingPostMessage](#) properties are set, the `Share` UI is hidden in Microsoft 365 for the web.

FileVersionPostMessage

A **Boolean** value that, when set to `true`, indicates the host expects to receive the `UI_FileVersions` `PostMessage` when the `Previous Versions` UI (*File* > *Info* > *Previous Versions*) in Microsoft 365 for the web is activated.

Hosts can also use the [FileVersionUrl](#) property to indicate that a new browser window should be opened when the Previous Versions UI is activated rather than sending a `PostMessage`. The [FileVersionUrl](#) property is ignored completely if the `FileVersionPostMessage` property is set to `true`.

If neither the [FileVersionUrl](#) nor the [FileVersionPostMessage](#) properties are set, the `Previous Versions` UI is hidden in Microsoft 365 for the web.

PostMessageOrigin

A **string** value indicating the domain that the [host page](#) is sending and receiving `PostMessages` to and from. Microsoft 365 for the web only sends outgoing

PostMessages to this domain, and only listens to PostMessages from this domain.

Tip

This value is used as the *targetOrigin* when Microsoft 365 for the web uses the [HTML5 Web Messaging protocol](#) . So, it must include the scheme and host name. If you're serving your pages on a non-standard port, you must include the port as well. The literal string `*`, while supported in the PostMessage protocol, isn't allowed by Microsoft 365 for the web.

WorkflowPostMessage

! Pre-release property - not yet used by any WOPI client

A **Boolean** value that, when set to `true`, indicates the host expects to receive the UI_Workflow PostMessage when the `Workflow` UI in Microsoft 365 for the web is activated.

Hosts can also use the [WorkflowUrl](#) property to indicate that a new browser window should be opened when the `Workflow` UI is activated rather than sending a PostMessage. The [WorkflowUrl](#) property is ignored completely if the `WorkflowPostMessage` property is set to `true`.

If neither the [WorkflowUrl](#) nor the [WorkflowPostMessage](#) properties are set, the `Workflow` UI is hidden in Microsoft 365 for the web.

Important

This value is ignored if [WorkflowType](#) isn't provided.

Best practices when using PostMessage properties

The WOPI protocol is designed for a variety of scenarios and environments. While PostMessage is a useful integration technique for web-browser-based WOPI clients such as Microsoft 365 for the web, it's not usable in other WOPI clients, such as Microsoft 365 for mobile.

To provide maximum compatibility with all types of WOPI clients, hosts should set corresponding URL properties when using PostMessage properties. For example, when

setting [FileSharingPostMessage](#) to `true`, hosts should also provide a [FileSharingUrl](#). This enables a WOPI client that can't use PostMessage to navigate the user to a URL that does allow them to manage sharing the file.

While the primary reason to provide corresponding URL properties for PostMessage properties is for non-browser-based WOPI clients, there are legitimate reasons to do this for Microsoft 365 for the web as well. In particular, users might use browsers that don't support PostMessage. While all officially supported Microsoft 365 for the web browsers support PostMessage, when users are on unsupported browsers, Microsoft 365 for the web strives to give them the best possible experience. Providing the URL properties enables users to access Microsoft 365 for the web features even in browsers where PostMessage won't work.

Customizing the Microsoft 365 for the web viewer UI using CheckFileInfo

The following table describes the available buttons and UI in the Microsoft 365 for the web viewer and what [CheckFileInfo](#) properties can be used to remove them.

 Expand table

Button	How to disable
Edit in Browser	Two options: 1. (preferred) Set UserCanWrite to <code>false</code> in the CheckFileInfo response (or omit it since the default for all boolean properties in CheckFileInfo is <code>false</code>) 2. Omit the HostEditUrl and EditModePostMessage properties from the <code>CheckFileInfo</code> response
Share	Omit the FileSharingUrl and FileSharingPostMessage properties from the <code>CheckFileInfo</code> response
Download / Download as PDF	Omit the DownloadUrl property from the <code>CheckFileInfo</code> response
Print	Set the DisablePrint property to <code>true</code> in the <code>CheckFileInfo</code> response
Comments	Can't be hidden
Find	Can't be hidden
Translate	Can't be hidden
Help	Can't be hidden

Button	How to disable
Give Feedback	Can't be hidden
Terms of Use	Can't be hidden
Privacy and Cookies	Can't be hidden
Accessibility Mode	Can't be hidden
Start Slide Show	Can't be hidden
Embed	Omit the HostEmbeddedViewUrl and HostEmbeddedEditUrl properties from the <code>CheckFileInfo</code> response
Refresh Selected Connection	Can't be hidden
Refresh All Connections	Can't be hidden
Calculate Workbook	Can't be hidden
Save a Copy	Set the UserCanNotWriteRelative property to <code>true</code> in the <code>CheckFileInfo</code> response

Integrate Microsoft 365 for the web with document workflow processes

Article • 02/27/2025

 Warning

This documentation is for an upcoming feature and might undergo dramatic changes prior to final release. Pre-release features are available in the [Test environment](#) only.

Hosts can integrate Microsoft 365 for the web into workflow processes, so that users can work with UI directly in Microsoft 365 for the web to manage documents in a workflow.

For example, consider a host that caters to customers in the education field. The host can provide a way for students to submit documents as part of an assignment. With the Microsoft 365 for the web workflow features, hosts can configure Microsoft 365 for the web to display a **Submit** button directly in the Microsoft 365 for the web UI that, when activated, displays host-specific UI letting the student submit the document.

Supporting workflows in Microsoft 365 for the web

In Microsoft 365 for the web, documents that are a part of workflows display a button that replaces the **Share** button. The button's text is specific to the [WorkflowType](#).

 Expand table

WorkflowType	Microsoft 365 for the web workflow button text
Assign	Manage Assignment
Submit	Turn In

 Important

In Microsoft 365 for the web, the *Assign* and *Submit* workflow types are mutually exclusive. Only one of the two workflow types should be set for a given document session.

Much like **Share**, when clicked, the workflow button can either navigate to a URL in a new browser tab or window or send a message to the [host frame](#).

To navigate to a URL, set the [WorkflowUrl](#) property to the URL to which Microsoft 365 for the web should navigate.

To post a message to the host frame instead, set the [WorkflowPostMessage](#) property to `true`. When clicked, Microsoft 365 for the web sends the `UI_Workflow` message to the host frame. The message includes the type of workflow that the document is a part of.

Tip

Microsoft 365 for the web always saves the latest copy of the document to the host when the workflow UI is activated. This ensures that the host always has the latest copy of the document to use in the workflow.

Important considerations

WOPI clients, including Microsoft 365 for the web, aren't designed to understand what the workflow is and how it behaves. In other words, when you use the [WorkflowUrl](#) or [WorkflowPostMessage](#) properties, Microsoft 365 for the web displays the workflow button and behaves appropriately when triggered. But if, for example, you want to prevent a user from submitting the same document multiple times, you must handle that in your own UI. In other words, Microsoft 365 for the web and other WOPI clients don't know anything about the workflow process except that a document is a part of a workflow and to display the appropriate UI. This provides a WOPI host great flexibility in their workflows, but the host is also responsible for managing the workflow as a whole.

Enable Microsoft 365 Add-ins provided by a WOPI host

Article • 02/27/2025

The WOPI Add-in user experience is different from that of a standard Microsoft 365 for the web session in three important ways:

- By default, Add-ins are not enabled for users connecting to Microsoft 365 through a WOPI host.
- If Add-ins are enabled, the Add-in button will be displayed in the Microsoft 365 ribbon. However, the Add-in store can not be used by a WOPI connected user. If the user clicks the Add-in button and attempts to sign in to the Add-in store, they will receive a message that the store has been disabled.
- Add-ins must be either preinstalled or sideloaded. See the Microsoft documentation for more information about [preinstalling](#) and [sideloading](#) Add-ins.

Getting started with WOPI Add-ins

1. Join the Cloud Storage Partner Program (CSPP). Learn more about the program from [Cloud Storage Partner Program Overview](#) and complete the [Office Cloud Storage Partner Program Application](#) [↗].
2. Use the [CSPP Environment Change Request form](#) [↗] to request that your CSPP WOPI environment be updated to have Add-ins enabled.
3. Create your Add-in. See the [Add-in developer documentation](#) for more information.
4. Complete the required security handshake with your host for Add-ins to run (see the [Security requirements to host Add-ins](#) section). This must be done for every Add-in.
5. Preinstall Add-ins to provide them for your users. Submit an array of one or more Add-ins as a Form field value as part of the POST request sent to Microsoft 365 for the web server (see the [Preinstall Add-ins](#) section).

Security requirements to host add-ins

Add-ins won't activate in the Microsoft 365 document without completing the appropriate check. This ensures the host isn't currently embedded in an unexpected

environment. The check outlined below is completed as part of [PostMessage](#).

The [App_IsFrameTrusted message](#) is posted by Microsoft 365 on the web application frame to initiate the handshake flow. The host is expected to determine whether the top frame is trusted and post a [Host_IsFrameTrusted message](#) back to the Microsoft 365 for the web application. The host first needs to append the query string `&sftc=1` to the URL of POST request sent to the Microsoft 365 on the web application. This message must be sent as a string.

Preinstall Add-ins

WOPI hosts can allow their choice of Add-ins to be enabled. All of these Add-ins can be [sideloaded](#), which is part of the development process. For an add-in to be preinstalled and ready for users to open, the Add-in must be available in [AppSource](#).

Send the list of Add-ins the host page provides as a Form field value. This is done as part of the [POST request body sent to Microsoft 365 for the web server](#). The data format of the Form is an encoded string version of the JSON. The Form data field name of data is `host_install_addins`.

The following code snippet shows the Form field name and value format of `host_install_addins`.

HTML

```
<body>
  <form id="office_form" name="office_form" target="office_frame" action="
<OFFICE_ACTION_URL>" method="post">
    <input name="access_token" value="<ACCESS_TOKEN_VALUE>"
type="hidden" />
    <input name="access_token_ttl" value="<ACCESS_TOKEN_TTL_VALUE>"
type="hidden" />
    <input name="host_install_addins" value="<JSON_OF_ADD-INS_LIST>"
type="hidden" />
  </form>
</body>
```

The value of `<JSON_OF_ADD-INS_LIST>` is a list of Add-ins reference data in the following stringified-JSON format. Note that you must use an [HTML-encoding tool](#) to convert the JSON string. You cannot send the string without encoding it.

JSON

```
[
  { addinId: "WA123456781", type: "TaskPaneApp" },
  { addinId: "WA123456782", type: "TaskPaneApp" },
  { addinId: "WA123456783", type: "TaskPaneApp" },
  { addinId: "WA123456784", type: "TaskPaneApp" }
]
```

Add-in restrictions

Currently, only Add-ins with [Add-in commands](#) are supported.

Testing

To troubleshoot enabling or preinstalling Add-ins, use a tool that can intercept and display the HTTP Requests from, and Responses to, your WOPI host. Test the setup by mimicking the data payload of Add-ins sent by your WOPI host page before you implement it in your page. This section explains how to do that with [Fiddler](#), but you should use whichever tool is familiar to you.

1. Get the ID for the Add-in from [AppSource](#). The ID is the unique part of the Add-in's URL. For example, the [Emoji Keyboard Add-in](#) has the ID WA104380121.
2. Record the JSON as you would for the POST command with the ID from step 1. For example, `[{"addinId":"WA104380121","type":"TaskPaneApp"}]`. The type is always `TaskPaneApp`.
3. Use an [HTML-encoding tool](#) to convert the JSON string from step 2. The previous example would end up like this: `%5B%7B%22addinId%22%3A%22WA104380121%22%2C%22type%22%3A%22TaskPaneApp%22%7D%5D`.
4. In Fiddler, add the following section to `FiddlerScript`, in the function `OnBeforeRequest`. The string appended to the request body is the encoded string from step 3.

JavaScript

```
if (oSession.PathAndQuery.StartsWith("/we/wordeditorframe.aspx?"))
{
    oSession.utilSetRequestBody(
        oSession.GetRequestBodyAsString() +

        "&host_install_addins=%5B%7B%22addinId%22%3A%22WA104380121%22%2C%22type
```



```
%22%3A%22TaskPaneApp%22%7D%5D" );  
}
```

5. To test with Excel for the web, use the following condition instead.

JavaScript

```
if  
(oSession.PathAndQuery.StartsWith("/x/_layouts/xlviewerinternal.aspx?")  
)
```

Troubleshooting

To troubleshoot any data format or encoding issue, examine the payload that Fiddler mimics in the Fiddler request panel. You can also check the `console.log` output in browser debugger console window.

Support

For support specific to Microsoft 365 Add-ins integration, visit [Microsoft 365 Add-ins additional resources](#) to find the appropriate support channel.

Frequently asked questions

Why don't I see the Add-in button in the ribbon?

The Add-in button is only displayed if Add-ins have been enabled by the CSPP team for your WOPI host.

Why can't I use an Add-in to insert another file into the current document in Microsoft 365 for the web - even though I can do that with the Microsoft 365 desktop apps?

In Excel, functionality was added to the Add-in platform at a different time and may require an update from the Add-in developer. In Word, this functionality is currently in preview and will be released after sufficient testing.

Why don't Add-ins persist from one session to the next? For example, if I install an Add-in while writing a document, then close the document, when I open Word again the Add-in isn't there and I have to reload it.

When a developer creates an Add-in, they often use a setting to enable persistence between sessions. This isn't something that occurs by default. Another reason would be that the Add-in being used was sideloaded by the end user, which is meant for developers to test their Add-ins. Finally, web browsers offer different settings for blocking information storage based on user privacy settings. What is stored in the browser cache has an impact on what Add-ins are available in Microsoft 365 on the web between sessions.

Can I send information (such as a user ID) from the host page to the web Add-in?

Add-ins use the Microsoft 365 JavaScript APIs to interact with the Microsoft 365 host and the document. You can find additional information in the [Microsoft 365 JavaScript API reference documentation](#).

Do I need to register my Add-in with the CSPP Team if I am not hosting my Add-in in the Store?

Yes. Please note that preinstall functionality will not work for Add-ins that are not submitted to AppSource.

Is there a way to stop my users from being prompted to trust Add-ins?

No. It is important for users to understand the activity happening in their Microsoft 365 session and the potential liabilities that come with web services.

Test Microsoft 365 for the web integration

Article • 01/29/2025

Before starting the [Launch process](#), do the following testing on your integration.

WOPI Validator tests

The automated tests in the following categories must be passing in the WOPI validator:

- HostFrameIntegration (ValidLanguagePlaceholderValues might be skipped if the host isn't using the [UI_LLCC](#) or [DC_LLCC](#) placeholder values)
- BaseWopiViewing
- CheckFileInfoSchema
- EditFlows
- Locks
- PutRelativeFile or PutRelativeFileUnsupported
- RenameFileIfCreateChildFileIsNotSupported or RenameFileIfCreateChildFileIsSupported (applicable only if the host supports [RenameFile](#))
- FileVersion
- PutUserInfo (applicable only if the host supports [PutUserInfo](#))
- ProofKeys (applicable only for hosts implementing [proof key validation](#))

Important

All of the applicable tests listed above *must* pass in order to go to production. We don't make exceptions to this requirement.

Tip

Hosts aren't required to support [RenameFile](#), [PutRelativeFile](#) or use [proof key validation](#). However, hosts that support these operations, or use application

features that rely on them, those test groups must be passing.

For example, [binary file conversion](#) requires the [PutRelativeFile](#) operation be implemented, so hosts that support conversion must also pass the PutRelativeFile tests.

Microsoft 365 for the web feature verification

Many Microsoft 365 for the web features rely on a host's WOPI implementation. You should test the following features to help ensure your WOPI implementation is correct and that the Microsoft 365 for the web integration is well-executed.

Co-authoring

Co-authoring support is a major boon to users, but it's also used to verify your implementation of file IDs and lock-related WOPI operations. **For this reason, multi-user co-authoring must be testable using the test accounts provided.**

Important

The Microsoft 365 for the web applications have unique behavior with respect to co-authoring. So, it's critical to test co-authoring in all three applications.

To check that co-authoring behaves as expected, you'll need at least two different user accounts. Then, follow these steps:

1. As User A, share a document with User B.
2. Open the document in edit mode as User A.
3. Open that same document in edit mode as User B.
4. Check that both instances of the Microsoft 365 for the web application are participating in the co-authoring session.
5. Make edits to the document as both users and ensure that both instances of the application remain connected to the co-authoring session.
6. After making some edits, leave the session and verify that the saved file contains the edits made by both User A and User B.

Common issues

1. If the users remain in different sessions (as in, co-authoring doesn't occur), it likely means your WOPI file IDs aren't consistent. See [file ID](#) for more information.

2. If one of the users is *kicked out* of the session while editing, it likely means that you're rejecting lock-related requests that come from a different user than the one who originally took the lock. WOPI locks aren't user-owned. For more information, see [Lock](#).

Single-user co-authoring

While the typical co-authoring scenario is two or more users collaborating on a single document in real-time, the feature also provides other benefits as outlined in [Benefits from co-authoring support](#).

To check that single-user co-authoring behaves as expected:

1. Open a document in edit mode.
2. Open a document in edit mode using the same user account originally used, but in a different browser.
3. Check that both instances of the Microsoft 365 for the web application are participating in the co-authoring session.

Downloaded files should have most recent document changes

If you are providing a [DownloadUrl](#), you should ensure that a file downloaded using the Microsoft 365 for the web **Download a Copy** buttons contain the most recent edits. To test this:

1. Open a document in edit mode.
2. Make an edit to the document and wait for the *Saved to <HostName>* text to display.
3. Click the **File > Save As > Download a Copy** button.
4. Check that the downloaded file has the most recent edits you made to the document.

Download a Copy should not re-direct

As describe in [DownloadUrl](#), Microsoft 365 for the web expects that when directing users to the [DownloadUrl](#), the file is immediately downloaded. This URL should not direct the user to some separate UI to download the file.

Rename

If you support renaming documents within Microsoft 365 for the web (as in, [SupportsRename](#) is `true`), you should check that the rename operation behaves as expected. To test this:

1. Open a document in edit mode.
2. Click on the document name in the top title bar.
3. Rename the document.
4. Exit the Microsoft 365 for the web application and check that the file was renamed.

If you're displaying the document name in the browser window and tab using the HTML `title` tag, you should check that the document name is updated after the file is renamed. If it's not, check that you're properly handling the `File_Rename` PostMessage.

Save As in Excel for the web

Excel for the web supports saving an open document as a new copy of that document using the **File > Save As > Save As** button. This feature uses the [PutRelativeFile](#) WOPI operation. You should test that this feature works as expected.

UI integration

Ensure you follow the [UI guidelines](#) as well as the Microsoft Cloud Storage Partner Program Integration Terms.

WOPI Validator

Article • 02/10/2025

📘 Important

Current status: Operational

💡 Tip

The WOPI Validator is also available as an open-source tool. See <https://github.com/Microsoft/wopi-validator-core> for more information.

To assist hosts in verifying their WOPI implementation, Microsoft 365 for the web provides a WOPI Validator that executes a test suite against a host's WOPI implementation. The test suite verifies a variety of things, including that the semantics for all of the WOPI operations ([CheckFileInfo](#), [GetFile](#), [PutFile](#), etc.) are correct and that request/response headers are set properly. New tests are added regularly.

The WOPI Validator is an Microsoft 365 for the web application similar to Word for the web or PowerPoint for the web. It uses the `.wopitest` file extension. The WOPI Validator is included in the [WOPI discovery](#) XML just like all other Microsoft 365 for the web applications.

📘 Important

WOPI hosts should use the [Test environment](#) to ensure that they are running the latest version of the WOPI Validator.

XML

```
<app name="WopiTest" checkLicense="true">
  <action name="view" urlsrc="https://onenote.officeapps-
df.live.com/hosting/WopiTestFrame.aspx" ext="wopitest"/>
  <action name="getinfo" urlsrc="https://onenote.officeapps-
df.live.com/hosting/GetWopiTestInfo.ashx" ext="wopitest"/>
</app>
```

The Validator provides two [WOPI actions](#), `view` and `getinfo`, which can be used to trigger the test suite.

Warning

The test suite will test operations like [PutFile](#), so the contents of the `.wopitest` file will be destroyed.

In addition, some tests create new files or containers using the [PutRelativeFile](#), [CreateChildFile](#), and [CreateChildContainer](#) operations. While the Validator attempts to clean up these files, if there are errors in the WOPI implementation, these clean up actions may fail, leaving behind these test files.

If that should happen, you must clean up these test files manually. If you don't, subsequent test runs may fail.

Interactive WOPI validation

The simplest way to use the Validator is to use the *view* action. To use the *view* action hosts should treat `.wopitest` files the same way other Microsoft 365 documents are treated. In other words, hosts should do the following:

1. Launch a [host page](#) pointed at the `.wopitest` file. Ideally, this should be the same host page used to host regular Microsoft 365 for the web sessions. This will allow the Validator to test things like PostMessage and do some validation on the way the Microsoft 365 for the web iframe was loaded.
2. The host page will create and navigate the Microsoft 365 for the web iframe to the *view* action URL provided in [WOPI discovery](#). The [WOPIsrc](#) and [access token](#) should be provided just like with all other actions.
3. The WOPI Validator will load and display a number of test groups. Each test group can be expanded to reveal the individual tests that it contains. You can run tests individually, by test group, or run all tests using the `Run All` button.

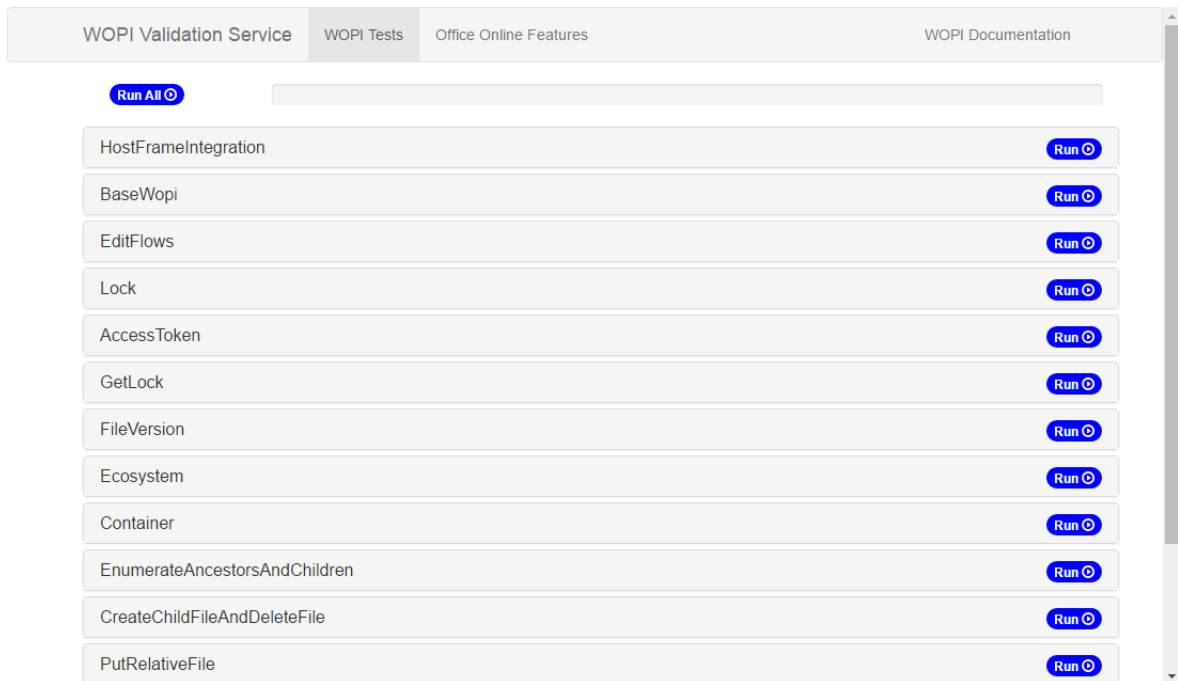


Figure 6 - WOPI Validator UI

Tests can either pass, fail, or be skipped. Before executing any tests, Microsoft 365 for the web will do some basic validation (e.g. confirm the file really has the `.wopitest` file extension) and check any applicable pre-requisites. Any test whose pre-requisites are not met will simply be skipped. For example, the tests in the `EditFlows` test group require the `SupportsUpdate` property to be set to `true`. If it is not, the tests in that group will all be skipped.

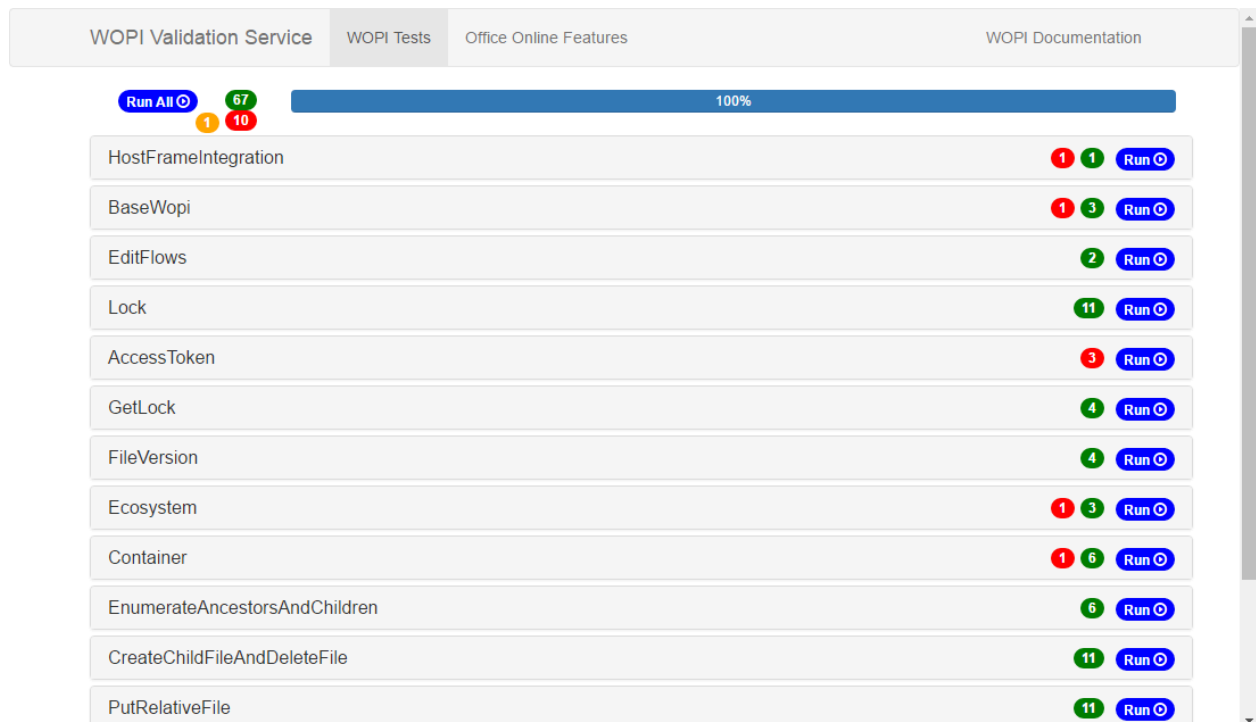


Figure 7 - Tests can pass, fail, or be skipped

Once a test has been run, you can click on it to see the each request that was issued by the test and the response data. If the test failed or was skipped, the reason will be displayed just under the test name. You can click on the specific request that failed and see more information about what the test was expecting. If you are implementing [proof key validation](#), you can use the `Current Proof Key Data` and `Old Proof Key Data` buttons to see the intermediate data on how the request was signed, which is extremely useful when debugging a proof key validation implementation.

The screenshot displays the 'WOPI Validation Service' interface. At the top, there are navigation links: 'WOPI Validation Service', 'WOPI Tests', 'Office Online Features', and 'WOPI Documentation'. The main heading is 'ecosystem.GetRootContainerInvalidAccessToken details'. Below this, it states 'GetRootContainer request failed.' There are two tabs: 'GetEcosystem' and 'GetRootContainer', with 'GetRootContainer' being the active tab. The 'Failures:' section lists two items: 'Incorrect StatusCode. Expected: 401, Actual: 200' and 'Incorrect StatusCode. Expected: 404, Actual: 200'. The 'TargetUrl:' is 'http://server.wopi.tylerbutler.com/wopi/wopi/ecosystem/root_container_pointer?access_token=INVALID'. The 'Time Taken:' is '78 ms'. The 'RequestHeaders:' section shows a scrollable list of headers including 'X-WOPI-Proof', 'X-WOPI-ProofOld', 'X-WOPI-TimeStamp', and 'Authorization: Bearer INVALID'. Below the headers are two buttons: 'Current Proof Key Data' (highlighted in green) and 'Old Proof Key Data' (highlighted in blue). The 'ResponseStatusCode:' is '200'. The 'ResponseHeaders:' section shows a scrollable list of headers including 'Content-Length', 'Cache-Control', 'Content-Type', 'Set-Cookie', 'Server', 'X-Powered-By', and 'Date'. The 'ResponseData (up to first 8000 characters):' section shows a JSON object with 'ContainerInfo' and 'ContainerPointer' fields.

WOPI Validation Service WOPI Tests Office Online Features WOPI Documentation

ecosystem.GetRootContainerInvalidAccessToken details

GetRootContainer request failed.

GetEcosystem GetRootContainer

Failures:

- Incorrect StatusCode. Expected: 401, Actual: 200
- Incorrect StatusCode. Expected: 404, Actual: 200

TargetUrl:

http://server.wopi.tylerbutler.com/wopi/wopi/ecosystem/root_container_pointer?access_token=INVALID

Time Taken: 78 ms

RequestHeaders:

X-WOPI-Proof: Kq6jhGHyKQjuR/I5qx3ZK8Q2CnH1wLFMzsKtKgO1vmhJ2585FCNCa/PhPFbeOY1Vo6RyndoupXh9VUXyo80a81SUBeNfcESfjgg1kaOT+naMHGaNheCN5XLZ
X-WOPI-ProofOld: ZTtykwDs04QK/pAjdcGu+upP9/Q8wEgFy80vHeEkZCSHpBqJQ6Pa+gs6vm7G2Iy6167XN9MTiz5M7OZ5T20+8Ax84aEYp340JF6eposEpdJ1iRZr+RUP>
X-WOPI-TimeStamp: 635991159286983829
Authorization: Bearer INVALID

Current Proof Key Data Old Proof Key Data

ResponseStatusCode:

200

ResponseHeaders:

Content-Length: 375
Cache-Control: private
Content-Type: application/json
Set-Cookie: session=eyJfaWQiOmsiIGIiOiJNemhtWVdwaFpqRTJPREpqT0Zd05ETmINV05t1RaaU5tUX1aV0ZoTTJRP5J9fQ.Ch0ZGQ.e29pAoPa3SCQhZRDpPg8onZf
Server: Microsoft-IIS/8.0
X-Powered-By: ASP.NET
Date: Tue, 17 May 2016 21:05:28 GMT

ResponseData (up to first 8000 characters):

```
{
  "ContainerInfo": {
    "Name": "content",
    "UserCanCreateChildContainer": true,
    "UserCanCreateChildFile": true,
    "UserCanDelete": true,
    "UserCanRename": false
  },
  "ContainerPointer": {
```

Figure 8 - Example WOPI validation results

Tip

For ease of testing, we strongly recommend that hosts support the `.wopitest` file extension just like all other file extensions supported by Microsoft 365 for the web and included in [WOPI discovery](#). This is especially important while testing, since it provides any user a quick and easy way to run the validation test suite.

Automated WOPI validation

The WOPI Validator exposes a second action, `getinfo`. The `getinfo` action is designed to be used server-to-server. Instead of launching a [host page](#), the host can simply do the following:

1. Issue a [GET](#) request to the `getinfo` action URL provided in WOPI discovery. The `WOPIsrc`, `access token`, and `access_token_ttl` should be provided just like with all other actions.

ⓘ Note

The `getinfo` action only supports [GET](#) requests, so the `access token`, an `access_token_ttl` values must be appended to the URL instead of being passed as [POST](#) parameters.

2. Microsoft 365 for the web will do some basic validation (e.g. confirm the file really has the `.wopitest` extension) and then return a JSON-formatted array of test URLs.
3. Hosts should then make a [GET](#) request to each test URL. Microsoft 365 for the web will run the specified test and return results in a simple JSON object. No changes to the URL are needed; the necessary parameters are included already on the URL returned from the Validator.

This is intended for automated use. For example, a host may wish to run this validation as part of rolling out new versions of their WOPI host.

Automated WOPI validation using a command-line tool

There are two options to automate WOPI validation. The first is to use the open-source validation tool at <https://github.com/Microsoft/wopi-validator-core>. This tool runs the same test suite as the hosted validator, but does not rely on any Microsoft 365 for the web server infrastructure. This makes it ideal for use in automated testing or for testing servers that are not connected to the internet.

The second option is to use the Python-based command-line tool at <https://github.com/Microsoft/wopi-validator-cli-python> instead of launching a [host page](#). This tool uses the `getinfo` action URL provided in [WOPI discovery](#) to execute the WOPI Validator, so it is simply an alternative way of executing the test cases in the hosted Validator.

Verify that requests originate from Microsoft 365 for the web by using proof keys

Article • 02/27/2025

When processing WOPI requests from Microsoft 365 for the web, you might want to verify that these requests are coming from Microsoft 365 for the web. To do this, you use proof keys.

Microsoft 365 for the web signs every WOPI request with a private key. The corresponding public key is available in the **proof-key** element in the WOPI discovery XML. The signature is sent with every request in the **X-WOPI-Proof** and **X-WOPI-ProofOld** HTTP headers.

The signature is assembled from information that's available to the WOPI host when it processes the incoming WOPI request. To verify that a request came from Microsoft 365 for the web, you:

- Create the expected value of the proof headers.
- Use the public key provided in WOPI discovery to decrypt the proof provided in the **X-WOPI-Proof** header.
- Compare the expected proof to the decrypted proof. If they match, the request originated from Microsoft 365 for the web.
- Ensure that the **X-WOPI-TimeStamp** header is no more than 20 minutes old.

When validating proof keys, if a request isn't signed properly, the host must return a [500 Internal Server Error](#) .

Note

Requests to the [FileUrl](#) aren't signed. The FileUrl is used exactly as provided by the host, so it doesn't necessarily include the access token, which is required to construct the expected proof.

Tip

The [Microsoft 365 for the web GitHub repository] (<https://github.com/Microsoft/Microsoft>  365-Online-Test-Tools-and-

Documentation) contains a set of unit tests that hosts can adapt to verify proof key validation implementations. For more information, see [Proof key unit tests](#).

Constructing the expected proof

To construct the expected proof, you assemble a byte array consisting of the access token, the URL of the request (in uppercase), and the value of the **X-WOPI-TimeStamp** HTTP header from the request. Each of these values must be converted to a byte array. In addition, you include the length, in bytes, of each of these values.

To convert the access token and request URL values, which are strings, to byte arrays, ensure the original strings are in UTF-8 first, then convert the UTF-8 strings to byte arrays. Convert the **X-WOPI-TimeStamp** header to a *long* and then into a byte array. Don't treat it as a string.

Then, assemble the data as follows:

- Four bytes that represent the length, in bytes, of the `access_token` on the request.
- The `access_token`.
- Four bytes that represent the length, in bytes, of the full URL of the WOPI request, including any query string parameters.
- The WOPI request URL in uppercase. All query string parameters on the request URL should be included.
- Four bytes that represent the length, in bytes, of the **X-WOPI-TimeStamp** value.
- The **X-WOPI-TimeStamp** value.

The following code samples show the construction of an expected proof in C#, Java, and Python.

Code sample 3 - Constructing the expected in proof in C#

C#

```
public bool Validate(ProofKeyValidationInput testCase)
{
    // Encode values from headers into byte[]
    var accessTokenBytes = Encoding.UTF8.GetBytes(testCase.AccessToken);
    var hostUrlBytes =
    Encoding.UTF8.GetBytes(testCase.Url.ToUpperInvariant());
    var timeStampBytes = EncodeNumber(testCase.Timestamp);

    // prepare a list that will be used to combine all those arrays together
    List<byte> expectedProof = new List<byte>(
        4 + accessTokenBytes.Length +
        4 + hostUrlBytes.Length +
```

```

        4 + timeStampBytes.Length);

expectedProof.AddRange(EncodeNumber(accessTokenBytes.Length));
expectedProof.AddRange(accessTokenBytes);
expectedProof.AddRange(EncodeNumber(hostUrlBytes.Length));
expectedProof.AddRange(hostUrlBytes);
expectedProof.AddRange(EncodeNumber(timeStampBytes.Length));
expectedProof.AddRange(timeStampBytes);

// create another byte[] from that list
byte[] expectedProofArray = expectedProof.ToArray();

// validate it against current and old keys in proper combinations
bool validationResult =
    TryVerification(expectedProofArray, testCase.Proof,
        _currentKey.CspBlob) ||
    TryVerification(expectedProofArray, testCase.OldProof,
        _currentKey.CspBlob) ||
    TryVerification(expectedProofArray, testCase.Proof,
        _oldKey.CspBlob);

// TODO:
// in real code you should also check that TimeStamp header is no more
// than 20 minutes old
// but because we're using predefined test cases to validate that the
// method works
// we can't do it here.
return validationResult;
}

```

Retrieving the public key

Microsoft 365 for the web provides two different public keys as part of the WOPI discovery XML: the current key and the old key. Two keys are necessary because the discovery data is meant to be cached by the host, and Microsoft 365 for the web periodically rotates the keys it uses to sign requests. When the keys are rotated, the current key becomes the old key, and a new current key generates. This helps minimize the risk that a host doesn't have updated key information from WOPI discovery when Microsoft 365 for the web rotates keys.

Both keys are represented in the discovery XML in two different formats. One format is for WOPI hosts that use the .NET framework. The other format can be imported in a variety of different programming languages and platforms.

Using .NET to retrieve the public key

If your application is built on the .NET framework, use the contents of the `value` and `oldvalue` attributes of the `proof-key` element in the WOPI discovery XML. These two attributes contain the Base64-encoded public keys that are exported by using the [RSACryptoServiceProvider.ExportCspBlob](#) method of the .NET Framework.

To import this key in your application, decode it from Base64 then import it by using the [RSACryptoServiceProvider.ImportCspBlob](#) method.

Using the RSA modulus and exponent to retrieve the public key

For hosts that don't use the .NET framework, Microsoft 365 for the web provides the RSA modulus and exponent directly. The modulus and exponent of the current key are found in the `modulus` and `exponent` attributes of the *proof-key* element in the WOPI discovery XML. The modulus and exponent of the old key are found in the `oldmodulus` and `oldexponent` attributes. All four of these values are Base64-encoded.

The steps to import these values differ based on the language, platform, and cryptography API that you're using.

The following examples show how to import the public key by using the modulus and exponent in both Java and Python (using the PyCrypto library).

Java

Code sample 4 - Generating a public key from a modulus and exponent in Java

Java

```
private static RSAPublicKey getPublicKey( String modulus, String exponent )
throws Exception
{
    BigInteger mod = new BigInteger( 1, DatatypeConverter.parseBase64Binary(
modulus ) );
    BigInteger exp = new BigInteger( 1, DatatypeConverter.parseBase64Binary(
exponent ) );
    KeyFactory factory = KeyFactory.getInstance( "RSA" );
    KeySpec ks = new RSAPublicKeySpec( mod, exp );

    return (RSAPublicKey) factory.generatePublic( ks );
}
```

Python

Code sample 5 - Generating a public key from a modulus and exponent in Python

Python

```
def generate_key(modulus_b64, exp_b64):  
    """  
    Generates an RSA public key given a base64-encoded modulus and exponent  
    :param modulus_b64: base64-encoded modulus  
    :param exp_b64: base64-encoded exponent  
    :return: an RSA public key  
    """  
    mod = int(b64decode(modulus_b64).encode('hex'), 16)  
    exp = int(b64decode(exp_b64).encode('hex'), 16)  
    seq = asn1.DerSequence()  
    seq.append(mod)  
    seq.append(exp)  
    der = seq.encode()  
    return RSA.importKey(der)
```

Verifying the proof keys

After you import the key, you can use a verification method provided by your cryptography library to verify incoming requests are signed by Microsoft 365 for the web. Because Microsoft 365 for the web rotates the current and old proof keys periodically, you have to check three combinations of proof key values:

- The **X-WOPI-Proof** value using the current public key
- The **X-WOPI-ProofOld** value using the current public key
- The **X-WOPI-Proof** value using the old public key

If any one of the values is valid, the request is signed by Microsoft 365 for the web.

The following examples show how to verify one of these combinations in C#, Java, and Python.

Verification in C#

Code sample 6 - Sample proof key validation code in C#

C#

```
private static bool TryVerification(byte[] expectedProof,  
    string signedProof,  
    string publicKeyCspBlob)  
{  
    using(RSACryptoServiceProvider rsaAlg = new RSACryptoServiceProvider())  
    {
```



```

byte[] publicKey = Convert.FromBase64String(publicKeyCspBlob);
byte[] signedProofBytes = Convert.FromBase64String(signedProof);
try
{
    rsaAlg.ImportCspBlob(publicKey);
    return rsaAlg.VerifyData(expectedProof, "SHA256",
signedProofBytes);
}
catch(FormatException)
{
    return false;
}
catch(CryptographicException)
{
    return false;
}
}
}

```

Verification in Java

Code sample 7 - Sample proof key validation code in Java

Java

```

public static boolean verifyProofKey( String strModulus, String strExponent,
String strWopiProofKey, byte[] expectedProofArray ) throws Exception
{
    PublicKey publicKey = getPublicKey( strModulus, strExponent );

    Signature verifier = Signature.getInstance( "SHA256withRSA" );
    verifier.initVerify( publicKey );
    verifier.update( expectedProofArray ); // Or whatever interface
specifies.

    final byte[] signedProof = DatatypeConverter.parseBase64Binary(
strWopiProofKey );

    return verifier.verify( signedProof );
}

```

Verification in Python

Code sample 8 - Sample proof key validation code in Python

Python

```

def try_verification(expected_proof, signed_proof_b64, public_key):
    """

```

```

    Verifies the signature of a signed WOPI request using a public key
    provided in
    WOPI discovery.
    :param expected_proof: a bytearray of the expected proof data
    :param signed_proof_b64: the signed proof key provided in the X-WOPI-
    Proof or
    X-WOPI-ProofOld headers. Note that the header values are base64-encoded,
    but
    will be decoded in this method
    :param public_key: the public key provided in WOPI discovery
    :return: True if the request was signed with the private key
    corresponding to
    the public key; otherwise, False
    """
    signed_proof = b64decode(signed_proof_b64)
    verifier = PKCS1_v1_5.new(public_key)
    h = SHA256.new(expected_proof)
    v = verifier.verify(h, signed_proof)
    return v

```

Proof key tests in the WOPI validator

The WOPI validator includes several tests that help verify proof key implementations.

ProofKeys.CurrentValid.OldValid

Tests that hosts accept requests where the **X-WOPI-Proof** value is correctly signed with the current proof key, and the **X-WOPI-ProofOld** value is signed with the old proof key.

ProofKeys.CurrentValid.OldInvalid

Tests that hosts accept with requests where the **X-WOPI-Proof** value is correctly signed with the current proof key, but the **X-WOPI-ProofOld** value is invalid. This scenario is unusual and shouldn't happen in a production environment, but since the **X-WOPI-Proof** value is signed with the current public key, the request should be accepted.

ProofKeys.CurrentInvalid.OldValidSignedWithCurrentKey

Tests that hosts accept with requests where the **X-WOPI-Proof** value is invalid but the **X-WOPI-ProofOld** value is signed with *current* public key. This can happen when a WOPI client such as Microsoft 365 for the web has rotated proof keys but the host hasn't re-run [WOPI discovery](#) yet.

ProofKeys.CurrentValidSignedWithOldKey.OldInvalid

Tests that hosts accept with requests where the **X-WOPI-ProofOld** value is invalid but the **X-WOPI-Proof** value is signed with *old* public key. This can happen when a WOPI client has rotated proof keys, the host has re-run [WOPI discovery](#) and has the updated keys, but the datacenter machine making the WOPI request doesn't yet have the updated keys.

ProofKeys.CurrentInvalid.OldValidSignedWithOldKey

Tests that hosts reject with requests where the **X-WOPI-Proof** value is invalid, and the **X-WOPI-ProofOld** value is signed with the *old* public key. This scenario is unusual and shouldn't happen in a production environment; such requests should be rejected.

ProofKeys.CurrentInvalid.OldInvalid

Tests that hosts reject that contain requests with invalid current and old proof keys.

ProofKeys.TimestampOlderThan20Min

Tests that hosts reject that contain requests with an **X-WOPI-Timestamp** value, which represents a time more than 20 minutes old.

Troubleshooting proof key implementations

If you're having difficulty with your proof key verification implementation, following are common issues to investigate:

- Verify you're converting the URL to uppercase.
- Verify you're including any query string parameters on the URL when transforming it for the purposes of building the expected proof value.
- Verify you're using the same encoding for any special characters that may be in the URL.
- Verify you're using an HTTPS URL if your WOPI endpoints are HTTPS. This is especially important if you have SSL termination in your network prior to your WOPI request handlers. In this case, the URL Microsoft 365 for the web uses to sign the requests will be HTTPS, but the URL your WOPI handlers ultimately receive will be HTTP. If you simply use the incoming request URL your expected proof won't match the signature provided by Microsoft 365 for the web.

In addition, use the Proof key unit tests to verify your implementation with sample data.

Launch your Microsoft 365 for the web integration

Article • 02/27/2025

Once you believe you are ready to launch your integration, you need to submit your solution for verification testing by Microsoft to begin the launch process.

To enable your WOPI host to use the Microsoft 365 for the web [production environment](#), Microsoft will perform a verification of your WOPI implementation and Microsoft 365 for the web integration.

Important

Be sure that you have thoroughly reviewed all the information here before you start the launch process!

The launch process consists of two phases:

1. **Verification** - The Microsoft test team will review your WOPI solution's protocol and user experience implementation to ensure compliance with all requirements as outlined in the CSPP WOPI documentation and the Microsoft Cloud Storage Partner Program Integration Terms. The duration of the verification testing process will vary based on the number of issues our test team discovers with your solution and the speed with which you are able to resolve these issues. Each verification pass will take up to 10 days for the Microsoft test team to complete.

Important

You can avoid delays in verification by ensuring that your implementation is consistent with this documentation and that you've done manual testing using our [testing guide](#) and that the [WOPI Validation application](#) tests are passing before beginning the launch process.

Important

A large percentage of verification failures are a result of the partner not following the [UI guidelines](#). Be sure that your solution is in complete compliance before you submit your solution for testing.

2. **Sign off and roll out** - Once Microsoft has signed off on your integration, your final domain configuration information will be entered into the WOPI production environment, which will take 4-5 weeks to deploy. When deployment is complete you can begin to roll out your WOPI solution to end-users. Depending on your traffic estimates, Microsoft may request that you roll out over a period of several days to ensure you do not overload Microsoft 365 for the web or your WOPI servers.

Use the [CSPP Verification Testing Submission form](#) to begin the verification testing process.

Before starting the launch process, you should read the below sections for an overview of the types of questions and issues that you may need to address during the process.

Preparing for the launch process

WOPI verification

As part of the verification testing process, Microsoft will test your WOPI implementation using the [WOPI Validator](#). To initiate verification testing you will need to provide a screen shot of the WOPI Validator showing that all of the tests in the following categories are passing:

- HostFrameIntegration (ValidLanguagePlaceholderValues might be skipped if the host isn't using the [UI_LLCC](#) or [DC_LLCC](#) placeholder values)
- BaseWopiViewing
- CheckFileInfoSchema
- EditFlows
- Locks
- PutRelativeFile or PutRelativeFileUnsupported
- RenameFileIfCreateChildFileIsNotSupported or RenameFileIfCreateChildFileIsSupported (applicable only if the host supports [RenameFile](#))
- FileVersion
- PutUserInfo (applicable only if the host supports [PutUserInfo](#))
- ProofKeys (applicable only for hosts implementing [proof key validation](#))

Important

All of the applicable tests listed above must pass in order to go to start verification testing. Do not submit your solution for verification testing until these tests are passing in the WOPI Validator.

Tip

Hosts are *not* required to support [RenameFile](#), [PutRelativeFile](#) or use [proof key validation](#). However, hosts that do support these operations, or use application features that rely on them, those test groups must be passing.

For example, [binary file conversion](#) requires the [PutRelativeFile](#) operation be implemented, so hosts that support conversion must also pass the PutRelativeFile tests.

Manual testing

In addition to checking the WOPI Validator results, Microsoft does some manual verification of scenarios that can't currently be tested using the WOPI validator. You should follow the steps in the [testing guide](#) and fully test these scenarios prior to starting the launch process.

Important

Even if you do not plan to use it in your current implementation, multi-user co-authoring must be implemented in all CSPP WOPI hosts. Your solution will not pass verification testing if we are unable to successfully complete multi-user co-authoring scenarios.

Test accounts and video

In order to perform verification testing, the Microsoft verification test team needs to sign in to your WOPI solution. Our test team requires that you provide Microsoft **test accounts** in addition to a **video demonstrating the testing**.

Requirements

You must provide the following information when submitting your integration for verification:

- The Microsoft CSPP Test Team must have **two accounts in your WOPI solution**. They should be associated with the email addresses:
 - csppverify1@microsoft.com
 - csppverify2@microsoft.com
 - **Each of the test accounts you provide must:**
 - **Upload new files** of any file type you open with Microsoft 365 for the web, including `.wopitest` and `.wopitestx` files used by the WOPI Validator
 - **Open .wopitest files in the WOPI Validator**
 - The accounts **must be capable of testing multi-user co-authoring**; co-authoring with a single user account is not sufficient
- A login URL to use to sign in to your test system
- A video (or several videos) demonstrating the following scenarios **using the test accounts provided**
 - Upload a new `.wopitest` file and open it in the WOPI validator
 - Run all the WOPI Validator tests and capture the passing results in your video; [all tests listed in the documentation](#) must be passing
 - Demonstrate your integration with **Word** by uploading a new Word document and editing it in Word on the web
 - Demonstrate your integration with **PowerPoint** by uploading a new PowerPoint document and editing it in PowerPoint for the web
 - Demonstrate your integration with **Excel** by uploading a new Excel document and editing it in Excel for the web
 - Open a document as two different user accounts and demonstrate multi-user coauthoring ([test details](#))
 - If an explicit "share" action is needed to share a document between two users, demonstrate how to do this in your video

Important

Before you submit your WOPI solution for verification testing, ensure that you have thoroughly reviewed all the information here and that you are ready to provide [test accounts and videos](#).

WOPI implementation questionnaire

There are some aspects of your WOPI implementation that are particularly critical to the success of your integration. In order to verify these parts of your implementation, Microsoft asks you to answer some questions regarding your specific WOPI implementation. These questions are included below.

Note

This list of questions is subject to change.

1. Confirm that your File IDs meet the [criteria listed in the documentation](#). Microsoft 365 for the web expects file IDs to be unique and consistent over time, as well as when accessed by different users or via different UI paths (for example, a given file might be available in two different parts of your UI, such as in a typical folder and also in search results. If the document is meant to be the same, then the file IDs should match. Otherwise, users will see unexpected behavior when they access the same file via different UI paths).
2. Confirm you're providing a user ID using the [UserId](#) field and that the ID is unique and consistent over time [as described here](#).
3. Confirm that the value in the [OwnerId](#) field represents the user who owns the document and is unique and consistent over time [as described here](#).
4. Are you sending the [SHA256](#) value in [CheckFileInfo](#)?
5. Are you using the [business user flow](#)?
6. What [WOPI host capabilities properties](#) are you passing in [CheckFileInfo](#)?
7. Do you use IPv6 in your datacenters?

Production settings check

Prior to enabling your integration in the [production environment](#), Microsoft asks you to verify your current [Microsoft-configured settings](#), including your entries in the [WOPI domain allow list](#) and [Redirect domain allow list](#).

Important

Changes to settings require time to make.

Any changes to Microsoft-configured settings **will take 4-5 weeks to fully roll out**. This includes changes to the [WOPI domain allow list](#), and applies to both the production and test environments.

Service management contacts

Microsoft 365 for the web is a worldwide cloud service, and is thus monitored at all times. As part of the launch process, Microsoft provides you with information regarding how to escalate service quality issues with the Microsoft 365 for the web on-call engineers.

In order to use the [production environment](#), you must also provide a contact for Microsoft's on-call engineers to reach if Microsoft 365 for the web detects an issue that we suspect is due to a problem on the host side. For example, Microsoft 365 for the web monitoring systems might detect error rates for sessions spiking, and the on-call engineer would contact the host to see if it's a known issue on the host side. Ideally this emergency contact can be reached 24x7, either by phone or email.

Rollout schedule and traffic estimates

Typically, Microsoft asks partners to roll out over a period of time - between a few days to two weeks - depending on the anticipated traffic. For smaller hosts this is not always necessary. If you're already planning on doing this, you should communicate the schedule to Microsoft (that is 10% day 1, 50% day 2, and so on.). If you're not, you must coordinate with Microsoft to ensure this is appropriate given your traffic estimates.

In order to best plan the rollout, be prepared to provide Microsoft with updated traffic estimates. Ideally, these estimates are broken down by view and edit, file type, and geography, but provide whatever you can.

Production access

Once you and Microsoft have agreed on a rollout plan and Microsoft has signed off on your WOPI implementation, your WOPI host is enabled in the [production environment](#). Plan to do some basic testing against the production environment prior to rollout to ensure there are no unique issues using that environment. Once you've completed that testing, you can roll your integration out to users according the agreed-upon rollout schedule.

Load Time Performance

Article • 02/27/2025

You can improve the load time performance of Microsoft 365 for the web applications by preloading the Microsoft 365 for the web static content.

Preload static content

You can preload the Microsoft 365 for the web static content (JavaScript, CSS, and images) into the user's browser cache. This ensures that when the user opens a document in Microsoft 365 for the web, they can use the previously cached static content. As in, they won't have to download the static content when they first load Microsoft 365 for the web.

To support preloading static content, Microsoft 365 for the web provides two WOPI actions for each Microsoft 365 for the web application in its discovery XML, one to preload static content for the `view` action (`preloadview`), and a second to preload static content for the `edit` action (`preloadedit`).

Hosts can use these URLs just like they use other [Action URLs](#), by pointing iframes in their pages at the action URL. Unlike most action URLs, the WopiSrc and access token don't need to be specified to use these actions.

Hosts can include both `preloadview` and `preloadedit` in their pages to preload static content for both. The static content preload actions contain the `UI_LLCC` placeholder value, which should be replaced with an appropriate language for the user so that the proper localized static content is preloaded.

Tip

If you want to preload static content for all applications and both view and edit modes, you must load multiple actions, one for each application and mode combination.

Considerations for security and privacy

Article • 01/29/2025

The considerations for security and privacy in Microsoft 365 for the web implementation follow.

Microsoft 365 for the web and customer data

Two classes of customer data flow through Microsoft 365 for the web servers: user metadata and customer content (documents, presentations, workbooks, and notebooks).

User metadata

User metadata consists of URLs, email addresses, user IDs, and so on. This data lives in memory and travels back and forth between Microsoft 365 for the web and the WOPI host through HTTPS. Microsoft 365 for the web goes to great lengths to scrub all personally identifying information (PII) from its logs. This is regularly audited to ensure Microsoft 365 for the web is compliant with several different privacy standards (such as FedRamp in the USA).

Customer content

In the case of customer content, Microsoft 365 for the web retrieves it from the host in order to render it for viewing or editing. In the following image, you can see how WOPI is used to fetch content from the host and send content changes to the host (ContosoDrive in this case).

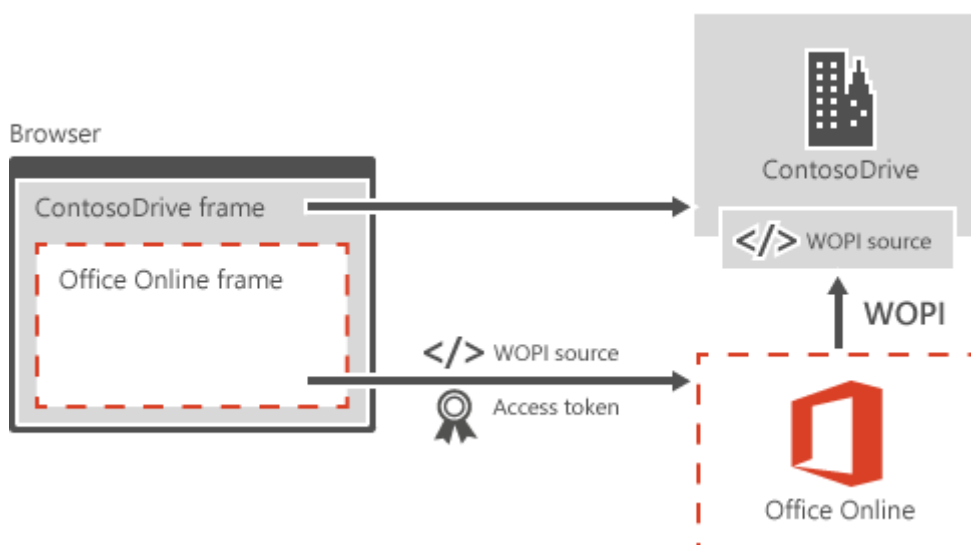


Figure 18 - WOPI protocol workflow

With the exception of caching (discussed below), it's reasonable to say that customer content only lives on Microsoft 365 for the web servers during a user session. That is, a user can reasonably expect that when they end an editing session, their content no longer lives anywhere on Microsoft 365 for the web servers once it's been saved to the host. The key exception here is the Microsoft 365 for the web viewing cache.

Microsoft 365 for the web viewing cache

In order to optimize view performance for PowerPoint for the web and Word for the web, Microsoft 365 for the web stores rendered documents in a local disk cache. This way, if more than one person wants to view a document, Microsoft 365 for the web only has to fetch it and render it once. It's important to note that a complete removal of the cache would significantly degrade the customer experience for many users and dramatically increase the cost of running Microsoft 365 for the web.

Documents in the cache are indexed using a [SHA256](#) hash that's generated based on the contents of the file (or some other unique attribute of the file). No user information is used to index the file. On every request for a file, if the WOPI host validates the access token, Microsoft 365 for the web uses the SHA256 hash returned by the host (or generated based on [file id](#) plus [version](#) value) to check for the file in the document cache.

Since the SHA256 hash is an enormous number that's generated using information unique to the file, there's essentially no chance of the same number being generated twice. Also, since the hash generates using information unique to the file and not based on any sort of user data, Microsoft 365 for the web can't retrieve information that's specific to a given user. Microsoft 365 for the web specifically doesn't log the SHA256 hash when the file is cached so that it's effectively impossible for Microsoft 365 for the web to retrieve information associated with a specific host or user without the participation of the host.

Documents live in the cache until they become unpopular. That is, the cache isn't time-based but rather based on available space and usage. Unpopular files might expire out of the cache in only a few days while popular documents might remain in the cache for up to 30 days.

As of May 2016, the contents of the cache are encrypted using a [FIPS 140-2](#) [compliant](#) encryption algorithm.

Compliance for Cloud Storage Partner Program Integrations

When you integrate with Microsoft 365 through the Cloud Storage Partner Program using the WOPI API, your requests hit the same infrastructure used for Microsoft 365 for the web and related services in Microsoft 365. Which means you and your customers benefit from the same strong security and privacy protections.

More information on the compliance of Microsoft services as related to the Cloud Storage Partner Program is available in the Microsoft 365 for the web portions of the audit reports related to Microsoft 365 provided in the [Microsoft Service Trust document portal](#) .

Note

Compliance for the overall end-to-end system for Cloud Storage Partner Program integrations requires that appropriate security and compliance requirements are also met by the cloud storage partner.

Integrate with Microsoft 365 for mobile

Article • 05/02/2023

You can integrate with Microsoft 365 for mobile to enable your users to view and edit Excel, PowerPoint, and Word files directly on their mobile devices.

If you offer both iOS and Android experience, we recommend integrating both experiences simultaneously. But you can choose to onboard either of the experience first depending on your priority. Once you've onboarded to WOPI APIs for either one experience, Android or iOS, integrating with the other is minimal work.

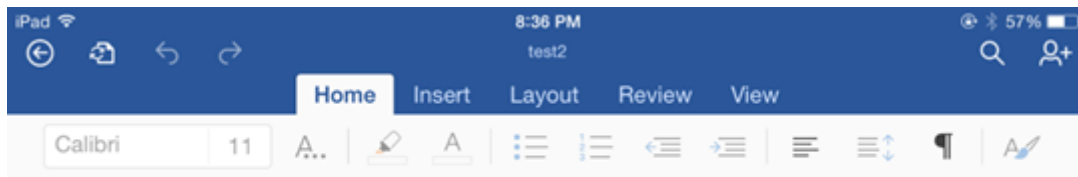


Figure 1 - Editing a file in Office for iOS



Figure 2 - Editing a file in Office for Android

If you deliver an iOS or Android experience that lets your users store Office files or include Office files as a key part of your solution, you can now integrate Microsoft 365 for mobile into your experience. This integration works directly against files stored by you. Your users won't need a separate storage solution to view and edit Office files.

Edit and view Office files

When you integrate with Microsoft 365 for mobile, your users can view and edit Excel, PowerPoint, and Word files directly on a mobile device. The level of editing support is determined by the type of user and whether they have an Office 365 subscription.

Integration process

Use the Web Application Open Platform Interface (WOPI) protocol to integrate Microsoft 365 for mobile with your application. The WOPI protocol enables Microsoft 365 for mobile to access and change files that are stored in your service.

To integrate your application with Microsoft 365 for mobile, do the following:

1. Become a member of the Office 365 - Cloud Storage Partner Program. Learn more about the program, as well as how to apply, at the [Office 365 Cloud Storage Partner Program](#) web site.
2. Provide the required on-boarding information as described in the section titled [Onboarding information](#).
3. Obtain your ProviderId and app store URLs from Microsoft. The app store URLs are the URLs you should use to launch Office on a platform's app store.
4. Add your domain to the WOPI domain [allow list](#). This is required even if you don't integrate with Office for the web so you can verify your WOPI integration with the [Validator](#).
5. Implement the WOPI protocol. The set of WOPI operations that must be supported is described in [WOPI implementation requirements for mobile integration](#).
6. Implement required changes to your app. Contact your Microsoft 365 for mobile integration contacts for directions on running Microsoft 365 for mobile in test mode for testing.
7. Run the [Validator](#) and fix any issues until the Validator reports a 100% pass rate.
8. Complete the manual testing outlined in the [Manual Validation section](#) to verify the integration works as expected.
9. Once all manual test cases have a 100% pass rate, send your [Manual Validation report](#) and directions to run your validator.
10. Once the Microsoft 365 for mobile team completes their manual testing, they work with you to schedule your launch date.

Authentication

Authentication is handled by passing Microsoft 365 for mobile an access token that you generate from a Bootstrapper URL. Assign this token a reasonable expiration date. Also, we recommend that tokens be valid for a single user against a single file to help mitigate the risk of token leaks.

Requirements

- Ensure that files are represented by a unique ID. See the full list of [File ID requirements](#).
- Have a mechanism for identifying file versions. See the [Version requirements](#).
- In order to integrate with Microsoft 365 for mobile, there are also promotional requirements which include:
 - Promoting Microsoft 365 for mobile integration somewhere within your app.
 - Promoting Microsoft 365 for mobile integration in the context of editing and viewing Office documents.

- Using Office as the default app for opening Office documents within your app.

Security considerations

Microsoft 365 for mobile is designed to work for enterprises with strict security requirements. To make your integration as secure as possible, ensure that:

- All traffic is SSL encrypted
- Server supports TLS 1.0+
- OAuth 2.0 is supported

Interested?

If you're interested in integrating your solution with Microsoft 365 for mobile, take a moment to register at [Office 365 Cloud Storage Partner Program](#).

Onboarding information

Article • 07/06/2022

Provide the following information to Microsoft to enable end-to-end testing and release of Microsoft 365 for Mobile integration. After initially onboarding to the [Microsoft Office 365 - Cloud Storage Partner Program](#), you are given access to the [CSPP Yammer Group](#) [↗]. You can work with the onboarding representatives there to provide your information to Microsoft.

Property	Data Type	Sample Value	Description
Localized Provider Name	String	Contoso	The name to display for this storage. provider
Icon Locations	String[]	<code>https://contoso.com/images/logo16.png</code> <code>https://contoso.com/images/logo32.png</code> <code>https://contoso.com/images/logo48.png</code> <code>https://contoso.com/images/logo64.png</code> <code>https://contoso.com/images/logo80.png</code> <code>https://contoso.com/images/logo96.png</code>	The path to the provider hosted icons, one for each of the following dimensions: • 16x16 • 32x32 • 48x48 • 64x64 • 80x80 • 96x96 Requirements: • must be PNG • icons must have at least a 1px white border to avoid bleed.
BootstrapperUri	String	<code>https://contoso.com/wopibootstrapper</code>	The full path to the Bootstrapper endpoint. Must end in wopibootstrapper.

Property	Data Type	Sample Value	Description
English Description	String	Free online storage. Store, access, and share thousands of documents.	Used to show more information about the service. Specify which part of the string shouldn't be localized. For example, your product or brand name if you include it in your description.
Client ID	String	s6BhdRkqt3	OAuth2 Client ID (for Office).
Client Secret	String	7FjfpZBr1K3tDRbnfVdmlw98	OAuth2 Client Secret (for Office). Don't provide this in email. This must be transferred securely.
Client App Name	String	Office	Please set this as the app name.
RedirectUri	String	<code>https://localhost</code>	The redirect URI that returns an authorization code. This URI is a known stop-URL and isn't loaded by Microsoft 365 for mobile.
ProviderId	String	TP_CONTOSO	Supplied by Microsoft.
TrustedDomain	String	<code>contoso.com</code> , <code>qa-contos.com</code>	Known domains for bootstrapper, authorization, and token issuance endpoints.
Scope (optional)	String	userprofile, editdocs	Set of comma-delimited scopes that are requested during authentication with the storage provider.
SupportRefresh	Bool	True	Denotes whether you'll issue a refresh token that can get a new access token.

Property	Data Type	Sample Value	Description
SiteUrl	String	<code>http://www.contoso.com</code>	Fully qualified address to your website.

Additional information required

- Three test accounts on each environment you want to onboard.
 - If your service has a commercial versus consumer offering, provide three of each.
- Website interface of each environment, if one exists.
- Whether each environment can be reached outside your network (if it's Internet accessible so we can use it).
- How many apps you have on each platform. (For example, on iOS, do you have a *for enterprise app* versus a *for consumer app*, or do you just have one?)
- If you've already integrated with Office for the web, provide directions to access the Validator.

Security Questions

- What's the expiry for access and refresh token for each environment?
- Do you actually check the redirect URI sent during OAuth2 flow against the one registered above, such that the authorization code would only ever be sent to the redirect URI?

WOPI implementation requirements for mobile integration

Article • 07/06/2022

The following describes the implementation requirements for Microsoft 365 for mobile integration.

ⓘ Note

This content applies to Microsoft 365 for mobile. Both of the endpoints require the same level of WOPI implementation.

This section documents specific requirements for your WOPI implementation for integration with Microsoft 365 for mobile beyond what's documented in general for WOPI. Learn more about how Microsoft 365 for mobile uses these operations in the [Operational flows for Microsoft 365 for mobile](#) section.

Required WOPI Operations for Microsoft 365 for mobile

The following WOPI operations are required for integrating with Microsoft 365 for mobile. Operations not listed here aren't currently called by Microsoft 365 for mobile.

If some operations aren't supported for a specific user, file, or container, just be sure to return the correct values in [CheckFileInfo](#) and [CheckContainerInfo](#).

File Operations

- [CheckFileInfo](#)
- [GetFile](#)
- [Lock](#)
- [GetLock](#)
- [RefreshLock](#)
- [Unlock](#)
- [UnlockAndRelock](#)
- [PutFile](#)
- [EnumerateAncestors \(files\)](#)

Container Operations

- [CheckContainerInfo](#)
- [CreateChildFile](#)
- [EnumerateAncestors](#) (containers)
- [EnumerateChildren](#) (containers)

Ecosystem Operations

- [CheckEcosystem](#)
- [GetRootContainer](#) (ecosystem)

Bootstrapper

- [Bootstrap](#)
- [GetNewAccessToken](#)
- [GetRootContainer](#) (bootstrapper)

Future Support

While these WOPI operations aren't currently used by Microsoft 365 for mobile, they must be implemented. Microsoft 365 for mobile might use these operations in the future.

- [RenameFile](#)
- [DeleteFile](#)
- [CreateChildContainer](#)
- [DeleteContainer](#)
- [RenameContainer](#)
- [GetEcosystem](#) (files)
- [GetEcosystem](#) (containers)

Other Requirements

- The X-WOPI-ItemVersion header must be included on [PutFile](#), [Lock](#), and [Unlock](#) responses.
- For the [Bootstrap](#) operation, the [Content-Type](#)[↗] response header must be set to `application/json`.
- [IsEduUser](#) and [LicenseCheckForEditIsEnabled](#) are required on [CheckFileInfo](#) and [CheckContainerInfo](#). The values from CheckFileInfo must match that of the file's parent container.

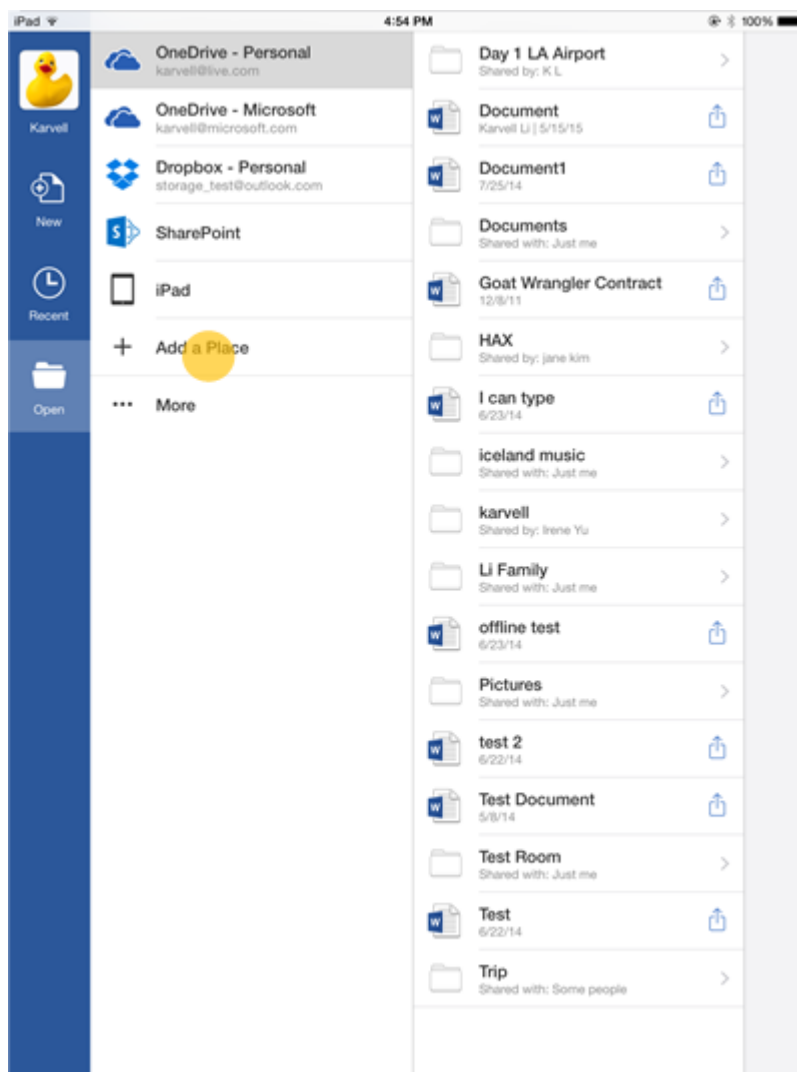
Browse, open, and save files in Microsoft 365 for mobile

Article • 07/06/2022

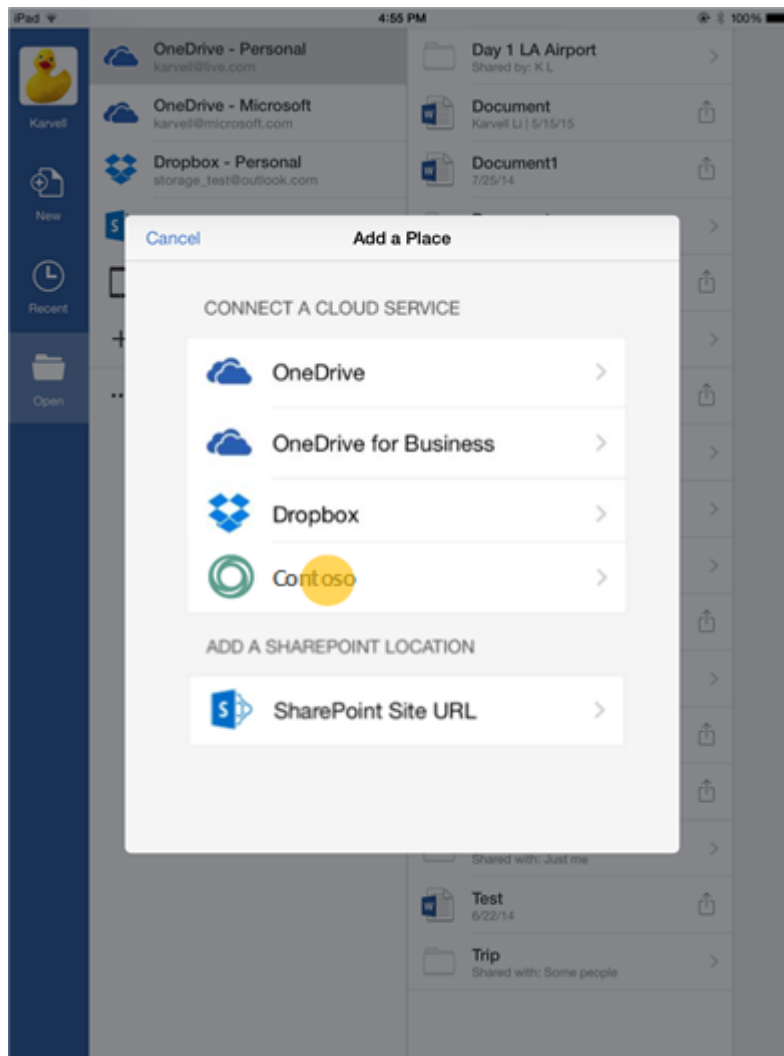
After you've integrated with Microsoft 365 for mobile, users can browse, open and save files from your storage location directly in the Microsoft Office apps. And they can open their Office files from your storage location. This section describes the user experience for browsing, opening and saving files from Microsoft 365 for mobile for Mobile.

Open and edit files from Microsoft 365 for mobile

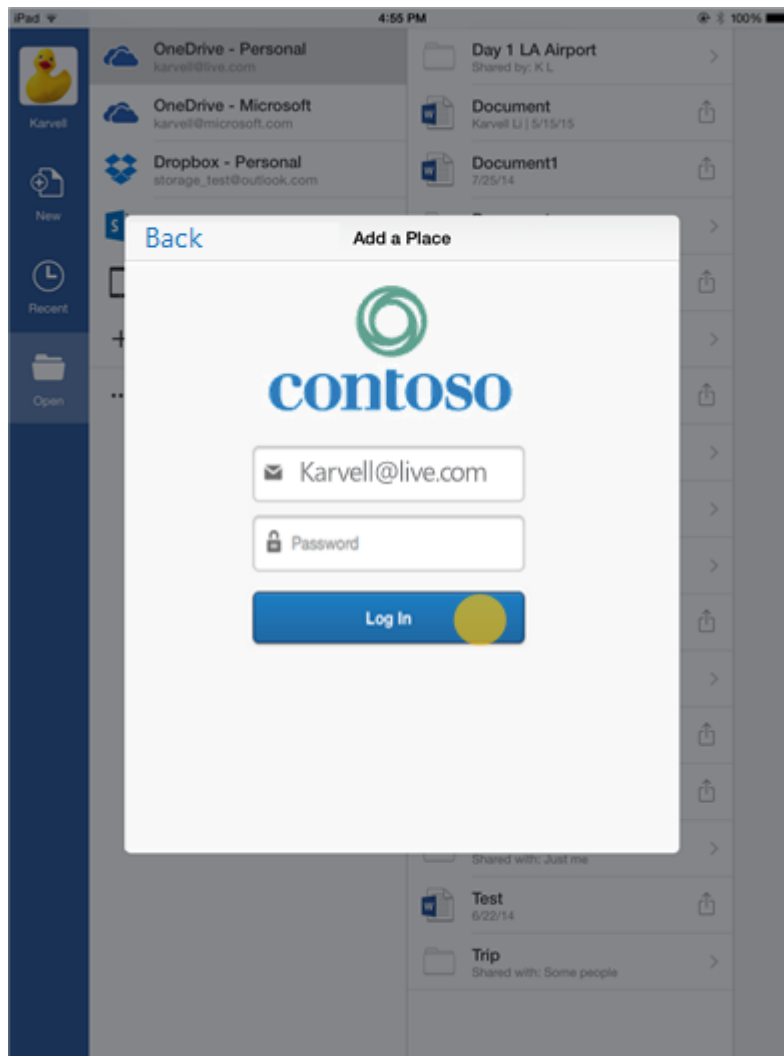
1. User selects **Add a Place** in their Office app.



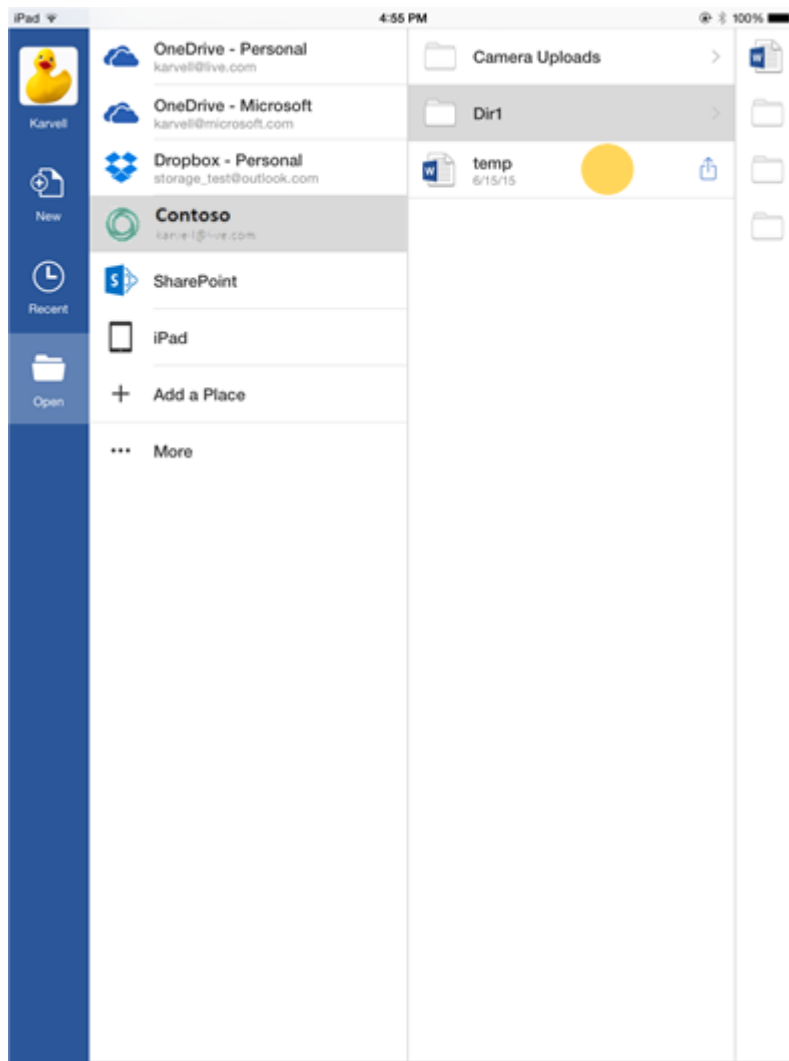
2. A list of places displays and the user selects **Contoso**.



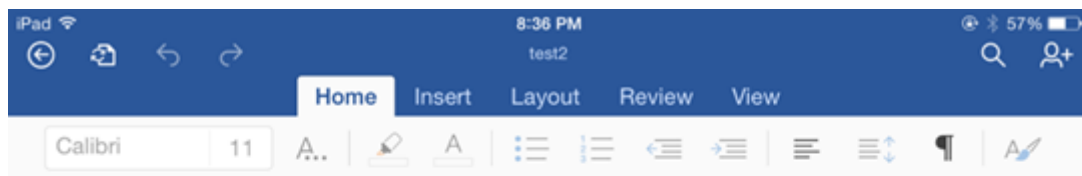
3. The user goes through Contoso's OAuth 2.0 flow.



4. The user can now browse files stored on Contoso and open them.

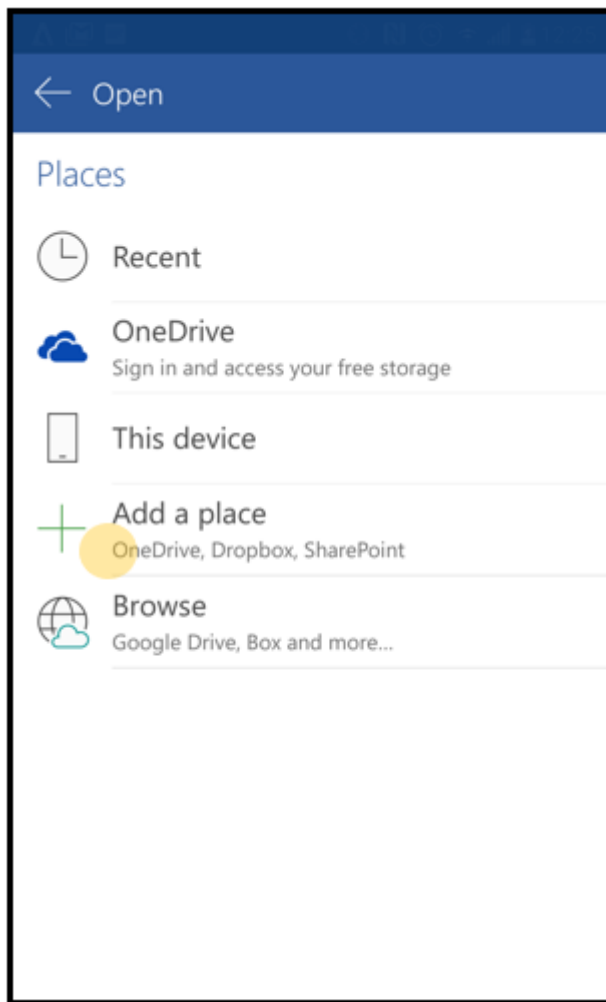


5. The user can edit the file.

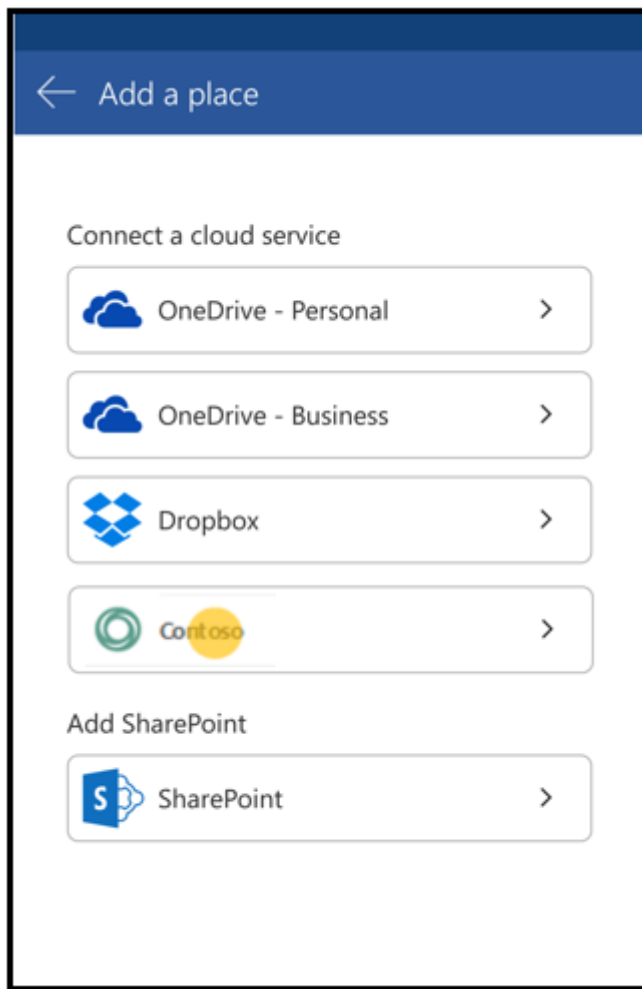


Open files from Office 365 for Android

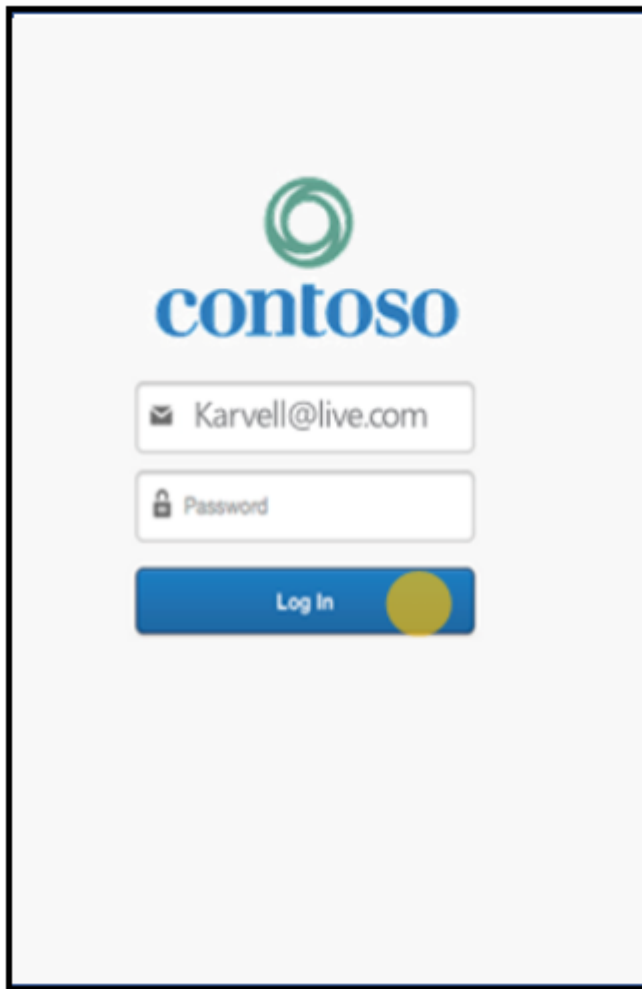
1. User selects **Add a Place** in their Office app.



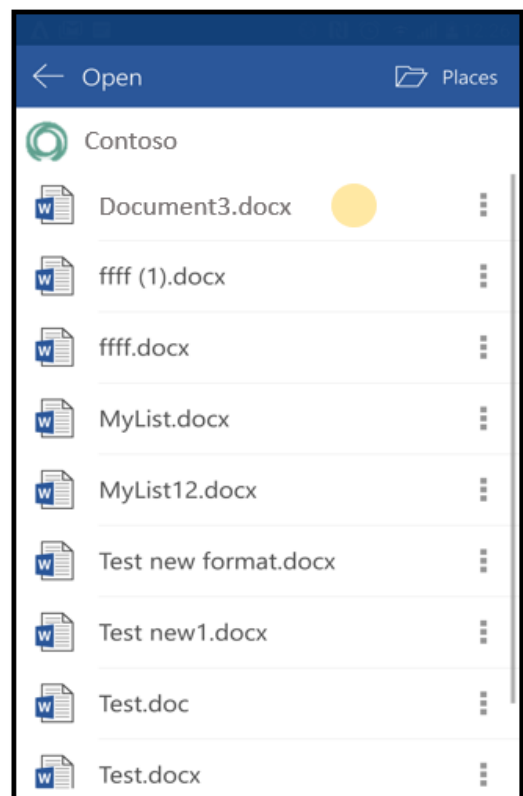
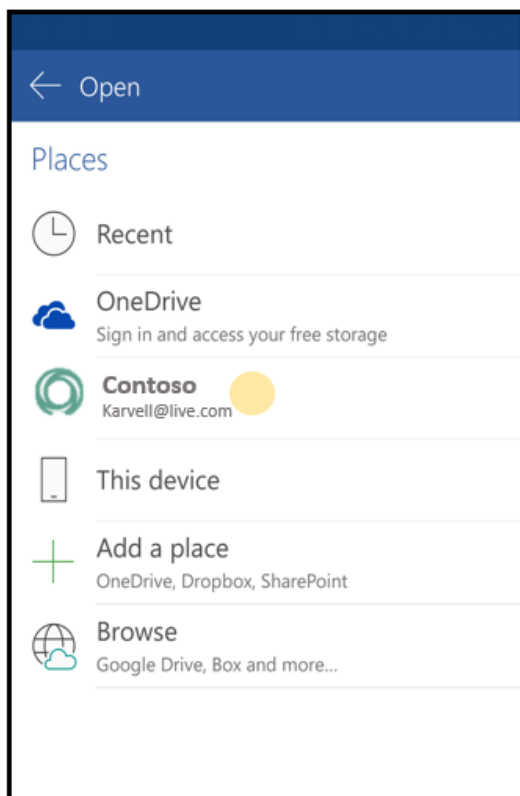
2. A list of places displays and the user selects **Contoso**.



3. The user goes through Contoso's OAuth 2.0 flow.



4. The user can now browse through files stored on Contoso and open them.

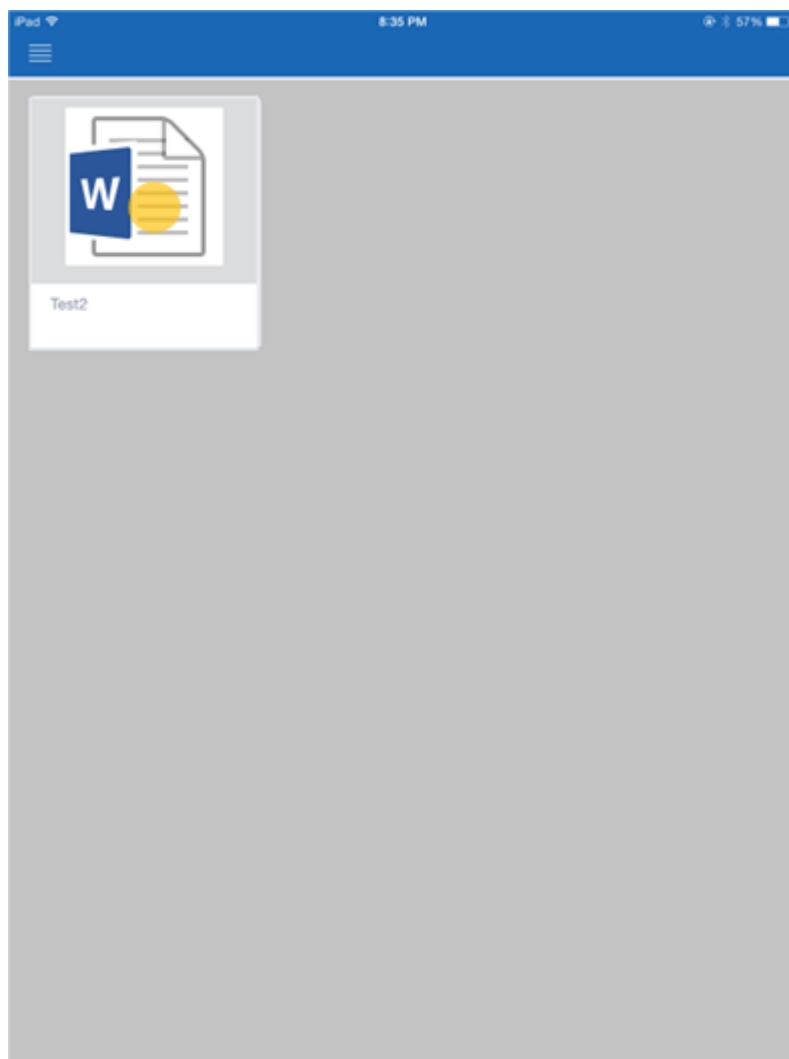


5. The user can edit the file.



Open and edit files from your Cloud Storage App

1. User browses to a file on your iOS or Android app.



2. User selects Edit.

wffsdafasdfa

! Note

If you do not have an in-app preview, you might open Office in other screens.

3. The file opens in Office and any saves go back to your service directly.

Open files from your app in Microsoft 365 for mobile

Article • 03/20/2023

In order to invoke Microsoft 365 for mobile when opening an Office file from your app, you must use the URL schemes that Word, Excel, and PowerPoint for iOS or Android register when they are installed.

URL schemes for invoking Microsoft 365 for mobile

The following is the list of scheme names used to invoke Microsoft 365 for mobile:

- `ms-word:`
- `ms-powerpoint:`
- `ms-excel:`

You must include the following information along with the URL scheme:

- The mode in which to open the file. Valid values are:
 - `ofv` for opening as read-only.
 - `ofe` for opening for editing.
- The [WOPISrc](#) to the file, denoted by the `|u|` parameter.
- The service identifier, denoted by the `|d|` parameter.
- The user identifier, denoted by the `|e|` parameter.
- The file name, with extension, denoted by the `|n|` parameter.
- The source of the open action with `|a|`. The value of this parameter can be either:
 - `Web` – to be used in the case where a website is invoking the protocol handler.
 - `App` – to be used in the case where a native app is invoking the protocol handler.

Example:

```
ms-word:ofe|u|https://contoso/wopi/file/12312|d|Contoso|e|a3243d|n|document1.docx|a|App
```

The URL used should be URL encoded.

Note that the file is opened directly against your service. Your app essentially passes the URL to the file to Microsoft 365 for mobile without passing the actual file. The Microsoft 365 for mobile app then opens the file directly using the WOPI protocol.

Identifying supported Office for iOS versions

Before using the URL schemes specified above to invoke Microsoft 365 for iOS, you must do the following:

1. Verify the particular Microsoft 365 for iOS application (Word, Excel, or PowerPoint) is installed.

Use the iOS `canOpenURL` method to determine whether your application can open the resource. This method takes the URL for the resource as a parameter, and returns `No` if the application that accepts the URL is not available. If `canOpenURL` returns `No`, you'll need to prompt the user to install Office for iOS. Use the following links to direct the user to the appropriate Microsoft 365 application:

- For iOS, use <https://aka.ms/m365ios3p>
- For Android, use <https://aka.ms/m365aos3p>

2. Check the version of Microsoft 365 for iOS installed is a supported version. You can check this by verifying whether the following URL schemes are registered:

- `ms-word-wopi-support-1605`
- `ms-powerpoint-wopi-support-1605`
- `ms-excel-wopi-support-1605`

Important

These URL schemes are only used to check if the currently installed Microsoft 365 for iOS supports opening files from a WOPI host, and not for invoking the Microsoft 365 for iOS apps.

(Optional) Passback protocol for Office for iOS

If you want Microsoft 365 for iOS to return users to your iOS application when they choose the in-app back arrow (distinct from the iOS back control), you will need to include the passback parameter when invoking Microsoft 365 for iOS. This is denoted by `|p|` followed by your app's registered URL scheme (without a colon).

You must ensure that your application can properly handle the response from Microsoft 365 for iOS.

Tip

You'll provide your app's registered URL scheme during the initial integration phase.

For security reasons, Microsoft 365 for iOS only returns users to the referring application if the file opened successfully. When the user chooses the back arrow, Microsoft 365 for iOS responds to the invoking application with the passback protocol, open mode, URL, upload-pending status, and `document context`. The upload-pending status uses the descriptor `|z|`, and is either yes or no.

`document context` is a string you provide via the `|c|` parameter when invoking Microsoft 365 for iOS. Microsoft 365 for iOS doesn't use this parameter; it's purely for your use, as needed by your app. Microsoft 365 for iOS doesn't limit the length of the string beyond any limits the operating system imposes.

Schema format invoking your app when user chooses back:

```
<app protocol>:ofe|u|<URL>|z|<yes or no>|c|<doc context>
```

Example:

```
contosodrive:ofe|u|https://contoso/Q4/budget.docx|z|no|c|folderviewQ4
```

Operational flows for Microsoft 365 for mobile

Article • 07/06/2022

This section outlines the implementation requirements and operational flows for user scenarios outlined in the [Browsing, opening, and saving files from Microsoft 365 for mobile](#) section.

Terminology

Term	Definition
User	Self-explanatory
Office Client	The Microsoft 365 for mobile app, i.e. Word/Excel/PowerPoint
Office Identity Service	The Office Identity service is a service Microsoft 365 for mobile uses for handling user-identity information. In OAuth flows, the Office Identity Service behaves as the External User Agent
Bootstrapper URL	URL that the Storage Provider hosts to allow an Microsoft 365 Client app with a valid OAuth 2.0 access token to retrieve a WOPI access token
Token Endpoint URL	URL that the Storage Provider hosts and uses in order to generate access or refresh tokens
Services Manager	Microsoft 365-owned service that stores a Service Catalog, listing all of the available services for an app or endpoint
Storage provider	Refers to a CSPP partner who has integrated their service into Microsoft 365 for mobile

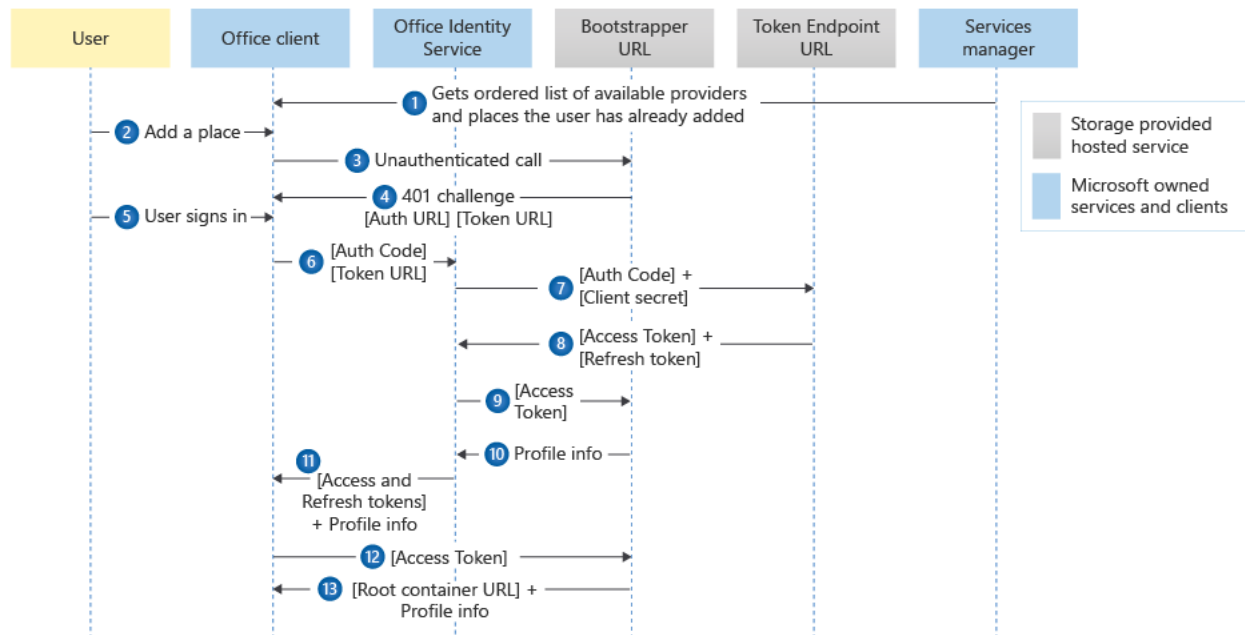
ⓘ Note

- Any time during these flows that Microsoft 365 for mobile tries to hit the bootstrapper and has expired or invalid OAuth credentials, Microsoft 365 for mobile will ask the user to log in again.
- Any time during these flows that Microsoft 365 for mobile has a missing, invalid, or expired WOPI access token, Microsoft 365 for mobile will attempt

to get a valid WOPI access token by calling `GetNewAccessToken`.

Add a Place

This describes the first time a user adds your storage provider using the `Add a Place` in Microsoft 365 for mobile. To correctly authenticate users, we use the following operational flow in order to connect a user's storage provider account with their Microsoft 365 for mobile account.



1. When the Microsoft 365 client boots, it calls the Services Manager for a list of available services. The Services Manager returns a Service Catalog, which is an alphabetically-sorted list of available providers and places to which the user can connect.
2. When the user clicks `Add a Place` in the Microsoft 365 for mobile backstage, they'll see your Storage Provider listed as an available place.
3. When the user adds the Storage Provider, the Microsoft 365 client makes an unauthenticated `Bootstrap` call.
4. The `Bootstrap` response includes the Authorization URI and token issue URI. Note that the Authorization URI must be provided as part of your [Onboarding information](#) so we can add it as a trusted domain.
5. Microsoft 365 for mobile loads the Authorization URI in a web view, which prompts the user to sign in with the service provider. Once the user finishes the sign-in process, the web view reaches a redirect URI, which causes it to close. The redirect URI also provides an auth code to Microsoft 365 for mobile.
6. Microsoft 365 for mobile sends the auth code and token issue URI to the Office Identity Service.

7. The Office Identity Service sends the auth code and the client secret to the Token endpoint.
8. The Token Endpoint issues an access token and a refresh token (if available) back to the Office Identity Service.
9. The Office Identity Service makes an authenticated [Bootstrap](#) call using the access token to retrieve the user's profile information.
10. The Office Identity Service sends the access and refresh tokens and the user's profile information to Microsoft 365 for mobile.
11. The user has now added the Storage Provider as a place. For the operational flow on browsing, opening, and saving files, see the next sections.

Browse and open files

Here is the operational flow for browsing and opening files:

1. *Get the Root Container URL:* Microsoft 365 for mobile calls [GetRootContainer \(bootstrapper\)](#) to obtain a Root Container URL.
2. *Get the contents of the container:* Microsoft 365 for mobile calls [EnumerateChildren \(containers\)](#) on the Root Container. The results are a set of containers and files in the root container. If the user wants to browse to another container within the current container, Microsoft 365 for mobile calls [CheckContainerInfo](#) on the other container to check permissions, then calls [EnumerateChildren \(containers\)](#) on that second container. This step is repeated as the user browses the container hierarchy until the user selects the file they want to open.
3. *Check file permissions:* Once the user selects a file, Microsoft 365 for mobile calls [CheckFileInfo](#) on that file to verify that the user has permissions to the file.
4. *Check file lock:*
 - If the earlier [CheckFileInfo](#) call returned `true` for [SupportsGetLock](#), Microsoft 365 for mobile calls [GetLock](#). If the [GetLock](#) response is a [409 Conflict](#) or includes an **X-WOPI-Lock** header, the file is locked, and Microsoft 365 for mobile does not continue opening it.
 - If the earlier [CheckFileInfo](#) call returned `true` for [SupportsGetLock](#), Microsoft 365 for mobile sends a [RefreshLock](#) request with a known invalid lock ID. If the [RefreshLock](#) response is a [409 Conflict](#) with a lock ID in the **X-WOPI-Lock** response header, the file is locked and Microsoft 365 for mobile does not continue opening it.
5. *Take a lock on the file:* Microsoft 365 for mobile calls [Lock](#) on the file, passing a lock ID it wishes to use in the **X-WOPI-Lock** request header. If the [Lock](#) call returns a

[200 OK ↗](#), the file is locked. Microsoft 365 for mobile will use the same lock ID when making future [PutFile](#) requests.

6. *Download the file*: Microsoft 365 for mobile makes a [GetFile](#) request on the file.

Save and close a file

1. *Save the file*: If the user has made changes to the file, Microsoft 365 for mobile will update the file's contents by calling [PutFile](#). The [PutFile](#) request will include the current WOPI lock ID that Microsoft 365 for mobile previously used to lock the file.
2. *Unlock the file*: Microsoft 365 for mobile will make an [Unlock](#) request against to unlock the file. This [Unlock](#) request will include the current WOPI lock ID that Microsoft 365 for mobile previously used to lock the file.

Bootstrap OAuth2

Article • 05/02/2023

The first step for the OAuth2 flow from Microsoft 365 for mobile to your application is an **unauthenticated** request to the bootstrapper endpoint.

Unauthenticated means that no access token is attached in the [Authorization](#) HTTP header, or that an expired or otherwise invalid token is attached.

Important

The bootstrapper URL must be an HTTPS endpoint, and connections to the bootstrapper must be made using TLS.

Important

The bootstrapper URL must be supplied as the `BootstrapUrl` property of the onboarding information described in the section titled **Onboarding information**.

The file and the bootstrapper can be at a different host and/or a different path, as long as they conform to the requirement that the endpoint is `/wopibootstrapper` and otherwise meets the requirements for the request/response. Additionally, it's important to note that the resource specified by the bootstrapper must be in a trusted domain configured as part of **provisioning your service entry with Microsoft**.

For full details, see the [Unauthenticated response](#) page.

Token issuance URL requirements

Article • 07/06/2022

This section describes requirements for your token issuance endpoint ([RFC 6749#section-3.2](#)):

- You must set the [Content-Type](#) header to `application/json`
- You should specify the OAuth2 access token expiration via the `expires_in` property, in seconds.
- If your access tokens do not expire, set the OAuth2 access to a value of 0 (zero).

Example Response Body:

text

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8

{
  "access_token": "123abcdefg",
  "token_type": "bearer",
  "expires_in": "86400"
}
```

Post-Authorization token endpoint

Article • 03/09/2023

The token endpoint URL ([RFC 6749#section-3.2](#)) is normally obtained from the initial unauthenticated call described in the [Bootstrap OAuth2](#) topic.

If the token endpoint URL cannot be determined before the end user has completed the sign-in process, an alternative token endpoint URL may be supplied.

This is done via a `tk=` URL parameter appended to the value of the [Location](#) header from the [302 Found](#) response at the end of the sign-in flow.

Important

The `tk=` parameter name is **case-sensitive** and its contents must be URL encoded.

For example, to return the following information:

Information	Value
Redirection URI	<code>https://localhost</code>
Authorization code (RFC 6749#section-4.1.2)	<code>"abcdefg"</code>
Token endpoint URL	<code>https://contoso.com/api/token/?extra=stuff</code>

The [Location](#) header in the [302 Found](#) response would be:

`Location: https://localhost?`

`code=abcdefg&tk=https%3A%2F%2Fcontoso.com%2Fapi%2Ftoken%2F%3Fextra%3Dstuff`

As a result, all calls to the token endpoint for obtaining access token via authentication-code exchange (or refresh flows using the refresh token) will hit this URL instead of the one initially returned as described in the [Bootstrap OAuth2](#) topic.

Optional Session Context String

Article • 07/06/2022

A Session Context String may be returned from the authentication process. Microsoft 365 for mobile will pass this string in an HTTP header in calls to the token endpoint URL ([RFC 6749#section-3.2](#)) and authenticated calls to the bootstrapper ([GetNewAccessToken](#), [Shortcut operations](#)).

The Session Context String is optional, and for the storage provider's use. A possible scenario would be to include a hint about a "tenant" so endpoints can know where they need to fetch and/or validate tokens.

Returning a Session Context string is done via an `sc=` URL parameter appended to the value of the [Location](#) header from the [302 Found](#) response at the end of the sign-in flow.

❗ Important

The contents of the `sc=` parameter must be URL encoded.

For example, to return the following information:

- Redirection URI is `https://localhost`
- Authorization code ([RFC 6749#section-4.1.2](#)) is "abcdefg"
- Session Context String is "hello:World"

The [Location](#) header in the [302 Found](#) response would be:

```
Location: https://localhost/?code=abcdefg&sc=hello%3AWorld
```

If present, the session context string will be included as an HTTP header when calls are made to the token exchange endpoint, and OAuth2 authenticated calls to the bootstrapper ([GetNewAccessToken](#), [Shortcut operations](#)) as follows:

```
X-WOPI-SessionContext: hello:World
```

Microsoft 365 for mobile OAuth2 sign-in URL parameters

Article • 07/06/2022

The HTTPS request to load the OAuth2 Authorization page ([RFC 6749#section-3.1](#)) includes standard OAuth2 parameters such as `client_id`, `redirect_uri`, etc. but may also include Office-specific parameters, documented here.

Important

Early releases of Microsoft 365 for mobile might not include some or all of these URL parameters, so callers should fall back to reasonable defaults.

Culture

The request includes the [Accept-Language](#) header. The iOS language settings determine value of this header. In addition, the following URL parameter is appended to the request URL:

```
rs=Culture
```

Culture contains the UI culture of the Microsoft 365 interface, e.g. `en-US`.

Build

The following URL parameter is appended to the request URL:

```
build=#
```

"#" will be a string showing the build of the Office application making the request.

Platform

The platform URL parameter communicates on which platform the app is running.

```
platform=iOS
```

Example

Here's a request to the sign-in endpoint showing all parameters in use:

```
https://contoso.com/api/oauth2/authorize?&rs=en-  
US&build=1.21.417&platform=iOS&client_id=abcdefg&redirect_uri=https%3A%2F%2Flocalho  
st&response_type=code
```

App to app sign in for Microsoft 365 for mobile

Article • 07/06/2022

The normal flow to sign in to your service from Microsoft 365 for mobile uses the OS WebView, where your web sign-in experience is rendered inside the Microsoft 365 for mobile app. Optionally, you can do an additional optimization where the user can sign in using your app.

The app-to-app flow involves Microsoft 365 for mobile invoking your app through your app's URL scheme to sign in to your service, and your app invoking Microsoft 365 for mobile when sign in is complete.

Sign in using your app

- When signing in to your service is required (i.e. the first time a file is opened from your app into Microsoft Office, or when the user explicitly adds your service as a place in Microsoft 365 for mobile), Microsoft 365 for mobile calls your bootstrapper to obtain the `authorization_uri`, which it displays in a UIWebView. In addition to the `authorization_uri`, you should return a set of UrlSchemes that can be used to invoke your app.
- Microsoft 365 for mobile will first attempt to use the UrlSchemes to invoke your app. If none are returned, or if none of them are registered (i.e. your app is not installed), Microsoft 365 for mobile will fall back to the UIWebView.
- If the user does have your app installed, your app will be invoked via the UrlSchemes and the user will be sent to your app to complete authentication. If the user declines to open your app, Microsoft 365 for mobile will fall back to the UIWebView.
- Once inside your app, you should do whatever is needed to obtain the user's auth code (for example, display a "Grant permission to Office" dialog or ask for additional sign ins).
- Return the user to the Microsoft 365 for mobile app with the auth code via the Office URL scheme (see next section).

Microsoft 365 for iOS Specific changes

URL scheme design

The data passed via the URL scheme is essentially the same as would be passed via `authorization_uri`.

Here is an example of a normal `authorization_uri` with parameters added. The parameters are described in [RFC6749](#).

Invoking the sign in screen:

```
https://contoso.com/api/oauth2/authorize?  
client_id=abcdefg&redirect_uri=https%3A%2F%2Flocalhost&response_type=code&scope=rs  
=en-US&Build=16.1.1234&Platform=iOS
```

Host's redirect URL that ends the OAuth flow:

```
https://localhost?  
state=&code=abcdefg&tk=https%3A%2F%2Fcontoso.com%2Fapi%2Ftoken%2F%3Fextra%3Dstuff
```

The same parameters are passed via the URL Schemes, with the addition of "action".

Microsoft 365 for iOS invoking your app ("To" URL):

```
Contoso:client_id=abcdefg&response_type=code&scope=wopi&rs=enUS&build=16.1.1234&pla  
tform=iOS&app=word&action=76d173ad-a43f-4e3c-a5e7-0e7276b4c624
```

Your app invoking Microsoft 365 for iOS ("Back" URL):

```
ms-word-  
tp:code=abcdefg&tk=https%3A%2F%2Fcontoso.com%2Fapi%2Ftoken%2F%3Fextra%3Dstuff&sc=xy  
z&action=76d173ad-a43f-4e3c-a5e7-0e7276b4c624
```

The values of the parameters should be URL encoded.

If authentication fails, the error parameters per [RFC6749](#) can also be passed back to Microsoft 365 for iOS via the URL schemes, just as they can be passed back to Microsoft 365 for iOS via the redirect URI in the `UIWebView` model.

New URL schemes registered by Microsoft 365 for iOS

- `ms-word-tp`
- `ms-excel-tp`
- `ms-powerpoint-tp`
- `ms-officemobile-tp`

These schemes are for Word, Excel, and PowerPoint respectively. Use these to invoke Microsoft 365 for iOS when the user is done with authentication on your side.

Action parameter

Action is a string passed to your app that should be passed back to Office unchanged.

Microsoft 365 for Android Specific changes

Service-side Changes

Adding information to the WWW-Authenticate response header on unauthenticated bootstrap requests

Microsoft 365 for Android will use Intent to invoke your App. The information Microsoft 365 needs is your App's Package name, Auth activity name, and Version code. Microsoft 365 will consider the provided Version code as the base version, and will assume that your App with this Version and above will support App-to-App authentication.

The information Microsoft 365 needs is passed via the URLScheme parameter in the WWW-Authenticate response header to unauthenticated bootstrap request.

Parameter	Value	Required	Example
Bearer	n/a	Yes	Bearer
authorization_uri	The URL of the OAuth2 Authorization Endpoint to begin authentication against as described at: RFC 6749#section-3.1	Yes	https://contoso.com/api/oauth2/authorize

Parameter	Value	Required	Example
<code>tokenIssuance_uri</code>	The URL of the OAuth2 Token Endpoint where authentication code can be redeemed for an access and (optional) refresh token. See Token EndPoint at: RFC 6749#section-3.2 	Yes	<code>https://contoso.com/api/oauth2/token</code>
<code>providerId</code>	A well-known string (as registered with with Microsoft 365) that uniquely identifies the host. Allowed characters: <code>[a-z,A-Z,0-9]</code>	No	<code>TP_CONTOSO</code>
<code>urlSchemes</code>	Information used to invoke your app (despite the name of the parameter, this may not always be URL schemes; e.g. on Android, intent is used). This is an ordered list by platform. Omit any platforms you do not support. Office will attempt to invoke these in order before falling back to the WebView auth.	No	<pre>{ "iOS": ["contoso", "contoso-EMM"], "Android": ["Package1VersionCode", "Package1Name", "Package1AuthActivityName", "Package2VersionCode", "Package2Name", "Package2AuthActivityName"], "UWP": [...] }</pre>

Client-side Changes

Invoking your App on Android

1. Microsoft 365 will create an intent, which will take the package name and auth activity name of your App. We'll set following two extras to intent:

- `AuthorizeUrlQueryParams`: It is exactly same as used in iOS without the `action` parameter. For example:
`client_id=abcdefg&response_type=code&scope=wopi&rs=enUS&build=16.1.1234&platform=android&app=word`
- `UserId`: It will be an optional parameter and will be set whenever we will have it. The third-party app should use this to verify that the sign in requested is for the User signed in to their App.

Code sample 1 - Sample intent creation code from [App2AppSigninIntent.java](#)

Java

```
protected void LaunchIntent()
{
    Intent authIntent = new Intent();
    authIntent.setComponent(new
ComponentName("com.android.thirdparty.packagename", "
com.android.thirdparty.packagename.AuthActivityName"));
    authIntent.putExtra(AuthorizeUrlQueryParams,
client_id=abcdefg&response_type=code&scope=wopi&rs=enUS&build=16.1.1234
&platform =android&app=word);
    authIntent.putExtra(UserId, User123);
    startActivityForResult(authIntent, uniqueRequestCode)
}
```

2. After this, Microsoft 365 will wait for the result and will expect following from the third-party App:

- `ResponseUrlQueryParams`: Again, this is exactly same as what we are getting in iOS minus the `action` parameter. The following are values of `ResponseUrlQueryParams` in success and failure cases:
 - `code=abcdefg&tk=http://contoso.com&sc=xyz` [during RESULT_OK]
 - `error=invalid_request&error_description="optional human readable message"` [during RESULT_CANCELLED or anything else]
- `UserId`: The third party should send which user it authenticated. Microsoft 365 will use this to show an error in case the `UserId` in the request and the response don't match.

Code sample 2 - Sample code from [App2AppSigninIntent.java](#)

Java

```
protected void onActivityResult(int requestCode, int resultCode, Intent result)
{
    // Check which request we're responding to
    if (requestCode == uniqueRequestCode)
    {
        // Make sure the auth request was successful
        if (resultCode == RESULT_OK)
        {
            //successfully authed
            // Here we will assume that result will contain
            ResonponseUrlQueryParams
            // which will look like this
            code=abcdefg&tk=http://contoso.com&sc=xyz
        }
        else if (resultCode == RESULT_CANCELED)
        {
            //auth request cancel
            // Here we will assume that result will contain
            ResonponseUrlQueryParams
            // But this time we will not get tk and session context but
            may get error and error_description.
        }
    }
    else
    {
        //failed
        // Here we will assume that result will contain
        ResonponseUrlQueryParams
        // But this time we will not get tk and session context but
        may get error and error_description.
    }
}
```

Work your App needs to do

1. Add an intent filter to AndroidManifest.xml:

Code sample 3 - Add an intent filter to [AndroidManifest.xml](#)

XML

```
<activity android:name="AuthActivityName">
    <intent-filter>
        <action android:name="android.intent.action.VIEW"/>
        <category android:name="android.intent.category.DEFAULT"/>
        <category android:name="android.intent.category.br /OWSABLE"/>
    </intent-filter>
</activity>
```

2. Handle the intent in AuthActivity:

Code sample 4 - Handle the intent using sample code from [HandleIntent.java](#) ↗

Java

```
protected void onCreate(Bundle savedInstanceState)
{
    super.onCreate(savedInstanceState);
    // Get the intent that started this activity
    Intent intent = getIntent();
    // Get the bundle of Extras, it will have data set by caller App
    Bundle extras = intent.getExtras();
}
```

3. Return the result:

Code sample 5 - Return the result using sample code from [HandleIntent.java](#) ↗

Java

```
protected void returnResult()
{
    // create intent to deliver result data
    Intent result = new Intent();
    result.putExtra("ResonponseUrlQueryParams",
        "code=abcdefg&tk=http://contoso.com&sc=xyz");
    setResult(Activity.RESULT_OK, result);
    finish();
}
```

You can find more details in [Return a Result on Android Developers](#) ↗.

Microsoft 365 for mobile Manual Validation

Article • 07/06/2022

Important

Contact csppteam@microsoft.com for instructions on how to access your mobile app in the QA environment.

You can test your WOPI Mobile integration using the steps outlined in [using the Validator to validate your WOPI implementation](#). However, the Validator will not be able to test the complete end-to-end experience. [You must run the following tests manually to verify the integration works correctly.](#) 

After all manual and validator tests have passed, you're ready for Microsoft to confirm your app is ready for the production environment.

You must provide Microsoft with the following so that we can perform verification testing:

1. A screenshot of your Validator results showing that all tests have passed
2. Test account information for use by Microsoft verification testers
3. The completed WOPI Mobile Testing Report
4. The exact name of your app as it appears in the Android and/or iOS stores

Email this information to csppteam@microsoft.com.

Note

- *Company app/application* refers to your app.
- *Company service* refers to your service.
- Where you see a reference to *Microsoft 365*, please substitute Word, Excel and PowerPoint app. The tests should be re-run against each Microsoft 365 application.
- Please note the version of the Microsoft 365 app against which you tested.

1. Open from Company app: first install

This test verifies the flow of using Microsoft 365 for the first time from the Company app. Repeat for each supported file type and each Microsoft 365 app.

1. Start with a fresh install of Company app. Ensure Microsoft 365 is not installed.
2. Boot up Company app and log in.

RESULT: General promotion for Microsoft 365 should be shown the first time after upgrading the company app.

1. Browse to a supported file type.

RESULT: "Open with Microsoft [app]" promotion, drawing attention to control and enabling open with Office once per first open for each supported file type. "Open with Microsoft [app]" should be the top choice if multiple choices are available. If a list is not shown, Microsoft 365 should be the default app for opening the file.

1. Activate control to open in Microsoft 365.

RESULT: The user should be sent to the app store page for the corresponding app.

1. Install the Microsoft 365 app.
2. After installation, go back to the Company app and activate the control to open in Microsoft 365 again.

RESULT: Microsoft 365 should start. The user should be prompted for credentials to Company service.

1. Enter credentials for the company service.

RESULT: File should open in read-only mode.

1. Select **Sign In** and sign in with a free Microsoft Account.

RESULT: File should open in edit mode if the user is a consumer and read-only mode if the user is a commercial user.

1. Make changes (you will need to sign in with a subscription account for testing a commercial user).
2. Select **Back (<-)**.
3. Select **Open**.

RESULT: Confirm that the company service is shown as a place.

2. Open from Microsoft 365 for mobile: fresh install

This test verifies the flow of using the Company service for the first time from Microsoft 365.

1. Launch a fresh install of Microsoft 365.
2. Go through the First Run Experience.
3. Skip **Sign In**.
4. Go to **Open** > **Add a Place**.

RESULT: The Company service shows up. Verify the name and icon of your service.

1. Select your Company service.
2. Enter your credentials.

RESULT: The root folder should show.

1. Browse around the folder structure in your service.

RESULT: Browse works as expected.

1. Open a file from **Browse**.

RESULT: The file should open in read-only mode.

1. Select **Sign In** and sign in with a free Microsoft Account.

RESULT: The file should open in edit mode if the user is a consumer and read-only mode if the user is a commercial user.

1. Make changes (you will need to sign in with a subscription account for testing a commercial user).
2. Select **Back (<-)**.
3. Select **Open**.

RESULT: The file should have the previously saved changes. Ensure changes are being saved on Company service.

3. Open from Company app: repeat usage

Repeat test 1, except with the Company service already added (from previous usage).

4. Open from Office for Mobile: repeat usage

Repeat test 2, except with the Company service already added (from previous usage).

5. Save as/duplicate

Verify the ability to duplicate a file to the Company service, both by adding a new place and by using an existing place.

6. Create new [name]

Verify the ability to create a new file saved to the Company service, both by adding a new place and by using an existing place.

7. Verify licensing

Verify that editing a file for a commercial user requires O365 subscription or else it opens read only.

Important

Go to *Settings > [Microsoft App] > Reset Word > Delete Sign-In Credentials* and restart Microsoft 365 before doing this test.

8. OAuth login page

Verify there is a link to the company's privacy statement on the company's login page when the user adds the Company service as a place.

Verify that the login page fits in the window of supported devices.

9. Verify file properties

Verify file properties from **Recent** and from an opened file. When opening the properties from the **Recent** tab or the **Open** tab, the *Author*, *Created*, *Modified By*, and *Company* fields will be empty.

10. Change passwords

This test verifies the flow of using the Company service after the user changes passwords.

ⓘ Note

This test changes based on how the Company service handles authentication and refresh/access tokens. If you invalidate the access and refresh token after the user changes their password, run this test. You can adapt this test to ensure the Microsoft 365 app is handling refresh and access tokens correctly.

1. Launch a fresh install of Microsoft 365.
2. Go through the First Run Experience.
3. Skip **Sign In**.
4. Go to **Open** > **Add a Place**.
5. Select your Company service.
6. Enter your credentials.
7. Browse around the folder structure in your service.
8. Open a file from **Browse**.
9. Select **Sign in** and sign in with a free Microsoft Account.
10. Make changes (you will need to sign in with a subscription account for testing a commercial user).
11. Select **Back**.
12. On the Company service app, change the user's password.
13. Open the Microsoft 365 for mobile app, browse to the Company service, and open a file.

RESULT: You should be prompted to enter credentials again.

Using the validator application to validate your WOPI implementation

Article • 05/02/2023

Because WOPI is used in both the Office for the web and Microsoft 365 for mobile integrations, you can verify your WOPI implementation by following the instructions in the [WOPI Validator](#) topic.

If you do not integrate with Office for the web, you can still verify your WOPI implementation using the above instructions by building a minimal [host page](#) and setting the `VALIDATOR_TEST_CATEGORY` to `OfficeNativeClient`. This will run only the tests that are necessary for Microsoft 365 for mobile integration.

The Validator will not be able to verify the end-user experience entirely, so you must also perform manual validation.

ⓘ Note

You will not be able to invoke any **WOPI actions** successfully unless your WOPI domain has been added to the **WOPI domain allow list**.

Cloud Storage Partner Program Plus (CSPP Plus) Overview

Article • 08/09/2023

CSPP Plus is a paid-program tier intended for a very limited set of highly qualified partners requiring deeper interoperability with Microsoft 365.

Please note that the qualifications listed here are not an exhaustive list, and entry to this program tier is extremely selective.

Program Qualifications

- Participation in the basic Microsoft CSPP program.
 - If you're not already a partner, [apply for basic CSPP membership](#) today.
- A non-refundable program entry fee, other fees may apply.
- Liability insurance including, but not limited to:
 - Commercial General Liability insurance.
 - Privacy and cybersecurity liability insurance.
 - Professional Liability/E&O insurance. Other Insurance: (4) Workers Compensation to meet statutory requirements, and (5) Employer's liability.
- Complete Microsoft 365 App Certification (including ISO-27001 certification).

Program Benefits

- Coauthoring in Word, Excel, and PowerPoint for Office for the web, Office Desktop, and Microsoft 365 for mobile.
- Access to Microsoft FedRamp environments from similarly compliant partner's environments
- Microsoft Intune mobile provisioning for partner storage locations
- Invitation to the semi-annual Microsoft CSPP Plus participant summit
- Increased file size limits for Office Online clients:

- Word - 100MB
- Excel - 100MB
- PowerPoint - 2GB

ⓘ **Note**

These file size limits apply only to Office for the web clients. Desktop and mobile clients do not have these limitations. However, extremely large files, network bandwidth, and other environmental factors can reduce overall performance.

Next steps

If you are already a basic CSPP member, meet the above Program Qualifications and are interested in applying to join CSPP Plus, please reach out to cspp-plus-partners@microsoft.com for more information.

WOPI extensions for coauthoring overview

Article • 03/09/2023

Here, you'll learn about changes and additions to the existing Web Application Open Platform Interface (WOPI) protocol to support real-time coauthoring with third-party hosts as part of Cloud Services Partner Program Plus (CSPP Plus). The documentation for WOPI extensions for coauthoring doesn't include elements related to uploading or downloading file contents, which are documented separately.

What's covered

These articles assume that the reader has the core knowledge of how the existing WOPI protocol works.

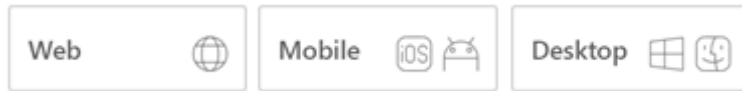
The articles refer to many concepts and terms that are defined by the existing protocol.

Next steps

- Learn about additional [CheckFileInfo](#) properties required and optional for CSPP Plus implementations.
- Learn what's new in the [GetNewAccessToken](#) WOPI coauthoring extensions.
- Learn about [additional lock types](#) provided by the WOPI coauthoring extensions.

Additional CheckFileInfo properties for CSPP Plus hosts

Article • 08/21/2024



To enable the WOPI coauthoring extensions for CSPP Plus, you must set the following existing optional properties and new properties.

If any of the required properties are set to their default value (`0` for int, `false` for Boolean, `String.Empty` or null for strings), the client must act as if the coauthoring extensions aren't available, effectively behaving as if `SupportsCoauth` is set to `false`. The exception is `AccessTokenExpiry`, for which `0` is a valid value.

New optional properties

For the WOPI coauthoring extensions to be enabled, the following new optional properties are *required* to be set to valid values:

- `SupportsCoauth`
- `SequenceNumber`
- `OfficeCollaborationServiceEndpointUrl`
- `RealTimeChannelEndpointUrl`
- `AccessTokenExpiry`
- `ServerTime`
- `SharingStatus`
- `FileGeoLocationCode`

SupportsCoauth

A *Boolean* value that indicates that the host provides the support for multiple WOPI clients to participate in document content modification in an overlapping manner. In addition to promising a non-default value for the following properties, it also indicates implementation of the following WOPI operations:

- `GetCoauthLock`
- `RefreshCoauthLock`
- `UnlockCoauthLock`

- `GetCoauthTable`
- `GetChunkedFile`
- `PutChunkedFile`
- `GetSequenceNumber`

SequenceNumber

An *int* value that indicates the current sequence number of the file's contents. The value controls the document coherency semantic for the update. For more information, see [GetSequenceNumber](#).

OfficeCollaborationServiceEndpointUrl

A *string* value that is a URI to the Collaboration Service endpoint that the WOPI client can use for collaboration. The value is the URL associated with the "collab" action provided in WOPI discovery.

RealTimeChannelEndpointUrl

A *string* value that is a URI to the Real Time Channel Service endpoint that the WOPI client can use for real-time updates. The value is the URL associated with the "rtc" action provided in WOPI discovery.

AccessTokenExpiry

A *long* value that indicates the time of access token expiry, represented as the number of milliseconds since January 1, 1970 UTC (the date epoch in JavaScript). For more information, see [access_token_ttl](#).

ServerTime

A *long* value that indicates the time on the server when responding to the request, represented as the number of milliseconds since January 1, 1970 UTC (the date epoch in JavaScript).

SharingStatus

ⓘ Note

`SharingStatus` is an existing property marked as "(Pre-release property - not yet used by any WOPI client)" in the currently available public documentation. The property is applicable for coauthoring extensions.

A *string* value that indicates whether the current document is shared with other users. The value can change when you add or remove permissions to other users. Clients might use this value to help choose how active to be in checking to see whether multiple clients are simultaneously working with the file. For more information, see [SharingStatus](#).

Possible values:

- **Private.** Only the document owner has permission to the file.
- **Shared.** At least one other user has access to the file via direct permissions or a sharing link.

The following *new* optional properties are *not required* for the coauthoring extensions to be enabled.

ComplianceDomainPrefix

A *string* value that is the regional compliance domain for this file, and which specifies that traffic should remain in a specific country or region. Don't return the property if there are no regional compliance requirements. Please contact CSPP Plus team for the supported values.

FileGeoLocationCode

A *string* value that indicates the geographical location where the file is hosted. If the `ComplianceDomainPrefix` property is returned, WOPI clients use that property to infer the file's location.

Please contact CSPP Plus team for the supported values.

Existing optional properties

To enable the coauthoring extensions, these *existing* optional properties are *required* to be set to valid values:

- `SupportsLocks`
- `SupportsUpdate`
- `SupportsUserInfo`

SupportsLocks

An existing *Boolean* property that must be `true`, indicating that the associated WOPI locking methods are supported.

SupportsUpdate

An existing *Boolean* property that must be `true`, indicating that the associated WOPI file-updating methods are supported.

SupportsUserInfo

An existing *Boolean* property that must be `true`, indicating that the associated WOPI method is supported.

GetNewAccessToken overview

Article • 07/06/2022

You *must* set the new `EndPoints` property in response to the existing [GetNewAccessToken](#) request. The contents of this property should be a nested JSON-formatted object. The object should have the following properties, with values that exactly match what would be returned in response to a [CheckFileInfo](#) request:

- `RealTimeChannelEndpointUrl`
- `OfficeCollaborationServiceEndpointUrl`

Sample response

The following sample response includes the new JSON properties:

JSON

```
{
  "Bootstrap": {
    "EcosystemUrl": "http://.../wopi*/ecosystem?access_token=",
    "UserId": "User ID",
    "SignInName": "user@contoso.com",
    "UserFriendlyName": "User Name"
  },
  "AccessTokenInfo": {
    "AccessToken": "1234567890abcdef",
    "AccessTokenExpiry": 1234567890
  },
  "EndPoints": {
    "RealTimeChannelEndpointUrl": "http://rtc",
    "OfficeCollaborationServiceEndpointUrl": "http://ocs"
  }
}
```

Next steps

- Learn what's new in the [CheckFileInfo](#) coauthoring extensions.
- Learn what's new in the [WOPI locks](#) coauthoring extensions.

Locks overview

Article • 07/06/2022

To support editing and coauthoring files, clients that use the WOPI protocol require the WOPI host to support locking files. For more information, see the full list of [WOPI lock APIs](#).

Lock types

- **WOPI locks:** Existing [WOPI locks](#) that are already used in Cloud Services Partner Program Plus (CSPP Plus), and which are extended with more properties for better support for clients. WOPI locks are mutually exclusive for use with all other lock types the host supports, including `Coauth` and `CoauthExclusive` locks.

Additional lock types provided by the coauthoring extensions:

- **`Coauth` lock:** New locks that are used to support the coauthoring extensions. The document must have `SupportsCoauth` enabled. The client must have edit permissions to request this lock. A client can acquire this lock only if one of the following scenarios is true:
 - The file isn't locked by any mechanism that the host understands (including but not limited to WOPI locks); or
 - The file is locked by any number of clients that hold `Coauth` lock references, and zero or one client holds a `CoauthExclusive` lock reference. If this lock is held, only clients that hold a `Coauth` lock may update the document (unless there is a `CoauthExclusive` lock).
- **`CoauthExclusive` lock:** A subtype of a `Coauth` lock that can be held by *at most* one client per document at any given time. The document must have `SupportsCoauth` enabled. The client must have edit permissions to request this lock. A client can acquire this lock only if one of the following scenarios is true:
 - The file isn't locked by any mechanism that the host understands (including but not limited to WOPI locks); or
 - The file is locked by any number of clients that hold `Coauth` lock references and no client holds a `CoauthExclusive` lock reference. This lock doesn't inhibit other client's abilities to acquire or release `Coauth` locks. If this lock is held, only the client that holds this lock may update the document.

📌 Note

Locks only affect modifying the file, and do not impact requests to read the file.

ⓘ Note

For the purposes of the existing WOPI lock semantics, the new `Coauth` locks are considered "another interface." As a result, if a `Coauth` or `CoauthExclusive` lock is held on the document, all **409 Conflict** [↗](#) responses should omit the `X-WOPI-Lock` response header for all WOPI methods.

How clients work with locks

A client can switch between a `Coauth` lock and a `CoauthExclusive` lock by calling the `GetCoauthLock` method and requesting the other type of lock. The same rules for evaluating the success of the `GetCoauthLock` call apply, regardless of the client's current lock status. `UnlockCoauthLock` removes the client's lock. When a client acquires a `CoauthExclusive` lock, only that client may update the document on the host. Requests for document updates that are made without the `CoauthExclusive` lock reference should fail.

`Coauth` lock and `CoauthExclusive` lock references are associated with their `CoauthLockId` value instead of with the user identity. A single user identity might have multiple lock references and a distinct `CoauthLockId` value, while a lock with a specific reference might be taken, refreshed, or released by calls that have different user identities.

Next steps

- Learn what's new in the [CheckFileInfo](#) coauthoring extensions.
- Learn what's new in the [GetNewAccessToken](#) coauthoring extensions.
- Learn what's new in the [WOPI locks](#) coauthoring extensions.

WOPI locks

Article • 07/06/2022

The existing WOPI lock implementation is extended to allow clients to specify the lock duration when they acquire locks and to refresh locks. If the new header isn't specified, the current 30-minute duration applies.

Lock/RefreshLock/UnlockAndRelock syntax

For full descriptions of these methods, see the existing API contracts:

- [Lock](#)
- [RefreshLock](#)
- [UnlockAndRelock](#)

Additional headers for existing Lock, RefreshLock, and UnlockAndRelock methods:

Request headers

- `X-WOPI-LockExpirationTimeout` (uint) – Optional. Represents the timeout duration for the lock expiration in seconds. The value can be part of a configuration value on the client side. The host must honor the lock expiration timeout duration the client provides. The timeout range must be within the values of 1 minute and 1 hour, inclusive of these values. If the value provided isn't within an acceptable range, the host should send 400 Bad Payload as a response.
- `X-WOPI-LockUserVisible` (Boolean) – Optional. If this header is set to `true`, the `UserFriendlyName` value that's associated with the access token for this request should be provided in `X-WOPI-ConflictingLockUsername` for responses that get a 409 Conflict response as a result of this lock being held. This value is valid only until the next time the lock is updated.

Response headers

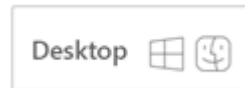
- `X-WOPI-ConflictingLockUsername` (string) – Optional. If the conflict is caused by a WOPI-exclusive lock being held, and if the `UserFriendlyName` value that's associated with the lock is visible (see `X-WOPI-LockUserVisible`), this header should contain that `UserFriendlyName` value when it responds to the request with 409 Conflict. This string may be presented to the user by the Microsoft 365 user interface.

Next steps

- Learn what's new in the [CheckFileInfo](#) WOPI coauthoring extensions.
- Learn what's new in the [GetNewAccessToken](#) WOPI coauthoring extensions.

RenameFile for CSPP Plus

Article • 03/04/2023



If a document is held by one or more `Coauth` or `CoauthExclusive` lock references, a valid `Coauth` lock reference must be provided for the call to succeed.

For a full description of the `RenameFile` API, see the existing API contract:

- [RenameFile](#)

Request headers

- `X-WOPI-CoauthLockId` (string) – Optional. A string provided by the client that the host must use to identify the `Coauth` lock on the file. Size limit: 1,024 ASCII characters.

Status codes

- [400 Bad Request](#) 
 - The specified name is illegal.
 - Occurs when both `X-WOPI-Lock` and `X-WOPI-CoauthLockId` are provided. Only one of the headers should be set.
 - The header size wasn't within the acceptable range.
- [409 Conflict](#) 
 - Lock mismatch - The file is locked but the supplied lock type (`X-WOPI-Lock`, `X-WOPI-CoauthLockId`) or value doesn't match the actual lock on the file.
 - The file is locked, but the client did not supply any lock (`X-WOPI-Lock` or `X-WOPI-CoauthLockId`)
 - Locked by another interface.

Next steps

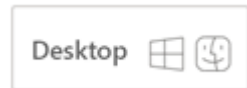
Learn about changes in these methods for WOPI coauthoring extensions:

- [GetCoauthLock](#)
- [UnlockCoauthLock](#)
- [RefreshCoauthLock](#)

- [GetCoauthTable](#)
- [WOPI locks](#)

GetCoauthLock

Article • 03/04/2023



Clients can request a `Coauth` lock by using a `GetCoauthLock` operation on a file. Clients can request `CoauthExclusive` locks if the document isn't exclusively locked and no other client has a `CoauthExclusive` lock.

Parameters

`file_id` – Required. Passed as part of the endpoint.

Endpoint

`/wopi/files/(file_id)` – Required.

Query parameters

`access_token` (string) – Required. An access token that the host uses to determine whether the request is authorized.

Request method

POST

Request headers

- `X-WOPI-Override` (string) – Required. The string is `GET_COAUTH_LOCK`.
- `X-WOPI-CoauthLockId` (string) – Required. A string provided by the client that the host must use to uniquely identify the client for the lock on the file. Size limit: 1,024 ASCII characters.
- `X-WOPI-CoauthLockType` (string) – Required. The string is `GET_COAUTH_LOCK`. The value can be `Coauth` or `CoauthExclusive`.
- `X-WOPI-CoauthLockMetadata` (string) – Optional. Set by the client to any value. If the header is omitted or empty, `CoauthLockMetadata` is set to `empty`. Size limit: 4 KB.

- `X-WOPI-CoauthLockExpirationTimeout` (uint) – Required. Represents the timeout duration in seconds for the lock expiration. Can be part of a configuration value on the client side. The host must honor the lock expiration timeout duration that the client provides. The timeout range must be within the values of 1 minute and 1 hour, inclusive of these values.

Response headers

- `X-WOPI-CoauthTableVersion` (string) – Required. The current version of the CoauthTable based on the host state of the CoauthTable ignoring expiration time for locks. This value must change when the CoauthTable changes. Change here indicates any property of any of the rows returned as part of CoauthTable. Please note that this value should not change if only the expiration time changes for one or more existing CoauthTable entries. This should be returned only as a part of successful response where response code is 200 OK.
- `X-WOPI-FailureReason` (string) – Optional. Used for logging purposes only. It should be set on non-200 OK responses.
- `X-WOPI-ConflictingLockUsername` (string) – Optional. If the conflict is caused by a WOPI-exclusive lock being held, and if the `UserFriendlyName` associated with the lock is visible (see `X-WOPI-LockUserVisible`), this header should contain that `UserFriendlyName` value when it responds to the request with 409 Conflict. This string might be presented to the customer by the Office UX.

Response body

A mandatory JSON-formatted object that contains the property `CoauthTable`. For the `CoauthTable` structure, see [GetCoauthTable](#). Returns `CoauthTable` in JSON format and contains array of entry of each client that has the unexpired `Coauth` or `CoauthExclusive` locks.

Status codes

- [200 OK](#)  – Success. `Coauth` lock was acquired on the file. The operation succeeds if the file is not exclusively locked by another entity. If a coauth lock already exists with the given `CoauthLockId` and same `CoauthLockType`, the request succeeds. The request in this case updates `CoauthLockMetadata` and `CoauthLockExpirationTimeout`. If a coauth lock already exists with the given `CoauthLockId` but a different `CoauthLockType`, the lock type is switched, except

when the request is for a `CoauthExclusive` type lock and another client has `CoauthExclusive` lock.

- [400 Bad Request](#)
 - The required parameters weren't provided.
 - The request was incorrectly formatted.
 - The `CoauthLockExpirationTimeout` that was provided wasn't within an acceptable range.
 - The header size wasn't within an acceptable range.
- [401 Unauthorized](#) – Invalid access token
- [404 Not Found](#)
 - The file wasn't found.
 - The access token doesn't have edit permissions.
- [409 Conflict](#)
 - The resource is exclusively locked by a different locking mechanism.
 - The resource is exclusively locked by another entity using the same locking mechanism.
- [500 Internal Server Error](#) – Server error
- [501 Not Implemented](#) – Operation not supported
- [503 Server Busy](#)

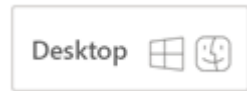
Next steps

Learn about changes in these `CoauthLocks` methods for WOPI coauthoring extensions:

- [UnlockCoauthLock](#)
- [RefreshCoauthLock](#)
- [GetCoauthTable](#)

UnlockCoauthLock

Article • 03/04/2023



A `Coauth` lock can be released by the clients using `UnlockCoauthLock` operation on a file. The host will determine the lock entry to be released by using the file ID and `CoauthLockId`.

Parameters

`file_id` – Required. Passed as part of the endpoint.

Endpoint

`/wopi/files/(file_id)` – Required.

Query parameters

`access_token` (string) – Required. An access token that the host uses to determine whether the request is authorized.

Request method

POST

Request headers

- `X-WOPI-Override` (string) – Required. The string is `UNLOCK_COAUTH_LOCK`.
- `X-WOPI-CoauthLockId` (string) – Required. A string provided by the client that the host must use to uniquely identify the client for the lock on the file. Size limit: 1,024 ASCII characters.

Response headers

- `X-WOPI-FailureReason` (string) – Optional. Used only for logging purposes. It should be set on non-200 OK responses.

Status codes

- [200 OK](#)  – Success. `Coauth` lock was released on the file for the client.
- [400 Bad Request](#) 
 - The required parameters weren't provided.
 - The request was incorrectly formatted.
 - The header size wasn't within an acceptable range.
- [401 Unauthorized](#)  – Invalid access token
- [404 Not Found](#) 
 - The file doesn't exist.
 - The access token doesn't have edit permissions.
- [409 Conflict](#) 
 - The lock doesn't exist.
- [500 Internal Server Error](#)  – Server error
- [501 Not Implemented](#)  – Operation not supported
- [503 Server Busy](#) 

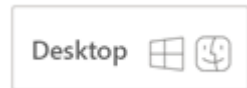
Next steps

Learn about changes in these `CoauthLocks` methods for WOPI coauthoring extensions:

- [GetCoauthLock](#)
- [RefreshCoauthLock](#)
- [GetCoauthTable](#)

RefreshCoauthLock

Article • 03/04/2023



A client can refresh a `Coauth` lock by using the `RefreshCoauthLock` operation on a file. The host determines the lock entry to be refreshed by using the file ID and the `CoauthLockId` value.

This call works only if there is an unexpired lock reference for the `Coauth` or `CoauthExclusive` lock.

Parameters

`file_id` – Required. Passed as part of the endpoint.

Endpoint

`/wopi/files/(file_id)` – Required.

Query parameters

`access_token` (string) – Required. An access token that the host uses to determine whether the request is authorized.

Request method

POST

Request headers

- `X-WOPI-Override` (string) – Required. The string is `REFRESH_COAUTH_LOCK`.
- `X-WOPI-CoauthLockId` (string) – Required. A string provided by the client that the host must use to uniquely identify the client for the lock on the file. Size limit: 1,024 ASCII characters.
- `X-WOPI-CoauthLockExpirationTimeout` (uint) – Required. Represents the timeout duration for the lock expiration in seconds. The value can be part of a

configuration value on the client side. The host must honor the lock expiration timeout duration the client provides. The timeout range must be within the values of 1 minute and 1 hour, inclusive of these values.

- `X-WOPI-CoauthLockMetadata` (string) – Optional. Set by the client to any value. If the header is omitted or empty, the `CoauthLockMetadata` value remains unchanged. If the header is set to a non-empty string, `CoauthLockMetadata` is overwritten with that value. Size limit: 4 KB.

Response headers

- `X-WOPI-CoauthTableVersion` (string) – Required. The current version of the CoauthTable based on the host state of the CoauthTable ignoring expiration time for locks. This value must change when the CoauthTable changes. Change here indicates any property of any of the rows returned as part of CoauthTable. Please note that this value should not change if only the expiration time changes for one or more existing CoauthTable entries. This should be returned only as a part of successful response where response code is 200 OK.
- `X-WOPI-FailureReason` (string) – Optional. Used for logging purposes only. It should be set on non-200 OK responses.

Response body

A mandatory JSON-formatted object that contains the `CoauthTable` property. For the `CoauthTable` structure, see [GetCoauthTable](#). Returns `CoauthTable` in JSON format and contains an array of entry of each client that has the unexpired `Coauth` locks.

Status codes

- [200 OK](#)  – Success. `Coauth` lock was refreshed on the file for the client.
- [400 Bad Request](#) 
 - The required parameters weren't provided.
 - The request was incorrectly formatted.
 - The `CoauthLockExpirationTimeout` provided was not within an acceptable range.
 - The header size wasn't within the acceptable range.
- [401 Unauthorized](#)  – Invalid access token
- [404 Not Found](#) 
 - The file wasn't found.
 - The access token doesn't have edit permissions.
- [409 Conflict](#) 

- The lock doesn't exist.
- [500 Internal Server Error](#)  – Server error
- [501 Not Implemented](#)  – Operation not supported
- [503 Server Busy](#) 

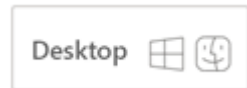
Next steps

Learn about changes in these `CoauthLocks` methods for WOPI coauthoring extensions:

- [GetCoauthLock](#)
- [UnlockCoauthLock](#)
- [GetCoauthTable](#)

GetCoauthTable

Article • 03/04/2023



Clients can call the `GetCoauthTable` operation to fetch the table that contains information about all the unexpired `Coauth` locks that the various clients acquired. This method doesn't require edit permissions to be called. This method is called frequently.

Parameters

`file_id` – Required. Passed as part of the endpoint.

Endpoint

`/wopi/files/(file_id)` – Required.

Query parameters

`access_token` (string) – Required. An access token that the host uses to determine whether the request is authorized.

Request method

POST

Request headers

- `X-WOPI-Override` (string) – Required. The string is `GET_COAUTH_TABLE`.
- `X-WOPI-CoauthTableVersion` (string) – Optional. `X-WOPI-CoauthTableVersion` value received from previous `GetCoauthTable` calls indicating the `CoauthTable` state client already has. If this is same as the latest state on the host, the host must not return any body for `GetCoauthTable` response and return HTTP code 200 with same `CoauthTableVersion` in response header. If this header is empty or not matching the current host state (`CoauthTableVersion`), the full `CoauthTable` would be returned.

Response headers

- `X-WOPI-CoauthTableVersion` (string) – Required. The current version of the CoauthTable based on the host state of the CoauthTable ignoring expiration time for locks. This value must change when the CoauthTable changes. Change here indicates any property of any of the rows returned as part of CoauthTable. Please note that this value should not change if only the expiration time changes for one or more existing CoauthTable entries. This should be returned only as a part of successful response where response code is 200 OK.
- `X-WOPI-FailureReason` (string) – Optional. Used for logging purposes only. It should be set on non-200 OK responses.

Response body

A mandatory JSON-formatted object that contains the `CoauthTable` property. For the `CoauthTable` structure, see [GetCoauthTable](#). Returns `CoauthTable` in JSON format and contains an array of entry of each client that has the unexpired `Coauth` locks:

- `CoauthLockId` (string) – The ID that's provided by the client.
- `CoauthLockMetadata` (string) – Set by the client when it requests the lock.
- `UserFriendlyName` (string) – The name of the user, suitable for displaying in the UI. Hosts should be able to populate this value based on the access token that was used to acquire the lock for that client ID.
- `CoauthLockTime` (uint) – A long value that indicates the time on the server when the lock was first acquired. The value shouldn't be updated in response to `RefreshCoauthLock` or `GetCoauthLock` when the lock already exists with a specific `CoauthLockId` value. Represented as the number of milliseconds since January 1, 1970 UTC (the date epoch in JavaScript).
- `CoauthLockType` (string) – `Coauth` or `CoauthExclusive`.

ⓘ Note

`None` is a value that's reserved for future use.

Status codes

- [200 OK](#)  – Success. Returns the `CoauthTable` value, even if it's empty.
- [400 Bad Request](#) 
 - The required parameters weren't provided.
 - The request was incorrectly formatted.
- [401 Unauthorized](#)  – Invalid access token
- [404 Not Found](#) 
 - The file doesn't exist.
 - The access token doesn't have edit permissions.
- [409 Conflict](#) 
 - The file is locked by another mechanism.
- [500 Internal Server Error](#)  – Server error
- [501 Not Implemented](#)  – Operation not supported
- [503 Server Busy](#) 

Next steps

Learn about changes in these `CoauthLocks` methods for WOPI coauthoring extensions:

- [GetCoauthLock](#)
- [UnlockCoauthLock](#)
- [RefreshCoauthLock](#)

Content sequence numbering and coherency

Article • 07/06/2022

The host stores the following parts of the content:

- Primary document content (`MainContent`)
- Multiple streams within the document in addition to `MainContent` (for example, the alternate stream used by Excel to write revision records)
- "Side car" information, which is used to store the last OCS save for the specific session.

ⓘ Note

All three of these content parts can be modified and fetched in a single request or response. Modifications must be processed within a transaction.

Strict content sequence numbering

The content parts are bundled together and associated with any given state of the file on the host. Each state on the host is assigned an integer sequence number, which must be assigned increasing values greater than zero. The maximum valid value is 2,147,483,647.

Coherency

All modification transactions with a `Coauth` lock must be gated with a sequence number. The request provides the expected sequence number. If the host's current state doesn't match that sequence number, then the request fails.

Next steps

[Chunk streams](#) for efficient file transfer.

Chunk streams

Article • 07/06/2022

To achieve incremental file transfer, the file contents are broken into chunks. How a binary stream is broken into chunks depends on the chunking scheme. The chunks constructed to be sent in the request or response stream may appear in any order. The file signature should be used to determine the order of the chunks.

Two chunking schemes are supported:

- `zip` – Applicable to zip files, which are the default format of Office files that support coauthoring.
- `FullFile` – Applicable to encrypted Office files. The full binary contents of the given stream are represented as a single chunk.

The host must support all of the chunking schemes. Zip chunking is the preferred scheme, except in scenarios with encrypted files that aren't in zip format.

ChunkId

Chunks are identified by `ChunkId`, which is the hash (128-bit Spooky hash) of the chunk binary payload.

File signature

The ordered list of `ChunkId` identifiers and the length of all the chunks for a given file. The file signature is designed to self-describe the full content of the file.

Request to get delta changes

1. Select the `chunkingScheme` option based on the file content.
2. Send a list of `ChunkId` identifiers that are already known to the WOPI client.

Response to get delta changes

1. Select the `chunkingScheme` option based on the file content.
2. Chunk the binary file contents based on `chunkingScheme`.
3. Compute `ChunkId` for each chunk and generate a file signature.
4. Decode the list of known chunks in the request.

5. Send the file signature and the chunks referred to in the file signature that are not in the request's list of known chunks.

Request to update file based on delta changes

1. Select the `chunkingScheme` option based on the file content.
2. The WOPI client chunks the updated binary file based on `chunkingScheme`.
3. Generate a new file signature.
4. Compare the new file signature with the last known file signature.
5. Send the new file signature and the chunks referred in the new file signature that aren't in the last known file signature.

Applying the delta changes

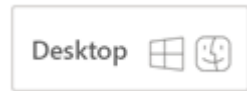
1. Select the `chunkingScheme` option based on the file content.
2. Chunk the base file based on `chunkingScheme`.
3. Assemble the new list of chunks based on the file signature, base file chunks, and delta chunks.
4. Write all the valid chunks in the order provided by the file signature you received.
This step doesn't require any zip semantics.

Next steps

- [CheckFileInfo](#)
- [GetChunkedFile](#)

CheckFileInfo and File Transfer Protocol

Article • 03/04/2023



The new *optional* `SupportsResumableFileTransfer` property enables resumable file transfers included as part of the WOPI coauthoring extensions.

Implementation

The `SupportsResumableFileTransfer` property is a Boolean value that indicates whether the host supports resumable file transfers. It also implements the following WOPI operations:

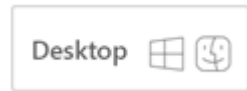
- `GetChunks`
- `CreateUploadSession`
- `UpdateUploadSession`

Next steps

- [GetChunkedFile](#)
- [CreateUploadSession](#)

GetChunkedFile

Article • 03/04/2023



POST /wopi/files/(file_id)

The `GetChunkedFile` operation retrieves the chunks for the file from the host. If the WOPI client already has a version of the file, it passes the list of `ChunkId` identifiers for already known chunks to the host.

Clients can use the `ContentFilters` filter to indicate which chunks are returned by the host.

The host needs to check if the file is a zip archive by following the [ZIP File Format Specification](#) [↗].

- If the file is not a zip archive, treat the full binary content of the given stream as a single chunk, and pass the file signature and the single chunk in the response body.
- If the file is a zip archive file and the client hasn't presented the list of known chunk IDs, pass the file signature and data for all chunks in the response body.
- If the file is a zip archive and the client presents the known chunk IDs, the host needs to compare the client's known chunk IDs passed in the request body with their own file signature. Then the host returns chunks not in the list.

In all of these cases, the host must pass a sequence number in the response header.

Parameters

- `file_id` (string) – Required. A string that specifies a file ID of a file managed by the host. This string must be URL safe.

Query parameters

- `access_token` (string) – Required. An access token that the host uses to determine whether the request is authorized.

Request headers

- `X-WOPI-Override` (string) – Required. The string is `GET_CHUNKED_FILE`.

Request body

Sample message:

JSON

```
{
  "ContentPropertiesToReturn": [
    "prop1",
    "prop2"
  ],
  "ContentFilters": [
    {
      "AlreadyKnownChunks": [
        "R3ZBz20okt9LmDaePUpB8Q=="
      ],
      "ChunkingScheme": "Zip",
      "StreamId": "MainContent",
      "ChunksToReturn": "All"
    },
    {
      "AlreadyKnownChunks": [],
      "ChunkingScheme": "FullFile",
      "StreamId": "AltStream",
      "ChunksToReturn": "All"
    }
  ]
}
```

Required request properties

The following properties must be present in all `GetChunkedFile` requests:

ContentPropertiesToReturn

This is a required property, but it can be an empty list.

ContentFilters

This allows the WOPI client to filter which streams and which chunks within those streams should be returned. This does not impact the `MessageJSON` frame returned, which will always contain the full signature of each stream being requested.

This should at least have one entry.

Valid values for `ChunksToReturn`:

- `All` – Receive all chunks that are not in the list of `AlreadyKnownChunkIds`. This is the default.
- `None` – Do not receive any chunk data.
- `LastZipChunk` – Receive only the last chunk of a zip archive. If the file is not a zip archive, this value behaves the same as `None`. If the host can't support zip chunking for the given file, treat this value as `None`.

The host only returns streams requested via `ContentFilters`. There is no way for the client to get all the streams without knowing the `StreamId` value in advance. In addition, there should only be one `ContentFilter` entry per `StreamId`. Otherwise, the host returns a 400 error. There should be at least one entry in the `ContentFilters` element. Otherwise, the request fails with a 400 return code. If the requested `StreamId` doesn't exist on the host, the request succeeds. In this case, the host returns a `Signature` entry with zero chunks in the MessageJSON. In this case, the `ChunkingScheme` should be set to 'Zip'. Anywhere `ChunkId` is used in a MessageJSON frame (`ChunkId` or `AlreadyKnownChunks`), the raw 16-byte spooky hash is encoded by using "Base64 encoding" (C# equivalent: `System.Convert.ToBase64String()`).

ⓘ Note

The `MainContent` stream should always exist after the file is created. It can be a zero-byte stream, which is represented as a `ChunkSignatures` array with one item. The `ChunkId` value for the zero-length item should be the Spooky hash of a zero-byte array.

Response headers

- `X-WOPI-SequenceNumber` (integer) – Required. An integer value that indicates the state of the file on the host. The value must be greater than 0, and should be increased if the host file is updated.
- `X-WOPI-ItemVersion` (string) – Required. A string value that indicates the file version. Its value should be the same as the `Version` value in `CheckFileInfo`.
- `X-WOPI-FailureReason` (string) – Optional. Used for logging purposes only. Set this value on non-200 OK responses.

Response body

If the host returns file contents to clients, data is encoded in the response body. The response body is composed of a list of frames. The first frame is `MessageJSON`, which contains the encoded file signature. File chunks are encoded to subsequent chunk frames. The response body stream ends with `EndFrame`.

Sample message:

JSON

```
{
  "ContentProperties": [
    {
      "Retention": "KeepOnContentChange",
      "Name": "prop1",
      "Value": "los"
    },
    {
      "Retention": "DeleteOnContentChange",
      "Name": "prop2",
      "Value": "los"
    },
  ],
  "Signatures": [
    {
      "StreamId": "MainContent",
      "ChunkingScheme": "Zip",
      "ChunkSignatures": [
        {
          "ChunkId": "R3ZBz20okt9LmDaePUpB8Q--",
          "Length": 4096
        },
        {
          "ChunkId": "ZZbM20yjAX0FHEnVJ7Q2cg//",
          "Length": 29999
        }
      ]
    },
    {
      "StreamId": "AltStream",
      "ChunkingScheme": "FullFile",
      "ChunkSignatures": [
        {
          "ChunkId": "Y/bM2oyjAX0FHEnVJ7Q2cg==",
          "Length": 4096
        }
      ]
    }
  ]
}
```

Status codes

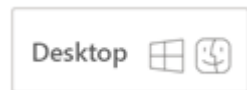
- [200 OK](#)  – Success.
- [400 Bad Request](#) 
 - The request was incorrectly formatted.
 - Frames in request stream cannot be deserialized properly.
 - The header size was not within the acceptable range.
- [401 Unauthorized](#)  – Invalid access token.
- [404 Not Found](#)  – Resource not found/user unauthorized.
- [500 Internal Server Error](#)  – Server error.
- [501 Not Implemented](#)  – Operation not supported.

Next steps

- [GetChunks](#)
- [PutChunkedFile](#)

GetChunks

Article • 03/04/2023



POST /wopi/files/(file_id)

The `GetChunks` operation allows clients to download specific chunks from the host. Clients can pass a list of `ChunkRange` values to be returned.

In response to the request, the host returns the sequence number along with the chunk data.

If the file on the host changes while the client is downloading the chunks, the client can detect the change by reading an updated sequence number in the response header. The host returns the list of actual chunks being returned in the response body `MessageJSON`. If the file is updated on the host, some of the requested chunks might no longer be valid. The `X-WOPI-SequenceNumber` header must be provided to reflect the latest host state.

Parameters

- `file_id` (string) – Required. A string that specifies a file ID of a file managed by the host. This string must be URL safe.

Query parameters

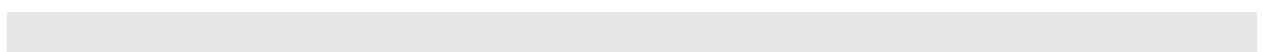
- `access_token` (string) – Required. An access token that the host uses to determine whether the request is authorized.

Request headers

- `X-WOPI-Override` (string) – Required. The string is `GET_CHUNKS`.

Request body

Sample message:



JSON

```
{
  "ContentFilters": [
    {
      "StreamId": "MainContent",
      "ChunkingScheme": "Zip",
      "ChunkRangesToReturn": [
        {
          "ChunkId": "ZZbM2oyjAX0FHEnVJ7Q2cg/",
          "Offset": 0,
          "Length": 4096
        },
        {
          "ChunkId": "--QQQbM2oyjAX0FHEnVJ7Q2cbb/",
          "Offset": 4096,
          "Length": -1
        }
      ]
    }
  ]
}
```

ⓘ Note

Setting the `Length` value of `ChunkRangeToReturn` to -1 indicates that the client is requesting the chunk data from `Offset` to the end of the chunk.

Status codes

- [200 OK](#) – Success
- [400 Bad Request](#)
 - The request was incorrectly formatted.
 - Frames in request stream can't be deserialized properly.
 - The header size wasn't within the acceptable range.
- [401 Unauthorized](#) – Invalid access token.
- [404 Not Found](#) – Resource not found/user unauthorized.
- [500 Internal Server Error](#) – Server error.
- [501 Not Implemented](#) – Operation not supported.

Response headers

- `X-WOPI-SequenceNumber` (integer) – Required. An integer value that indicates the latest state of the file on the host. The value must be greater than 0, and should be

increased if the host file is updated.

- `X-WOPI-ItemVersion` (string) – Required. A string value that indicates the version of the file. Its value should be the same as the `Version` value in `CheckFileInfo`.
- `X-WOPI-FailureReason` (string) – Optional. Used for logging purposes only. It should be set on non-200 OK responses.

Response body

Sample message:

JSON

```
{
  "ChunkRangeCollection": [
    {
      "StreamId": "MainContent",
      "ChunkIds": [
        "=="QQQbM2oyjAX0FHEnVJ7Q2cbb/",
        "ZZbM2oyjAX0FHEnVJ7Q2cg//"
      ]
    }
  ]
}
```

The response body is composed of `MessageJSON`, several `PartialChunk` frames, and `EndFrame`. The `ChunkId` values of the chunks that are passed in subsequent `PartialChunk` frames are encoded in `MessageJSON`. This can facilitate asynchronous processing on the receiver. Chunk data is encoded in `PartialChunk` frames. Each frame contains the binary content for a chunk range. `ChunkId`, `offset`, `length` of a partial chunk, and `flag` (`EndOfChunk`) are stored in the extended header field, and data bytes are stored in the frame payload. The response body ends with `EndFrame`.

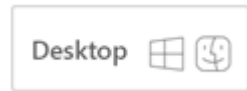
The response body encoding protocol is introduced in this section.

Next steps

- Learn about [PutChunkedFile](#).
- Learn about [GetSequenceNumber](#).

PutChunkedFile

Article • 03/04/2023



POST /wopi/files/(file_id)/contents

The `PutChunkedFile` operation updates the file's contents.

`PutChunkedFile` supports two types of locks: WOPI lock and `Coauth` lock.

- If the `X-WOPI-Lock` header isn't empty, the WOPI client uses a WOPI Lock.
- If the `X-WOPI-CoauthLockId` header isn't empty, the WOPI client uses `Coauth` lock.
- If both headers are set in a request, the host returns a 400 error.

When a host receives a `PutChunkedFile` request on a file that isn't locked, the host must check the current size of the file. If it's 0 bytes, the `PutChunkedFile` request is valid and should proceed. If it's any value other than 0 bytes, or is missing, the host responds with a "409 Conflict" message. For more information, see [Creating new files using Office for the web](#).

If the file is currently locked by a WOPI lock and the `X-WOPI-Lock` value doesn't match the lock currently on the file, the host returns a "lock mismatch" response (409 Conflict) and includes an `XWOPI-Lock` response header with the value of the current lock on the file. If the file is unlocked, the host must set `X-WOPI-Lock` to the empty string.

If the file is currently locked by a WOPI lock and the client provides a `Coauth` lock, the host returns a "lock mismatch" response (409 Conflict) and includes an `X-WOPI-ConflictingMechanism` response header with a `WOPI-Lock` value. If the file is unlocked, the host must set `X-WOPI-Lock` to an empty string.

If the file is currently locked by `Coauth` lock and the `X-WOPI-CoauthLockId` value isn't in the `CoauthTable`, the host returns a "lock mismatch" response (409 Conflict).

If the file is currently locked by another entity with a `CoauthExclusive` lock, the host returns a "lock mismatch" response (409 Conflict) with an empty response body. The `X-WOPI-ConflictingMechanism` header should be set to `WOPI-CoauthExclusiveLock`.

If the file is currently locked by another entity without a WOPI lock or a WOPI `Coauth` lock, the host must return a "lock mismatch" response (409 Conflict) with an empty response body. The `X-WOPI-ConflictingMechanism` header should be set to `Other`.

Host file lock state	Client presented lock	X-WOPI-ConflictingMechanism response header	X-WOPI-Lock response header	Client retry strategy
Another entity beyond WOPI locks	WOPI lock or Coauth or CoauthExclusive lock	Other	Empty	No retry
WOPI lock	Mismatching WPI lock or any Coauth or CoauthExclusive lock	WOPI-Lock	Value of current WOPI lock on the file	No retry
WOPI CoauthExclusive lock	Any WOPI lock or Coauth lock or mismatching CoauthExclusive lock	WOPI-CoauthExclusiveLock	Empty	No Retry
WopiCoauthLock	CoauthLockId isn't in CoauthTable or is expired	N/A	N/A	Client must send a GetCoauthLock request and follow with a PutChunkedFile request if the lock is successfully acquired.

When the client has a valid WOPI, Coauth, or CoauthExclusive lock, but the sequence number provided by the WOPI client doesn't match the latest value on the host, this is considered a coherency failure. The host must return "412 Precondition Failed" with an empty response body and pass its latest sequence number in the response header.

A new sequence number should be generated and returned in the response header.

Parameters

- file_id (string) – Required. A string that specifies the file ID of a file managed by the host. This string must be URL safe.

Query parameters

- `access_token` (string) – Required. An access token that the host uses to determine whether the request is authorized.

Request headers

- `X-WOPI-Override` (string) – Required. The string is `PUT_CHUNKED_FILE`.
- `X-WOPI-SequenceNumber` (integer) – Required. An integer value provided by the WOPI client that indicates the sequence number of the state of the file client expects on the host. The value must be greater than 0.
- `X-WOPI-Lock` (string) – Optional. A string provided by the WOPI client in a previous lock request.
- `X-WOPI-CoauthLockId` (string) – Optional. A string provided by the client that the host must use to uniquely identify the client for the lock on the file. The size limit is 1024 characters. This is the string provided by the client to a `GetCoauthLock` request.
- `X-WOPI-Editors` (string) – Optional. A comma-delimited string of `UserId` values that represent the users who contributed changes to the document in the `PutChunkedFile` request. The host must validate the proof-key if this header is set. For a full description of proof-key validation, see [Verify that requests originate from Office for the web by using proof keys](#).

Request body

Sample message:

JSON

```
{
  "ContentProperties": [],
  "Signatures": [
    {
      "StreamId": "MainContent",
      "ChunkingScheme": "Zip",
      "ChunkSignatures": [
        {
          "ChunkId": "YKWPmdVdMY14qK003yTMXg==",
          "Length": 9999
        },
        {
          "ChunkId": "ZZbM2oyjAX0FHEnVJ7Q2cg//",
          "Length": 6999
        },
        {
          "ChunkId": "==QQQbM2oyjAX0FHEnVJ7Q2cbb",

```

```

        "Length": 3999
      }
    ]
  },
  {
    "StreamId": "AlternateStream",
    "ChunkingScheme": "FullFile",
    "ChunkSignatures": [
      {
        "ChunkId": "R3ZBz2Ookt9LmDaePUpB8Q==",
        "Length": 102499
      }
    ]
  }
]
}

```

When processing a `PutChunkedFile` request, the host only updates the streams included in the request body `Signatures` array. Other streams for the file are left untouched and carried forward. These unmodified streams are in the updated file at the new sequence number. If the client includes a `Signature` entry with zero chunks for any stream other than `MainContent`, the host deletes the stream.

When the request specifies an upload session via `UploadSessionTokenToCommit`, the host includes the upload session binary stream while processing the request as if it was passed inline in the `PutChunkedFile` request.

Response headers

- `X-WOPI-SequenceNumber` (integer) – Required. An integer value that indicates the latest state of the file on the host. The value must be greater than 0, and should be increased if the host file is updated.
- `X-WOPI-ItemVersion` (string) – Required. A string value that indicates the file version. Its value should be the same as the `Version` value in `CheckFileInfo`.
- `X-WOPI-FailureReason` (string) – Optional. Used for logging purposes only. It should be set for non-200 OK responses.
- `X-WOPI-Lock` (string) – Optional. A string value that identifies the current lock on the file. This header must always be included when responding to the request with "409 Conflict" and when the file is locked by a WOPI Lock. Don't include it when responding to the request with "200 OK".
- `X-WOPI-ConflictingMechanism` (string) – Optional. A string value that indicates the current lock on the file. This value must be set when responding to the request with "409 Conflict" due to lock mismatch. Don't include it when responding to the

request with "200 OK". Valid values of this header are `WOPI-Lock`, `WOPI-CoauthExclusiveLock`, and `Other`.

Status codes

- [200 OK](#) – Success
- [400 Bad Request](#)
 - The required parameters were not provided.
 - The request was incorrectly formatted.
 - Both X-WOPI-Lock and X-WOPI-CoauthLockId were provided.
 - The host does not have a ChunkId but the WOPI client did not pass any chunk payload.
 - Frames in request stream cannot be deserialized properly.
 - X-WOPI-Editors header is set but proof-key is not correctly signed.
 - Client did not include MainContent StreamId in the request
 - The header size was not within the acceptable range.
- [401 Unauthorized](#) – Invalid access token.
- [404 Not Found](#) – Resource not found/user unauthorized.
- [409 Conflict](#)
 - The resource is exclusively locked using a different locking mechanism (other than WOPI/`Coauth` locks).
 - The resource is exclusively locked by another entity using a different lock type (WOPI versus `Coauth`). This failure would be indicated by X-WOPI-Lock response header.
 - The resource is exclusively locked by another entity using `CoauthExclusiveLock` (`Coauth` versus `CoauthExclusive`). This failure would be indicated by X-WOPI-ConflictingMechanism response header.
- [412 Precondition Failed](#)
 - The sequence number provided by WOPI client does not match the state of the document on the host. The current sequence number of the file on the host must be provided in the X WOPI SequenceNumber response header in this case.
- [413 Request Entity Too Large](#) – File is too large; the maximum file size is host-specific.
- [500 Internal Server Error](#) – Server error.
- [501 Not Implemented](#) – Operation not supported.

The maximum number of alternate streams (streams other than `MainContent`) supported for a given file at any time is 256. An alternate stream will expire 30 days after it was last updated.

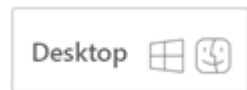
The maximum number of unexpired content properties supported for a given file at any time is 256. A content property will expire 30 days after it was last updated.

Next steps

[GetSequenceNumber](#)

GetSequenceNumber

Article • 03/04/2023



POST /wopi/files/(file_id)

The `GetSequenceNumber` operation retrieves the sequence number of the file on the host.

Parameters

- `file_id` (string) – Required. A string that specifies the file ID of a file managed by the host. This string must be URL safe.

Query parameters

- `access_token` (string) – Required. An access token that the host uses to determine whether the request is authorized.

Request headers

- `X-WOPI-Override` (string) – Required. The string is `GET_SEQUENCE_NUMBER`.

Response headers

- `X-WOPI-SequenceNumber` (integer) – Required. An integer value that indicates the latest state of the file on the host. The value should be greater than 0, and should be increased if the host file is updated.
- `X-WOPI-FailureReason` (string) – Optional. Used for logging purposes only. It should be set on non-200 OK responses.

Status codes

- [200 OK](#) – Success.
- [401 Unauthorized](#) – Invalid access token.
- [404 Not Found](#) – Resource not found/user unauthorized.
- [500 Internal Server Error](#) – Server error.

- [501 Not Implemented](#) [↗](#) – Operation not supported.

Next steps

[File content data model](#)

File content data model

Article • 07/06/2022

These parts of the content are to be stored on the host:

- Primary document content: Identified by StreamId = "MainContent"
- Multiple alternate streams within the document
- Set of content properties

All three of these content parts can be modified or fetched in a single request or response, and modifications must be processed within a transaction. In addition, if the document is updated using any mechanism other than the `PutChunkedFile` API, the host should clear the following parts of the content:

- Multiple alternate streams within the document (except MainContent stream)
- All Content Properties with `ContentPropertyRetention` == `DeleteOnContentChange`

ContentProperty

A JSON-formatted object containing the following properties:

- `ContentPropertyRetention` – A string value that indicates whether the content properties should (or shouldn't) be maintained after changes. Valid values are `DeleteOnContentChange` and `KeepOnContentChange`.
- `Name` – A string value that indicates the name of the content property. The maximum length of this value is 256 characters.
- `Value` – A string value of the content property. The maximum length of this value is 1 KB.

JSON

```
{
  "ContentPropertyRetention": "KeepOnContentChange",
  "Name": "Property Name",
  "Value": "Property Value"
}
```

The maximum number of unexpired content properties supported for a file is 256. A content property expires 30 days after it was last updated.

Chunk streams for efficient transfer

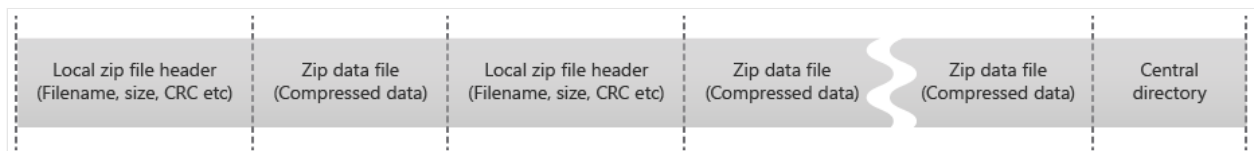
To achieve incremental file transfer, the file contents are broken into chunks. How a binary stream is broken into chunks depends on the `chunkingScheme` value.

Two chunking schemes are currently supported:

- `Zip` – Zip files are the default format of Office files that support coauthoring.
- `FullFile` – Encrypted Office files. Full binary contents of a stream are represented as a single chunk.

For the `Zip` chunking scheme, `ZipLocalFileHeader`, `ZipPayload`, and the central directory are separate chunks. Delta chunks are transferred in the `PutChunkedFile` and `GetChunkedFile` methods.

Chunks are identified by `ChunkId` (128-bit [Spooky hash](#)). The order of chunks for each file stream is also specified by the protocol.



On processing a `PutChunkedFile` request, if the file on the host is a zip archive, the host should update the file based on the client's file signature and the delta chunks in the request body.

If the file is not a zip archive, the host needs to update the file with the single full file chunk in the request body.

Next steps

[Chunk streams](#)

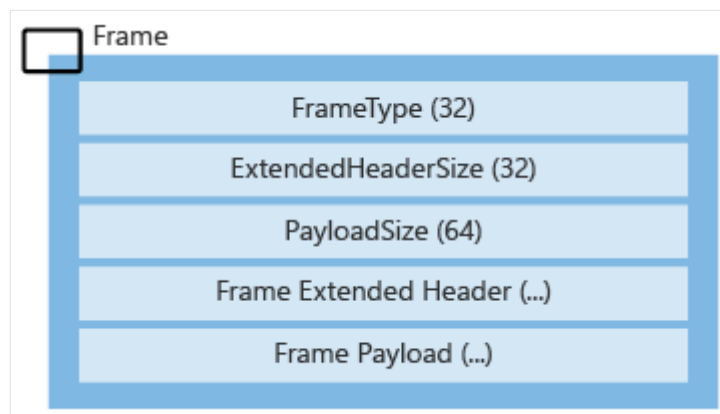
Encode and send data on the wire

Article • 07/06/2022

This article explains how to encode and send data on the wire.

Frame protocol for sending chunks

The request/response payload is broken into frames to send the chunks. Each frame has a header that declares its frame type, extended header size, payload size, and extended header. All binary structures must be provided in big-endian byte order.



Frame type

Four frame types are supported:

- **MessageJSON** – Must be the first frame; contains request or response-body structured data serialized via JSON.
- **Chunk** – Contains the full binary content for a given chunk. **ChunkId** is stored in the extended header field, and chunk data bytes are stored in the frame payload.
 - Frame Extended Header – Used for additional data for **Chunk** and **ChunkRange** frames.
 - Frame Payload – The frame payload is all the data in binary format.
- **ChunkRange** – Contains the partial binary content for a chunk. **ChunkId**, **offset**, **length**, and **Flags** are stored in the extended header field, and chunk data bytes are stored in the frame payload.
- **EndFrame** – Must be the last frame; indicates the end of the request payload.

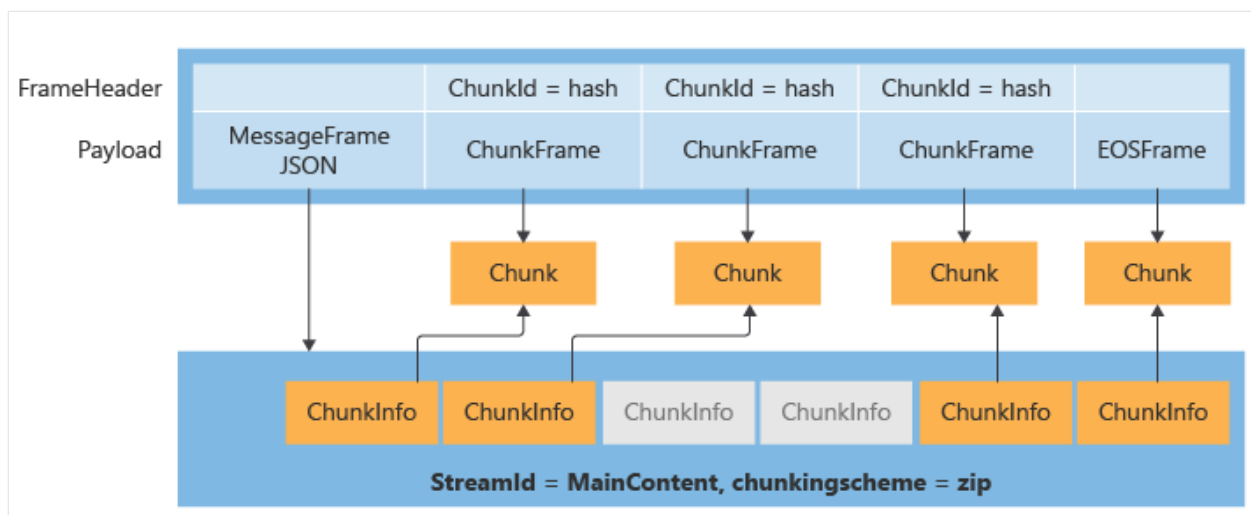
The table lists all frame types and expected values for the fields:

FrameType Name	FrameType	ExtendedHeader Size	Payload Size	Extended Header Contents	Extended Header Encoding	Frame Payload
MessageJSON	2	0	64-bit integer	Empty	N/A	Structured data applicable to that request or response
Chunk	3	16	64-bit integer	ChunkId (128-bit spooky hash)	Raw bytes	All chunk data bytes
ChunkRange	4	36	64-bit integer	- ChunkId (16 bytes) - Offset (64-bit unsigned integer) - Length (64-bit unsigned integer) - Flags (32-bit unsigned integer; indicates EndOfChunk)	Raw bytes (all numbers in big-endian byte order)	Chunk range bytes
EndFrame	1	0	0	Empty	N/A	Empty

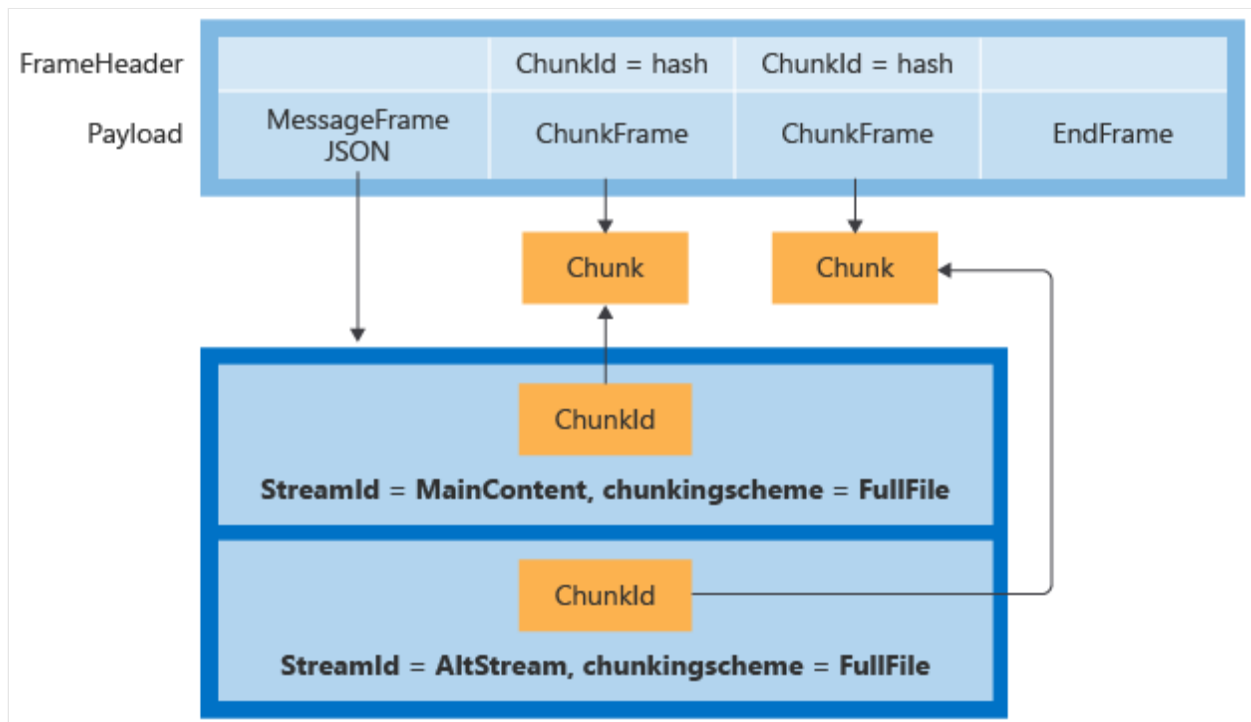
ChunkRange extended header: This data is laid out in bytes as detailed in the preceding table. For the Flags field, these are the only valid values (for the 32-bit unsigned integer) currently:

- 0 - Indicates a ChunkRange between the chunk.
- 1 – Indicates the ChunkRange, which includes the end of the chunk.

Upload request stream example for zip archive file



Upload request stream example for non-zip archive file



Sample `PutChunkedFile` MessageJSON for a non-zip archive file:

JSON

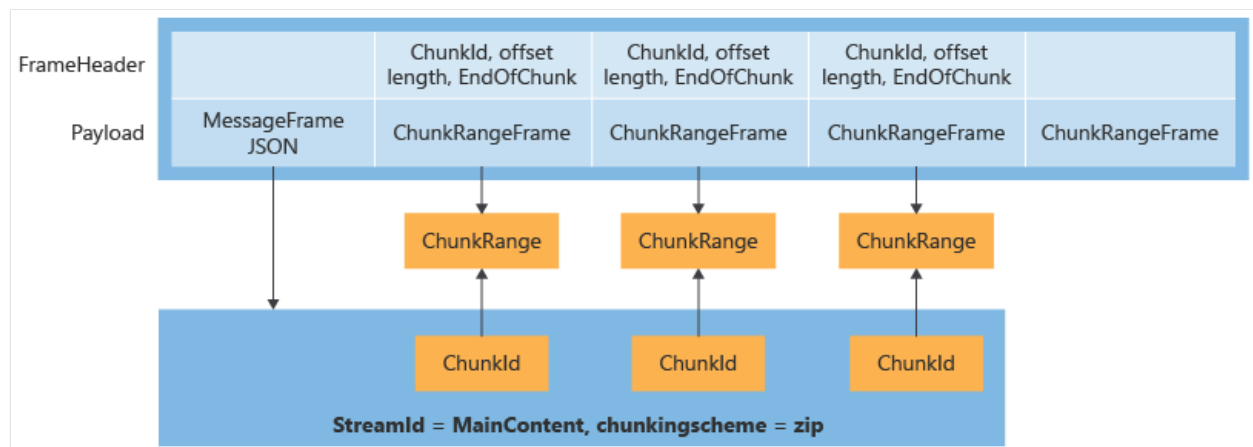
```
{
  "ContentProperties": [],
  "Signatures": [
    {
      "StreamId": "MainContent",
      "ChunkingScheme": "FullFile",
      "ChunkSignatures": [
        {
          "ChunkId": "P4oXyRH4V/AHALoD7GZ9Yw==",
          "Length": 199999
        }
      ]
    }
  ]
}
```

```

    }
  ]
},
{
  "StreamId": "Alternate",
  "ChunkingScheme": "FullFile",
  "ChunkSignatures": [
    {
      "ChunkId": "YKWPmdVdMY14qK003yTMXg==",
      "Length": 2999
    }
  ]
}
],
"UploadSessionTokenToCommit": null
}

```

GetChunks response stream example for zip archive file:



ⓘ Note

The MessageJSON for all the requests and responses are encoded by using JSON.

Next steps

Learn about [Multi-request file uploads](#).

Multi-request file uploads

Article • 07/06/2022

Clients can create an upload session to allow large upload payloads to be broken up and transferred to the host over multiple requests. Requests must provide the payload in sequential order. Each request resumes, in byte order, where the previous request finished. The upload session payload contains only chunk frames, and the entire upload session is referred to by a final `PutChunkedFile` call that contains all of the frames in the upload session. If the `PutChunkedFile` call fails (such as in the case of a sequence number mismatch), the upload session remains, allowing the client to try a new `PutChunkedFile` call without having to retransmit the entire upload session payload.

To upload a set of chunks using an upload session, follow the steps in these articles:

- [Create an upload session](#)
- [Upload bytes to the upload session](#)
- [Commit an upload session](#)

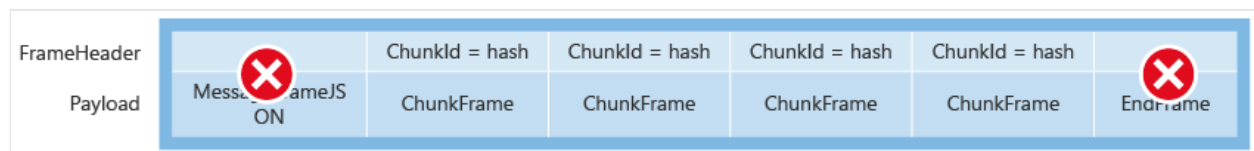
Create an upload session

Article • 03/04/2023

To begin uploading a set of chunks, the client must first request a new upload session. This creates a temporary storage location where the bytes of the chunks are saved until the complete chunk payload is uploaded.

Upload session binary stream

The binary stream for an upload session is a collection of chunks (`FullChunk`) encoded in similar way as `PutChunkedFile`, where a list of chunks to be uploaded is encoded by using `Frames`. Make sure there isn't `MessageJSON` at the beginning or `EndFrame` at the end.



CreateUploadSession

POST /wopi/files/(file_id)

Parameters

- `file_id` (string) – Required. A string that specifies a file ID of a file managed by the host. This string must be URL safe.

Query parameters

- `access_token` (string) – Required. An access token that the host uses to determine whether the request is authorized.

Request headers

- `X-WOPI-Override` (string) – Required. The string is `CREATE_UPLOAD_SESSION`.

The response to this request provides the details of the newly created `UploadSession`, which includes the `sessiontoken` string used for uploading the parts and the expiry time for the upload session. This session token must be a unique identifier tied to this specific

upload session. A given user ID might have more than one upload session active at a time. Every upload session has a 30-minute expiry interval from the time it was last updated.

Response headers

- `X-WOPI-FailureReason` (string) – Optional. Used for logging purposes only. It should be set on non-200 OK responses.

Response body

The response is [JSON](#) containing the following required properties:

- `UploadSessionToken` – A string identifier for the newly created upload session. This is used to reference the upload session for any future operations.

Sample response:

JSON

```
{
  "UploadSessionToken": "sessiontoken"
}
```

Status codes

- [200 OK](#) – Success.
- [401 Unauthorized](#) – Invalid access token.
- [404 Not Found](#) – Resource not found/user unauthorized.
- [500 Internal Server Error](#) – Server error.
- [501 Not Implemented](#) – Operation not supported.

Next steps

- [Upload bytes to the upload session](#)
- [Commit an upload session](#)

Upload bytes to an upload session

Article • 07/06/2022

To upload a set of chunks, the client makes an `UpdateUploadSession` request by passing the session token value received in the `createUploadSession` response. You can upload the entire chunk frame payload, or split it into multiple byte ranges. The byte ranges of individual upload payloads are not required to align to frame chunks within the overall upload session binary stream.

The upload session binary stream fragments must be uploaded in order. Uploading fragments out of order results in an error. If the content range offset of the request is not equal to the total number of bytes that have already been sent in previous requests for a session, the request fails with a "400 - Bad request" error.

If the request body length doesn't match `X-WOPI-ContentRangeLength`, the request fails with a "400 - Bad request" error.

After every successful update, the host extends the upload session expiry time by 30 minutes.

UpdateUploadSession

`POST /wopi/files/(file_id)`

Parameters

- `file_id` (string) – Required. A string that specifies a file ID of a file managed by the host. This string must be URL safe.

Query parameters

- `access_token` (string) – Required. An access token that the host uses to determine whether the request is authorized.

Request headers

- `X-WOPI-Override` (string) – Required. The string is `UPDATE_UPLOAD_SESSION`.
- `X-WOPI-UploadSessionToken` (string) – Required. A string identifier for the upload session that's being updated.

- `X-WOPI-ContentRangeOffset` (long) – Required. A long value (64-bit signed integer) that represents the offset within the upload session stream this request starts at.
- `X-WOPI-ContentRangeLength` (long) – Required. A long value (64-bit signed integer) that indicates the length of the content being sent in the body.

Request body

The request body must be the full binary contents of the part of the request stream being sent.

Response headers

- `X-WOPI-UploadSessionCurrentOffset` (long) – Required. A long value (64-bit signed integer) that represents the current offset within the upload session stream next request starts at.
- `X-WOPI-FailureReason` (string) – Optional. Used for logging purposes only. It should be set on non-200 OK responses.

Status codes

- [200 OK](#)  – Success
- [400 Bad Request](#) 
 - The content range offset of the request is not equal to the total number of bytes that have already been sent in previous requests for this session.
 - Request body length does not match `X-WOPI-ContentRangeLength`.
 - The header size was not within the acceptable range.
- [401 Unauthorized](#)  – Invalid access token.
- [404 Not Found](#)  – Resource not found/user unauthorized.
- [410 Gone](#)  – Upload session token is invalid (for example, the upload session expired).
- [500 Internal Server Error](#)  – Server error.
- [501 Not Implemented](#)  – Operation not supported.

Next steps

- [Create an upload session](#)
- [Commit an upload session](#)

Commit an upload session

Article • 07/06/2022

This article describes how to commit an upload session.

Method

After the data is uploaded for the session, the client can commit that session by using the `PutChunkedFile` method, which passes the session token.

Success

If the commit is successful, the upload session and associated temporary storage are automatically cleaned up by the host.

Fail

If the commit fails for an upload session due to coherency failure, the WOPI client can update the upload session by appending new chunks. It will then retry the commit.

Next steps

- [Create an upload session](#)
- [Upload bytes to the upload session](#)

Sync client integration overview

Article • 08/18/2022

To open Microsoft Office files stored in a third-party sync provider while supporting coauthoring and to support opening and display contents based on the local file without requiring a network round trip to download content, Office needs additional metadata and content from the third-party sync engine. This section describes how Office can retrieve the information it needs to open the locally synced files from a third-party sync engine's Windows File Explorer or macOS Finder folder. It describes how Office can do so without requiring a server round trip, while also supporting coauthoring scenarios.

How sync client integration works

Office needs to get the `WOPISrc` where the file is stored on the server. It also needs some other data about the file to determine how to open it and where to open it from. In the open-via-local-path flow, we'll use the contents of the file already downloaded by the sync engine so the contents of the file display quickly without a full file download. When successfully opened third-party files are edited, Office uploads edits directly to the Web Application Open Platform Interface (WOPI) service. The third-party sync client continues to keep the local file synced.

Next steps

- Learn about [terms](#) you'll need to know.
- Understand the [scope](#) of sync client integration.

Sync client integration scope

Article • 07/06/2022

This article describes the scope of sync client integration.

Out of scope

Note the following scope limitations for sync client integration:

- There's no support for iOS or Android.
- There's no Windows Runtime/Project Centennial app support.
- Open requests started from a service URL, including opens from a protocol handler or from a URL in the most recently used (MRU) list and backstage, are opened as server-only with coauthoring support. However, they don't benefit from opening and displaying contents based on the local file without requiring a network round trip using the sync-client integration described in this section.
- If a third-party sync engine isn't running, responding, installed, or signed in on Windows:
 - Coauthoring is disabled if files are opened from the local path. Files are opened as local-only.
 - If it's not turned on, the third-party sync engine might not attempt to start or sign in when we CoCreate a handle on `IStorageProviderHandler`. This situation will create additional delays in the open flow that will be perceived as "Office is slow to open."
- If there are dirty changes pending upload on the locally synced folder and the file is opened via the local path, Microsoft Office will open files as local-only without coauthoring support.
- When a file is closed, if there are no pending uploads, Office will delete the file entry in our internal local cache. This deletion allows support for opening and displaying contents based on the local file without requiring a network round trip for Excel files while preventing merge conflicts on future open operations. This is possible because the cache isn't kept in sync with the file when Office isn't running.
- If a file is a placeholder (dehydrated), the sync engine must download it before it starts Office if the file is being opened from the local path.

- Placeholder files that are opened while the user is offline will fail to open.

Next steps

- Learn how Office gets [metadata](#) from third-party sync engines.

Scenarios and edge cases

Article • 09/08/2022

Learn how sync-client integration works in key scenarios and edge cases.

Open in-sync third-party synced file via File Explorer for the first time (Windows)

User actions	Product behavior
1. In File Explorer or the Common File Dialog (CFD), double-clicks the third-party synced file. 2. Edits or coauthors the file. 3. Closes the file.	1. Office uses the local path to the local file to open and display contents without requiring a network zero round trip <i>if the file supports coauthoring</i>. Otherwise, the file opens from the local file (no coauthoring). 2. The local path is converted to a service URL via registry and COM calls. The hash of the local content is verified to match the hash from the sync client. 3. Edits are uploaded to and downloaded from the service. 4. The third-party sync client keeps the local file in sync. 5. When the file is closed, it's deleted from the Office cache.

Open in-sync third-party synced file via the macOS Finder for the first time (macOS)

User actions	Product behavior
1. In the Finder, double-clicks the third-party synced file. 2. Edits or coauthors the file. 3. Closes the file.	1. The third-party sync client is expected to set its sync domain with the OS so that a call to <code>NSURLIsUbiquitousItemKey</code> succeeds. 2. The XPC service converts the local path to a service URL after the service verifies that the local hash matches the hash from the sync client. 3. Office uses the local path and the local file to open and display the file contents without requiring a network round trip <i>if the file supports coauthoring</i>. Otherwise, the file opens from the local file (no coauthoring). 4. Edits are uploaded to and downloaded from the service. 5. The third-party sync client keeps the local file in sync. 6. When the file is closed, it's deleted from the Office cache.

Open third-party synced file while offline

In this scenario, while a user is offline, the user opens a third-party synced file in Microsoft Office by double-clicking the file in File Explorer or the macOS Finder.

User actions	Product behavior
1. Edits the file, then saves it. 2. Closes Office.	1. The third-party sync client is expected to respond, even when the user is offline. 2. Office identifies the file as a third-party synced file via registry values. Then, it gets properties from the sync client via COM/XPC. <i>If the properties returned by the host in <code>CheckFileInfo</code> indicate that the file doesn't support coauthoring (the <code>SupportsCoauth</code> property is <code>false</code>), Office opens the file as a local file (with no coauthoring).</i> 3. The file is in sync based on the <code>FileChecksum</code> property and the hash on disk. 4. Office opens with coauthoring support without requiring a network round trip. 5. Edits remain in the upload branch pending upload. Edits aren't persisted to the file on disk. While the user is offline, edits remain only in the Office cache. 6. <i>Pending Upload</i> is shown in the title bar throughout the session. 7. When the file is closed, the file remains in the Office cache because of the pending uploads. 8. When the file is closed, a final modal prompt tells the user that contents aren't yet uploaded.

Open file with pending changes in Office file cache from previous session

User actions	Product behavior
--------------	------------------

User actions	Product behavior
1. Completes the <i>Open third-party synced file while offline</i> scenario. 2. In File Explorer or the macOS Finder, double-clicks to open Office with a pending upload from the previous offline session file. 3. Edits the file, and then saves it. 4. Closes Office.	1. <i>If the file doesn't support coauthoring, the file is opened as a local file and pending changes aren't uploaded.</i> 2. If the file does support coauthoring, Office loads the contents from the Office cache. 3. The contents from the local file in the synced folder are not used by Office, and they are not displayed to the user. 4. The contents in the Office cache, including pending edits, are loaded and displayed to the user. 5. Edits are uploaded to the service, along with edits from the previous session left in the cache. 6. If uploads are successful, the file is deleted from the Office cache when the file is closed.

Open out-of-sync third-party synced file via File Explorer or macOS Finder

In this scenario, while a user is offline, the user edits a local third-party synced file via OpenOffice.

User actions	Product behavior
1. Closes OpenOffice. 2. In File Explorer or the macOS Finder, double-clicks the file to open the same file with Office. 3. Closes Office.	1. Office identifies the file as a third-party synced file via registry values, and then gets the file properties from the sync client via COM/XPC. <i>If the properties returned by the host in CheckFileInfo indicate that the file doesn't support coauthoring (the <code>SupportsCoauth</code> property is <code>false</code>), Office opens the file as a local file (with no coauthoring).</i> 2. The <code>FileChecksum</code> value that's retrieved from the sync client doesn't match the hash of the file on disk, which indicates that the local file isn't synced. 3. Office opens the file as local-only with no coauthoring support. 4. No entry is made for this file in the Office cache. 5. The sync client maintains the local file in sync.

Open third-party synced file via path from the MRU list

In this scenario, the local path is stored in the most recently used (MRU) list from the previous sync-backed file open (the most common case).

User actions	Product behavior
1. Opens the file from the MRU list.	1. The local path is mapped with help from the sync engine to the WOPI URL.
2. Edits the file.	2. Office opens the file without requiring a network round trip (including Excel) from the local disk, with coauthoring support. <i>If the file properties returned by the host in CheckFileInfo indicate that the file doesn't support coauthoring (the <code>SupportsCoauth</code> property is <code>false</code>), Office opens the file as a local file (with no coauthoring).</i>
3. Closes the file.	3. If everything is in sync, the file is deleted from the Office cache when the file is closed.

Open third-party synced file via URL from the MRU list

In this scenario, the service URL is stored from the previous URL open.

User actions	Product behavior
--------------	------------------

User actions	Product behavior
1. Opens the file from the MRU list. 2. Edits the file. 3. Closes the file.	1. The URL isn't mapped to the local path. The local sync client isn't involved at all. 2. Office doesn't open the file without a network round trip to download content. Because it's a first open, Office downloads the entire file prior to opening it. 3. The file is opened with coauthoring support and edits are uploaded to the service. <i>If the properties returned by the host in CheckFileInfo indicate that the file doesn't support coauthoring (the <code>SupportsCoauth</code> property is <code>false</code>), Office won't open the file.</i> 4. The file remains in the Office file cache because it was opened via a URL.

Open third-party synced file via File Explorer or the macOS Finder when the sync client doesn't respond

User actions	Product behavior
1. Opens Task Manager and stops the third-party sync client process. 2. Double-clicks the file or CFD to open the Office file in the synced folder. 3. Closes Office.	1. When the file is opened, Office detects via registry values or the <code>NSURLIsUbiquitousItemKey</code> API that the path belongs to a third-party sync client. 2. Office attempts to cocreate the COM object to communicate with the sync client by using the <code>IStorageProvider</code> APIs or an XPC equivalent. 3. Any failure that occurs while attempting to use <code>CoCreate</code>, <code>GetPropertyHandlerFromPath</code>, <code>RetrieveProperties</code>, or XPC equivalents from the sync client results in a local-only open without coauthoring support*. Includes scenarios in which: • The COM call times out. • The sync client isn't running. • The sync client failed to respond with all required properties. *If the sync client becomes responsive mid-session, nothing changes after the file is opened. Office writes only to the local file on disk without coauthoring support.

Save As to a third-party service offline

User actions	Product behavior
1. Opens an existing local file on the desktop. 2. Saves as via Save A Copy > Browse > CFD > C:\users\syncclient. 3. Enters a file name. 4. Selects Save. 5. Closes the file.	1. Office doesn't find an existing file in the synced folder. 2. Office doesn't attempt to communicate with the sync client. 3. Office creates the file in the local synced folder. 4. Office opens the file as local-only without coauthoring support. 5. After the sync client is in sync, a subsequent open has coauthoring support.

Save As to a third-party synced folder via the macOS Finder or CFD

User actions	Product behavior
1. Drags a large file to a synced folder. 2. Before the sync client uploads the file, double-clicks the file. 3. Closes the file.	1. Office detects that the file belongs to a sync root and attempts to get the file properties via COM or XPC. 2. The sync client is expected to fail <code>RetrieveProperties</code> or the XPC call with <code>NotFound</code> or an equivalent error. 3. Office opens the file as local-only without coauthoring support. 4. The sync client continues to sync the file to the service while the file is open and after it's closed.

Drag a file to a third-party synced folder and immediately open

User actions	Product behavior
--------------	------------------

User actions	Product behavior
1. Drags a large file to a synced folder. 2. Before the sync client uploads the file, double-clicks the file. 3. Closes the file.	1. Office detects that the file belongs to a sync root and attempts to get the file properties via COM or XPC. 2. The sync client is expected to fail <code>RetrieveProperties</code> or the XPC call with <code>NotFound</code> or an equivalent error. 3. Office opens the file as local-only without coauthoring support. 4. The sync client continues to sync the file to the service while the file is open and after it's closed.

Open a file with more changes from the server

In this scenario, a third-party synced file is in sync locally, but pending changes on the service haven't yet been downloaded by the sync client.

User actions	Product behavior
1. Opens the file. 2. Waits for changes to appear. 3. Closes the file.	1. The hash that's retrieved from the sync client and the hash of the file on disk matches. 2. The file opens without requiring a network round trip and displays the contents on disk. 3. Office uses the contents on disk to ask the service for any deltas. 4. Server changes are downloaded and displayed to the user. The local file in the synced folder remains stale. 5. The sync client independently keeps the local file in sync. 6. When the file is closed, it's deleted from the Office cache.

Rename a file outside the app when the file is already in the Office cache with pending changes

User actions	Product behavior
--------------	------------------

User actions	Product behavior
1. Opens the file with third-party sync integration. 2. Closes the file (leaving file in the Office cache). 3. Renames the file via File Explorer or the macOS Finder and waits for the sync client to propagate changes to the service. 4. Double-clicks to open the file in File Explorer or the macOS Finder.	1. Office detects that the file is in the sync folder and gets the usual metadata through COM/XPC. 2. <code>WOPIUrl</code> is critical because it indicates that the file exists on the service and allows the file to be mapped in the Office cache.* 3. Office finds the existing row entry in its cache by using the <code>WopiSrc</code> value. 4. The local contents from the disk aren't shown to the user. 5. The contents in the cache are shown to the user and the file opens online with coauthoring support. * <code>WOPIUrl</code> contains the unique <code>fileId</code>, which doesn't change when a file is moved or renamed.

Open the locally synced file that's left after the sync client is uninstalled and remove fails

User actions	Product behavior
1. Stops syncing a root or uninstalls the sync client. 2. The sync client is expected to remove synced files from client, but if failure occurs or while uninstalling the file, some files might be left over. 3. Syncs the client folder and double-clicks the file in File Explorer to open the file.	Windows: 1. After Office fails to find the right handler in the registry, it opens the file as local-only. 2. If a failure leaves the registry entries in place, Office fails while cocreating the nonexistent process. The file is again opened as local-only without coauthoring support. Mac: • In macOS, failures in <code>NSURLIsUbiquitousItemKey</code> after the engine is uninstalled are expected.

Next steps

- Learn how Office gets [data](#) from third-party sync engines.

- Learn how Office gets [metadata](#) from third-party sync engines.

How Office gets metadata from third-party sync engines

Article • 07/06/2022

This article describes how Microsoft Office gets metadata from third-party sync engines.

Windows metadata

In Windows, Office uses `IStorageProviderHandler` over COM to retrieve properties from the third-party sync engine. For more information, see [storageprovider.h header - Win32 apps](#) in the Windows documentation. As we learned from first-party integration, using `StorageFile` directly can be slow in edge cases. To minimize performance impact and avoid the need to hard-code third-party CLSID GUIDs in Office code, the third-party provider needs to write COM information we can retrieve in a shared registry location. Third-party sync engine providers should write their unique CLSID and hash algorithm in this registry location:

```
Computer\HKEY_CURRENT_USER\SOFTWARE\SyncEngines\Providers\<Third-party sync engine>
    Handler -> REG_SZ
    HashAlgorithm -> REG_SZ
```

They should write their local path root folder, Web Application Open Platform Interface (WOPI) Service ID (assigned by Microsoft), and WOPI User ID in this location:

```
Computer\HKEY_CURRENT_USER\SOFTWARE\SyncEngines\Providers\<Third-party sync engine>\<Sub engine>
    MountPoint -> REG_SZ
    WOPIServiceId -> REG_SZ
    WOPIUserId -> REG_SZ
```

The `<Sub engine>` path allows third-party providers to register more than one provider, if applicable. Third-party sync engines that have several service locations that sync data can list them all here. Office will parse the registry folder structure to find the right provider to CoCreate based on `MountPoint`, just as we do for OneDrive integration in the same registry location. For example, OneDrive registers several business folders uniquely identified by a GUID and one labeled "Personal."

When resolving a local file as connected with a cloud file URL, early in the Office open stack, Office checks if the path of the local file passed in subsumes the path under the third-party mountpoint, after it iterates through all registered mountpoints. If it does, Office will CoCreate an `IStorageProviderHandler` instance, using the CLSID listed under `Handler`, by using `CoCreateInstance(<Handler CLSID>, NULL, CLSCTX_ALL)`. After the instance is CoCreated, Office calls `GetPropertyHandlerFromPath` with the local file path to retrieve an `IStorageProviderPropertyHandler` object. By using this object, Office calls `RetrieveProperties`, looking for two properties:

```
// Name: System.StorageProviderFileRemoteUri --
PKEY_StorageProviderFileRemoteUri
// Type: String -- VT_LPWSTR (For variants: VT_BSTR)
// FormatID: {FCEFF153-E839-4CF3-A9E7-EA22832094B8}, 112
//
// The storage provider's remote URI for this file.
DEFINE_PROPERTYKEY(PKEY_StorageProviderFileRemoteUri, 0xFCEFF153, 0xE839,
0x4CF3, 0xA9, 0xE7, 0xEA, 0x22, 0x83, 0x20, 0x94, 0xB8, 112);
#define INIT_PKEY_StorageProviderFileRemoteUri { { 0xFCEFF153, 0xE839,
0x4CF3, 0xA9, 0xE7, 0xEA, 0x22, 0x83, 0x20, 0x94, 0xB8 }, 112 }

// Name: System.StorageProviderFileChecksum --
PKEY_StorageProviderFileChecksum
// Type: Buffer -- VT_VECTOR | VT_UI1 (For variants: VT_ARRAY | VT_UI1)
// FormatID: {B2F9B9D6-FEC4-4DD5-94D7-8957488C807B}, 5
//
// The checksum computed by the storage provider for the file. Files with
the same checksum value will have the same contents.
DEFINE_PROPERTYKEY(PKEY_StorageProviderFileChecksum, 0xB2F9B9D6, 0xFEC4,
0x4DD5, 0x94, 0xD7, 0x89, 0x57, 0x48, 0x8C, 0x80, 0x7B, 5);
#define INIT_PKEY_StorageProviderFileChecksum { { 0xB2F9B9D6, 0xFEC4,
0x4DD5, 0x94, 0xD7, 0x89, 0x57, 0x48, 0x8C, 0x80, 0x7B }, 5 }

// Name: System.StorageProviderFileFlags -- PKEY_StorageProviderFileFlags
// Type: UInt32 -- VT_UI4
// FormatID: {B2F9B9D6-FEC4-4DD5-94D7-8957488C807B}, 8
//
// Information specified by the storage provider about a file or a folder.
DEFINE_PROPERTYKEY(PKEY_StorageProviderFileFlags, 0xB2F9B9D6, 0xFEC4,
0x4DD5, 0x94, 0xD7, 0x89, 0x57, 0x48, 0x8C, 0x80, 0x7B, 8);
#define INIT_PKEY_StorageProviderFileFlags { { 0xB2F9B9D6, 0xFEC4, 0x4DD5,
0x94, 0xD7, 0x89, 0x57, 0x48, 0x8C, 0x80, 0x7B }, 8 }

// Name: System.StorageProviderSyncClientId --
PKEY_StorageProviderSyncClientId
// Type: String -- VT_LPWSTR (For variants: VT_BSTR)
// FormatID: {0142C8EA-B555-4562-BC82-4466E3D415FF}, 1
//
// The storage provider's SyncClientId, an optional property, uniquely
identifies an installed sync client per user per machine.
```

```

DEFINE_PROPERTYKEY(PKEY_StorageProviderSyncClientId, 0x142c8ea, 0xb555,
0x4562, 0xbc, 0x82, 0x44, 0x66, 0xe3, 0xd4, 0x15, 0xff, 1);
#define INIT_PKEY_StorageProviderSyncClientId { { 0x142c8ea, 0xb555, 0x4562,
0xbc, 0x82, 0x44, 0x66, 0xe3, 0xd4, 0x15, 0xff }, 1 }

// Name: System.StorageProviderSessionId -- PKEY_StorageProviderSessionId
// Type: String -- VT_LPWSTR (For variants: VT_BSTR)
// FormatID: {57C1C5FF-C98B-436D-A284-E48FE6628276}, 1
//
// The storage provider's SessionId, an optional property, uniquely
identifies a communication session with Office client.
DEFINE_PROPERTYKEY(PKEY_StorageProviderSessionId , 0x57c1c5ff, 0xc98b,
0x436d, 0xa2, 0x84, 0xe4, 0x8f, 0xe6, 0x62, 0x82, 0x76, 1);
#define INIT_PKEY_StorageProviderSessionId { { 0x57c1c5ff, 0xc98b, 0x436d,
0xa2, 0x84, 0xe4, 0x8f, 0xe6, 0x62, 0x82, 0x76 }, 1 }

```

In the `PROPVARIANT` from `StorageProviderFileRemoteUri`, we expect a string with the unique `WopiUr1` for a file. This `PROPVARIANT` is defined by Windows, but the third-party provider holds that information and will respond to the COM call.

In the `PROPVARIANT` from `StorageProviderFileChecksum`, we expect a string value for the last known hash of the file on the service. In other words, the hash of the file when it was last in sync with the service. `HashAlgorithm` in the sync engine provider's registry allows the third-party provider to specify the hash algorithm to use from one of the algorithms defined in [CNG algorithm identifiers \(Bcrypt.h\) - Win32 apps](#) in the Windows documentation. For example, SHA1 or SHA512 for convenience, if the provider already hashes the files a certain way for its own sync needs. If the value isn't valid, Office defaults to SHA512. This `PROPVARIANT` is defined by Windows, but the third-party provider holds that information and will respond to the COM call.

The Office client uses the `StorageProviderFileChecksum` property to determine if the file is dirty. If the file is dirty, Office doesn't open the file from the service because doing so would create conflicts that the sync engine wouldn't be able to resolve without forking the file. Instead, Office opens the file local-only like any other file that's not in a sync engine location. To determine if the file is dirty, Office will hash the contents of the file on disk and compare it against the hash retrieved from the sync engine. This process creates a perceivable performance impact, depending on file size and the hash algorithm used.

The Office client uses the `StorageProviderWOPIServiceId` and `StorageProviderWOPIUserId` properties internally to retrieve any previously stored authentication info for the file to authenticate with the WOPI service.

`StorageProviderFileFlags` is a bit-flag field used by the sync client to inform the Office client whether the file supports coauthoring. If `0x1` is set, coauthoring is supported.

Otherwise, it isn't. If the file supports coauthoring, it will be opened with coauthoring extensions. If not, the Office client will open the file locally without coauthoring.

`StorageProviderSyncClientId` and `StorageProviderSessionId` are custom properties. They are not defined by Windows. Therefore, third-party sync client providers must hard-code these properties using the same GUID and property identifier listed in the `FormatID` for these properties. The Office client uses the `StorageProviderSyncClientId` and `StorageProviderSessionId` properties to associate with the integration debugging information. No customer or service Personal Identifiable Information should be present in these properties. When troubleshooting integration issues, third-party sync client providers can use the `StorageProviderSyncClientId` and `StorageProviderSessionId` to correlate with the providers' side of debugging data.

To support this flow, the third-party sync engine must implement the *StorageHandler* APIs.

Resolving a local file as connected with a cloud file URL starting from the URL isn't supported for this scenario. This will result in a server-only open regardless of the dirty-file status.

Mac metadata

macOS has security features that prohibit inter-process communication calls between apps that aren't located in the same publisher's AppGroup. But Apple has FileProvider APIs designed for sync-provider scenarios like this one. By using these APIs, we can communicate cross process with third-party apps.

The APIs let us connect to custom-defined "services" for a given a local file path that belongs to a third-party sync engine. This means third-party partners can implement protocols defined by Office to retrieve the additional information needed to support real-time coauthoring. This technology is similar to COM on Windows. The Apple APIs are available in macOS 10.13 (High Sierra) or later.

As on Windows, using XPC can have performance implications. We must first determine that we really need to use XPC before invoking a performance-intensive API. As with the MountPoint registry shortcut we use in Windows, Office must first determine, early in the file-open flow, if the given file path is backed by a sync engine. Third-party sync engines must register their domains with the OS as part of using Apple's FileManager APIs for sync. After successful registration, a call made to `NSURLIsUbiquitousItemKey` returns true if the third party's registered domain is a subpath of the local file path.

For more information, see [NSURLIsUbiquitousItemKey](#). Although Apple's documentation implies that this API is applicable only to iCloud files, that's no longer the case. The same property now supports third-party synced files.

Here's the declaration for `NSURLIsUbiquitousItemKey`:

```
const NSURLResourceKey NSURLIsUbiquitousItemKey;
```

After Office determines that a file is "ubiquitous," we look for available services that are supported by the file. If our required service name is in the list of services returned by `getFileProviderServicesForItemAtURL`, we can start the XPC connection with the third-party sync engine.

Office defines a new service named `com.microsoft.office.CoauthoringService.v1`. It will be published in a header file and shared with third-party partners to consume:

```
- (void)getFileProviderServicesForItemAtURL:(NSURL *)url
                                completionHandler:(void
(^)(NSDictionary<NSFileProviderServiceName,NSFileProviderService *>
*services, NSError *error))completionHandler;
```

For more information, see [getFileProviderServicesForItemAtURL:completionHandler:](#) in the Apple developer documentation.

On the Office call to `getFileProviderServicesForItemAtURL`, the OS determines that the item belongs to a File Provider. It internally calls `supportedServiceSourcesForItemIdentifier`. The third-party partner must implement `supportedServiceSourcesForItemIdentifier` to return the preceding service in the list of available services:

```
- (NSArray<id<NSFileProviderServiceSource>>
*)supportedServiceSourcesForItemIdentifier:
(NSFileProviderItemIdentifier)itemIdentifier
```

For more information, see [supportedServiceSourcesForItemIdentifier:error:](#) in the Apple developer documentation.

After Office iterates through available services and finds the new service, Office initializes a `NSFileProviderService` with the name, and then calls `getFileProviderConnectionWithCompletionHandler` to initialize a `NSXPCConnection`. When

those initializations succeed, Office sets the `remoteObjectInterface` to the Office-defined protocol (`OfficeCoauthoringService`) shared with third-party partners in the header file mentioned previously.

```
+ (NSXPCInterface *)interfaceWithProtocol:(Protocol *)protocol
```

For more information, see [interfaceWithProtocol:](#) in the Apple developer documentation.

The Office-defined XPC protocol allows us to retrieve the five properties we need for a given file in one call. All XPC protocol calls are asynchronous, so all functions must be void. To receive a response, we need a reply/completion block that we can wait for. Although these calls are asynchronous by nature, Office must synchronously wait for their completion in the critical file-open flow of third-party synced files.

To allow flexibility in the third party's implementation of the XPC service, we again pass in the local path with which we initialized the XPC connection as part of `retrievePropertiesAtItem`. Doing so lets the implementation of the protocol be a single instance for all files instead of one per file, if the third-party provider prefers it for simplicity and performance.

Here's the Office header file to be shared with and implemented by third-party partners:

```
/*
OfficeCoauthoringService.h
Header for XPC between Microsoft Office and third-party sync engines
*/

#import <Foundation/Foundation.h>

#define OfficeCoauthoringServiceName
"com.microsoft.office.CoauthoringService.v1"

#define wopiSrcProperty "wopiSrc" // expects NSString
#define wopiUserIdProperty "wopiUserId" // expects NSString
#define wopiServiceIdProperty "wopiServiceId" // expects NSString
#define hashProperty "hash" // expects NSData
#define hashAlgorithmProperty "hashAlgorithm" // expects NSString
#define supportsCoauthAlgorithmProperty "supportsCoauth" // expects NSData
#define synClientIdProperty "syncClientId" // expects NSString
#define sessionIdProperty "sessionId" // expects NSString

@protocol OfficeCoauthoringService
```

```
- (void)retrievePropertiesAtItem: (NSURL*)localPath  
  
reply : (void(^)(NSDictionary * result)) reply;  
  
@end
```

Third-party partners that want to support coauthoring must import and implement the `OfficeCoauthoringService` protocol. They must return an `NSDictionary` that contains all five of the required properties in the reply after Office requests those properties during the file-open flow. After Office receives the response, there's no further interaction between Office and third-party sync engines for the rest of the file session.

If any of the required properties aren't present in the response, Office will open the file local-only by using just the local path. If all required properties are present, Office will compute the hash of the file content on disk and compare it to the `FileChecksum` property.

The sync engine is expected to update the `FileChecksum` after every change from the service is written to the local disk. An update is also expected after changes from disk are successfully uploaded to the service. This means Office will treat `Hash of the local file != FileChecksum` as dirty and `Hash of the local file == FileChecksum` as not dirty. It will continue with a shared implementation with Windows. Dirty files will open local-only by using just the local path, and non-dirty files will open without requiring a network round trip with the URL and coauthoring support.

The Office apps internally use `WOPIServiceId` and `WOPIUserId` properties to find authentication info for the WOPI service.

The sync client uses `SupportsCoauth` to notify the Office client whether the file supports coauthoring. If it supports coauthoring, the Office client will open the file and coauthoring will take place. If not, the Office client will open the file locally with no coauthoring.

The Office client uses the `SyncClientId` and `SessionId` properties to associate with the integration debugging information. No customer or service Personal Identifiable Information should be present in these properties. When troubleshooting integration issues, third-party sync client providers can use the `SyncClientId` and `SessionId` to correlate with the providers' side of debugging data.

Next steps

- Learn how Office gets [data](#) from third-party sync engines.

- Learn how sync client integration works in key [scenarios and edge cases](#).

How Office gets data from third-party sync engines

Article • 07/06/2022

In addition to metadata about a file, Microsoft Office also needs the contents of the local file. The data populates the Office file cache with the local copy of the file. The local copy allows users to immediately start reading and editing the file. No full-file downloads or round trips are needed in the critical open path. However, offline open operations in the same integrated mode are supported.

How data is populated

Placeholder files must be hydrated by the sync engine at this point. The attempt to read the contents of the file should be the latest opportunity for the sync engine to hydrate it. No other calls from Office should be needed to hydrate a placeholder. A read attempt should be enough.

While resolving a local file as connected with a cloud file URL, after a file is determined to be a third-party synced file, Office will use the stream on disk to populate the Office file cache. It will then display file content to the user for read and edit. At this point, Office will also issue an async request to retrieve changes from the service, if there are any.

After the file is open, Office doesn't interact with the file-only disk or with the third-party sync client. Local user edits are uploaded to the service directly. They aren't written to the file on disk. The sync engine is expected to keep the file on disk in sync. Remote user edits during coauthoring also aren't persisted to the local file on disk.

After a successful open operation backed by sync-client integration, the local path used is stored in our MRU. This stored path allows future open operations of the same file via MRU to open sync-backed and without requiring a network round trip.

When a file is closed, the cache entry and file contents are generally deleted. There's an exception when the user has been working offline and there are changes in the cache that haven't been uploaded. This exception prevents the existence of stale data that's unsynced in our cache. Excel can have trouble merging data that's kept stale for a long time. For follow-up open operations for which there's already an entry for a given file in our cache, the branches aren't replaced with the local copy. Instead, the cached copy is opened and displayed to the user after MapPaths. This scenario implies that the

previous session didn't save changes to the server, so we can't override it with local content.

Next steps

- Learn how Office gets [metadata](#) from third-party sync engines.
- Learn how sync-client integration works in key [scenarios and edge cases](#).

Add-a-Place Provisioning for Desktop

Article • 08/24/2023

Initial View

When **File** / **Open** is selected, the desktop application "backstage" with the user's Most Recently Used (MRU) document list and provisioned service locations are shown to the user.

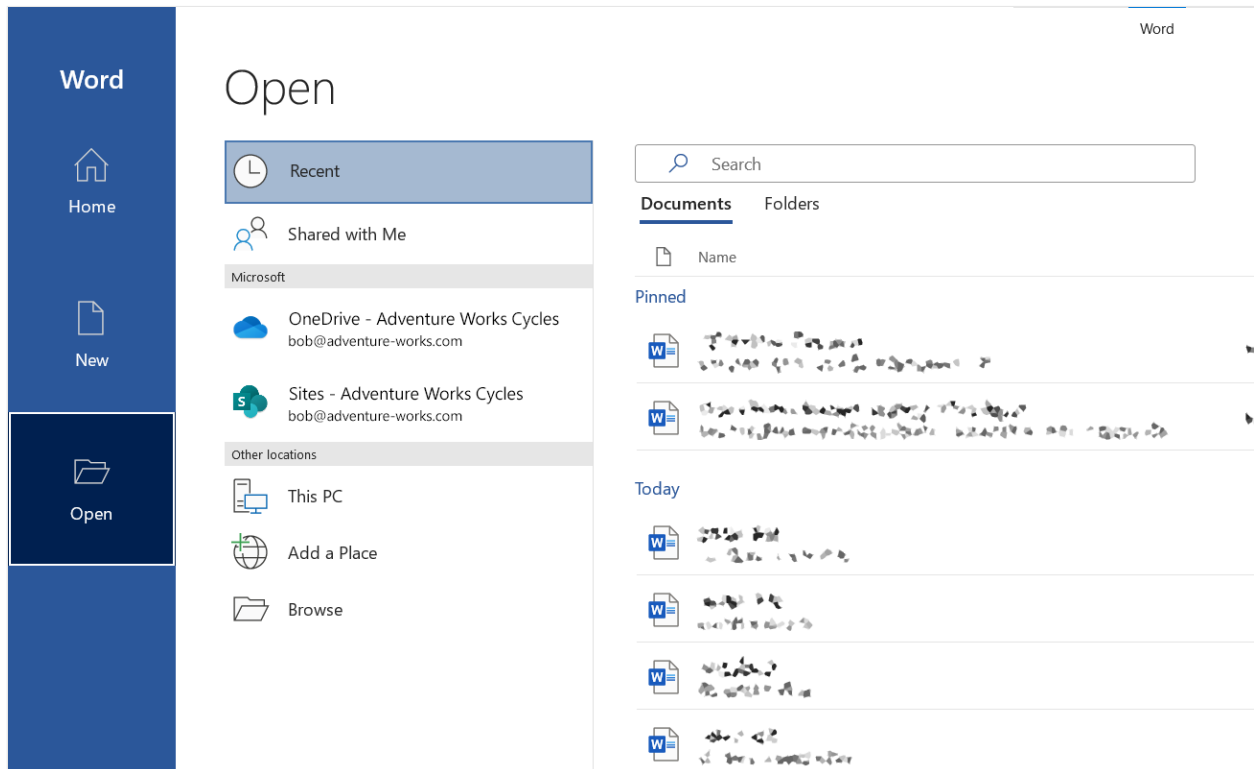


Figure 1a - Initial installation - Windows

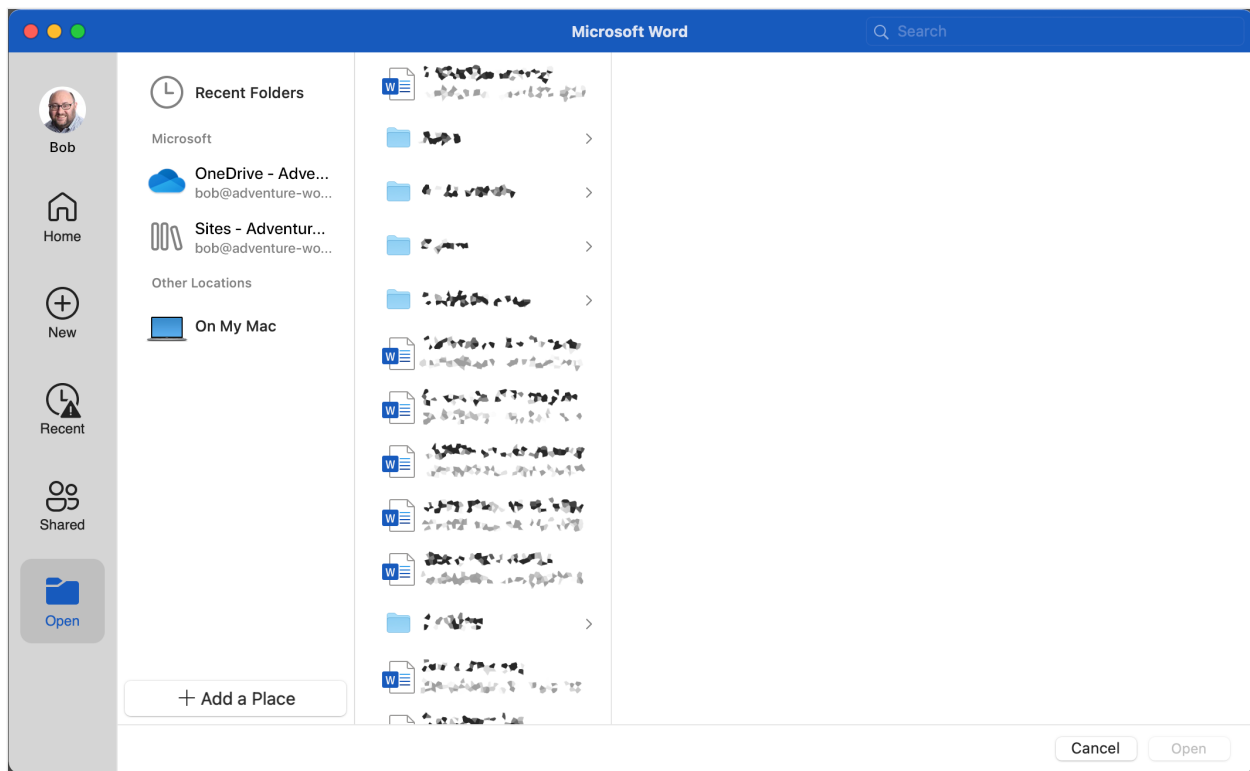


Figure 1b - Initial installation - Mac

In the example above, OneDrive for Business and SharePoint have been previously provisioned for this user.

Adding a Service

To enable opening, editing, and coauthoring of files stored in CSPP Plus third-party services using the Office Desktop applications, the user must first provision the third-party services for their device. This involves selecting the third-party service from the list shown in **Add a Place** and successfully signing in to that service. The same results can be obtained from **Account** / **Add a service** and selecting the third-party service from there.

Once successfully added, the third-party service will show up in the Open section of the application "backstage" across Word, Excel, and PowerPoint on that device.

Step 1: Select a service

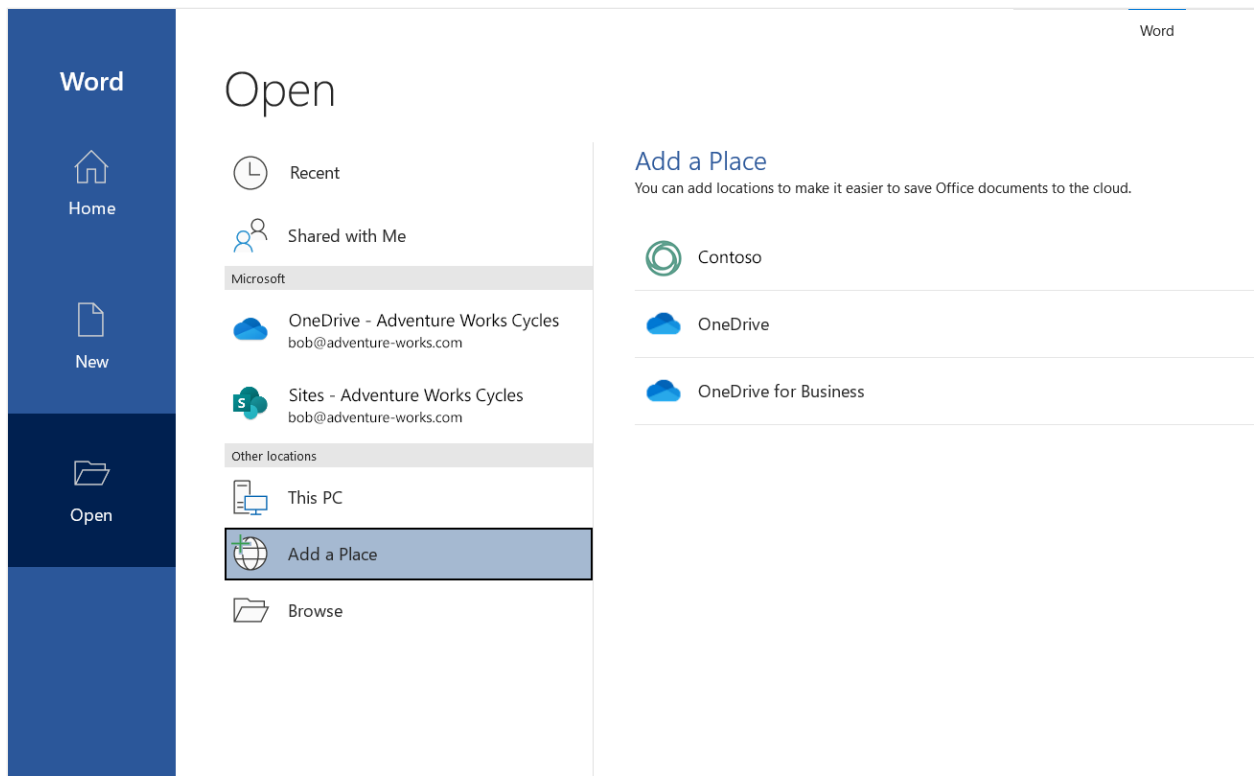


Figure 2a - List of available services in Add-a-Place - Windows

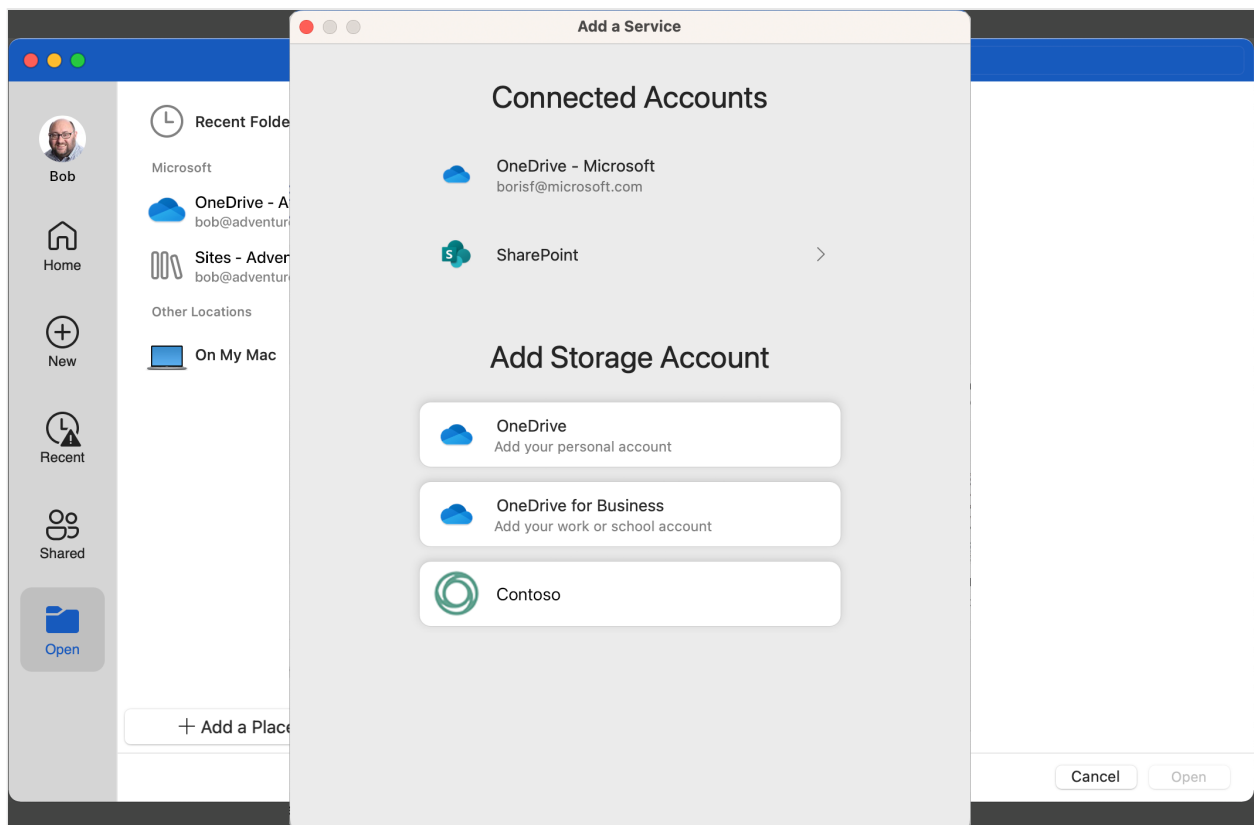


Figure 2b - List of available services in Add-a-Place - Mac

Step 2: Sign in

When a service is selected in the list, the user is prompted to sign in. This experience is provided by the third-party host.

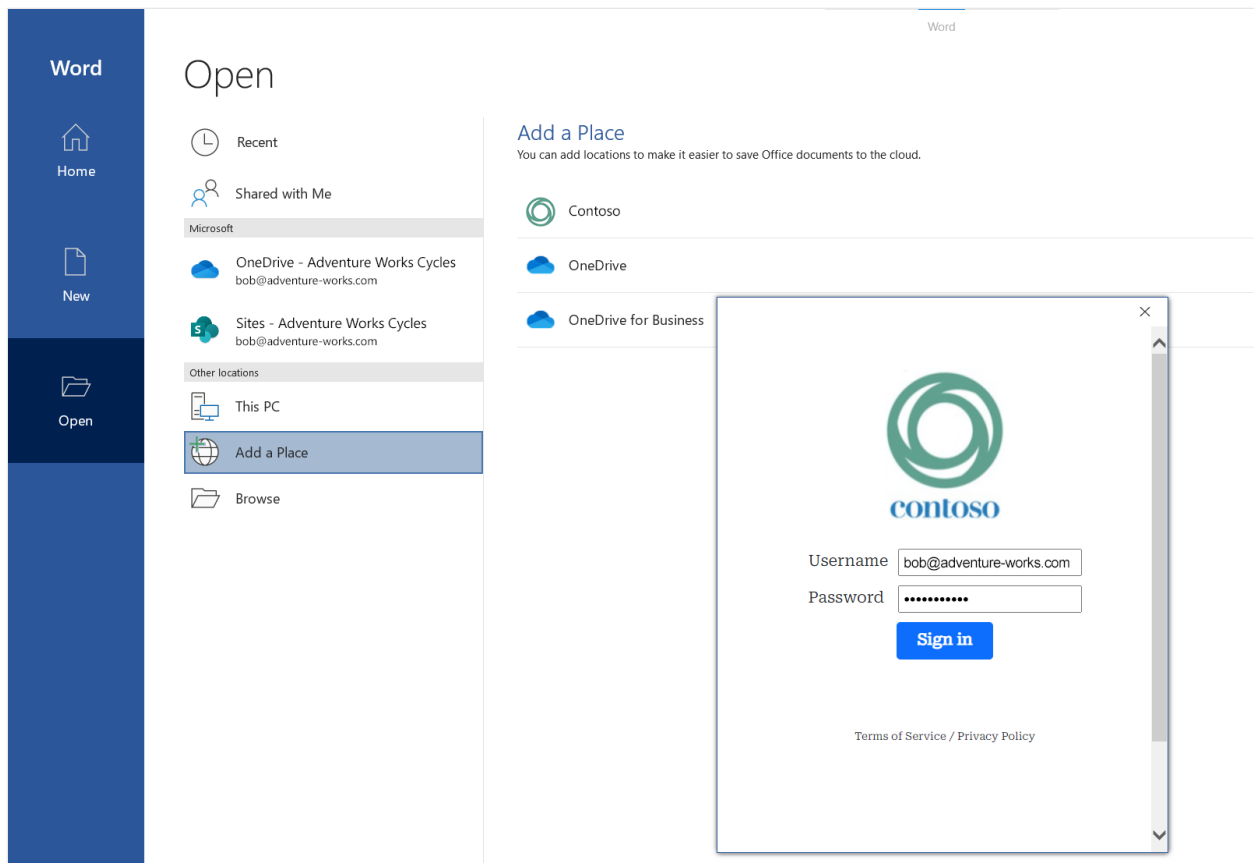


Figure 3a - Third-party service sign-in - Windows

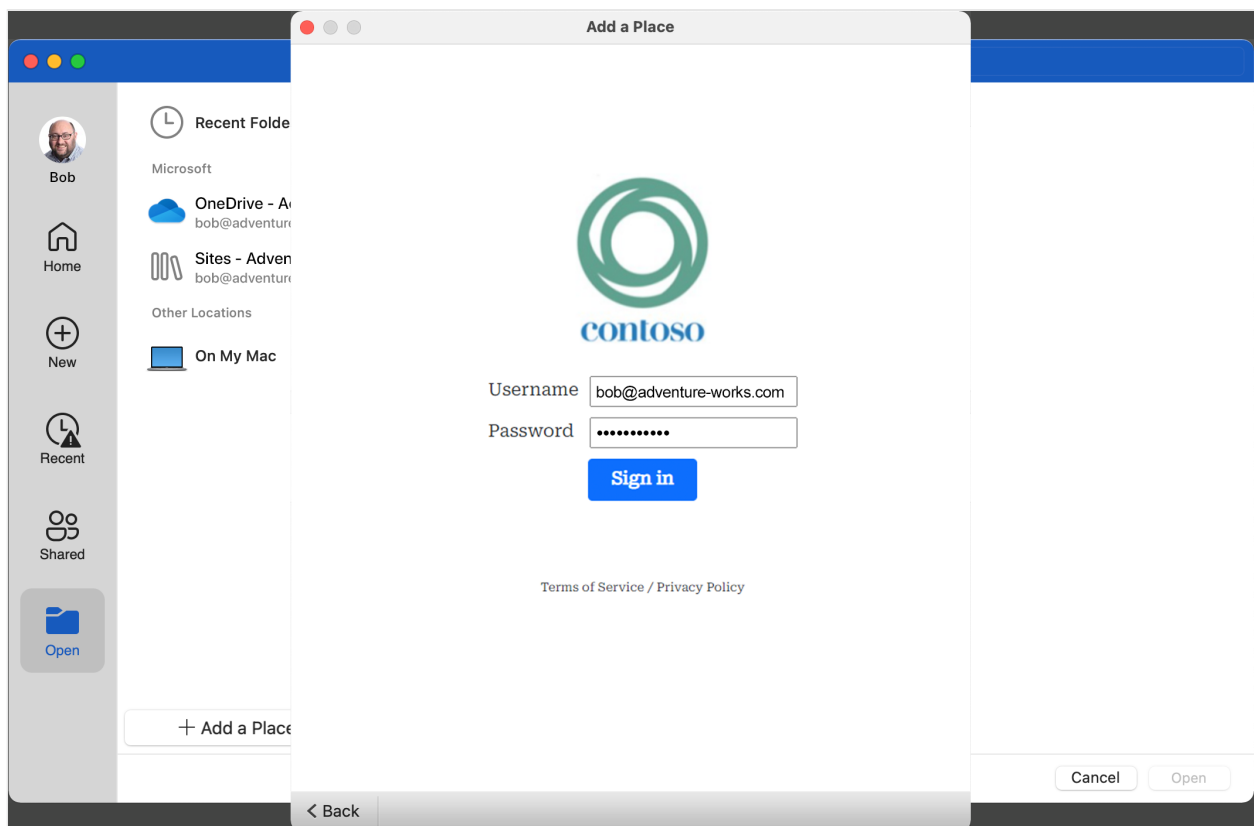


Figure 3b - Third-party service sign-in - Mac

Provisioning Complete

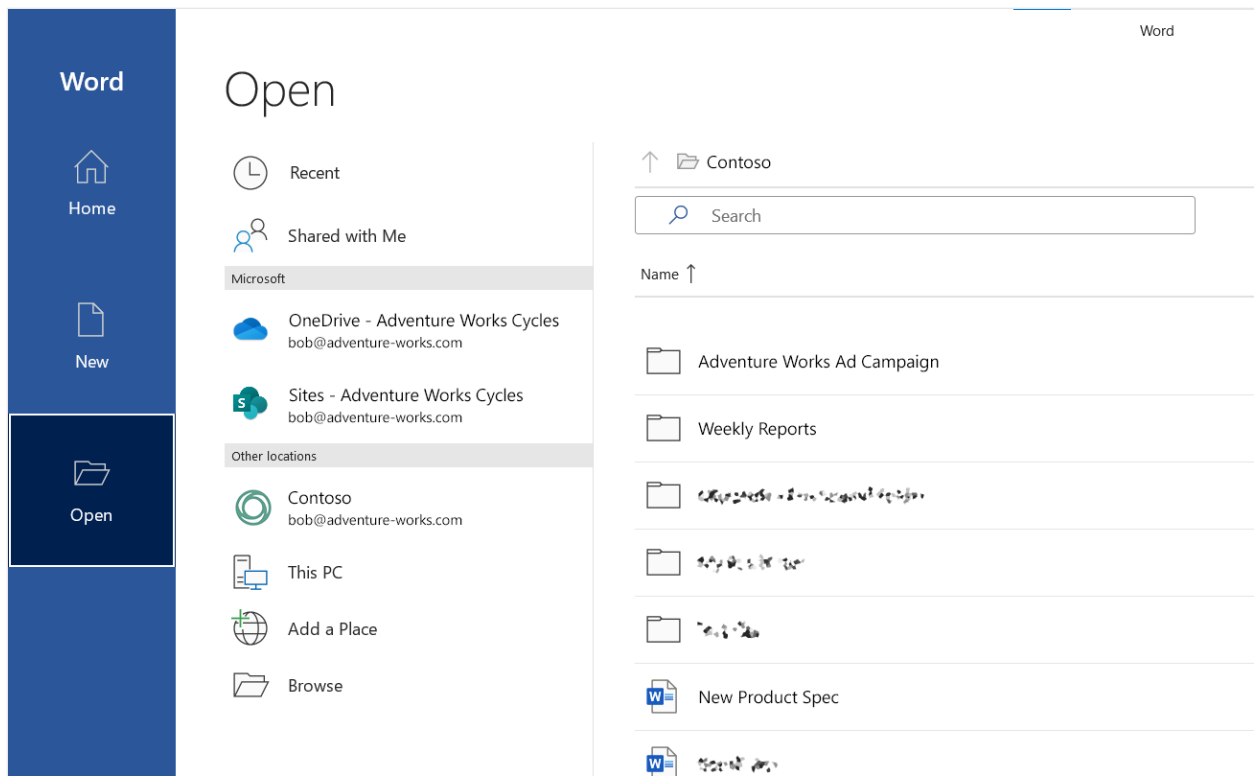


Figure 4a - After a successful login, the service appears in the list of provisioned services (similar to Figure 1a) - Windows

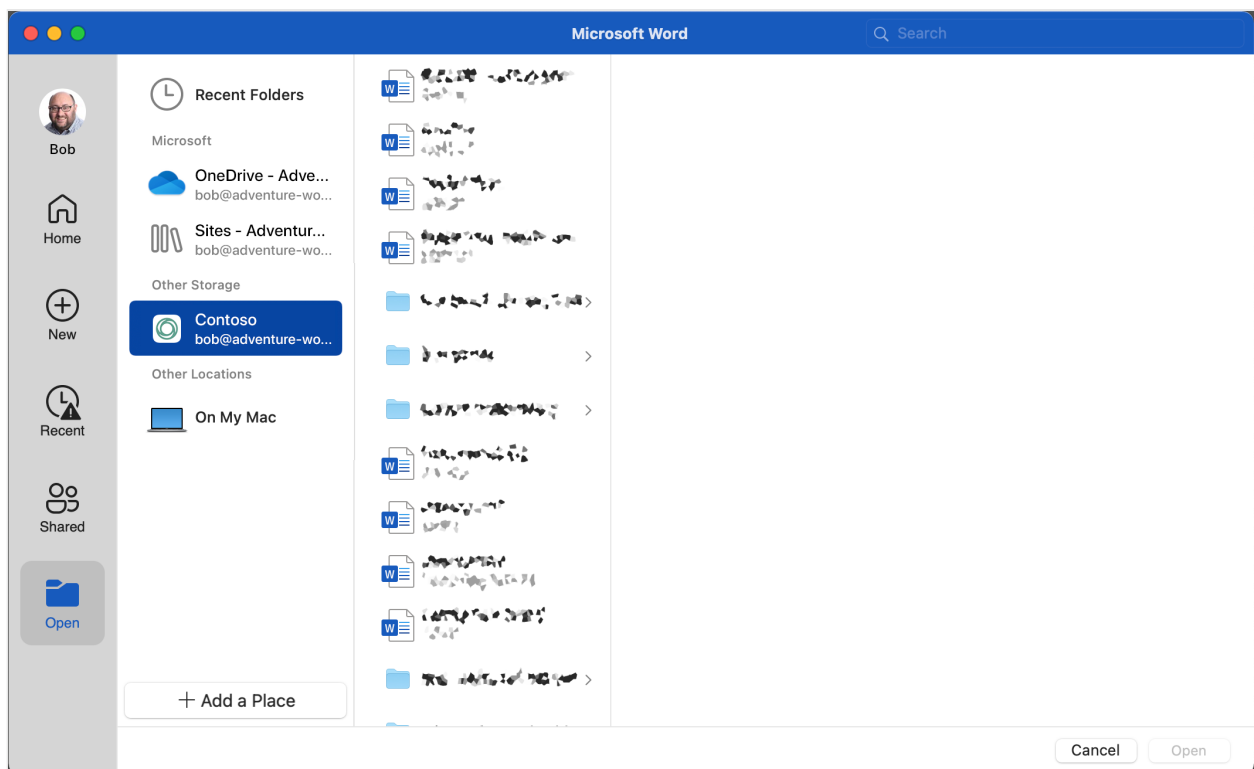


Figure 4b - After a successful login, the service appears in the list of provisioned services (similar to Figure 1b) - Mac

At this point, the service is provisioned and can be used without additional sign-in or provisioning requirements.

Users can use the application backstage to browse documents stored in their file hierarchy on the third-party service, save files to that service, and so on.

If the third-party provides a “sync-client” application that the end-user has installed, and that third-party sync-client has implemented the documented interfaces, Microsoft Office Desktop users are able to double-click a file in Windows File Explorer or macOS Finder to edit or coauthor a cloud file for a service that has been previously provisioned.

Similarly, using the defined protocol-handler URL pattern, the third-party service can open a particular cloud file in an Office Desktop application triggered from the web (for example, “Open in Desktop”).

Removing a Provisioned Service

If the end-user wants to remove a provisioned account from their device, they can easily do so via the Account tab in the application backstage.

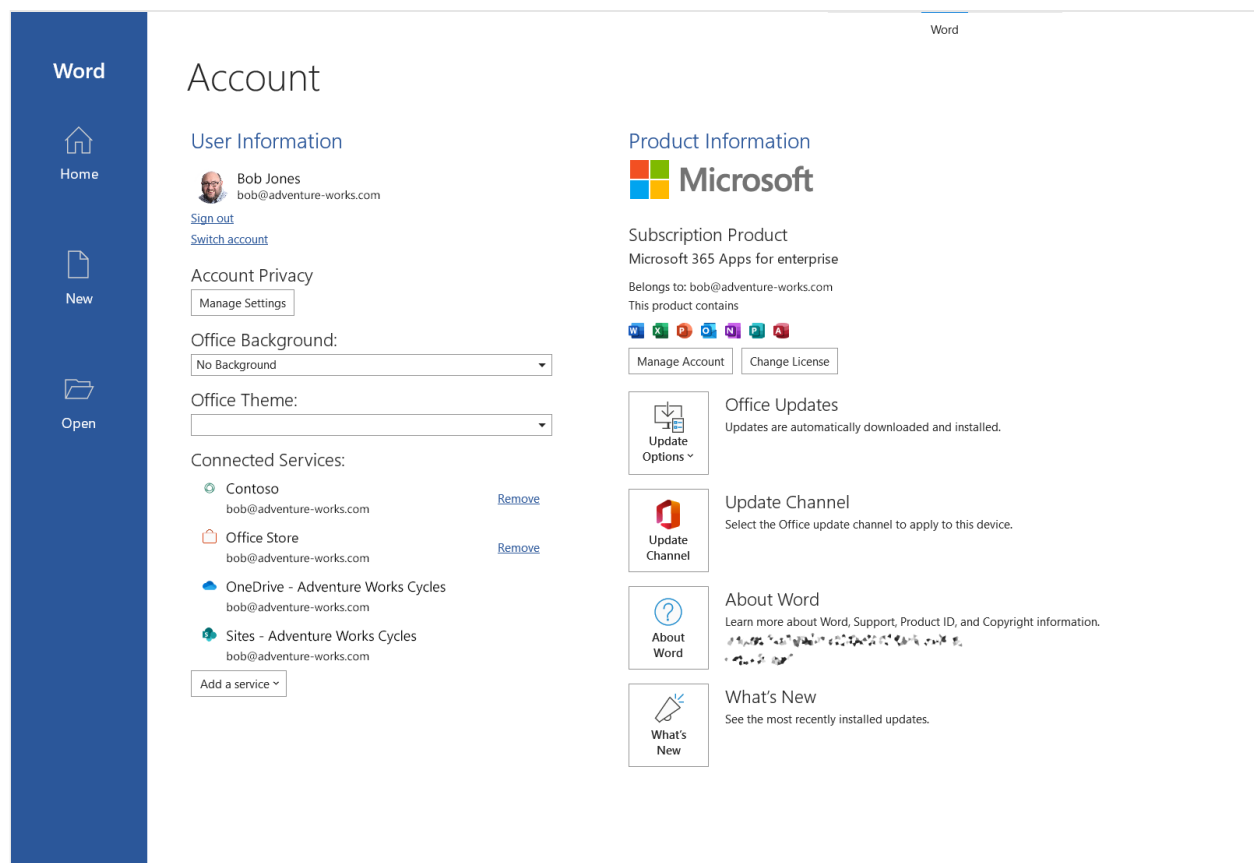


Figure 5a – Removing a provisioned account – Windows

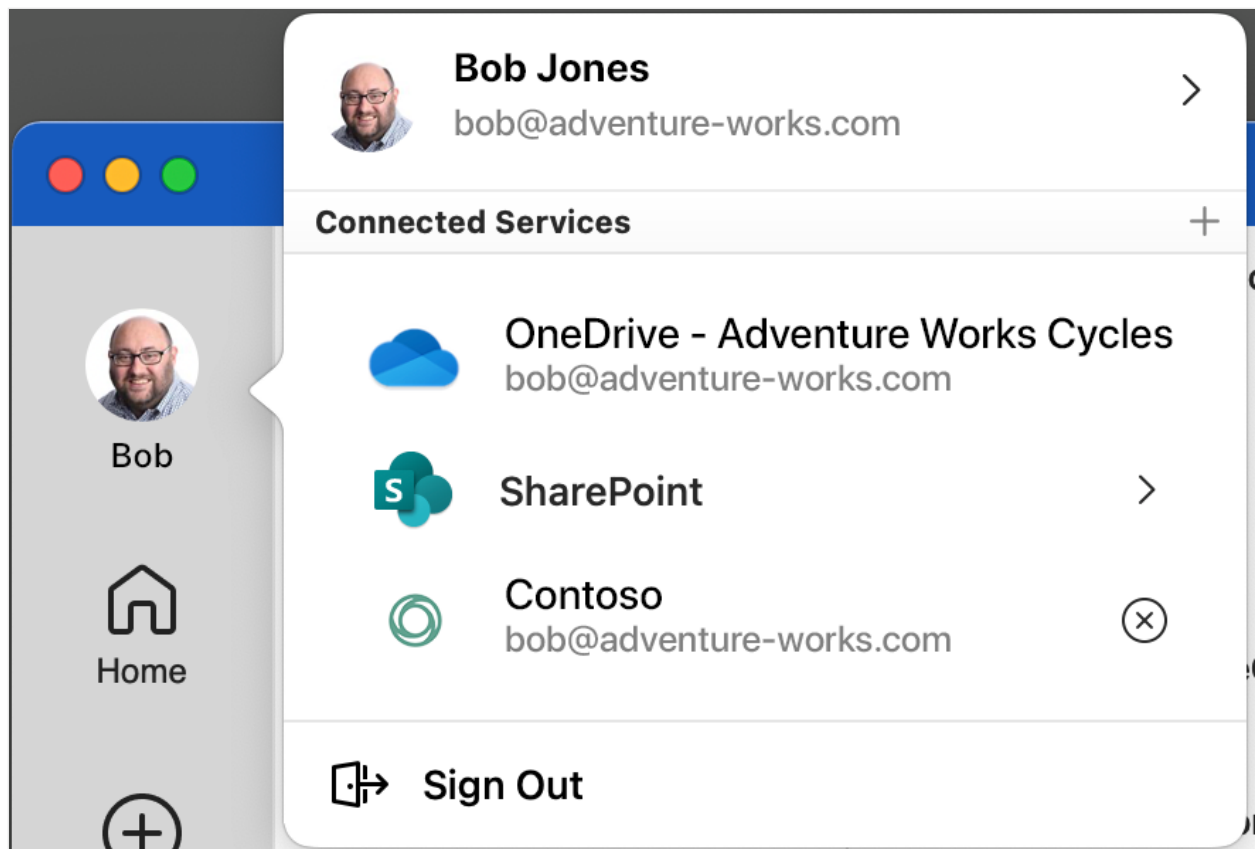


Figure 5b - Removing a provisioned account - Mac

Auto-Provisioning by the IT Administrator

To facilitate provisioning of third-party services at scale, Microsoft has made it possible for IT administrators to "auto-provision" the feature within their enterprise via script.

Windows

Microsoft provides third-party partners a PowerShell (auto-provisioning) script they can distribute to IT administrators. IT administrators can deploy this script to their users' Windows machines, which will partially provision the service on behalf of their users.

macOS

Similarly, on macOS, an additional CFPreferences-compatible preference has been added for Office to enable automatic provisioning of a third-party service. You can set this by using enterprise management software for Mac, such as Jamf Pro.

Category	Details
Domain	com.microsoft.office
Key	OfficePrePopulatedThirdPartyCloudStorageProviders

Category	Details
Data Type	Array of Dictionary elements (see sample plist XML)

Here is a sample plist XML for provisioning a third-party service with the `TP_CONTOSO` identifier:

```
XML

<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN"
"http://www.apple.com/DTDs/PropertyList-1.0.dtd">
<plist version="1.0">
  <dict>
    <key>OfficePrePopulatedThirdPartyCloudStorageProviders</key>
    <array>
      <dict>
        <key>CSPServiceID</key>
        <string>TP_CONTOSO</string>
      </dict>
    </array>
  </dict>
</plist>
```

Alternatively, IT administrators on macOS can also set this preference is by using the `defaults` command. For example, to provision the Contoso service with the `TP_CONTOSO` identifier, you can open Terminal and enter the following command:

```
Console

defaults write com.microsoft.office
OfficePrePopulatedThirdPartyCloudStorageProviders -array '{
CSPServiceID=TP_CONTOSO; }'
```

Auto-provisioning user experience

When a third-party service is auto-provisioned by the IT administrator, the user must complete provisioning by signing in to the service on their device before the service is available for use.

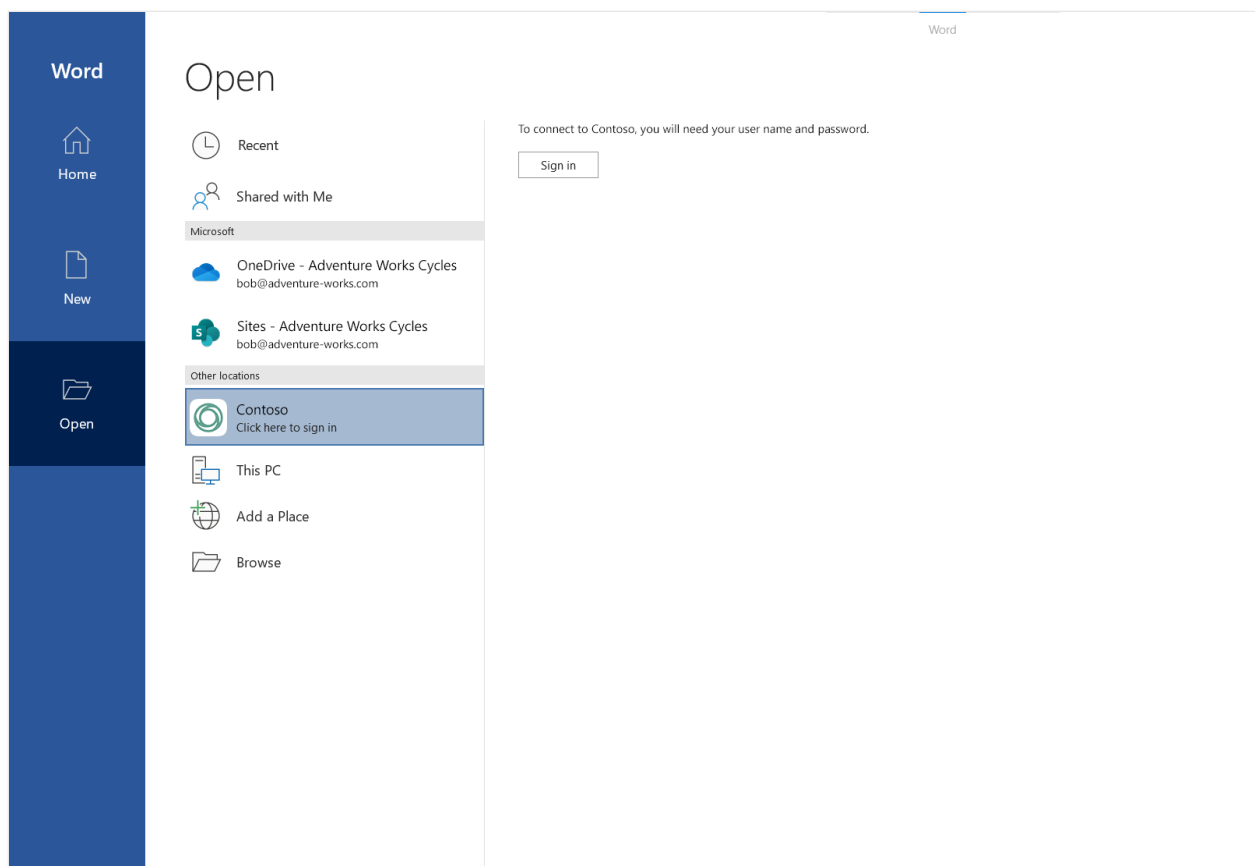


Figure 6a - Partially auto provisioned third-party service entry - Windows

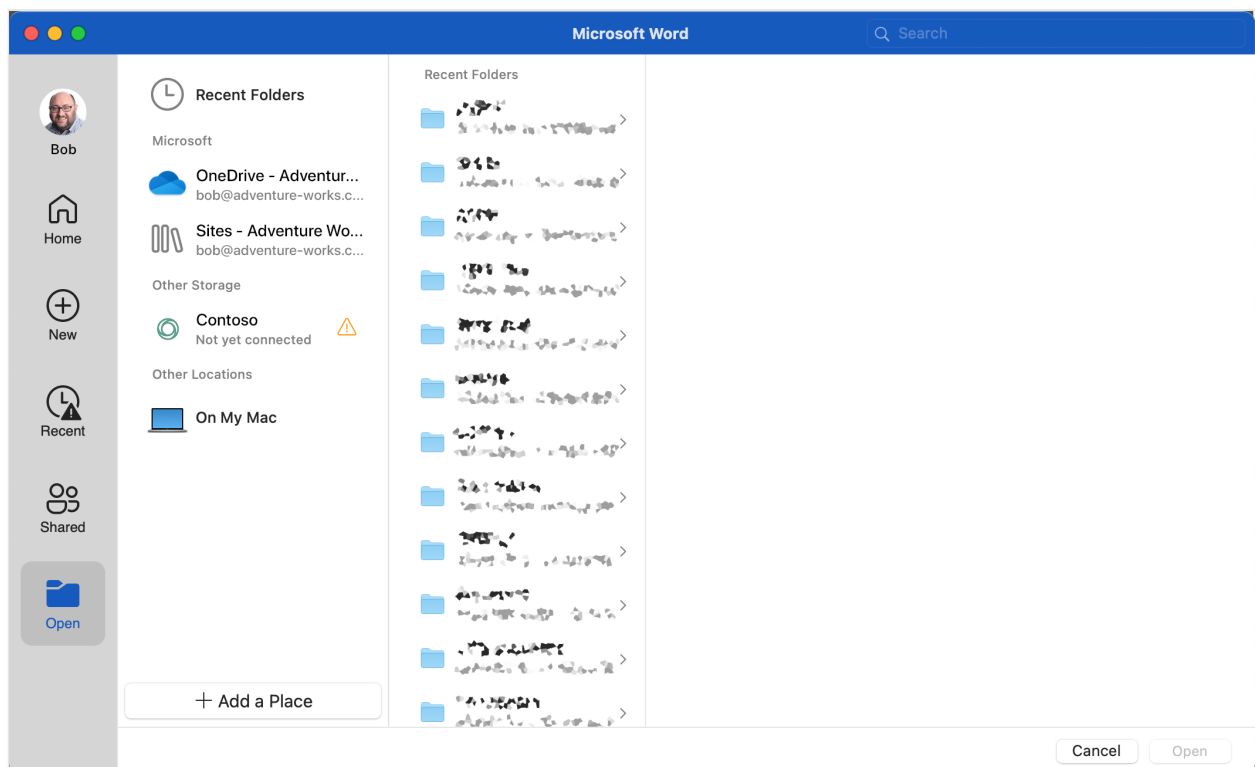


Figure 6b - Partially auto provisioned third-party service entry - Mac

Once the user clicks **Sign in** in the application backstage (Windows) or on the third-party service entry labeled **Not yet connected** (Mac), they are prompted to authenticate in the same way as if they had selected to add the service manually (see Steps 2a/2b: Sign in [above](#)).

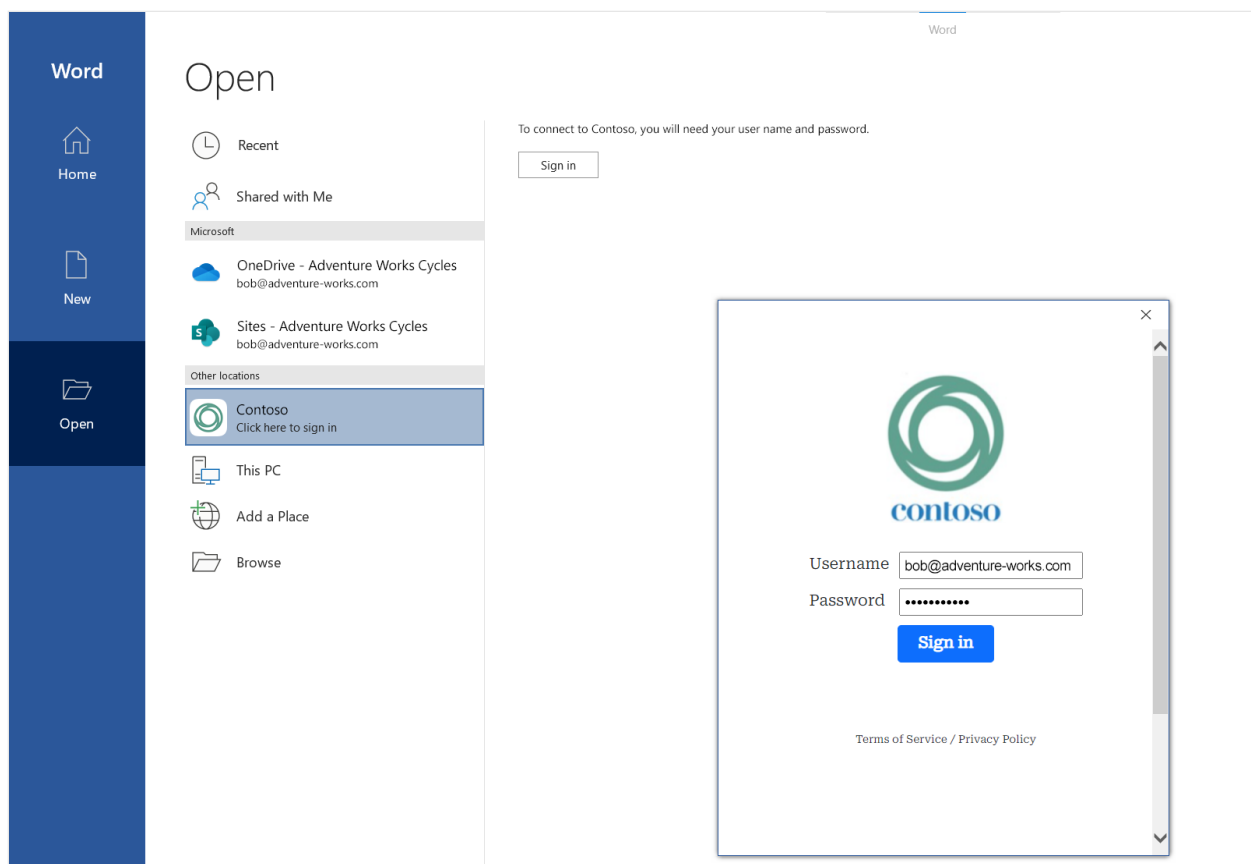


Figure 7 - Sign-in prompt after selecting **Sign in** for an auto-provisioned service (this experience is provided by the third-party host)

Once they complete their login, the service is provisioned and ready to use, as detailed earlier in this article:

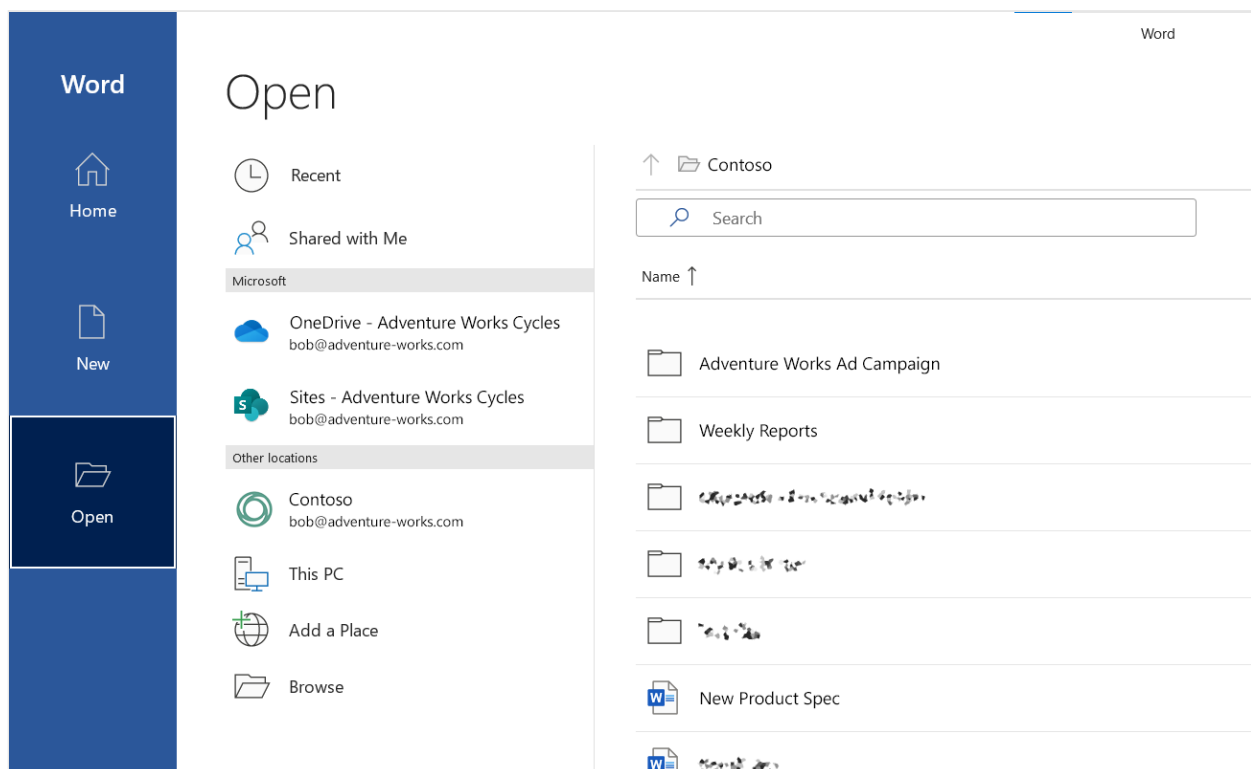
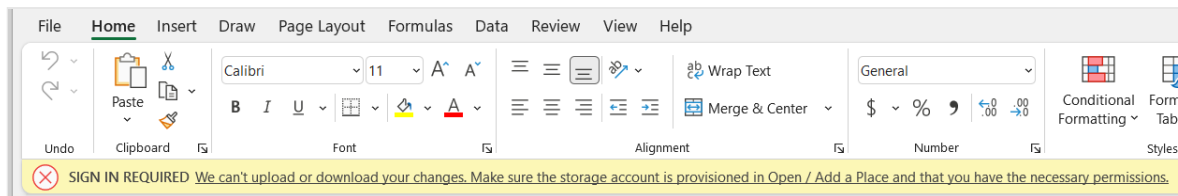


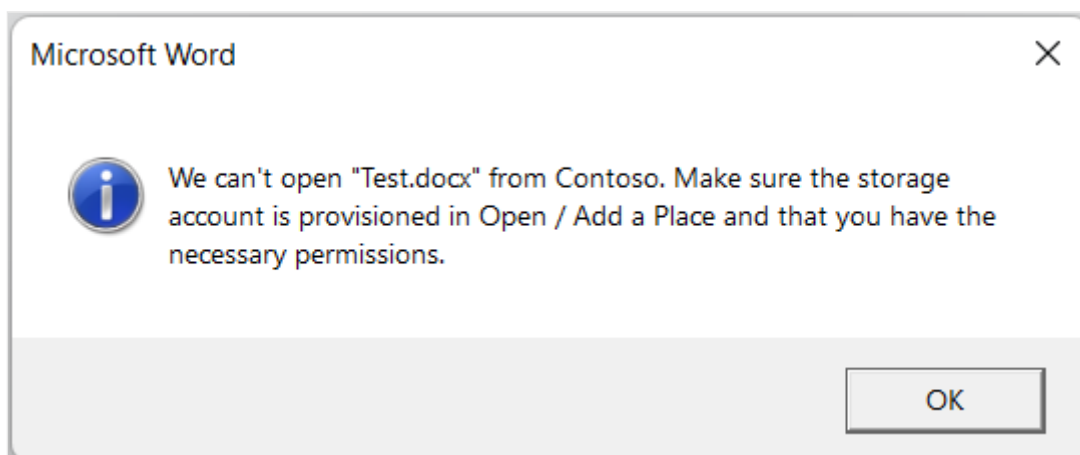
Figure 8 - After a successful login, the service appears in the list of provisioned services (same as Figure 4a)

Auto-provisioning considerations

- After the administrator deploys and runs the auto-provisioning script on a user's machine (Windows) or sets the preference (macOS), the third-party service provisioning is not complete until the user signs in to the third-party service within Office on that device. This will impact access to the third-party service through the sync-client integration and through the "Open in Desktop" functionality.
- Both the PowerShell script for Windows and the preference for macOS are applied when the Word, Excel, or PowerPoint applications launch, and should not be used while any Office applications are running.
- The third-party service settings are specific to a user's login profile per device, and can't be applied machine-wide. For example, on Windows, IT administrators should deploy the script as part of the [one-time login flow](#) for their users.
- Sync client integration – Opening a file within the user's sync client-managed folder using File Explorer or Finder will open the file in Local Only mode, and coauthoring will not be available.



- Open in Desktop – Trying to open a file in the Desktop app from a web browser using the protocol handler will result in the following error message:



Related administrative settings

The following administrative settings may impact the availability of CSPP Plus third-party services in the Office Desktop applications:

- [How to block OneDrive use from within Microsoft 365 Apps for enterprise and Office 2016 applications](#)
- ["This feature has been disabled by your administrator" error in Microsoft Office](#)
- [Use policy settings to manage privacy controls for Microsoft 365 Apps for enterprise](#)

Related articles

- [Configuration Profile Reference \(Apple developer documentation\)](#) 
- [Deploy preferences for Office for Mac](#)

Protected view for documents on CSPP Plus hosts

Article • 03/20/2023

Protected view ([Protected View](#)) and Application Guard ([Application Guard for Office](#)) are mechanisms embedded into Microsoft Office products to help protect users' computers from attacks in files sourced from the Internet. A user's license controls the availability of these tools. When enabled, Office opens files from potentially unsafe locations in Protected View or Application Guard.

As part of the CSPP Plus program, desktop Office clients can open, edit, and save files stored on selected third-party hosts. Like other files originating from the Internet, these files are considered potentially unsafe and, by default, open in a restricted form:

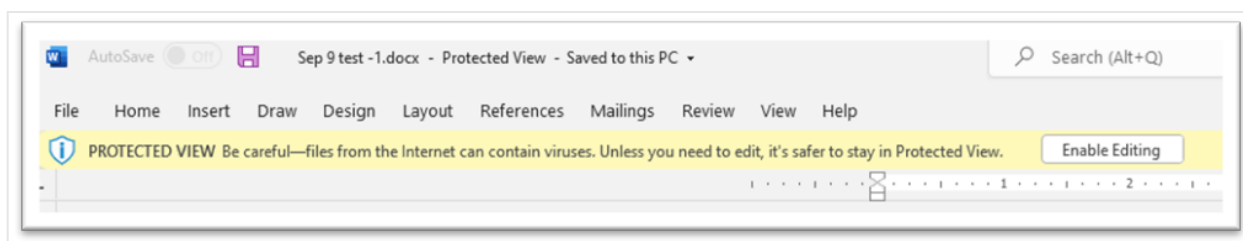


Figure 1 - File opened in Protected View

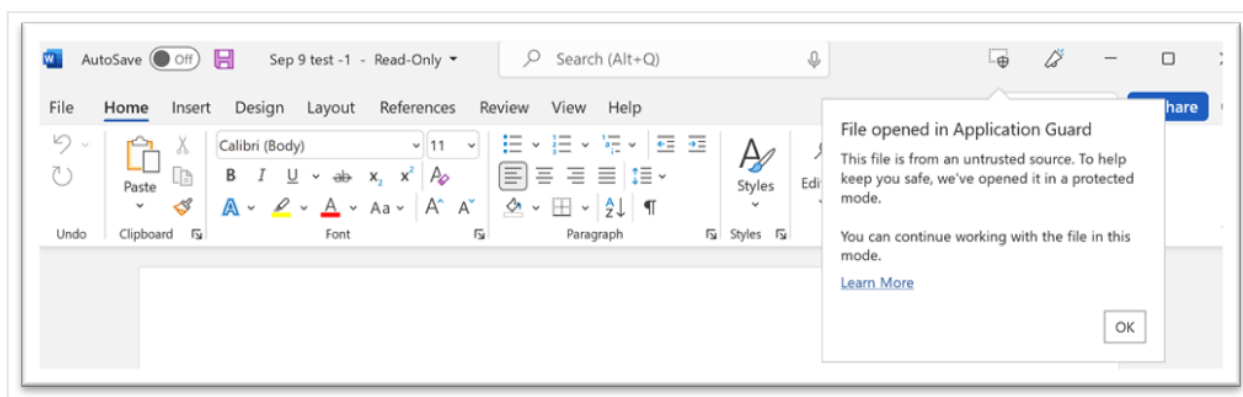


Figure 2 - File opened in Application Guard

It's possible to remove the protection provided by Protected View and Application Guard to enable unrestricted access to files, converting the documents to Trusted Documents.

ⓘ Note

Organizations can implement policies that prevent removing Protected View and Application Guard protection from a file.

Individual (per file) authorization

Users can trust individual documents to prevent opening them in Protected View and Application Guard:

Warning

Users are advised to only do this if they are confident that the file, and its source, is trustworthy.

For Protected View:

- On the Message Bar, click **Enable Editing**.

For Application Guard:

- Go to **File > Info** and select **Remove protection**.

Manual authorization (per machine)

Users can trust specific Internet locations to prevent Protected View and Application Guard to act on those files based on location.

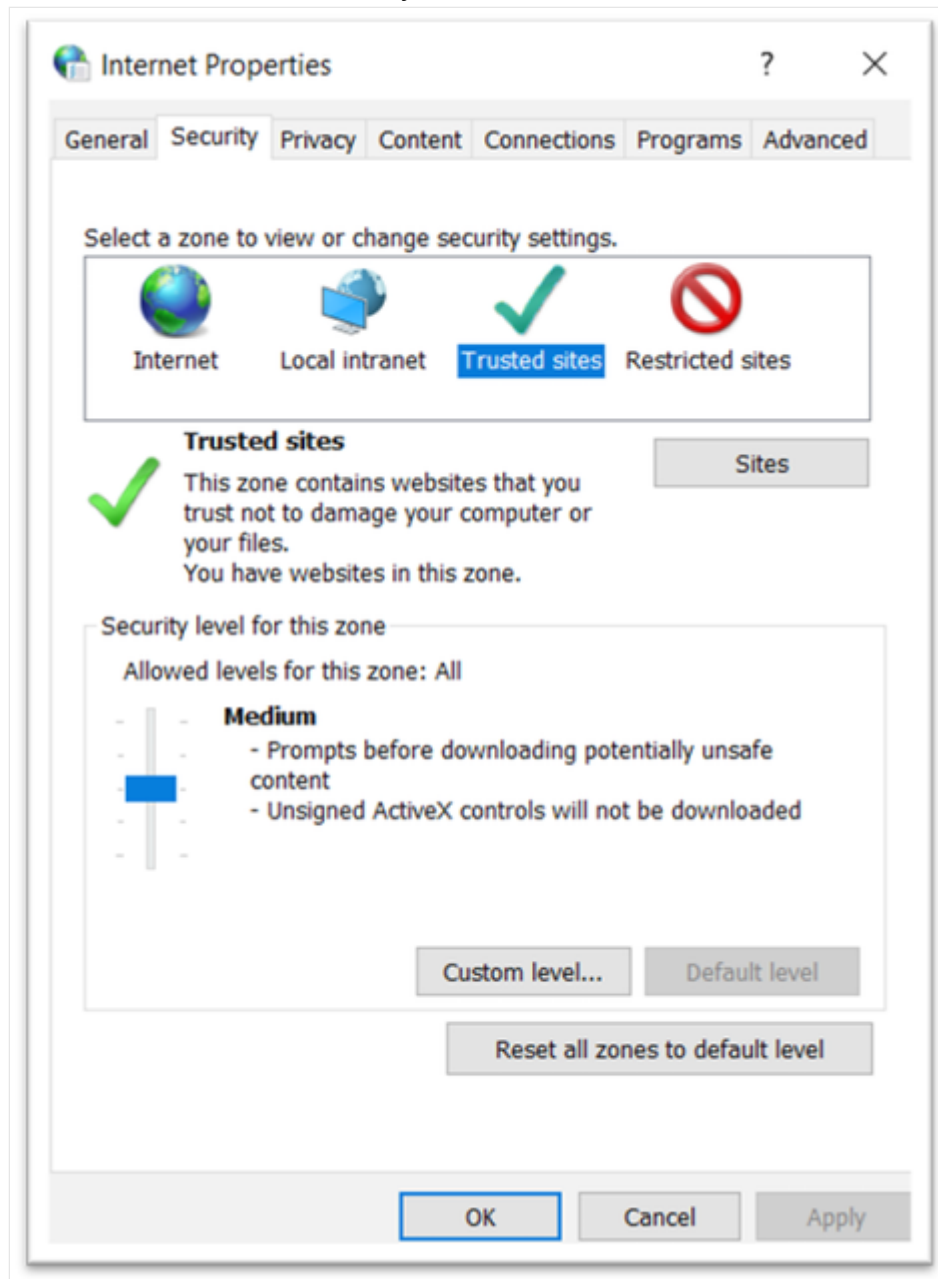
Warning

Only do this if you're confident that the location is trustworthy.

Add the host internet address as a trusted site:

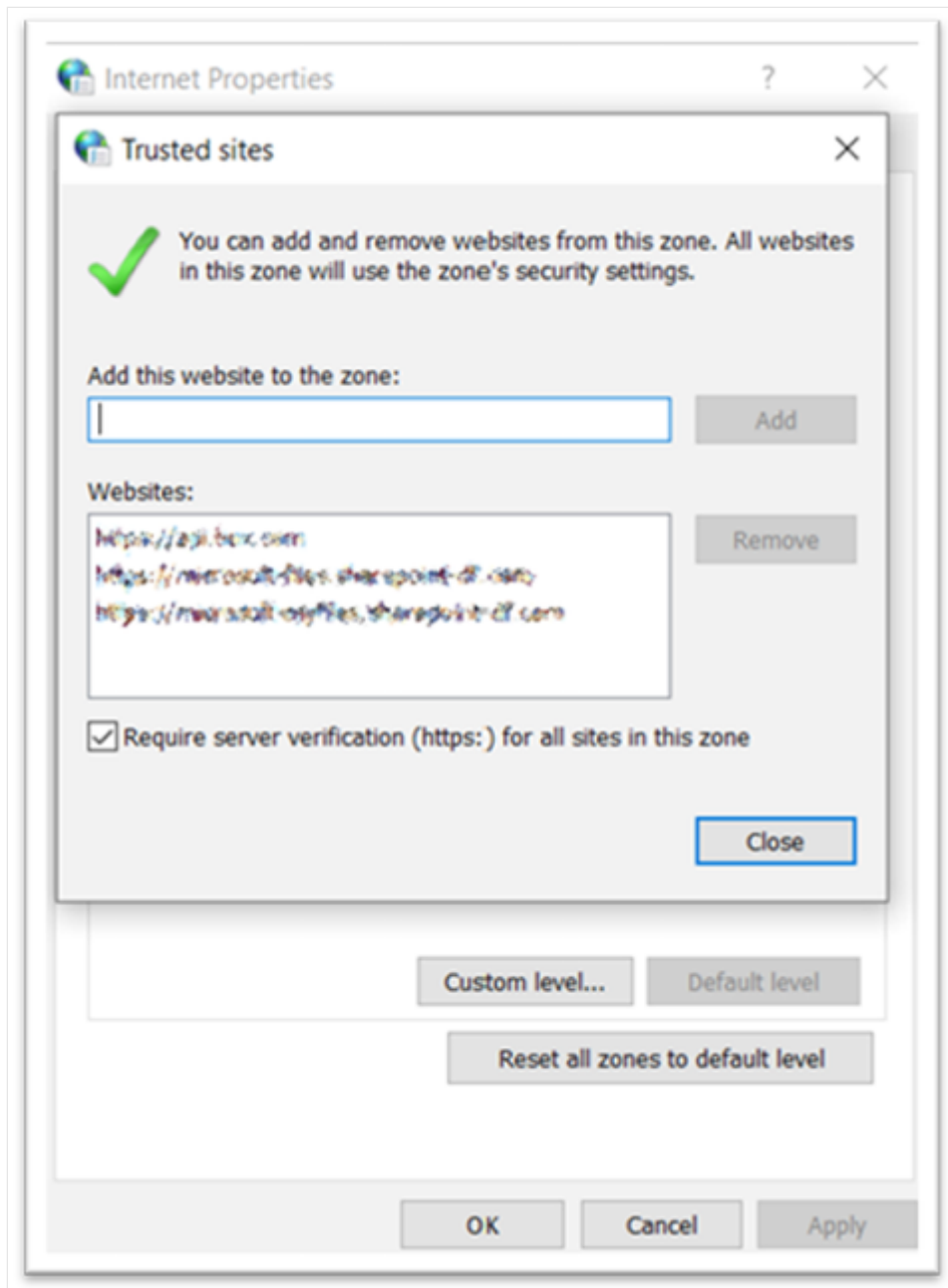
1. Open the **Control Panel**.
2. Open the **Internet Options** object.
3. In the Internet Properties window, click the **Security** tab.

4. Select the **Trusted sites** entry and click the **Sites** button.



5. In the **Add this website to the zone** field, enter the address for the trusted location starting with *https*..

6. Click **Add**.



7. Close the windows.

ⓘ Note

Organizations can implement policies that prevent users from modifying these settings.

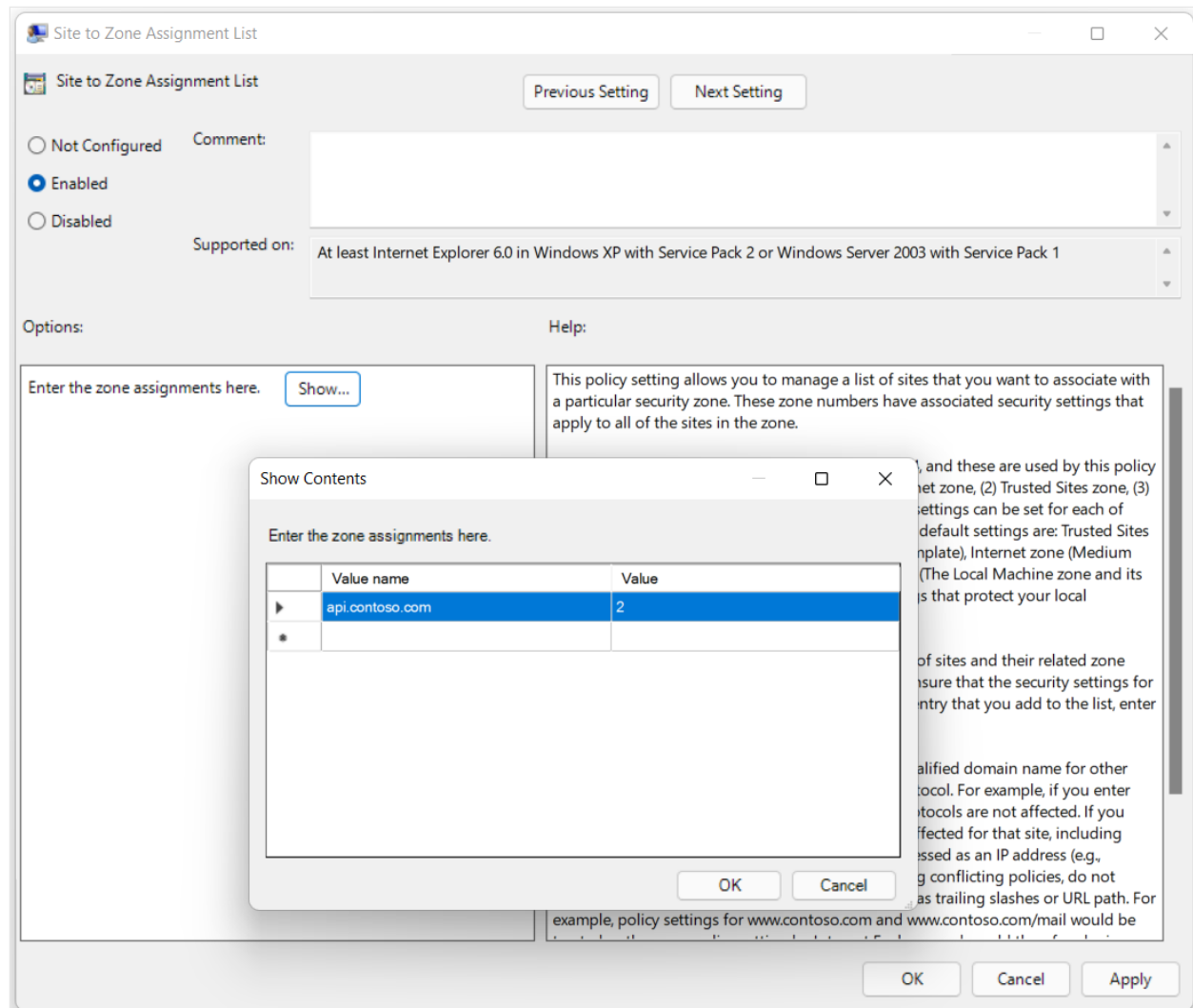
Global authorization (GPO)

IT admins can enforce assignments of Internet Sites to Security Zones across organizations using Group Policy Objects (GPO). This includes the classification of a particular hosts as a trusted location.

To enforce mapping hosts to certain zones, IT admins can enable and configure this GPO: **Site to Zone Assignment List**.

You can find this GPO in the Group Policy Editor under:

Computer Configuration / Administrative Templates / Windows Components / Internet Explorer / Internet Control Panel / Security Page / Site to Zone Assignment List.



ⓘ Note

api.contoso.com in the previous screenshot should be replaced by the appropriate domain(s) provided by the CSPP Plus host.

After enabling the GPO, IT Admins can assign websites to each of the security zones according to this map:

1. Intranet zone
2. Trusted Sites zone
3. Internet zone

4. Restricted Sites zone

To prevent Protected View and Application Guard to act on files based on location, the host address, starting with https:, should be assigned to Trusted Sites zone (2).

Note

Enabling the Site to Zone Assignment List GPO overwrites all previously configured assignments. It prevents users from modifying the assignments. And sets it to only consider the mappings included in the GPO across all the zones.

Related articles

- [Using Group Policy Preferences to add a web site to local intranet, trusted site, and still allow them to add their own](#) .

Supported File Types and Extensions for Coauthoring

Article • 06/17/2023

	Word	Excel	PowerPoint
Coauthoring-Enabled • The list of WOPI+ Open and Save a Copy is filtered to this list	<code>.docx</code>	<code>.xlsx</code> <code>.xlsm</code> <code>.xlsb</code>	<code>.pptx</code> <code>.ppsx</code>
Local-only • All other extensions (including but not limited to) • Open and edit via sync-client on the local machine, not via WOPI+	<code>.doc</code> <code>.docm</code> <code>.dot</code> <code>.dotx</code> <code>.dotm</code> <code>.odt</code>	<code>.xls</code> <code>.xla</code> <code>.ods</code> <code>.csv</code> <code>.xml</code> <code>.xmlx</code> <code>.mht,</code> <code>.mhtml</code> <code>.htm,</code> <code>.html</code> <code>.txt</code> <code>.dif</code> <code>.slk</code> <code>.pdf</code> <code>.xps</code>	<code>.ppt</code> <code>.pot</code> <code>.potm</code> <code>.potx</code> <code>.pps</code> <code>.ppsm</code> <code>.pptm</code> <code>.odp</code>

Coauthoring-Enabled

You can open and save these files to the host via WOPI+.

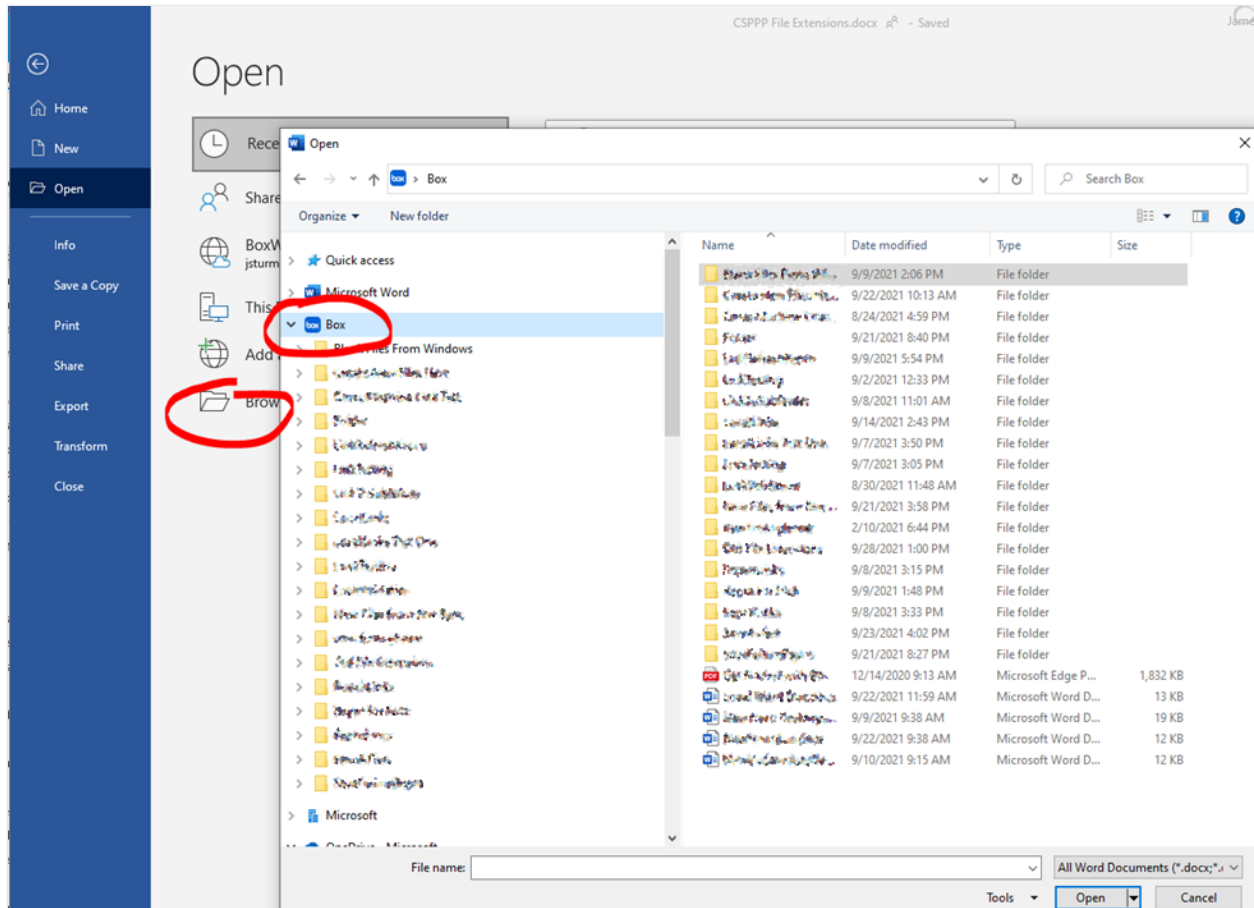
The Add-a-Place file browsing experience and the Save-a-Copy experience are intentionally filtered to only these file extensions.

Local-only file types

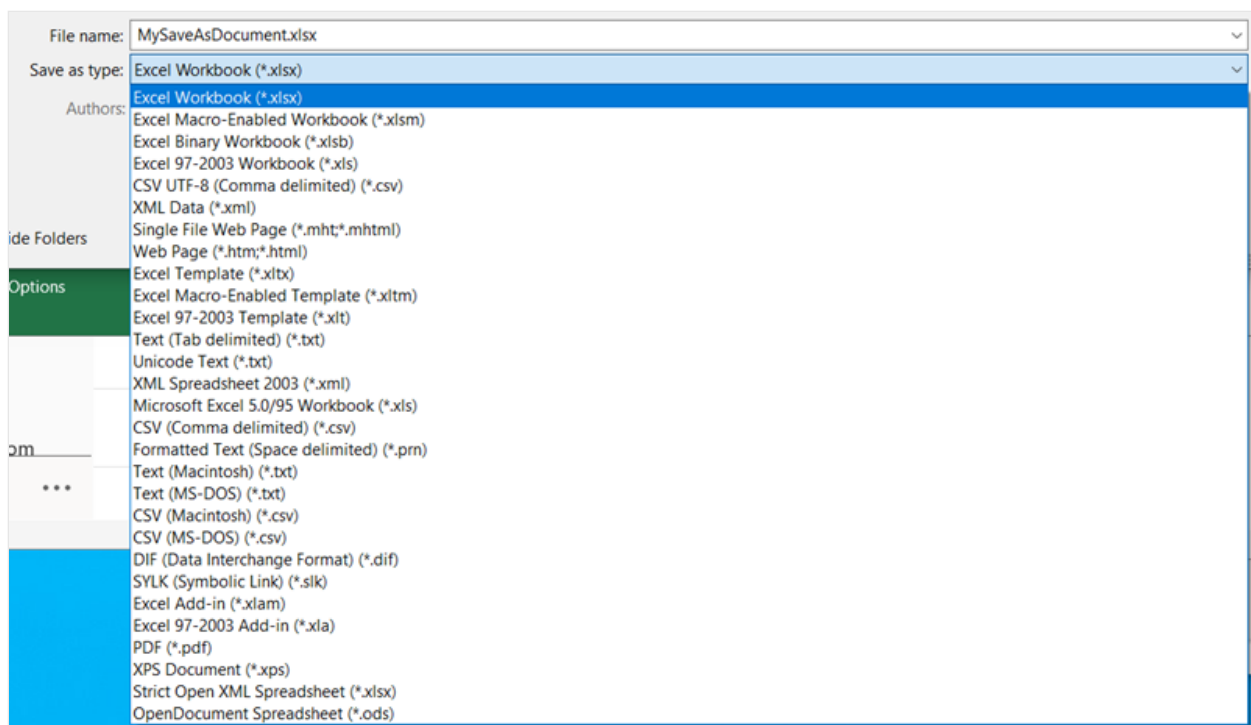
These are non-collaboration file types that are editable by the Microsoft Office Desktop apps, but are not supported for coauthoring.

As noted above, these file types are filtered out from the in-app file browsing experience via Add-a-Place.

In the case of an open from the local disk (including from a sync-backed folder) files with these extensions will always open in Local Only mode. Similarly, if the user wants to save to a file of this type, they can use the Browse functionality to open the native Common File Dialog.



This will provide a file type drop-down (as shown in Excel) that's typical for the local file experience:



Strict Open XML Document format

Coauthoring is not supported for files originally saved in the Strict Open XML Document format. Attempting to open a file stored in this format will result in the error, *"Someone has this workbook locked"*.

Unfortunately, it's not always easy to determine if a file was saved in the Strict Open XML Document format as these files have the same extensions as the default file types. This means that Word, Excel, and PowerPoint files saved in the Strict Open XML Document format will still have the `.docx`, `.xlsx`, and `.pptx` extensions.

If your users are seeing error this when attempting to coauthor, use the following steps to try and resolve the issue:

1. Have all but one user close the file
2. The remaining user should save the file in any format other than Strict XML
3. All users can now open the document and coauthor

More information about coauthoring and Office file types

- For general information about coauthoring that is not WOPI specific, see the [Document collaboration and co-authoring](#) documentation.
- You can find answers to common co-authoring issues on the [Troubleshoot co-authoring in Office](#) page.

- See the [File format reference for Word, Excel, and PowerPoint](#) to get detailed information about Office file formats.

CSPP Plus Host Quality Requirements

Article • 07/06/2022

The quality metrics described below should be monitored by CSPP Plus hosts on an ongoing basis to ensure a quality end-to-end experience for Office customers.

Reliability

Microsoft and each CSPP Plus host will measure the reliability of the host and sync-client integration at the API layer.

Host API Reliability

The host APIs are documented in this documentation set, including the CSPP Plus WOPI extensions.

The host must deliver a highly reliable service before the end-to-end experience is generally available:

- 99.95% reliable per API
- All APIs included
- Failures include server timeouts and HTTP status codes as described in the following table:

Result	Description
400	Bad Requests that cannot be attributed to caller defects as defined by the protocols
500 Internal Server Error	Server error
501 Not Implemented	Operation not supported
50x	Other server failure codes

Sync Client API Reliability

The sync client must deliver a highly reliable integration with Microsoft Office applications before the end-to-end experience is generally available:

- 99.95% reliable per API

See [Sync client integration overview](#) for information about the sync client APIs for Windows and macOS.

Performance

Host API Performance

Microsoft and each CSPP Plus host will measure the performance of the host at the WOPI API layer. The host must deliver acceptable performance on a select subset of performance-critical APIs:

Fixed Size APIs

Achieve less than 1.5 second response times at 95th percentile, as measured from the client (including network latency) for the following APIs:

- [GetCoauthTable](#)
- [CheckFileInfo](#)
- [GetCoauthLock](#)

Chunk Size APIs

Achieve less than 2 second completion times at the following 95th percentile performance, as measured from the client (including network latency) for the following APIs:

- [GetChunkedFile](#)
- [GetChunks](#)

Note

Chunk size isn't directly related to the overall file size and can't be controlled by the host or the client; the chunk sizes are based on the files' content. Assuming files from third-party hosts will look about the same as SharePoint/OneDrive files in aggregate, the 95th percentiles should be roughly the same.

Sync Client API Performance

Sync client performance has to be effectively real-time (no in-line network calls) and so should be less than 10 ms at 95th percentile per API call.

Scale

To get an accurate measure of quality, the reliability measure above will be reviewed for the trailing 10,000 data points per requirement.

Supporting services

Article • 07/06/2022

The following services are used internally within the Microsoft environment to support coauthoring provided as part of the Cloud Storage Partner Program (CSPP) Plus program tier.

Office Collaboration Service (OCS)

The Office Collaboration Service (OCS) is part of CSPP Plus when two or more users are collaborating on a document at the same time. OCS isn't required for coauthoring.

OCS improves coauthoring performance, reduces load on the host by allowing clients to exchange coauthoring edits quickly amongst themselves, and greatly increases the number of supported active coauthors. OCS can also merge changes from multiple users, as needed. OCS saves these accumulated edits to the host on a regular basis.

Clients initially connect to the host directly, then transition to OCS when real-time collaboration is needed. In some cases, clients can transition back out of OCS into host-mode collaboration. Most document editing doesn't start an OCS session and the use of the OCS service is transparent to both end-users and hosts.

OCS is a part of CSPP Plus for Office for Desktop, Office for the web, and Microsoft 365 for mobile.

OCS session start

An OCS *session*, which is an internal construct within the OCS service, keeps active coauthoring document state on behalf of one or more end-users.

A session starts in the following scenarios:

- A second client joins the document. This could be another user, the same user editing the same document on a different device, or, if on the web, a different browser tab. Additional clients that open the document can connect to the already established OCS session.
- Local internet connectivity has recently been restored.
- Five minutes have elapsed since the last OCS session failure.

Applications can disable OCS transitions in certain scenarios where they can't support OCS coauthoring. For example, Word doesn't support OCS for .docm documents. In

these cases, applications would continue to save their changes to the host directly.

Client transition out of OCS

Clients can transition out of an OCS session for the following reasons:

- The OCS session has ended due to a fatal service error.
- The client's transition to OCS has failed.
- All other users have left the document and five minutes have elapsed.
- OCS incompatible content was added to the document.
- The user has lost permissions to the document or the user's access token for the document is about to expire.

Real Time Channel (RTC) service

The Real Time Channel (RTC) service provides real-time peer-to-peer connectivity between clients participating in an Office document coauthoring session.

The RTC service supports the following:

- Accelerated transfer of coauthoring changes and typing between clients.
- Information about presence in the document by other users, such as cursor location, display of users in the *Coauth Gallery*, and so on.

CSPP UI Guidelines

Article • 01/29/2025

Important

These guidelines are not exhaustive. Hosts are expected to follow the terms of the Microsoft Cloud Storage Partner Program Integration Terms with respect to Microsoft 365 for the web integration.

Requirements

The following requirements apply to an Microsoft 365 for the web integration.

1. **Don't display any UI on or around the Microsoft 365 editor.** The Microsoft 365 for the web editors must always be displayed 'edge-to-edge', with no surrounding UI. The editor cannot be 'light-boxed' or integrated as a component in host UI. The editor is a standalone application.

The Microsoft 365 for the web viewers can be 'light-boxed' or otherwise embedded in your web experience. However, if the user transitions to a Word, Excel, or PowerPoint editor, the editor must be edge-to-edge.

Embedding the Microsoft 365 for the web viewers or editors into native applications using a webview is not allowed.

Note

Refer to the support article titled [Which browsers work with Microsoft 365 for the web and Microsoft 365 Add-ins](#)  for more information on browser support for Microsoft 365 for the web across platforms.

2. **Use Microsoft-provided application and file type icons.** For more information, see [Application and file type icons](#).
3. **Provide favicons for the Microsoft 365 applications.** Whenever the editor or the viewer are displayed full-window or full-tab, respectively, the favicon for the page must be set to the appropriate favicon. The preferred method is to use the URLs provided in WOPI discovery. For more information, see [Favicon URLs](#).

4. **Use Microsoft 365 application names in UI that activates Microsoft 365.** For example, if you have UI in your application that reads, `Open`, this UI would read `Open in PowerPoint for the web`, `Open in Microsoft 365 for the web`, Or `Open in Microsoft 365 in a browser`. For more information, see [here](#) .
5. **Provide breadcrumb and breadcrumb URL values.** Breadcrumbs help users understand where their document is, as well as how to get back to where they were. For more information, see [Breadcrumbs](#).

Other recommendations

While the following guidelines are not required, Microsoft 365 for the web strongly recommends partners do the following:

1. **Provide support for sharing within Microsoft 365.** Microsoft 365 for the web provides a mechanism to share documents with other users directly within the Microsoft 365 for the web applications. You should take advantage of this capability so that users can access sharing controls directly within Microsoft 365 for the web. For more information, see [FileSharingPostMessage](#) and [FileSharingUrl](#).
2. **Provide an in-app Edit in Browser button.** If you are using the Microsoft 365 for the web viewer and the current user has permissions to edit the document, you should provide a [HostEditUrl](#) so that the `Edit in Browser` button is always displayed. This helps provide a more seamless transition for users.

Note

You can also use the [EditModePostMessage](#) property to receive a `PostMessage` when the `Edit in Browser` button is clicked, so you can handle the transition to edit mode yourself.

3. **Include the document name in the HTML title tag so it displays in the browser tab/window text.** This is especially important in cases where users may open multiple documents in different browser tabs/windows.

Application and file type icons

As part of the Cloud Storage Partner Program, Microsoft provides a branding toolkit that includes proper file type and application icons in various sizes, as well as vector-based image formats for re-sizing.

You can find the branding toolkit in the in the *Network Resources* section of the Office 365 Cloud Storage Partner Program Yammer group.

Important

We update product icons from time-to-time and you are responsible for keeping your WOPI solution up-to-date. Check the branding toolkit on a regular basis to ensure your solution remains compliant with the latest CSPP UI requirements.

Use the icons as follows:

1. When displaying an Microsoft file, either individually or as part of a list of files, use the file type icons. Don't use the application (product) icons for this purpose.
2. When displaying a button or other UI element that opens an Microsoft 365 for the web application, use the application icons. For example, if you display an `Open in Word on the web` button, use the Word application icon.

Important

If you re-size or otherwise modify the provided icons, you must use the vector source files to maintain the high image quality of the icons.

Breadcrumbs

Breadcrumbs are an important navigational tool for users. They dramatically improve the user experience by providing helpful anchors so users can both understand where the document they're working on is located and more easily navigate in and out of the Microsoft 365 for the web applications.

WOPI supports [two levels of breadcrumbs](#) only:

BreadcrumbBrandName/BreadcrumbBrandUrl

Set these properties to the root of your navigational hierarchy. A basic rule of thumb is that clicking the breadcrumb takes the user to their logical *home* within your WOPI host.

In some cases, you might have several different siloed hierarchies within your application. In such cases, it might make more sense to set these properties to the root of the particular hierarchy in which the current document is located.

Ultimately, pick a location most appropriate for your users and application structure.

BreadcrumbFolderName/BreadcrumbFolderUrl

Set these properties to the container in which the current document is located. A basic rule of thumb is that clicking this breadcrumb takes the user back to the same location they were in prior to opening the document.

Tip

If you support multiple paths to get to a file, you might wish to expose different breadcrumb properties depending on how the user navigated to the file. You can do this by using the [session context](#) to customize your [CheckFileInfo](#) response.

Example

Consider a logical hierarchy like this:

```
text

Documents
├── Reviews
│   ├── Data
│   │   ├── Aggregate Data.xlsx
│   │   └── Raw Data.xlsx
│   └── Monthly Review.pptx
└── Deals
    ├── Integration Plans.docx
    └── Leads.xlsx
```

In this case, if the user opens `Aggregate Data.xlsx`, `BreadcrumbBrandName/BreadcrumbBrandUrl` should be set to `Documents`, while the `BreadcrumbFolderName/BreadcrumbFolderUrl` should be set to `Data`.

Similarly, if the user opens `Integration Plans.docx`, `BreadcrumbBrandName/BreadcrumbBrandUrl` should be set to `Documents`, while the `BreadcrumbFolderName/BreadcrumbFolderUrl` should be set to `Deals`.

WOPI discovery

Article • 01/29/2025

WOPI discovery is the process where a WOPI host identifies Microsoft 365 for the web capabilities and how to set up Microsoft 365 for the web applications within a site. WOPI hosts use the discovery XML to determine how to interact with Microsoft 365 for the web. The WOPI host should cache the data in the discovery XML. Although this XML doesn't change often, we recommend that you issue a request for the XML periodically to ensure you always have the latest version. 12-24 hours is a good cadence to refresh, although in practice it's updated much less frequently.

A more dynamic option is to re-run discovery when [proof key validation](#) fails, or when it succeeds using the old key. That implies that the keys have been rotated, so discovery should definitely be re-run to get the new public key.

Lastly, another option is to run discovery whenever one of your machines restart. All these approaches, as well as combinations of them, have been used by hosts in the past. Whichever approach makes the most sense depends on your infrastructure.

Important

Hosts shouldn't rely on the [Expires](#)  HTTP header on the WOPI discovery URL to know when to re-run WOPI discovery. While this might change in the future, currently, the value in the [Expires](#)  header isn't appropriate for this purpose.

Tip

For more information on the URLs to retrieve discovery XML for Microsoft 365 for the web test and production environments, see [WOPI discovery URLs](#).

WOPI discovery actions

The **action** element and its attributes in the discovery XML provides some important information about Microsoft 365 for the web.

 Expand table

Element or attribute	Description
action element	Represents: • Operations that you can do on an Microsoft 365 document. • The file formats (in the form of file extensions) that are supported for the action.
requires attribute	The WOPI REST endpoints that are required to use the actions.
urlsrc attribute	The URI that you use to invoke the action on a particular file.

The following example shows an **action** element in the Microsoft 365 for the web discovery XML:

XML

```
<action name="edit" ext="docx" requires="locks,update"
  urlsrc="https://word-
edit.officeapps.live.com/we/wordeditorframe.aspx?
  <ui=UI_LLCC&><rs=DC_LLCC&><showpagestats=PERFSTATS&>" />
```

This example defines an action called `edit` that's supported for `docx` files. The edit action requires the `locks` and `update` capabilities. To invoke the edit action on a file, navigate to the URI included in the `urlsrc` attribute. You must parse the `urlsrc` value and add some parameters. For a full description of this process, see [Action URLs](#).

ⓘ Note

Some actions aren't supported within the Microsoft 365 - Cloud Storage Partner Program. These operations are marked  Not supported in the CSPP.

WOPI actions

All WOPI actions require hosts to implement [CheckFileInfo](#) and [GetFile](#).

view

An action that renders a non-editable view of a document.

edit

An action that lets users edit a document.

Requires: `update`, `locks`

editnew

An action that creates a new document using a blank file template appropriate to the file type, then opens that file for editing in Microsoft 365 for the web.

Requires: `update`, `locks`

convert

An action that converts a document in a binary format, such as `doc`, into a modern format, like `docx`, so that it can be edited in Microsoft 365 for the web. For more information about this action, see [Editing binary document formats](#).

Requires: `update`, `locks`

getinfo

An action that returns a set of URLs for running automated test cases. This action is only used by the [WOPI Validator](#) and is meant to be used in an [automated fashion](#).

interactivepreview



Not supported in the CSPP

An action that provides an interactive preview of the file type.

mobileView

An action that renders a non-editable view of a document that's optimized for viewing on mobile devices such as smartphones.

Tip

Microsoft 365 for the web automatically redirects `view` to `mobileView` when needed, so typically hosts don't need to use this action directly.

embedview

An action that renders a non-editable view of a document that's optimized for embedding in a web page.

imagepreview()



Not supported in the CSPP

An action that provides a static image preview of the file type.

formsubmit

An action that supports accepting changes to the file type via a form-style interface. For example, a user might be able to use this action to change the content of a workbook even if they didn't have permission to use the `edit` action.

formedit

An action that supports editing the file type in a mode better suited to working with files that have been used to collect form data via the `formsubmit` action.

rest

An action that supports interacting with the file type via additional URL parameters that are specific to the file type in question.

present



Not supported in the CSPP

An action that presents a [broadcast](#) of a document.

presentservice



Not supported in the CSPP

This action provides the location of a [broadcast](#) endpoint for broadcast presenters. Interaction with the endpoint is described in [\[MS-OBPRS\]](#).

attend

 Not supported in the CSPP

An action that attends a [broadcast](#) of a document.

attendservice

 Not supported in the CSPP

This action provides the location of a [broadcast](#) endpoint for broadcast attendees. Interaction with the endpoint is described in [\[MS-OBPAS\]](#) [↗](#).

preloadedit

An action to [preload static content](#) for Microsoft 365 for the web edit applications.

preloadview

An action to [preload static content](#) for Microsoft 365 for the web view applications.

syndicate

 Not supported in the CSPP

legacywebservice

 Not supported in the CSPP

rtc

 Not supported in the CSPP

collab

 Not supported in the CSPP

documentchat

 Not supported in the CSPP

Action requirements

The WOPI protocol exposes a number of different REST endpoints and operations that you can perform via those endpoints. You don't have to implement all of these for all actions. Actions define their requirements as part of the discovery XML. The requirements themselves are groups of WOPI operations that must be supported in order for the action to work.

update

Requires: [PutFile](#), [PutRelativeFile](#)

locks

Requires: [Lock](#), [RefreshLock](#), [Unlock](#), [UnlockAndRelock](#)

Action URLs

The URI values provided in the `urlsrc` attribute in the discovery XML aren't in a valid format. Simply navigating to them results in errors. A WOPI host must transform the URIs provided in order to make them valid action URLs that you can use to invoke actions on a file. To transform the `urlsrc` attribute into a proper action URL, the host must parse and replace [Placeholder values](#) with appropriate values or discard them completely.

After the URL is transformed, it's a valid URL. When the URL is opened, the action invokes against the file indicated by the [WOPISrc](#).

Transforming the urlsrc parameter

Some WOPI actions expose parameters that hosts can use to customize the behavior of the Microsoft 365 for the web application. For example, most actions support optional query string parameters that tell Microsoft 365 for the web what language to render the application UI in.

These parameters are exposed in the `urlsrc` attribute in the discovery XML. Each of these optional parameters are contained within angle brackets (`<` and `>`), and conform to the pattern `<name=PLACEHOLDER_VALUE[&]>`, where `name` is the name of the query string parameter and `PLACEHOLDER_VALUE` is a value that can be replaced by the host. By convention all placeholder values in Office for the web action URIs are capitalized.

The list of all placeholder values used by Microsoft 365 for the web and what values are valid replacements for each placeholder are listed in the [Placeholder values](#) section.

The placeholders are replaced as follows:

- If the `PLACEHOLDER_VALUE` is unknown to the host, the entire parameter, including the angle brackets, is removed.
- Similarly, if the `PLACEHOLDER_VALUE` is known but the host wishes to ignore it or use the default value for that parameter, you should remove the entire parameter, including the angle brackets.
- If the `PLACEHOLDER_VALUE` is known, the angle brackets are removed, the `name` value is left intact, and the `PLACEHOLDER_VALUE` string is replaced with an appropriate value. If present, you must keep the optional `&`.

The following section contains a list of all current placeholder values that Microsoft 365 for the web exposes in its discovery XML.

Note

Office for the web can add new placeholders and actions at any time. Hosts must ignore - and thus remove from the URL per the instructions above - any placeholder values they don't explicitly understand.

Placeholder values

The following are the placeholder values.

BUSINESS_USER

You can set this value to `1` to indicate that the current user is a business user. This placeholder value must be used by hosts that support the business user flow. For more information, see [Supporting document editing for business users](#).

DC_LLCC

This value represents the language that Microsoft 365 for the web should use for the purposes of data calculation. For a list of currently supported languages, see [What languages does Office for the web support?](#)

In addition to the values provided in the Locale ID column, you can supply any language, provided it's in the format described in [RFC 1766](#). Typically, this value is the

same as the value provided for [UI_LLCC](#), and defaults to that value if not provided.

DISABLE_ASYNC

ⓘ Note

This value is used in the `attend` action only.

You can set this value to `true` to prevent a [broadcast](#) attendee from navigating a file independently.

DISABLE_BROADCAST

ⓘ Note

This value is used in [broadcast](#)-related actions only.

You can set this value to `true` to load a view of a document that doesn't start or join a [broadcast](#) session. This view looks and behaves like a regular broadcast frame.

DISABLE_CHAT

You can set this value to `1` to disable in-document chat functionality.

EMBEDDED

ⓘ Note

This value is used in [broadcast](#)-related actions only.

You can set this value to `true` to indicate that the output of the action is embedded in a web page.

FULLSCREEN

ⓘ Note

This value is used in [broadcast](#)-related actions only.

You can set this value to `true` to load the file type in full-screen mode.

HOST_SESSION_ID

This value is passed by hosts to associate an Microsoft 365 for the web session with a host session identifier. This helps Microsoft 365 for the web engineers more quickly find logs for troubleshooting purposes based on a host-specific session identifier.

RECORDING

ⓘ Note

This value is used in [broadcast](#)-related actions only.

You can set this value to `true` to load the file type with a minimal user interface.

SESSION_CONTEXT

You can replace this placeholder by any string value. If provided, this value is passed back to the host in subsequent [CheckFileInfo](#) and [CheckFolderInfo](#) calls in the **X-WOPI-SessionContext** request header. There is no defined limit for the length of this string; however, since it's passed on the query string, it's subject to the overall Microsoft 365 for the web URL length limit of 2048 bytes.

New in version 2018.12.15: Prior to this version, session context was supported but hosts were required to add it to the action URL manually using the `sc` query parameter. This placeholder enables hosts to handle session context in the same way as other URL parameters.

THEME_ID

ⓘ Note

This value is used in [broadcast](#)-related actions only.

You can set this value to either `1` or `2` to designate a specific user interface appearance. `1` denotes a light-colored theme and `2` denotes a darker theme.

UI_LLCC

This value represents the language the Microsoft 365 for the web application UI should use. Microsoft 365 for the web doesn't support all languages, and might use a substitute language if the language requested isn't supported. For a list of currently supported languages, see [What languages does Office for the web support?](#).

In addition to the values provided in the Locale ID column, you can supply any language, provided it's in the format described in [RFC 1766](#). If you don't set a value for this placeholder, Office for the web tries to use the browser language setting (`navigator.language`). If no valid language can be determined, Office for the web defaults to `en-US` (US English).

VALIDATOR_TEST_CATEGORY

ⓘ Note

This value runs the [WOPI Validator](#) in different modes.

You can set this value to `All`, `OfficeOnline` or `OfficeNativeClient` to activate the tests specific to Office for the web and Microsoft 365 for mobile. If omitted, the default value is `All`.

`All`: activates all [WOPI Validator](#) tests.

`OfficeOnline`: activates all tests necessary for Office for the web integration.

`OfficeNativeClient`: activates all tests necessary for Microsoft 365 for mobile integration.

WOPI_SOURCE

This placeholder must be replaced by a [WopiSrc](#) value.

ⓘ Important

Unlike other placeholders, replacing this placeholder is required.

New in version 2018.12.15: Prior to this version, hosts were required to add the [WopiSrc](#) to the action URL for most (but not all) actions. This placeholder enables hosts to handle the [WopiSrc](#) in the same way as other URL parameters.

Additional notes

Depending on the specific scenario where action URLs are invoked, there are additional relevant components to action URLs. Since action URLs are typically invoked from the host page, these are covered in the [Building a host page](#) section.

Favicon URLs

The discovery XML includes a URL to an appropriate [favicon](#) for all Microsoft 365 for the web applications in the `favIconUrl` attribute of the `app` element. For example:

XML

```
<app name="Excel"
  favIconUrl="https://excel.officeapps.live.com/x/_layouts/resources/FavIcon_Excel.ico"
  checkLicense="true">
  ...
</app>
```

Hosts should use this URL as the favicon for their host page, so that the appropriate application icon appears when Microsoft 365 for the web is used.

Microsoft 365 for the web environments

Article • 01/29/2025

Microsoft 365 for the web provides two environments that cloud storage partners can use. The [WOPI discovery URLs](#) for each environment are provided below.

Initially you're given access to the test environment (also called *Dogfood*), that you use when building and testing your integration.

Once you are ready to release your integration, you can start the [Launch process](#). As part of that process, your application is given access to the production environment.

Test environment

The test environment is updated frequently - usually at least once per day - and all initial testing should be done against it. Initially, this is the only environment you can access.

The test environment runs the most recent Microsoft 365 for the web code available. This means that you might see features there that aren't yet available in production. However, the WOPI interactions between Microsoft 365 for the web and your WOPI host should not differ dramatically between test and production.

Because builds are deployed frequently to this environment, you might see regressions in behavior. However, the deployment cadence supports pushing changes quickly, so contact Microsoft if you experience any strange or unexpected behavior when using the test environment.

Important

End users shouldn't access the test environment. If you're making Microsoft 365 for the web integration available to your end-users, you must be in the production environment.

Production environment

The production environment is updated weekly.

Primary requirements for production environment

ⓘ Note

This is not an exhaustive list.

- Hosts must use HTTPS in the production Microsoft 365 for the web environment. HTTP isn't supported.
- Domains on the [WOPI domain allow list](#) must be a WOPI-dedicated subdomain in production.
- Domains must be owned by the partner.
- Production changes can take **4-5 weeks** to fully roll out; this includes domain changes.

WOPI discovery URLs

 Expand table

Environment	Discovery URL
Production	https://onenote.officeapps.live.com/hosting/discovery 
Test/Dogfood	https://ffc-onenote.officeapps.live.com/hosting/discovery 

💡 Tip

These URLs are publicly accessible. However, you won't be able to invoke any [WOPI actions](#) successfully unless your WOPI domain is added to the [WOPI domain allow list](#).

Microsoft-configured settings

Article • 01/29/2025

Most Microsoft 365 for the web behavior is configurable based on properties provided in [CheckFileInfo](#). However, there are some Microsoft 365 for the web settings that must be changed by Microsoft.

Important

Changes to these settings require time to propagate.

Any changes to Microsoft-configured settings **will take 4-5 weeks to fully roll out**. This includes changes to the [WOPI domain allow list](#), and applies to both the production and test environments.

WOPI domain allow list

Important

Any domains added to the allow list **must be owned by the partner**. Microsoft doesn't permit domains associated with a partner that aren't owned and controlled by that partner.

Note

Only subdomains are allowed in the domain allow list. For example: 'test.contoso.com' is acceptable. '*.contoso.com' is not acceptable and will be rejected.

Microsoft 365 for the web only makes WOPI requests to trusted partner domains. This domain list is called the *WOPI domain allow list*. It contains entries of the form:

- `wopi.contoso.com`
- `qa-wopi.contoso.com`

Entries in the WOPI domain allow list are implicitly wild-carded. In other words, the entry `wopi.contoso.com` is equivalent to `*.wopi.contoso.com`. This entry allows WOPI requests to be made to any subdomain under `wopi.contoso.com`.

Tip

If you ever generate [WopiSrc](#) values that point to a subdomain, it needs to be on the allow list. The [WopiSrc](#) represents the domain that a WOPI client uses to run WOPI operations against.

If you don't ever generate [WopiSrc](#) values that point to a subdomain, that subdomain doesn't need to be on the WOPI domain allow list (but it might need to be on the [Redirect domain allow list](#)).

Test Environment

Microsoft 365 for the web has different allow lists for the production and test environments. When you're first given access to the test environment, Microsoft adds the domains you provide to the test-only WOPI domain allow list.

In the Microsoft 365 for the web test environment, hosts must use a WOPI-dedicated subdomain for handling WOPI traffic. This subdomain is typically something like `wopi-test.hostdomain.com`, though that's just a name convention and hosts can use other names if needed. This approach ensures that Microsoft 365 for the web can't make WOPI requests to arbitrary domains.

Note

The [domain\(s\) configured](#) for your Test environment can't match the ones you will be using in Production

For testing and development using the Microsoft 365 for the web test environment, a WOPI-dedicated subdomain isn't required.

Production Environment

In the Microsoft 365 for the web production environment, hosts must use a WOPI-dedicated subdomain for handling WOPI traffic. This subdomain is typically something like `wopi.hostdomain.com`, though that's just a name convention and hosts can use other names if needed. This approach ensures that Microsoft 365 for the web can't make WOPI requests to arbitrary domains.

Warning

A production WOPI subdomain shouldn't ever surface user-provided content. In other words, a user shouldn't be able to upload something to the host and trick Microsoft 365 for the web into making WOPI requests to the user-controlled content. That would represent a potential security hole.

For example, consider a service that uses the URL `https://www.contosodrive.com` for their main website. Users can upload and control content that's served out of the `www.contosodrive.com` domain. If the Microsoft 365 for the web allow list contains `contosodrive.com`, then a nefarious user could upload content and then create a **WOPISrc** pointing to it, like this: `?`

`WOPISrc=https://www.contosodrive.com/my-content/wopi/files/attack.json`. They could then provide an arbitrary CheckFile and possibly GetFile response by using the **FileUrl** property. This means that an attacker can abuse the trust between the Microsoft 365 for the web service and the host. In one possible example, the attacker could change links in the Microsoft 365 for the web UI like the button controlled by the **FileSharingUrl** property to lead to malicious sites.

This threat is mitigated by requiring a dedicated subdomain for WOPI traffic that's separate from the domain used when serving user content.

Tip

All domains on the production allow list are automatically allowed in the test environment. The inverse isn't true.

You can request changes to your domain allow list by submitting an [Environment Change Request form](#). Any changes to Microsoft-configured settings **will take 4-5 weeks to fully roll out**. This applies to both the production and test environments.

'Saved to...' name

Microsoft 365 for the web displays a message in the bottom status bar when saving files.

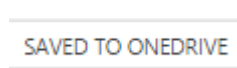


Figure 1 - The 'Saved to...' UI for OneDrive

By default, the name listed in this UI will match the Microsoft 365 for the web partner ID for your host. In most cases, this is the appropriate value. However, there may be cases where you wish to use a different name here than your partner ID. For example, you may

have a specific product brand that you want to display here such as 'ContosoDrive' instead of 'Contoso.' In that case, you can provide your preferred name to Microsoft.

Important

This value is not localized.

You can request changes to your 'Saved to...' name by submitting an [Environment Change Request form](#). Any changes to Microsoft-configured settings **will take 4-5 weeks to fully roll out**. This applies to both the production and test environments

Redirect domain allow list

Note

This setting is only relevant for hosts using the [business user flow](#).

When validating that a business user has an Microsoft 365 subscription, Microsoft 365 for the web navigates the user off of the host site so they can sign in to their Microsoft 365 account. Once the user has signed in, Microsoft 365 for the web will navigate back to the [HostEditUrl](#) provided by the host initially.

In order for that redirection to happen, the domain of the [HostEditUrl](#) must be on the redirect domain allow list. Like the WOPI domain allow list, entries on this list are implicitly wild-carded. The entries on this list might be different than those on the WOPI domain allow list.

Microsoft 365 for the web also uses the [DownloadUrl](#) as part of the [business user flow](#), and the domain for this URL must also be included on the redirect domain allow list.

You can request changes to your Redirect domain allow list by submitting an [Environment Change Request form](#). Any changes to Microsoft-configured settings **will take 4-5 weeks to fully roll out**. This applies to both the production and test environments

Homepage URL

Note

This setting is only relevant for hosts using the [business user flow](#).

As part of the [business user flow](#), Microsoft 365 for the web may determine that a user does not have the Microsoft 365 subscription necessary for editing documents using Microsoft 365 for the web. In this case Microsoft 365 for the web offers to redirect the user back to the host. However, navigating the user to the [HostEditUrl](#) does not make sense in this case since the user does not have the appropriate subscription to edit, so Microsoft 365 for the web will not load properly.

Thus, in this case Microsoft 365 for the web navigates the user to a URL determined by the host, called the Homepage URL. This is a single setting per WOPI host and must be changed by Microsoft.

In this scenario, Microsoft 365 for the web navigates the user to a URL determined by the host, called the Homepage URL. This is a single setting per WOPI host and must be changed by Microsoft.

WOPI set up requirements for Microsoft 365 for the web integration

Article • 01/29/2025

A WOPI host doesn't need to implement every WOPI operation. WOPI hosts express their capabilities using [properties in CheckFileInfo](#), such as [SupportsLocks](#). Also, WOPI actions specify the WOPI operations that must be supported in order to use that action, in the form of [Action requirements](#).

But practically speaking, there's a minimum set of operations required to support the two major Office for the web WOPI scenarios - viewing and editing.

Important

While the following lists cover the WOPI operations to be implemented, hosts must also provide [file IDs](#) and [access tokens](#) as part of a basic WOPI implementation.

View

In order to support viewing documents using Microsoft 365 for the web, WOPI hosts implement:

- [CheckFileInfo](#)
- [GetFile](#)

Important

[GetFile](#) must be implemented even if a host is using the [FileUrl](#) property.

Edit

In order to support editing documents using Microsoft 365 for the web, WOPI hosts implement:

- [CheckFileInfo](#)
- [GetFile](#)
- [PutFile](#)
- [PutRelativeFile](#)

- Lock
- Unlock
- UnlockAndRelock
- RefreshLock

OneNote

Article • 01/29/2025

OneNote for the web integration isn't included in the Microsoft 365 - Cloud Storage Partner Program.

Folders endpoint

The Folders endpoint provides access to folder-level operations.

OneNote Online

ⓘ Note

The operations exposed by this endpoint are only used by OneNote for the web and aren't needed to integrate with Microsoft 365 for the web. They're included here for completeness but don't need to be implemented.

OneNote for the web integration isn't included in the Microsoft 365 - Cloud Storage Partner Program.

The Folders endpoint provides access to folder-level operations. This endpoint exposes the [CheckFolderInfo](#) operation, which returns information about a folder, the permissions that the user has on that folder, and the capabilities that the WOPI host has on the folder.

Folder contents endpoint

OneNote for the web

ⓘ Note

The operations exposed by this endpoint are only used by OneNote for the web and aren't needed to integrate with Microsoft 365 for the web. They're included here for completeness but don't need to be implemented.

OneNote for the web integration isn't included in the Microsoft 365 - Cloud Storage Partner Program.

The Folder contents endpoint provides access to folder contents. This endpoint exposes the [EnumerateChildren \(folders\)](#) operation, which returns the contents of a folder on the WOPI server.

WOPI REST API Reference

Article • 02/10/2025

The Web Application Open Platform Interface Protocol (WOPI) defines a set of operations that enable clients to access and change files stored by a server. WOPI is a REST-based protocol. It defines a set of [REST endpoints](#) and operations that are run by issuing HTTP requests to those endpoints.

The WOPI protocol is used by Microsoft 365 applications, such as Microsoft 365 for the web, to view and edit files that are stored in a cloud service. Thus, Microsoft 365 for the web is a [WOPI client](#), while the cloud service that stores the files is a [WOPI server](#), often referred to as a *host*.

This documentation is a reference for all the defined WOPI operations. However, clients don't have to use every operation. WOPI is modular. A WOPI server need only run the operations required by the WOPI client(s) that the server intends to integrate with. To learn more about the specific requirements for integrating with Microsoft 365 for the web or Microsoft 365 for mobile, see the following sections:

- [Using the WOPI protocol to integrate with Microsoft 365 for the web](#)
- [Integrating with Microsoft 365 for mobile](#)

Standard WOPI request and response headers

Article • 02/27/2025

Following are the standard WOPI request and response headers.

GET /wopi/

All WOPI requests might contain the following request and response headers. Individual WOPI operations might send additional request headers or require additional response headers. These unique headers are described in the documentation for each WOPI operation.

Tip

HTTP header names are case-insensitive. For more information, see [RFC 7230#section-3.2](#).

Request headers

- [Authorization](#) – The **string** value `Bearer <token>` where `<token>` is the [access token](#) for the request.

Important

WOPI clients aren't required to pass the [access token](#) in the [Authorization](#) header, but they must send it as a URL parameter in all WOPI operations. So, for maximum compatibility, WOPI hosts should either use the URL parameter in all cases, or fall back to it if the [Authorization](#) header isn't included in the request.

- *X-Request-ID* – A **string** that the host should log when logging server activity to correlate that request with a specific WOPI call to the host.
- *X-WOPI-AppEndpoint* – A **string** that indicates the endpoint of the WOPI client sending the request. This is typically used to indicate geographic location, datacenter, and so on. This string mustn't be used for anything other than logging.

- *X-WOPI-RequestingApplication* – A **string** that indicates the WOPI client sending the request. This string mustn't be used for anything other than logging.
- *X-WOPI-ClientVersion* – A **string** that the host should log and which indicates the version of the WOPI client making the request. There's no standard for how this string is formatted, and it mustn't be used for anything other than logging.
- *X-WOPI-CorrelationId* – A **string** that the host should log when logging server activity to correlate that activity with WOPI client activity.

Tip

For more information on how this ID is used in Microsoft 365 for the web, see [Troubleshooting interactions with Microsoft 365 for the web](#).

- *X-WOPI-DeviceId* – A **string** that the host should log and which indicates the ID of the device making the request. This string mustn't be used for anything other than logging.
- *X-WOPI-SessionId* – A **string** that the host should log to correlate WOPI client activity within a session. This string mustn't be used for anything other than logging.
- *X-WOPI-MachineName* – A **string** indicating the name of the WOPI client machine making the request. This string mustn't be used for anything other than logging.
- *X-WOPI-PerfTraceRequested* – This header is reserved for future use.
- *X-WOPI-Proof* – A **string** representing data signed using a SHA256 (A 256 bit SHA-2-encoded [FIPS 180-2 [↗](#)]) encryption algorithm. For more information on the use of this header value, see [Verifying that requests originate from Microsoft 365 for the web](#).
- *X-WOPI-ProofOld* – A **string** representing data signed using a SHA256 (A 256 bit SHA-2-encoded [FIPS 180-2 [↗](#)]) encryption algorithm. For more information on the use of this header value, see [Verifying that requests originate from Microsoft 365 for the web](#).
- *X-WOPI-TimeStamp* – A **64-bit integer** that represents the number of 100-nanosecond intervals that have elapsed between 12:00:00 midnight, January 1, 0001, and the time of the request. You can set this value in .NET using the following C# code: `DateTime.UtcNow.Ticks`.

For more information, see [DateTime.Ticks Property \[↗\]\(#\)](#).

Response headers

- [Content-Type](#) – This header should be set to a value appropriate to the type of data being included in the response. For example, when responding to a WOPI request with JSON-encoded data in the response body, the [Content-Type](#) header should be set to `application/json`. WOPI clients might ignore a response with a [Content-Type](#) header that doesn't match the expected type.
- *X-WOPI-HostEndpoint* – A **string** that indicates the endpoint of the WOPI host handling the request. This is analogous to the **X-WOPI-AppEndpoint** request header and typically indicates geographic location, datacenter, and so on. This string mustn't be used for anything other than logging.
- *X-WOPI-MachineName* – A **string** indicating the name of the WOPI host server handling the request. This string mustn't be used for anything other than logging.
- *X-WOPI-PerfTrace* – This header is reserved for future use.
- *X-WOPI-ServerError* – A **string** indicating that an error occurred while processing the WOPI request. This header should be included in a WOPI response if the status code is [500 Internal Server Error](#), but might be returned on any response with a non-200 status code. The value should contain details about the error. This string mustn't be used for anything other than logging.
- *X-WOPI-ServerVersion* – A **string** indicating the version of the WOPI host server handling the request. There's no standard for how this string is formatted, and it must not be used for anything other than logging.

WOPI REST endpoints

Article • 06/02/2022

A WOPI host needs to provide some information about the files it stores, as well as the binary contents of those files. Because WOPI is a REST-based callback interface, this information is provided via specific URLs. A WOPI host provides a small REST API around its files. WOPI clients such as Microsoft 365 for the web then use those REST API to work with the files.

Not all operations and endpoints are required. All actions require the [CheckFileInfo](#) and [GetFile](#) operations.

Each WOPI endpoint supports a number of operations specified by the caller in the **X-WOPI-Override** request header. Parameters for each operation are also passed in HTTP headers, which all begin with `X-WOPI-`. This way, executing a WOPI operation is as simple as issuing a request to the appropriate REST endpoint and passing appropriate HTTP header values with the request.

ⓘ Note

[Standard WOPI request and response headers](#)

Endpoint URLs

All WOPI host endpoints must be located at a URL that *starts* with `/wopi`. For example, all of the following URLs are **valid** URLs for the [File contents endpoint](#) for a file with the ID `abc123`:

- `https://api.contoso.com/modules/wopi/files/abc123/contents`
- `https://test.wopi.contoso.com/wopi_test/files/abc123/contents`

However, the following URLs are **not valid**:

- `https://api.contoso.com/api_wopi/files/abc123/contents` (invalid because the endpoint URL does not *start* with `/wopi`)
- `https://api.contoso.com/files/abc123/contents` (invalid because the endpoint URL does not *start* with `/wopi`)
- `https://test.wopi.contoso.com/officeonline/files/abc123/contents` (invalid because the endpoint URL does not *start* with `/wopi`)

- `https://wopi.contoso.com/wopi/files/ids/abc123/contents` (invalid because the endpoint URL contains `/ids`, which is not permitted)

Files endpoint

The Files endpoint provides access to file-level operations.

URL

`/wopi/files/(file_id)`

The following operations are exposed through this endpoint:

- [CheckFileInfo](#)
- [PutRelativeFile](#)
- [Lock](#)
- [Unlock](#)
- [RefreshLock](#)
- [UnlockAndRelock](#)
- [DeleteFile](#)
- [RenameFile](#)

File contents endpoint

The File contents endpoint provides access to retrieve and update the contents of a file.

URL

`/wopi/files/(file_id)/contents`

The following operations are exposed through this endpoint:

- [GetFile](#)
- [PutFile](#)

Containers endpoint

URL

`/wopi/containers/(container_id)`

The following operations are exposed through this endpoint:

- [CheckContainerInfo](#)
- [CreateChildContainer](#)
- [CreateChildFile](#)
- [DeleteContainer](#)
- [EnumerateAncestors \(files\)](#)
- [EnumerateChildren \(containers\)](#)
- [RenameContainer](#)

Ecosystem endpoint

The Ecosystem endpoint serves as a bridge for WOPI clients that do not have a File or Container ID that they are operating on.

URL

```
/wopi/ecosystem
```

The following operations are exposed through this endpoint:

- [CheckEcosystem](#)
-  [GetFileWopiSrc \(ecosystem\)](#)
- [GetRootContainer \(ecosystem\)](#)

Bootstrapper endpoint

The following operations are exposed through this endpoint:

- [Bootstrap](#)
- [GetNewAccessToken](#)
- [Shortcut operations](#)

Security considerations

Article • 07/06/2022

The security considerations for WOPI access tokens follow.

Preventing token trading

Some WOPI operations, such as [GetEcosystem \(files\)](#), [EnumerateAncestors \(files\)](#), and [EnumerateChildren \(containers\)](#), return URLs that include WOPI [access tokens](#). WOPI access tokens must always be treated as per-resource, per-user by a WOPI client, and they should always expire after a period of time. WOPI deliberately doesn't define a means for a client to refresh a WOPI access token given another WOPI access token.

But WOPI is also deliberately designed to support navigation from a file or container to the [Ecosystem endpoint](#), then back to a container or file. This means that it's possible for a client to *refresh* their WOPI tokens indefinitely unless the WOPI host is careful when issuing new access tokens to mitigate the threats. We refer to this threat as *token trading*.

To illustrate the threat of token trading, consider the following scenario:

1. A WOPI client, on behalf of User A, is issued a WOPI access token, `TOKEN1`, for the file `Document.docx`. The token has a lifetime of 12 hours.
2. While `TOKEN1` is still valid, the client calls the [EnumerateAncestors \(files\)](#) operation using `TOKEN1` as the access token.
3. The server responds with a URL to the parent container along with a new WOPI access token for that container, `TOKEN2`.
4. The client then calls [EnumerateChildren \(containers\)](#) using `TOKEN2` as the access token.
5. The server responds with the URL for the children files, including `Document.docx`, along with a new access token for `Document.docx`, `TOKEN3`.

At this point, the client has effectively traded `TOKEN1` for `TOKEN3`. If each token issued has a lifetime of 12 hours, this means a client can effectively refresh their access token for a particular file by going through the container hierarchy, without actually authenticating with the server using a non-WOPI authentication mechanism.

If unmitigated, this scenario greatly increases the potential damage caused by token leakage. If a malicious attacker gains access to `TOKEN1`, they can potentially access

`Document.docx` indefinitely, as long as User A still has access to it. In other words, the attacker can impersonate User A indefinitely.

In addition, an attacker could use [EnumerateAncestors \(files\)](#) and [EnumerateChildren \(containers\)](#) to trade `TOKEN1` for a token that's valid for any other document in the container hierarchy that the user has access to, then impersonate User A indefinitely to access those other documents and containers as well.

Mitigation

The preferred way to mitigate this threat is to create new WOPI access tokens with the same token lifetime as the WOPI access token that was used when the operation was called. In other words, in the scenario described above, the lifetime of `TOKEN2` and `TOKEN3` should be the same as `TOKEN1`. All three tokens should expire at the same time.

This should only apply when a WOPI access token is used to create another WOPI access token. In other cases where WOPI access tokens are issued, such as from [Bootstrap](#) or other OAuth-authenticated [Shortcut operations](#), or, in the case of web-based WOPI clients, by visiting a host page that issues the access token, the host-defined default access token lifetime can apply. This is safe in those cases because there's a separate primary authentication method that's also in use. In the case of [Bootstrap](#), it's an OAuth token. In the case of web-based WOPI clients, it's the host's standard web authentication system.

Operations only used by OneNote Online

Article • 02/10/2025

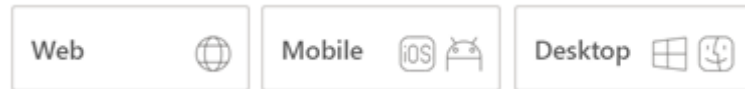
The following operations are only used by OneNote for the web and aren't required to integrate with Microsoft 365 for the web or Microsoft 365 for mobile. These are included for completeness but don't need to be implemented.

OneNote for the web integration isn't included in the Microsoft 365 - Cloud Storage Partner Program.

- CheckFolderInfo
- EnumerateChildren (folders)
- ExecuteCellStorageRelativeRequest
- ExecuteCellStorageRequest

CheckFileInfo

Article • 03/04/2023



The `CheckFileInfo` operation returns information about a file, a user's permissions on that file, and general information about the capabilities that the WOPI host has on the file.

GET /wopi/files/(file_id)

The `CheckFileInfo` operation is one of the most important WOPI operations.

`CheckFileInfo` must be implemented for all WOPI actions. This operation returns information about a file, a user's permissions on that file, and general information about the capabilities that the WOPI host has on the file. Also, some `CheckFileInfo` properties can influence the appearance and behavior of WOPI clients.

Tip

Some properties aren't supported within the Microsoft 365 - Cloud Storage Partner Program. These operations are marked  `Not supported in the CSPP`.

Parameters

- `file_id` (*string*) – A string that specifies a [file ID](#) of a file managed by the host. This string must be URL safe.

Query parameters

- `access_token` (*string*) – An [access token](#) that the host uses to determine whether the request is authorized.

Request headers

- `X-WOPI-SessionContext` – The value of the [session context](#), if provided on the initial WOPI action URL.

Status codes

- [200 OK](#) [↗] – Success.
- [401 Unauthorized](#) [↗] – Invalid [access token](#).
- [404 Not Found](#) [↗] – Resource not found or user unauthorized.
- [500 Internal Server Error](#) [↗] – Server error.

In addition to the request and response headers listed here, this operation might also use the Standard WOPI request and response headers. For more information see [Standard WOPI request and response headers](#).

Response properties

`CheckFileInfo` response properties are a key way that a WOPI host communicates capabilities and expected behaviors to WOPI clients. Many of the properties in the `CheckFileInfo` response are required. Even properties that are optional provide important ways for the host to direct the end-user client experience.

[Response property reference](#)

PostMessage properties for web-based WOPI clients

`CheckFileInfo` supports a number of properties that web-based WOPI clients can use, such as Microsoft 365 for the web to customize the user interface and experience when using those clients. For more information on these properties and how to use them, see [PostMessage properties](#).

CheckFileInfo properties for CSPP Plus

CSPP Plus extends `CheckFileInfo` with additional properties intended to support WOPI+ hosts.

For more information, see [CSPP Plus CheckFileInfo properties](#).

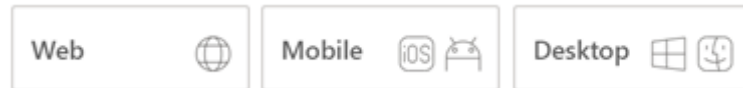
Other Properties

These are additional properties supported by `CheckFileInfo` but aren't central to the operation of a WOPI host. Properties that are pre-release and not supported by any WOPI client or have been previously deprecated are also listed here.

For more information on other supported properties, see [Other CheckFileInfo property reference](#).

CheckFileInfo - Response properties

Article • 03/09/2023



The response to a `CheckFileInfo` call is [JSON](#) containing a number of properties, most of which are optional.

All optional values default to the following values based on their type:

Type	Default value
Boolean	<code>false</code>
String	The empty string
Integer/Long	Varies; see individual properties for details
Array	Empty array

ⓘ Important

No properties should be set to `null`. If you don't wish to set a property, omit it from the response and WOPI clients will use the default value.

Required response properties

The following properties must be present in all `CheckFileInfo` responses:

BaseFileName

The **string** name of the file, including extension, without a path. Used for display in user interface (UI), and determining the extension of the file.

OwnerId

A **string** that uniquely identifies the owner of the file. In most cases, the user who uploaded or created the file is considered the owner.

ⓘ Important

This ID is subject to uniqueness and consistency requirements. For more information, see [Requirements for user identity properties](#).

Size

The size of the file in bytes, expressed as a **long**, a 64-bit signed integer.

UserId

A **string** value uniquely identifying the user currently accessing the file.

Important

This ID is subject to uniqueness and consistency requirements. See [Requirements for user identity properties](#) for more information.

Version

The current version of the file based on the server's file version schema, as a **string**. This value must change when the file changes, and version values must never repeat for a given file.

Important

This value must be a **string**, even if numbers are used to represent versions.

Requirements for user identity properties

Hosts use the [OwnerId](#) and [UserId](#) properties to provide user ID data to WOPI clients. User identity properties are intended for telemetry purposes, and thus should not be shown in any WOPI client UI. These properties must meet the following requirements:

- Unique to a single user.
- Consistent over time. For example, if a particular user uses a WOPI client to view a document on Monday, then returns and views another document on Tuesday, the value of the user identity properties should match.

Important

Hosts must only use alphanumeric (A-Z, a-z, and 0-9) characters in user identity properties.

WOPI host capabilities properties

The WOPI host capabilities properties indicate to the WOPI client what WOPI capabilities that the host supports for a file. All of these properties are optional and thus default to `false`; hosts should set them to `true` if their WOPI implementation meets the requirements for a particular property.

Important

If a WOPI server sets any capabilities properties to `true`, WOPI clients assume that all of the operations represented by that capability property are supported. So, a WOPI host must implement all operations represented by that capability property if they set the property to `true`.

SupportedShareUrlTypes

An **array** of strings containing the [Share URL](#) types supported by the host.

These types are passed in the **X-WOPI-UrlType** request header to signify which Share URL type to return for the [GetShareUrl \(files\)](#) operation.

<i>Possible Values</i>	<i>Description</i>
ReadOnly	This type of Share URL allows users to view the file using the URL, but does not give them permission to edit the file.
ReadWrite	This type of Share URL allows users to both view and edit the file using the URL.

SupportsCobalt

A **Boolean** value that indicates that the host supports the following WOPI operations:

- [ExecuteCellStorageRequest](#)
- [ExecuteCellStorageRelativeRequest](#)

SupportsContainers

A **Boolean** value that indicates that the host supports the following WOPI operations:

- [CheckContainerInfo](#)
- [CreateChildContainer](#)
- [CreateChildFile](#)
- [DeleteContainer](#)
- [DeleteFile](#)
- [EnumerateAncestors \(containers\)](#)
- [EnumerateAncestors \(files\)](#)
- [EnumerateChildren \(containers\)](#)
- [GetEcosystem \(containers\)](#)
- [RenameContainer](#)

Tip

SupportsContainers is a superset of **SupportsDeleteFile**. However, WOPI hosts should explicitly return `true` for both properties if SupportsContainers is `true`.

SupportsDeleteFile

A **Boolean** value that indicates that the host supports the [DeleteFile](#) operation.

SupportsEcosystem

A **Boolean** value that indicates that the host supports the following WOPI operations:

- [CheckEcosystem](#)
- [GetEcosystem \(containers\)](#)
- [GetEcosystem \(files\)](#)
- [GetRootContainer \(ecosystem\)](#)

SupportsExtendedLockLength

A **Boolean** value that indicates that the host supports lock IDs up to 1024 ASCII characters long. If not provided, WOPI clients will assume that lock IDs are limited to 256 ASCII characters.

❗ Important

While the 256 ASCII character lock length is currently sufficient, longer lock IDs will likely be required to support future scenarios, so we recommend hosts support extended lock lengths as soon as possible. See [lock ID lengths](#) for more information.

SupportsFolders

A **Boolean** value that indicates that the host supports the following WOPI operations:

- [CheckFolderInfo](#),
- [EnumerateChildren](#) (folders)
- [DeleteFile](#)

SupportsGetFileWopiSrc

A **Boolean** value that indicates that the host supports the  [GetFileWopiSrc](#) (ecosystem) operation.

SupportsGetLock

A **Boolean** value that indicates that the host supports the [GetLock](#) operation.

SupportsLocks

A **Boolean** value that indicates that the host supports the following WOPI operations:

- [Lock](#),
- [Unlock](#)
- [RefreshLock](#)
- [UnlockAndRelock](#) operations for this file.

SupportsRename

A **Boolean** value that indicates that the host supports the [RenameFile](#) operation.

SupportsUpdate

A **Boolean** value that indicates that the host supports the following WOPI operations:

- [PutFile](#)
- [PutRelativeFile](#)

SupportsUserInfo

A **Boolean** value that indicates that the host supports the [PutUserInfo](#) operation.

User metadata properties

The following properties are used to provide additional information about users.

IsAnonymousUser

A **Boolean** value indicating whether the user is authenticated with the host or not. Hosts should always set this to `true` for unauthenticated users, so that clients are aware that the user is anonymous.

When setting this to `true`, hosts can choose to omit the [UserId](#) property, but must still set the [OwnerId](#) property.

IsEduUser

A **Boolean** value indicating whether the user is an education user or not.

LicenseCheckForEditIsEnabled

A **Boolean** value indicating whether the user is a business user or not.

ⓘ Note

Supporting document editing for business users

UserFriendlyName

A **string** that is the name of the user, suitable for displaying in UI.

UserInfo

A **string** value containing information about the user. This string can be passed from a WOPI client to the host by means of a [PutUserInfo](#) operation. If the host has a UserInfo string for the user, they must include it in this property. See the [PutUserInfo](#) documentation for more details.

User permissions properties

A WOPI client must always assume that users have limited permissions to documents. If a host does not set the appropriate user permissions properties, users will not be able to perform operations such as editing documents using a WOPI client.

Ultimately, the host has final control over whether WOPI operations attempted by the client should succeed or fail based on the [access token](#) provided in the WOPI request. Thus, these properties do not act as an authorization mechanism. Rather, these properties help WOPI clients tailor their UI and behavior to the specific permissions a user has. For example, a WOPI client can hide file renaming UI if the [UserCanRename](#) property is `false`.

However, a WOPI client expects that even if that UI were somehow made available to a user without appropriate permissions, the WOPI [RenameFile](#) request would fail since the host would determine the action was not permissible based on the [access token](#) passed in the request.

Note that there is no property that indicates the user has permission to read/view a file. This is because WOPI requires the host to respond to any WOPI request, including [CheckFileInfo](#), with a [401 Unauthorized](#) [↗](#) or [404 Not Found](#) [↗](#) if the access token is invalid or expired.

ReadOnly

A **Boolean** value that indicates that, for this user, the file cannot be changed.

RestrictedWebViewOnly

A **Boolean** value that indicates that the WOPI client should restrict what actions the user can perform on the file. The behavior of this property is dependent on the WOPI client.

UserCanAttend

A **Boolean** value that indicates that the user has permission to view a [broadcast](#) of this file.

UserCanNotWriteRelative

A **Boolean** value that indicates the user does not have sufficient permission to create new files on the WOPI server. Setting this to `true` tells the WOPI client that calls to [PutRelativeFile](#) will fail for this user on the current file.

UserCanPresent

A **Boolean** value that indicates that the user has permission to [broadcast](#) this file to a set of users who have permission to broadcast or view a broadcast of the current file.

UserCanRename

A **Boolean** value that indicates the user has permission to rename the current file.

UserCanWrite

A **Boolean** value that indicates that the user has permission to alter the file. Setting this to `true` tells the WOPI client that it can call [PutFile](#) on behalf of the user.

File URL properties

Hosts can return a number of URLs for the file that WOPI clients may navigate to in various scenarios.

Tip

These properties can be used to customize the user experience of the Office for the web applications. See [Customize Office for the web with CheckFileInfo properties](#) for more information about how each property is used in Office for the web.

CloseUrl

A URI to a web page that the WOPI client should navigate to when the application closes, or in the event of an unrecoverable error.

DownloadUrl

A user-accessible URI to the file intended to allow the user to download a copy of the file.

📘 Important

This URI should directly download the file. In other words, WOPI clients expect that when directing users to this URL, the file will be immediately downloaded. This URL should not direct the user to some separate UI to download the file.

📘 Important

This URI should always provide the most recent version of the file.

FileEmbedCommandUrl

A URI to a location that allows the user to create an embeddable URI to the file.

FileSharingUrl

A URI to a location that allows the user to share the file.

FileUrl

A URI to the file location that the WOPI client uses to get the file. If this is provided, the WOPI client may use this URI to get the file instead of a [GetFile](#) request. A host might set this property if it is easier or provides better performance to serve files from a different domain than the one handling standard WOPI requests. WOPI clients must not add or remove parameters from the URL; no other parameters, including the [access token](#), should be appended to the FileUrl before it is used.

📘 Important

The `FileUrl` is meant as a performance enhancement. The [GetFile](#) operation must still be supported for the file even when the `FileUrl` property is provided.

ⓘ Note

Requests to the `FileUrl` can not be signed using **proof keys**. The `FileUrl` is used exactly as provided by the host, so it does not necessarily include the access token, which is required to construct the expected proof.

FileVersionUrl

A URI to a location that lets the user view the version history for the file.

HostEditUrl

A URI to a [host page](#) that loads the `edit` WOPI action.

HostEmbeddedViewUrl

A URI to a web page that provides access to a viewing experience for the file that can be embedded in another HTML page. This is typically a URI to a [host page](#) that loads the `embedview` WOPI action.

HostViewUrl

A URI to a [host page](#) that loads the view WOPI action. This URL is used by Office for the web to navigate between view and edit mode.

SignoutUrl

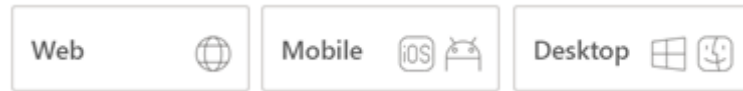
A URI that signs the current user out of the host's authentication system.

ⓘ Note

See also [SignInUrl](#)

CheckFileInfo - Other properties

Article • 03/09/2023



Miscellaneous Properties

AllowAdditionalMicrosoftServices

A **Boolean** value that indicates a WOPI client may connect to Microsoft services to provide end-user functionality.

In Office for the web, setting this property to `true` enables the following features:

- Bing spelling and proofing - Office for the web will use the [Bing Spell Check API](#) to provide better spelling and proofing suggestions for supported languages.
- Smart Lookup - Office for the web will use Bing to power the [Smart Lookup](#) feature, which provides quick access to definitions, Wiki articles, and top related searches from the web.

Additional features might be added in the future.

AllowErrorReportPrompt

A **Boolean** value that indicates that in the event of an error, the WOPI client is permitted to prompt the user for permission to collect a detailed report about their specific error. The information gathered could include the user's file and other session-specific state.

💡 Tip

This value should be omitted (or explicitly set to `false`) if no additional collection should be done on errors, or if the user has opted out of telemetry collection.

AllowExternalMarketplace

A **Boolean** value that indicates a WOPI client may allow connections to external services referenced in the file (for example, a marketplace of embeddable JavaScript apps).

ClientThrottlingProtection

A **string** value offering guidance to the WOPI client as to how to differentiate client throttling behaviors between the user and documents combinations from the WOPI host. Under times of stress, the WOPI client may choose to make use of this field to vary the level of impact of client side throttling behaviors within the set of active host documents. If the WOPI client chooses to differentiate throttling of client behaviors that are not necessarily tied to WOPI calls to the host, it may apply the most reduced quality of service to the LeastProtected document/users and the least reduced quality of service to the MostProtected documents/users. As in the case of [RequestedCallThrottling](#), it is advised that hosts sharing this value between responses for distinct users of the same document at any given time may yield more deterministic results from the clients.

<i>Possible Values</i>	<i>Description</i>
MostProtected	The most protected documents/users.
Protected	The documents/users that should be protected more than the average ones.
Normal	The documents/users with the standard level of protection from throttling.
LessProtected	The documents/users that can be throttled more heavily than a typical than the average ones.
LeastProtected	The least protected documents/users.

If no value is specified, or an invalid value is provided, the default behavior is to assume the WOPI host intended the value of None.

CloseButtonClosesWindow

A **Boolean** value that indicates the WOPI client should close the window or tab when the user activates any `Close` UI in the WOPI client.

Tip

This property may not behave as expected in Office for the web.

Note

Why should I avoid using the `CloseButtonClosesWindow` property in Office for the web?

CopyPasteRestrictions

ⓘ Note

Pre-release property - not yet used by any WOPI client

A **string** value indicating whether the WOPI client should disable Copy and Paste functionality within the application. The default is to permit all Copy and Paste functionality, i.e. the setting has no effect.

<i>Possible Values</i>	<i>Description</i>
BlockAll	Copy and Paste are completely disabled within the application.
CurrentDocumentOnly	Copy and Paste are enabled but content can only be copied and pasted within the file currently open in the application.

Any values other than those listed above must be ignored by the WOPI client.

💡 Tip

This property is only respected by Excel for the web. It has no effect in PowerPoint for the web or Word for the web.

DisablePrint

A **Boolean** value that indicates the WOPI client should disable all print functionality.

DisableTranslation

A **Boolean** value that indicates the WOPI client should disable all machine translation functionality.

FileExtension

A **string** value representing the file extension for the file. This value must begin with a .. If provided, WOPI clients will use this value as the file extension. Otherwise the extension will be parsed from the [BaseFileName](#).

Tip

While this property is not required, hosts should set it rather than relying on the [BaseFileName](#) parsing.

FileNameMaxLength

An **integer** value that indicates the maximum length for file names that the WOPI host supports, excluding the file extension. The default value is 250. Note that WOPI clients will use this default value if the property is omitted or if it is explicitly set to 0.

Tip

This property is optional; however, hosts wishing to enable file renaming within Office for the web should verify that the default value is appropriate and set it accordingly if not. See the [RenameFile](#) operation for more details.

LastModifiedTime

A **string** that represents the last time that the file was modified. This time must always be a must be a UTC time, and must be formatted in ISO 8601 round-trip format. For example, `"2009-06-15T13:45:30.000000Z"`.

RequestedCallThrottling

A **string** value indicating whether the WOPI host is experiencing capacity problems and would like to reduce the frequency at which the WOPI clients make calls to the host. Each WOPI application may choose how best to respect the expressed desire from the host. WOPI applications may respond in manners such as reducing the frequency of CheckFileInfo calls and extending the window between when a user makes a change and the updated document gets saved back to the WOPI host. It is advised that hosts sharing this value between responses for distinct users of the same document at any given time may yield more deterministic results from the clients.

<i>Possible Values</i>	<i>Description</i>
Normal	The WOPI host is healthy and does not want any additional request throttling.
Minor	The WOPI host is requesting a small amount of throttling from the WOPI client.
Medium	The WOPI host is requesting a medium amount of throttling from the WOPI client.
Major	The WOPI host is requesting a large amount of throttling from the WOPI client.
Critical	The WOPI host is requesting the WOPI client to apply the largest amount of throttling possible.

If no value is specified, or an invalid value is provided, the default behavior is to assume the WOPI host intended the value of None.

SHA256

A 256 bit SHA-2-encoded [\[FIPS 180-2\]](#) hash of the file contents, as a Base64-encoded string. Used for caching purposes in WOPI clients.

Tip

See **Optimizing document viewing for high volume** for more details on how this property is used in Office for the web.

SharingStatus

Note

Pre-release property - not yet used by any WOPI client

A **string** value indicating whether the current document is shared with other users. The value can change upon adding or removing permissions to other users. Clients should use this value to help decide when to enable collaboration features as a document must be Shared in order to multi-user collaboration on the document.

<i>Possible Values</i>	<i>Description</i>
Private	Only the document owner has permission to the file.

<i>Possible Values</i>	<i>Description</i>
Shared	At least one other user has access to the file via direct permissions or a sharing link.

If no value is specified, or an invalid value is provided, the default behavior is to assume the WOPI host intended the value of None.

TemporarilyNotWritable

A **Boolean** value that indicates that if host is temporarily unable to process writes on a file

UniqueContentId



Not supported in CSPP

In special cases, a host may choose to not provide a [SHA256](#), but still have some mechanism for identifying that two different files contain the same content in the same manner as the [SHA256](#) is used.

This string value can be provided rather than a [SHA256](#) value if and only if the host can guarantee that two different files with the same content will have the same UniqueContentId value.

💡 Tip

See [Optimizing document viewing for high volume](#) for more details on how this property is used in Office for the web.

Unused and future properties

The following properties are defined as valid CheckFileInfo response properties. However, they are not used, either because they are pending deprecation or they are designated for future features of WOPI clients and WOPI servers.

ClientUrl

A user-accessible URI directly to the file intended for opening the file through a client. Can be a DAV URL ([RFC 5323](#)), but may be any URL that can be handled by a client

that can open a file of the given type.

CobaltCapabilities

A array of strings that contains the Cobalt capabilities supported by the host. If `SupportsCobalt` is set to `false`, this property must be ignored by the WOPI client. Any value other than the possible values listed below must be ignored by the WOPI client.

Possible Values	Description
DownloadStreaming	This type of CobaltCapabilities indicates the host supports Cobalt streaming for download at the moment it handles the request.

DisableBrowserCachingOfUserContent

A **Boolean** value that indicates that the WOPI client should disable caching of file contents in the browser cache. Note that this has important performance implications for web browser-based WOPI clients.

EditAndReplyUrl

Not yet documented.

HostAuthenticationId

A **string** value uniquely identifying the user currently accessing the file.

Warning

This property should not be used. Hosts should use the **UserId** property instead.

HostEmbeddedEditUrl

A URI to a web page that provides access to an editing experience for the file that can be embedded in another HTML page. For example, a page that provides an HTML snippet that can be inserted into the HTML of a blog.

HostNotes

A **string** that is used by the host to pass arbitrary information to the WOPI client. The client must ignore this string if it does not recognize its contents. A host must not require that a client understand the contents of this string to operate.

HostRestUrl

A URI that is the base URI for REST operations for the file.

IrmPolicyDescription

A **string** that the WOPI client should display to the user indicating the IRM policy for the file. This value should be combined with [IrmPolicyTitle](#).

IrmPolicyTitle

A **string** that the WOPI client should display to the user indicating the IRM policy for the file. This value should be combined with [IrmPolicyDescription](#).

PresenceProvider

A **string** that identifies the provider of information that a WOPI client may use to discover information about the user's online status (for example, whether a user is available via instant messenger).

PresenceUserId

A **string** that identifies the user in the context of the [PresenceProvider](#).

ProtectInClient

A **Boolean** value that indicates that the WOPI client should take measures to prevent copying and printing of the file. This is intended to help enforce IRM.

SignInUrl

A URI that will allow the user to sign in using the host's authentication system. This property can be used when supporting anonymous users. If this property is not provided, no sign in UI will be shown in Office for the web.

SignoutUrl

A URI that will sign the current user out of the host's authentication system.

SupportsFileCreation

A **Boolean** value that indicates that the host supports creating new files using the WOPI client.

SupportsScenarioLinks

A **Boolean** value that indicates that the host supports scenarios where users can operate on files in limited ways via restricted URLs.

SupportsSecureStore

A **Boolean** value that indicates that the host supports calls to a secure data store utilizing credentials stored in the file.

TenantId

A **string** value uniquely identifying the user's 'tenant,' or group/organization to which they belong.

⊗ Caution

The presence of this property does not remove the uniqueness and consistency requirements listed above. User properties are expected to be unique per user and consistent over time regardless of the presence of a **TenantId**.

TimeZone

A **string** that is used to pass time zone information to a WOPI client. The format of this value is determined by the host.

UserPrincipalName

A **string** value uniquely identifying the user currently accessing the file.

WebEditingDisabled

A **Boolean** value that indicates that the WOPI client must not allow the user to edit the file.

ⓘ **Note**

This does not mean that the user doesn't have rights to edit the file. Hosts should use the **UserCanWrite** property for that purpose.

Breadcrumb properties

Breadcrumb properties are used by some WOPI clients to display breadcrumb-style navigation elements within the WOPI client UI.

BreadcrumbBrandName

A **string** that indicates the brand name of the host.

BreadcrumbBrandUrl

A URI to a web page that the WOPI client should navigate to when the user clicks on UI that displays [BreadcrumbBrandName](#).

BreadcrumbDocName

A **string** that indicates the name of the file. If this is not provided, WOPI clients may use the [BaseFileName](#) value.

BreadcrumbFolderName

A **string** that indicates the name of the container that contains the file.

BreadcrumbFolderUrl

A URI to a web page that the WOPI client should navigate to when the user clicks on UI that displays [BreadcrumbFolderName](#).

Workflow properties

Warning

This documentation is for an upcoming feature and may undergo dramatic changes prior to final release. Pre-release features are available in the **Test environment** only.

WorkflowType

Note

Pre-release property - not yet used by any WOPI client

An **array of strings** containing the workflow types available for the document. Possible values are:

- `Assign`
- `Submit`

This property must always be specified if [WorkflowUrl](#) or [WorkflowPostMessage](#) are provided. If this property is not supplied, then both [WorkflowUrl](#) and [WorkflowPostMessage](#) must be ignored by the WOPI client.

Conversely, if the WorkflowType property is provided but neither [WorkflowUrl](#) nor [WorkflowPostMessage](#) are provided, then the WorkflowType value must be ignored by the WOPI client.

Important

While this property is an array of strings, note that specific values of WorkflowType may be mutually exclusive depending on the WOPI client. WOPI clients must use the following guidelines when handling values in the WorkflowType array:

- If no supported values are provided, the WOPI client must behave as though the WorkflowType property was not provided.
- If multiple values are provided and the WOPI client does not support multiple values, the client may use the first supported value provided in the array or behave as though the WorkflowType property was not provided.

WorkflowUrl

ⓘ Note

Pre-release property - not yet used by any WOPI client

A URI to a location that allows the user to participate in a workflow for the file.

ⓘ Important

This value will be ignored if **WorkflowType** is not provided.

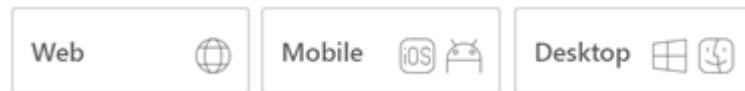
Deprecated properties

The following properties are deprecated and should no longer be used by WOPI clients or servers.

Property	Deprecated since version
BreadcrumbDocUrl	2014.06.01
EditingCannotSave	2014.06.01
HostName	2016.01.12
PrivacyUrl	2015.06.01
TermsOfUseUrl	2015.06.01

GetFile

Article • 03/04/2023



The `GetFile` operation retrieves a file from a host.

GET /wopi/files/(file_id)/contents

The `GetFile` operation retrieves a file from a host.

Tip

By default, Microsoft 365 for the web uses the `GetFile` WOPI request to retrieve files from the host. However, hosts can override this behavior by providing a direct URL to the file using the [FileUrl](#) property in the [CheckFileInfo](#) response.

Parameters

- **file_id** (*string*) – A string that specifies a [file ID](#) of a file managed by host. This string must be URL safe.

Query parameters

- **access_token** (*string*) – An [access token](#) that the host uses to determine whether the request is authorized.

Request headers

- **X-WOPI-MaxExpectedSize** – An **integer** specifying the upper bound of the expected size of the file being requested. Optional. The host should use the maximum value of a 4-byte integer if this value isn't set in the request. If the file requested is larger than this value, the host must respond with a [412 Precondition Failed](#) .

Response headers

- **X-WOPI-ItemVersion** – An optional **string** value indicating the version of the file. Its value should be the same as [Version](#) value in [CheckFileInfo](#).

Response body

The response body must be the full binary contents of the file.

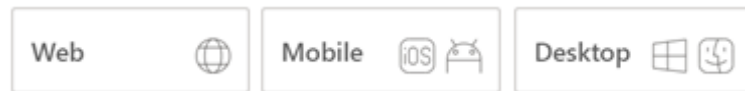
Status codes

- [200 OK](#)  – Success.
- [401 Unauthorized](#)  – Invalid [access token](#).
- [404 Not Found](#)  – Resource not found or user unauthorized.
- [412 Precondition Failed](#)  – File is larger than **X-WOPI-MaxExpectedSize**.
- [500 Internal Server Error](#)  – Server error.

In addition to the request and response headers listed here, this operation might also use the Standard WOPI request and response headers. For more information see [Standard WOPI request and response headers](#).

Lock

Article • 03/04/2023



The **Lock** operation locks a file for editing by the WOPI client application instance that requested the lock.

POST (/wopi/files/(file_id))

The **Lock** operation locks a file for editing by the WOPI client application instance that requested the lock. To support editing files, WOPI clients require that the WOPI host supports locking files. When locked, a file must not be writable by other applications.

If the file is currently unlocked, the host should lock the file and return [200 OK](#).

If the file is currently locked and the **X-WOPI-Lock** value matches the lock currently on the file, the host should treat the request as if it's a [RefreshLock](#) request. That is, the host should refresh the lock timer and return [200 OK](#).

In all other cases, the host must return a *lock mismatch* response ([409 Conflict](#)) and include an **X-WOPI-Lock** response header containing the value of the current lock on the file.

In cases where the file is locked by someone other than a WOPI client, hosts should still always include the current lock ID in the **X-WOPI-Lock** response header. However, if the current lock ID isn't representable as a WOPI lock (for example, it's longer than the [maximum lock length](#)), the **X-WOPI-Lock** response header should be set to the empty string or omitted completely.

For more general information about locks, see [Lock](#).

Parameters

- **file_id** (*string*) – A string that specifies a [file ID](#) of a file managed by host. This string must be URL safe.

Query parameters

- **access_token** (*string*) – An [access token](#) that the host uses to determine whether the request is authorized.

Request headers

- *X-WOPI-Override* – The **string** `LOCK`. This header is required.
- *X-WOPI-Lock* – A **string** provided by the WOPI client that the host uses to identify the lock on the file. This header is required.

Response headers

- *X-WOPI-Lock* – A **string** value identifying the current lock on the file. This header must always be included when responding to the request with [409 Conflict](#). It shouldn't be included when responding to the request with [200 OK](#).
- *X-WOPI-LockFailureReason* – An optional **string** value indicating the cause of a lock failure. This header might be included when responding to the request with [409 Conflict](#). There's no standard for how this string is formatted, and it must only be used for logging purposes.
- *X-WOPI-LockedByOtherInterface* – **Deprecated: Deprecated since version 2015-12-15:** This header is deprecated and should be ignored by WOPI clients.
- *X-WOPI-ItemVersion* – An optional **string** value indicating the version of the file. Its value should be the same as [Version](#) value in [CheckFileInfo](#).

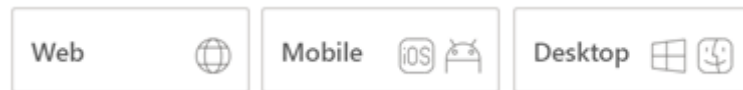
Status codes

- [200 OK](#) – Success.
- [400 Bad Request](#) – **X-WOPI-Lock** was not provided or was empty.
- [401 Unauthorized](#) – Invalid [access token](#).
- [404 Not Found](#) – Resource not found or user unauthorized.
- [409 Conflict](#) – Lock mismatch or locked by another interface. You must always include an **X-WOPI-Lock** response header containing the value of the current lock on the file when using this response code.
- [500 Internal Server Error](#) – Server error.
- [501 Not Implemented](#) – Operation not supported.

In addition to the request and response headers listed here, this operation might also use the Standard WOPI request and response headers. For more information see [Standard WOPI request and response headers](#).

GetLock

Article • 03/04/2023



The `GetLock` operation retrieves a lock on a file.

POST /wopi/files/(file_id)

The `GetLock` operation retrieves a lock on a file. This operation *doesn't create a new lock*. Rather, it always returns the current lock value in the **X-WOPI-Lock** response header. Because of this, this operation's semantics differ slightly from the other lock-related operations.

If the file is currently unlocked, the host must return a [200 OK](#) and include an **X-WOPI-Lock** response header set to the empty string.

If the file is currently locked, the host should return a [200 OK](#) and include an **X-WOPI-Lock** response header containing the value of the current lock on the file. If the current lock ID isn't representable as a WOPI lock (for example, it's longer than the [maximum lock length](#)), the host should return a [409 Conflict](#) and set the **X-WOPI-Lock** response header to the empty string or omit it completely.

💡 Tip

While a [409 Conflict](#) is technically a valid response to this operation, it's rarely needed in practice, and hosts should respond with a [200 OK](#) in most cases.

For more general information about locks, see [Lock](#).

Parameters

- **file_id** (*string*) – A string that specifies a [file ID](#) of a file managed by host. This string must be URL safe.

Query parameters

- **access_token** (*string*) – An [access token](#) that the host uses to determine whether the request is authorized.

Request headers

- *X-WOPI-Override* – The **string** `GET_LOCK`. This string is required.

Response headers

- *X-WOPI-Lock* – A **string** value identifying the current lock on the file. Unlike other lock operations, this header is required when responding to the request with either [200 OK](#) or [409 Conflict](#).
- *X-WOPI-LockFailureReason* – An optional **string** value indicating the cause of a lock failure. This header might be included when responding to the request with [409 Conflict](#). There's no standard for how this string is formatted, and it must only be used for logging purposes.
- *X-WOPI-LockedByOtherInterface* – **Deprecated: Deprecated since version 2015-12-15:** This header is deprecated and should be ignored by WOPI clients.

Status codes

- [200 OK](#) – Success. You must include an **X-WOPI-Lock** response header containing the value of the current lock on the file when using this response code.
- [401 Unauthorized](#) – Invalid [access token](#)
- [404 Not Found](#) – Resource not found/user unauthorized.
- [409 Conflict](#) – Lock mismatch or locked by another interface. You must include an **X-WOPI-Lock** response header containing the value of the current lock on the file when using this response code.
- [500 Internal Server Error](#) – Server error.
- [501 Not Implemented](#) – Operation not supported.

In addition to the request and response headers listed here, this operation might also use the Standard WOPI request and response headers. For more information see [Standard WOPI request and response headers](#).

RefreshLock

Article • 03/04/2023



The `RefreshLock` operation refreshes the lock on a file.

POST /wopi/files/(file_id)

The `RefreshLock` operation refreshes the lock on a file by resetting its automatic expiration timer to 30 minutes. The refreshed lock must expire automatically after 30 minutes unless it's modified by a subsequent WOPI operation, such as [Unlock](#) or [RefreshLock](#).

WOPI clients usually make a [Lock](#) request to lock a file prior to calling this operation. The WOPI client passes the lock ID established by that previous [Lock](#) operation in the **X-WOPI-Lock** request header.

If the file is currently locked and the **X-WOPI-Lock** value doesn't match the lock currently on the file, or if the file is unlocked, the host must do the following:

- Return a *lock mismatch* response ([409 Conflict](#) [↗](#))
- Include an **X-WOPI-Lock** response header containing the value of the current lock on the file

In cases where the file is unlocked, the host must set **X-WOPI-Lock** to the empty string.

In cases where the file is locked by someone other than a WOPI client, hosts should still always include the current lock ID in the **X-WOPI-Lock** response header. However, if the current lock ID isn't representable as a WOPI lock (for example, it's longer than the [maximum lock length](#)), the **X-WOPI-Lock** response header should be set to the empty string or omitted completely.

For more general information regarding locks, see [Lock](#).

Parameters

- **file_id** (*string*) – A string that specifies a [file ID](#) of a file managed by host. This string must be URL safe.

Query parameters

- **access_token** (*string*) – An [access token](#) that the host uses to determine whether the request is authorized.

Request headers

- *X-WOPI-Override* – The **string** `REFRESH_LOCK`. This header is required.
- *X-WOPI-Lock* – A **string** provided by the WOPI client that the host must use to identify the existing lock on the file. This header is required.

Response headers

- *X-WOPI-Lock* – A **string** value identifying the current lock on the file. This header must always be included when responding to the request with [409 Conflict](#). It should not be included when responding to the request with [200 OK](#).
- *X-WOPI-LockFailureReason* – An optional **string** value indicating the cause of a lock failure. This header might be included when responding to the request with [409 Conflict](#). There's no standard for how this string is formatted, and it must only be used for logging purposes.
- *X-WOPI-LockedByOtherInterface* –

Deprecated: Deprecated since version 2015-12-15: This header is deprecated and should be ignored by WOPI clients.

Status codes

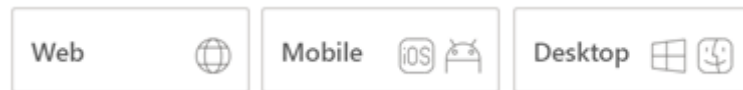
- [200 OK](#) – Success.
- [400 Bad Request](#) – **X-WOPI-Lock** was not provided or was empty.
- [401 Unauthorized](#) – Invalid [access token](#).
- [404 Not Found](#) – Resource not found or user unauthorized.
- [409 Conflict](#) – Lock mismatch or locked by another interface. An **X-WOPI-Lock** response header containing the value of the current lock on the file must always be included when using this response code.
- [500 Internal Server Error](#) – Server error.

- [501 Not Implemented](#)  – Operation not supported.

In addition to the request and response headers listed here, this operation might also use the Standard WOPI request and response headers. For more information see [Standard WOPI request and response headers](#).

Unlock

Article • 03/04/2023



The `Unlock` operation releases the lock on a file.

POST /wopi/files/(file_id)

The `Unlock` operation releases the lock on a file.

WOPI clients usually make a [Lock](#) request to lock a file prior to calling this operation. The WOPI client passes the lock ID established by that previous [Lock](#) operation in the **X-WOPI-Lock** request header.

If the file is currently locked and the **X-WOPI-Lock** value doesn't match the lock currently on the file, or if the file is unlocked, the host must:

- Return a *lock mismatch* response ([409 Conflict](#))
- Include an **X-WOPI-Lock** response header containing the value of the current lock on the file.

In cases where the file is unlocked, the host must set **X-WOPI-Lock** to the empty string.

In cases where the file is locked by someone other than a WOPI client, hosts should still always include the current lock ID in the **X-WOPI-Lock** response header. However, if the current lock ID isn't representable as a WOPI lock (for example, it's longer than the [maximum lock length](#)), the **X-WOPI-Lock** response header should be set to the empty string or omitted completely.

For more general information about locks, see [Lock](#).

Parameters

- **file_id** (*string*) – A string that specifies a [file ID](#) of a file managed by host. This string must be URL safe.

Query parameters

- **access_token** (*string*) – An [access token](#) that the host uses to determine whether the request is authorized.

Request headers

- *X-WOPI-Override* – The **string** `UNLOCK`. This string is required.
- *X-WOPI-Lock* – A **string** provided by the WOPI client that the host uses to identify the lock on the file. This string is required.

Response headers

- *X-WOPI-Lock* – A **string** value identifying the current lock on the file. You must always include this header when responding to the request with [409 Conflict](#). It shouldn't be included when responding to the request with [200 OK](#).
- *X-WOPI-LockFailureReason* – An optional **string** value indicating the cause of a lock failure. This header might be included when responding to the request with [409 Conflict](#). There's no standard for how this string is formatted, and it must only be used for logging purposes.
- *X-WOPI-LockedByOtherInterface* –

Deprecated: Deprecated since version 2015-12-15: This header is deprecated and should be ignored by WOPI clients.
- *X-WOPI-ItemVersion* – An optional **string** value indicating the version of the file. Its value should be the same as [Version](#) value in [CheckFileInfo](#).

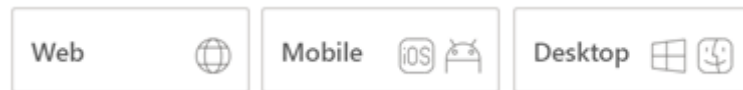
Status codes

- [200 OK](#) – Success.
- [400 Bad Request](#) – **X-WOPI-Lock** wasn't provided or was empty.
- [401 Unauthorized](#) – Invalid [access token](#).
- [404 Not Found](#) – Resource not found or user unauthorized.
- [409 Conflict](#) – Lock mismatch or locked by another interface. You must include an **X-WOPI-Lock** response header containing the value of the current lock on the file when using this response code.
- [500 Internal Server Error](#) – Server error.
- [501 Not Implemented](#) – Operation not supported.

In addition to the request and response headers listed here, this operation might also use the Standard WOPI request and response headers. For more information see [Standard WOPI request and response headers](#).

UnlockAndRelock

Article • 03/04/2023



The `UnlockAndRelock` operation releases a lock on a file, and then immediately places a new lock on the file.

POST /wopi/files/(file_id)

The `UnlockAndRelock` operation releases a lock on a file, and then immediately places a new lock on the file.

Important

This operation must be atomic.

`UnlockAndRelock` is similar semantically to the [Lock](#) operation. The two operations share the same **X-WOPI-Override** value. So, hosts must differentiate the two operations based on the presence, or lack of, the **X-WOPI-OldLock** request header.

Unlike the [Lock](#) operation, the `UnlockAndRelock` operation passes the current expected lock ID in the **X-WOPI-OldLock** request header. The **X-WOPI-Lock** value is the lock ID for the new lock.

If the file is currently locked and the **X-WOPI-OldLock** value doesn't match the lock currently on the file, or if the file is unlocked, the host must:

- Return a *lock mismatch* response ([409 Conflict](#) )
- Include an **X-WOPI-Lock** response header containing the value of the current lock on the file

In cases where the file is unlocked, the host must set **X-WOPI-Lock** to the empty string.

In cases where the file is locked by someone other than a WOPI client, hosts should still always include the current lock ID in the **X-WOPI-Lock** response header. However, if the current lock ID isn't representable as a WOPI lock (for example, it's longer than the [maximum lock length](#)), you should set the **X-WOPI-Lock** response header to the empty string or omitted completely.

For more general information about locks, see [Lock](#).

Parameters

- **file_id** (*string*) – A string that specifies a [file ID](#) of a file managed by host. This string must be URL safe.

Query parameters

- **access_token** (*string*) – An [access token](#) that the host uses to determine whether the request is authorized.

Request headers

- **X-WOPI-Override** – The **string** `LOCK`. Required.
- **X-WOPI-Lock** – A **string** provided by the WOPI client that the host must use to identify the new lock on the file. The maximum length of a lock ID is 1024 ASCII characters. Required.
- **X-WOPI-OldLock** – A **string** provided by the WOPI client that is the existing lock on the file. Required. If **X-WOPI-OldLock** is not provided, the request is identical to a [Lock](#) request.

Response headers

- **X-WOPI-Lock** – A **string** value identifying the current lock on the file. This header must always be included when responding to the request with [409 Conflict](#). It should not be included when responding to the request with [200 OK](#).
- **X-WOPI-LockFailureReason** – An optional **string** value indicating the cause of a lock failure. This header Might be included when responding to the request with [409 Conflict](#). There's no standard for how this string is formatted, and it must only be used for logging purposes.
- **X-WOPI-LockedByOtherInterface** –

Deprecated: Deprecated since version 2015-12-15: This header is deprecated and should be ignored by WOPI clients.

Status codes

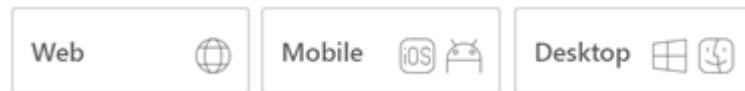
- [200 OK](#) – Success.

- [400 Bad Request](#) – **X-WOPI-Lock** was not provided or was empty.
- [401 Unauthorized](#) – Invalid [access token](#)
- [404 Not Found](#) – Resource not found/user unauthorized
- [409 Conflict](#) – Lock mismatch or locked by another interface. You must include an **X-WOPI-Lock** response header containing the value of the current lock on the file when using this response code.
- [500 Internal Server Error](#) – Server error.
- [501 Not Implemented](#) – Operation not supported.

In addition to the request and response headers listed here, this operation might also use the Standard WOPI request and response headers. For more information see [Standard WOPI request and response headers](#).

PutFile

Article • 03/04/2023



The `PutFile` operation updates a file's binary contents.

POST /wopi/files/(file_id)/contents

The `PutFile` operation updates a file's binary contents.

WOPI clients usually make a [Lock](#) request to lock a file prior to calling this operation. The WOPI client passes the lock ID established by that previous [Lock](#) operation in the **X-WOPI-Lock** request header.

When a host receives a `PutFile` request on a file that's not locked, the host checks the current size of the file. If it's 0 bytes, the `PutFile` request should be considered valid and should proceed. If it's any value other than 0 bytes, or missing altogether, the host should respond with a [409 Conflict](#). For more information, see [Creating new files using Microsoft 365 for the web](#).

If the file is currently locked and the **X-WOPI-Lock** value doesn't match the lock currently on the file, the host must

- Return a *lock mismatch* response ([409 Conflict](#))
- Include an **X-WOPI-Lock** response header containing the value of the current lock on the file.

In cases where the file is unlocked, the host must set **X-WOPI-Lock** to the empty string.

In cases where the file is locked by someone other than a WOPI client, hosts should still always include the current lock ID in the **X-WOPI-Lock** response header. However, if the current lock ID isn't representable as a WOPI lock (for example, it's longer than the [maximum lock length](#)), the **X-WOPI-Lock** response header should be set to the empty string or omitted completely.

Parameters

- **file_id** (*string*) – A string that specifies a [file ID](#) of a file managed by host. This string must be URL safe.

Query Parameters

- `access_token` (*string*) – An [access token](#) that the host uses to determine whether the request is authorized.

Request headers

- *X-WOPI-Override* – The **string** `PUT`. This header is required.
- *X-WOPI-Lock* – A **string** provided by the WOPI client in a previous [Lock](#) request. This header isn't included during [document creation](#).
- *X-WOPI-Editors* – A comma-delimited **string** of [UserId](#) values representing all the users who contributed changes to the document in this `PutFile` request.

Request body

The request body must be the full binary contents of the file.

Response headers

- *X-WOPI-Lock* – A **string** value identifying the current lock on the file. You must always include this header when responding to the request with [409 Conflict](#). It shouldn't be included when responding to the request with [200 OK](#).
- *X-WOPI-LockFailureReason* – An optional **string** value indicating the cause of a lock failure. This header might be included when responding to the request with [409 Conflict](#). There's no standard for how this string is formatted, and it must only be used for logging purposes.
- *X-WOPI-LockedByOtherInterface* – **Deprecated: Deprecated since version 2015-12-15:** This header is deprecated and should be ignored by WOPI clients.
- *X-WOPI-ItemVersion* – An optional **string** value indicating the version of the file. Its value should be the same as [Version](#) value in [CheckFileInfo](#).

Tip

For `PutFile` responses, this should be the version of the file after the `PutFile` operation.

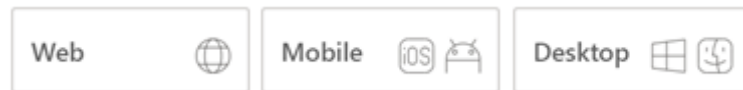
Status codes

- [200 OK](#)  – Success.
- [401 Unauthorized](#)  – Invalid [access token](#).
- [404 Not Found](#)  – Resource not found or user unauthorized.
- [409 Conflict](#)  – Lock mismatch or locked by another interface. You must include an **X-WOPI-Lock** response header containing the value of the current lock on the file when using this response code.
- [413 Request Entity Too Large](#)  – File is too large. The maximum file size is host-specific.
- [500 Internal Server Error](#)  – Server error.
- [501 Not Implemented](#)  – Operation not supported.

In addition to the request and response headers listed here, this operation might also use the Standard WOPI request and response headers. For more information see [Standard WOPI request and response headers](#).

PutRelativeFile

Article • 03/04/2023



The `PutRelativeFile` operation creates a new file on the host based on the current file.

POST (/wopi/files/(file_id))

The `PutRelativeFile` operation creates a new file on the host based on the current file. The host must use the content in the [POST](#) body to create a new file.

The `PutRelativeFile` operation has two distinct modes: *specific* and *suggested*. The primary difference between the two modes is whether the WOPI client expects the host to use the file name provided exactly (*specific* mode) or if the host can adjust the file name in order to make the request succeed (*suggested* mode).

Hosts can determine the mode of the operation based on which of the mutually exclusive **X-WOPI-RelativeTarget** (indicates *specific* mode) or **X-WOPI-SuggestedTarget** (indicates *suggested* mode) request headers is used. The expected behavior for each mode is described in detail below.

The `PutRelativeFile` operation might be called on a file that is not locked, so the **X-WOPI-Lock** request header is not included in this operation. An example of when this might occur is if a user uses the *Save As* feature when viewing a document in read-only mode. The source file won't be locked in this case, but the `PutRelativeFile` operation is invoked on it.

However, a file matching the target name might be locked, and in *specific* mode, the host must respond with a [409 Conflict](#) and include a **X-WOPI-Lock** response header as described below.

❗ Important

If a WOPI host sets the **SupportsUpdate** property in **CheckFileInfo** to `true`, the host is expected to implement the `PutRelativeFile` operation. However, a host might choose not to implement this operation even though **SupportsUpdate** is `true` and they must do the following:

1. Set the **UserCanNotWriteRelative** property to `true` always.

2. Return a **501 Not Implemented** [↗](#) to all `PutRelativeFile` requests.

Excel for the web

Excel for the web uses this operation in the following two ways:

- As part of the **Save As** feature. If `PutRelativeFile` is not supported, the **Save As** feature doesn't work in Excel for the web.
- To support editing of some Excel files in Excel for the web. Some files might contain content that's not currently supported in Excel for the web. In this case, Excel for the web prompts a user to save an editable copy of the document, removing all unsupported content. The file can then be edited in Excel for the web. If `PutRelativeFile` isn't supported, files with unsupported content aren't editable in Excel Online.

Parameters

- `file_id` (*string*) – A string that specifies a [file ID](#) of a file managed by host. This string must be URL safe.

Query parameters

- `access_token` (*string*) – An [access token](#) that the host uses to determine whether the request is authorized.

Request headers

- `X-WOPI-Override` – The **string** `PUT_RELATIVE`. This header is required.
- `X-WOPI-SuggestedTarget` –

A UTF-7 encoded **string** specifying either a file extension or a full file name, including the file extension. Hosts can differentiate between full file names and extensions as follows:

- If the string begins with a period (`.`), it's a file extension.
- Otherwise, it's a full file name.

If only the extension is provided, the name of the initial file without extension should be combined with the extension to create the proposed name.

The response to a request including this header must never result in a [400 Bad Request](#) or [409 Conflict](#). Rather, the host must modify the proposed name as needed to create a new file that's both legally named and doesn't overwrite any existing file, while preserving the file extension.

This header must be present if **X-WOPI-RelativeTarget** is not present. The two headers are mutually exclusive. If both headers are present, the host should respond with a [400 Bad Request](#) or [501 Not Implemented](#).

- *X-WOPI-RelativeTarget* –

A UTF-7 encoded **string** that specifies a full file name including the file extension. The host must not modify the name to fulfill the request.

If the specified name is illegal, the host must respond with a [400 Bad Request](#).

If a file with the specified name already exists, the host must respond with a [409 Conflict](#), unless the **X-WOPI-OverwriteRelativeTarget** request header is set to `true`. When responding with a [409 Conflict](#) for this reason, the host might include an **X-WOPI-ValidRelativeTarget** specifying a file name that's valid.

If the **X-WOPI-OverwriteRelativeTarget** request header is set to `true` and a file with the specified name already exists and is locked, the host must respond with a [409 Conflict](#) and include an **X-WOPI-Lock** response header containing the value of the current lock on the file.

This header must be present if **X-WOPI-SuggestedTarget** is not present. The two headers are mutually exclusive. If both headers are present, the host should respond with a [400 Bad Request](#) or [501 Not Implemented](#).

- *X-WOPI-OverwriteRelativeTarget* –

A **Boolean** value that specifies whether the host must overwrite the file name if it exists. The default value is `false`. In other words, if **X-WOPI-OverwriteRelativeTarget** is not explicitly included on the request, hosts must behave as though its value is `false`.

This header is only valid if the **X-WOPI-RelativeTarget** is also included on the request. It must be ignored in all other cases.

If the user is not authorized to overwrite the target file, the host must respond with a [501 Not Implemented](#).

- *X-WOPI-Size* – An **integer** that specifies the size of the file in bytes.

- *X-WOPI-FileConversion* –

A header whose presence indicates that the request is being made in the context of a [binary document conversion](#). This header is only included on the request in that case. Thus, if **X-WOPI-FileConversion** isn't explicitly included on the request, hosts must behave as if the `PutRelativeFile` request is not being made as part of a binary document conversion.

For more information on the conversion process and how to use this header, see [Editing binary document formats](#).

Request body

The request body must be the full binary contents of the file.

Response headers

- *X-WOPI-ValidRelativeTarget* – A UTF-7 encoded **string** that specifies a full file name including the file extension. This header might be used when responding with a [409 Conflict](#), because a file with the requested name already exists, or when responding with a [400 Bad Request](#), because the requested name contained invalid characters. If this response header is included, the WOPI client should automatically retry the `PutRelativeFile` operation using the contents of this header as the **X-WOPI-RelativeTarget** value and shouldn't display an error message to the user.
- *X-WOPI-Lock* – A **string** value identifying the current lock on the file. This header must always be included when responding to the request with [409 Conflict](#). It shouldn't be included when responding to the request with [200 OK](#).
- *X-WOPI-LockFailureReason* – An optional **string** value indicating the cause of a lock failure. This header might be included when responding to the request with [409 Conflict](#). There's no standard for how this string is formatted, and it must only be used for logging purposes.
- *X-WOPI-LockedByOtherInterface* –

Deprecated: Deprecated since version 2015-12-15: This header is deprecated and should be ignored by WOPI clients.

Status codes

- [200 OK](#) – Success.
- [400 Bad Request](#) – Specified name is illegal.
- [401 Unauthorized](#) – Invalid [access token](#).
- [404 Not Found](#) – Resource not found or user unauthorized.
- [409 Conflict](#) – Target file already exists or the file is locked. If the target file is locked, you must include an **X-WOPI-Lock** response header containing the value of the current lock on the file.
- [413 Request Entity Too Large](#) – File is too large. The maximum size is host-specific.
- [500 Internal Server Error](#) – Server error.
- [501 Not Implemented](#) – Operation not supported. If the host sets the [SupportsUpdate](#) and `UserCanNotWriteRelative` properties to `true` in [CheckFileInfo](#), you must use this response code when this operation is called.

In addition to the request and response headers listed here, this operation might also use the Standard WOPI request and response headers. For more information see [Standard WOPI request and response headers](#).

Response

The response to a `PutRelativeFile` call is [JSON](#) containing a number of properties, some of which are optional.

All optional values default to the following values based on their type:

Type	Default value
Boolean	<code>false</code>
String	The empty string
Integer/Long	Varies; see individual properties for details
Array	Empty array

Important

No properties should be set to `null`. If you don't wish to set a property, simply omit it from the response and WOPI clients then use the default value.

Required response properties

The following properties must be present in all `PutRelativeFile` responses:

- **Name** (*string*) - The name of the file, including extension, without a path.
- **Url** (*string*) - A URL of the form `http://server/<...>/wopi/files/(file_id)?access_token=(access token)`, of the newly created file on the host. This URL is the [WOPISrc](#) for the new file with an [access token](#) appended. Or, stated differently, it's the URL to the host's Files endpoint for the new file, along with an [access token](#). A [GET](#) request to this URL invokes the [CheckFileInfo](#) operation.

⊗ Caution

This property includes an **access token** so it has important security implications. For more information, see **Preventing 'token trading'**.

Optional response properties

Following are the optional response properties.

- **HostViewUrl** - The [HostViewUrl](#), as a **string**, for the newly created file.
- **HostEditUrl** - The [HostEditUrl](#), as a **string**, for the newly created file.

RenameFile

Article • 03/04/2023



The `RenameFile` operation renames a file.

POST /wopi/files/(file_id)

Important

Renaming the file must not cause the [File ID](#), and by extension, the [WOPISrc](#), to change.

If the host can't rename the file because the name requested is invalid or conflicts with an existing file, the host should try to generate a different name based on the requested name that meets the file name requirements.

If the host can't generate a different name, it should return an HTTP status code [400 Bad Request](#)[↗]. The response must include an `X-WOPI-InvalidFileNameError` header that describes why the file name was invalid.

If the file is currently unlocked, the host should respond with a [200 OK](#)[↗] and proceed with the rename.

If the file is currently locked and the `X-WOPI-Lock` value doesn't match the lock currently on the file the host must return a *lock mismatch* response ([409 Conflict](#)[↗]) and include an `X-WOPI-Lock` response header containing the value of the current lock on the file.

Tip

Microsoft 365 for the web includes contains UI that users can use to rename files. In order to activate this UI in Office for the web, you must implement the `RenameFile` operation, and also do the following:

- Set [SupportsRename](#) and [UserCanRename](#) to `true` in your [CheckFileInfo](#) response.

- Set a [FileNameMaxLength](#) value if the default value isn't correct for your WOPI host.

Parameters

- **file_id** (*string*) – A string that specifies a [file ID](#) of a file managed by host. This string must be URL safe.

Query parameters

- **access_token** (*string*) – An [access token](#) that the host uses to determine whether the request is authorized.

Request headers

- *X-WOPI-Override* – The **string** `RENAME_FILE`. This header is required.
- *X-WOPI-Lock* – A **string** provided by the WOPI client that the host uses to identify the lock on the file.
- *X-WOPI-RequestedName* – A UTF-7 encoded **string** that's a file name, *not including the file extension*.

Response headers

- *X-WOPI-InvalidFileNameError* – A **string** describing the reason the rename operation couldn't be completed. This header should only be included when the response code is [400 Bad Request](#). This value must only be used for logging purposes.
- *X-WOPI-Lock* – A **string** value identifying the current lock on the file. This header must always be included when responding to the request with [409 Conflict](#). It shouldn't be included when responding to the request with [200 OK](#).
- *X-WOPI-LockFailureReason* – An optional **string** value indicating the cause of a lock failure. This header might be included when responding to the request with [409 Conflict](#). There's no standard for how this string is formatted, and it must only be used for logging purposes.
- *X-WOPI-LockedByOtherInterface* –

Deprecated: Deprecated since version 2015-12-15: This header is deprecated and should be ignored by WOPI clients.

Status Codes

- [200 OK](#) – Success.
- [400 Bad Request](#) – Specified name is illegal.
- [401 Unauthorized](#) – Invalid [access token](#).
- [404 Not Found](#) – Resource not found or user unauthorized.
- [409 Conflict](#) – Lock mismatch or locked by another interface. You must always include an **X-WOPI-Lock** response header containing the value of the current lock on the file when using this response code.
- [500 Internal Server Error](#) – Server error.
- [501 Not Implemented](#) – Operation not supported.

In addition to the request and response headers listed here, this operation might also use the Standard WOPI request and response headers. For more information, see [Standard WOPI request and response headers](#).

Response

The response to a `RenameFile` call is [JSON](#) containing a single required property:

- **Name** (*string*) - The name of the renamed file *without a path or file extension*.

Important

The **Name** property returned can't include the file extension. This is different than other similar WOPI operations such as [PutRelativeFile](#) and [CreateChildFile](#).

DeleteFile

Article • 03/04/2023



The `DeleteFile` operation deletes a file from a host.

ⓘ Note

This WOPI operation isn't required to integrate with Office for the web or Microsoft 365 for mobile at this time.

POST /wopi/files/(file_id)

The `DeleteFile` operation deletes a file from a host.

If the file is currently locked, the host should return a [409 Conflict](#) and include an **X-WOPI-Lock** response header containing the value of the current lock on the file. If the current lock ID is not representable as a WOPI lock (for example, it's longer than the [maximum lock length](#)), the host should return a [409 Conflict](#) and set the **X-WOPI-Lock** response header to the empty string or omit it completely.

Parameters

- **file_id** (*string*) – A string that specifies a [file ID](#) of a file managed by host. This string must be URL safe.

Query parameters

- **access_token** (*string*) – An [access token](#) that the host uses to determine whether the request is authorized.

Request headers

- **X-WOPI-Override** – The string `DELETE`. Required.

Status codes

- [200 OK](#) – Success.
- [401 Unauthorized](#) – Invalid [access token](#).
- [404 Not Found](#) – Resource not found or user unauthorized.
- [409 Conflict](#) – Lock mismatch or locked by another interface. You must include an **X-WOPI-Lock** response header containing the value of the current lock on the file when using this response code.
- [500 Internal Server Error](#) – Server error.
- [501 Not Implemented](#) – Operation not supported.

In addition to the request and response headers listed here, this operation might also use the Standard WOPI request and response headers. For more information see [Standard WOPI request and response headers](#).

EnumerateAncestors (files)

Article • 03/04/2023



The `EnumerateAncestors` operation represents an ancestry file or folder being operated on via WOPI operations. A host must issue a unique ID for any file used by a WOPI client. The client will, in turn, include the file ID when making requests to the WOPI host. Thus, a host must be able to use the file ID to locate a particular file. For more information on File ID strings and requirements for integration, see [Key concepts](#).

ⓘ Note

`EnumerateAncestors` (containers)

GET /wopi/files/(file_id)/ancestry

Parameters

- `file_id` (*string*) – A string that specifies a [file ID](#) of a file managed by host. This string must be URL safe.

Query parameters

- `access_token` (*string*) – An [access token](#) that the host uses to determine whether the request is authorized.

Response headers

- `X-WOPI-EnumerationIncomplete` –

An optional header indicating that the enumeration of the container's ancestry is incomplete. If set, the value of this header must be the string `true`.

A WOPI client might choose to issue additional `EnumerateAncestors` requests to complete the enumeration.

Status codes

- [200 OK](#) [↗] – Success.
- [401 Unauthorized](#) [↗] – Invalid [access token](#).
- [404 Not Found](#) [↗] – Resource not found or user unauthorized.
- [500 Internal Server Error](#) [↗] – Server error.
- [501 Not Implemented](#) [↗] – Operation not supported.

In addition to the request and response headers listed here, this operation might also use the Standard WOPI request and response headers. For more information see [Standard WOPI request and response headers](#).

Response

The response to a `EnumerateAncestors` call is [JSON](#) [↗] containing the following required properties:

- **AncestorsWithRootFirst** - An array of JSON-formatted objects containing the following properties:
 - **Name** - The name of the container without a path. This value should match the Name property in a [CheckContainerInfo](#) response. This value is required.
 - **Url** - A URL to the container, including a valid [access token](#). This property is required.

⊗ Caution

This property includes an **access token** so it has important security implications. See **Preventing 'token trading'** for more details.

The array must always be ordered such that the ancestor closest to the root is the first element.

If there are no ancestor containers, this property should be an empty array.

Consider a file in the following container hierarchy:

`/root/grandparent/parent/myfile.docx`. When called on this file, the

`EnumerateAncestors` operation should return the following:

JSON

```
{
  "AncestorsWithRootFirst": [
    {
      "Url": "http://.../wopi*/containers/<containerId>?access_token=
<per_container_token>",
      "Name": "root"
    },
    {
      "Url": "http://.../wopi*/containers/<containerId2>?access_token=
<per_container_token2>",
      "Name": "grandparent"
    },
    {
      "Url": "http://.../wopi*/containers/<containerId3>?access_token=
<per_container_token3>",
      "Name": "parent"
    }
  ]
}
```

GetShareUrl (files)

Article • 03/04/2023



The `GetShareUrl` operation returns a share URL that's suitable for viewing a shared file when launched in a web browser.

! Note

`GetShareUrl` (containers)

POST (/wopi/files/(file_id))

The `GetShareUrl` operation returns a [Share URL](#) that's suitable for viewing a shared file when launched in a web browser. A host can support multiple Share URL types, as described by the [SupportedShareUrlTypes](#) property. The `X-WOPI-UrlType` request header contains the Share URL type that should be returned.

If the `X-WOPI-UrlType` header is not present or contains a value that's invalid or not supported by the host, the host should respond with a [501 Not Implemented](#) [↗](#).

Parameters

- `file_id` (*string*) – A string that specifies a [file ID](#) of a file managed by host. This string must be URL safe.

Query parameters

- `access_token` (*string*) – An [access token](#) that the host uses to determine whether the request is authorized.

Request headers

- `X-WOPI-Override` – The **string** `GET_SHARE_URL`. This header is required.
- `X-WOPI-UrlType` – A **string** indicating what Share URL type to return. This header is required.

Status codes

- [200 OK](#) – Success
- [401 Unauthorized](#) – Invalid [access token](#)
- [404 Not Found](#) – Resource not found/user unauthorized
- [500 Internal Server Error](#) – Server error
- [501 Not Implemented](#) – Operation not supported

ⓘ Note

Standard WOPI request and response headers

In addition to the request/response headers listed here, this operation may also use the [Standard WOPI request and response headers](#).

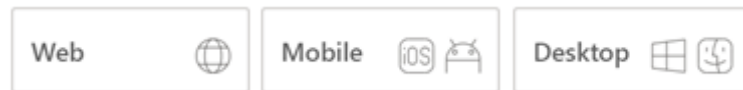
Response

The response to a `GetShareUrl` call is [JSON](#) containing the following required property:

- **ShareUrl** - A URL that points to a webpage, which lets a user access the file. This URL is required. For more information, see [Share URL](#).

PutUserInfo

Article • 03/04/2023



The `PutUserInfo` operation stores basic user information on the host.

POST /wopi/files/(file_id)

The `PutUserInfo` operation stores some basic user information on the host. When a host receives this request, they must store the `UserInfo` string, which is contained in the body of the request. The `UserInfo` string should be associated with a particular user, and should be passed back to the WOPI client in subsequent `CheckFileInfo` responses in the `UserInfo` property.

The `UserInfo` string is provided in the body of the request, and has a maximum size of 1024 ASCII characters.

WOPI clients only call this WOPI operation if the host sets the `SupportsUserInfo` property to `true` in the `CheckFileInfo` response.

Parameters

- `file_id` (*string*) – A string that specifies a [file ID](#) of a file managed by host. This string must be URL safe.

Query parameters

- `access_token` (*string*) – An [access token](#) that the host uses to determine whether the request is authorized.

Request headers

- `X-WOPI-Override` – The string `PUT_USER_INFO`. This header is required.

Body

The request body must be the full `UserInfo` string.

Status codes

- [200 OK](#)  – Success.
- [401 Unauthorized](#)  – Invalid [access token](#).
- [404 Not Found](#)  – Resource not found or user unauthorized.
- [500 Internal Server Error](#)  – Server error.
- [501 Not Implemented](#)  – Operation not supported.

CheckContainerInfo

Article • 07/12/2023



GET /wopi/containers/(container_id)

The `CheckContainerInfo` operation is similar to the [CheckFileInfo](#) operation, but operates on containers instead of files. `CheckContainerInfo` returns information about a container and a user's permissions on that container.

Parameters

- **container_id** (*string*) – A string that specifies a container ID of a container managed by host. This string must be URL safe.

Query Parameters

- **access_token** (*string*) – An [access token](#) that the host will use to determine whether the request is authorized.

Status Codes

- [200 OK](#) – Success
- [401 Unauthorized](#) – Invalid [access token](#)
- [404 Not Found](#) – Resource not found/user unauthorized
- [500 Internal Server Error](#) – Server error
- [501 Not Implemented](#) – Operation not supported

Note

In addition to the request/response headers listed here, this operation may also use the **Standard WOPI request and response headers**.

Response

The response to a `CheckContainerInfo` call is [JSON](#).

All optional values default to the following values based on their type:

Type	Default value
Boolean	<code>false</code>
String	The empty string
Integer/Long	Varies; see individual properties for details
Array	Empty array

Important

No properties should be set to `null`. If you do not wish to set a property, simply omit it from the response and WOPI clients will use the default value.

Required response properties

The following properties must be present in all `CheckContainerInfo` responses:

- **Name** - The name of the container without a path. This value will be displayed in the WOPI client UI.

Other response properties

- **HostUrl** - A URI to a webpage for the container.
- **IsAnonymousUser** (*Boolean*) - A Boolean value indicating whether the user is authenticated with the host or not. This should match the [IsAnonymousUser](#) value returned in [CheckFileInfo](#).
- **IsEduUser** (*Boolean*) - A Boolean value indicating whether the user is an education user or not. This should match the [IsEduUser](#) value returned in [CheckFileInfo](#).
- **LicenseCheckForEditIsEnabled** (*Boolean*) - A Boolean value indicating whether the user is a business user or not. This should match the [LicenseCheckForEditIsEnabled](#) value returned in [CheckFileInfo](#).

- **SharingUrl** - A URI to a webpage to allow the user to control sharing of the container. This is analogous to the [FileSharingUrl](#) in [CheckFileInfo](#).
- **SupportedShareUrlTypes** - An array of strings containing the [Share URL](#) types the host supports. The types indicate the sharing options available for the container itself, and not on the files in the container.

These types can be passed in the **X-WOPI-UrlType** request header to signify which Share URL type to return for the [GetShareUrl \(containers\)](#) operation.

<i>Possible Values</i>	<i>Description</i>
ReadOnly	This type of Share URL allows users to view the container using the URL, but does not give them permission to make changes to the container.
ReadWrite	This type of Share URL allows users to both view and make changes to the container using the URL.

- **UserCanCreateChildContainer** (*Boolean*) - A Boolean value that indicates the user has permission to create a new container in the container.
- **UserCanCreateChildFile** (*Boolean*) - A Boolean value that indicates the user has permission to create a new file in the container.
- **UserCanDelete** (*Boolean*) - A Boolean value that indicates the user has permission to delete the container.
- **UserCanRename** (*Boolean*) - A Boolean value that indicates the user has permission to rename the container.

CreateChildContainer

Article • 03/04/2023



POST (/wopi/containers/(container_id))

The `CreateChildContainer` operation creates a new child container in the provided parent container.

The `CreateChildContainer` operation has two distinct modes: *specific* and *suggested*. The primary difference between the two modes is whether the WOPI client expects the host to use the container name provided exactly (*specific* mode), or if the host can adjust the container name in order to make the request succeed (*suggested* mode).

Hosts can determine the mode of the operation based on which of the mutually exclusive **X-WOPI-RelativeTarget** (indicates *specific* mode) or **X-WOPI-SuggestedTarget** (indicates *suggested* mode) request headers is used. We'll describe the expected behavior for each mode in detail below.

Parameters

- **container_id** (*string*) – A string that specifies a container ID of a container managed by host. This string must be URL safe.

Query Parameters

- **access_token** (*string*) – An [access token](#) that the host will use to determine whether the request is authorized.

Request Headers

- **X-WOPI-Override** – The **string** `CREATE_CHILD_CONTAINER`. Required.
- **X-WOPI-SuggestedTarget** –

A UTF-7 encoded **string** that specifies a full container name. Required.

The response to a request including this header must never result in a [400 Bad Request](#) or [409 Conflict](#). Rather, the host must modify the proposed name as

needed to create a new container that is legally named.

This header must be present if **X-WOPI-RelativeTarget** is not present; the two headers are mutually exclusive. If both headers are present, the host should respond with a [501 Not Implemented](#).

- *X-WOPI-RelativeTarget* –

A UTF-7 encoded **string** that specifies a full container name. The host must not modify the name to fulfill the request.

If the specified name is illegal, the host must respond with a [400 Bad Request](#).

If a container with the specified name already exists, the host must respond with a [409 Conflict](#). When responding with a [409 Conflict](#) for this reason, the host may include an **X-WOPI-ValidRelativeTarget** specifying a container name that is valid.

This header must be present if **X-WOPI-SuggestedTarget** is not present; the two headers are mutually exclusive. If both headers are present, the host should respond with a [501 Not Implemented](#).

Response Headers

- *X-WOPI-InvalidContainerNameError* – A **string** describing the reason the `CreateChildContainer` operation could not be completed. This header should only be included when the response code is [400 Bad Request](#). This string is only used for logging purposes.
- **Status Codes**
 - [200 OK](#) – Success
 - [400 Bad Request](#) – Specified name is illegal
 - [401 Unauthorized](#) – Invalid [access token](#)
 - [404 Not Found](#) – Resource not found/user unauthorized
 - [409 Conflict](#) – Target container already exists
 - [500 Internal Server Error](#) – Server error
 - [501 Not Implemented](#) – Operation not supported

ⓘ Note

In addition to the request/response headers listed here, this operation may also use the **Standard WOPI request and response headers**.

Response

The response to a `CreateChildContainer` call is [JSON](#) [↗].

All optional values default to the following values based on their type:

Type	Default value
Boolean	<code>false</code>
String	The empty string
Integer/Long	Varies; see individual properties for details
Array	Empty array

ⓘ Important

No properties should be set to `null`. If you do not wish to set a property, simply omit it from the response; WOPI clients will use the default value in these cases.

Required response properties

The following properties must be present in all `CreateChildContainer` responses:

- **ContainerPointer** - A JSON-formatted object containing the following properties:
- **Name** - The name of the container without a path. This value should match the Name property in a [CheckContainerInfo](#) response. Required.
- **Url** - A URI to the container, including a valid [access token](#). Required.

⊗ Caution

This property includes an **access token**, and thus has important security implications. See **Preventing 'token trading'** for more details.

Other response properties

- **ContainerInfo** - Hosts can optionally include the ContainerInfo property, which should match the [CheckContainerInfo](#) response for the newly created container.

If not provided, the WOPI client will call [CheckContainerInfo](#) to retrieve it. Including this property in the response is strongly recommended so that the WOPI client does not need to make an additional call to [CheckContainerInfo](#).

Sample response:

JSON

```
{
  "ContainerPointer" : {
    "Url" : "http://.../wopi*/containers/<containerId>?access_token=
<per_container_token>",
    "Name" : "Container Name"
  },
  "ContainerInfo" : {
    "Name" : "Container Name",
    "HostUrl" : "",
    "SharingUrl" : "",
    "UserCanCreateChildContainer" : false,
    "UserCanCreateChildFile" : false,
    "UserCanDelete" : false,
    "UserCanRename" : false
  }
}
```

CreateChildFile

Article • 03/04/2023



POST /wopi/containers/(container_id)

The `CreateChildFile` operation creates a new file in the provided container. The resulting file must be zero bytes in length.

The `CreateChildFile` operation has two distinct modes: *specific* and *suggested*. The primary difference between the two modes is whether the WOPI client expects the host to use the file name provided exactly (*specific* mode), or if the host can adjust the file name in order to make the request succeed (*suggested* mode).

Hosts can determine the mode of the operation based on which of the mutually exclusive **X-WOPI-RelativeTarget** (indicates *specific* mode) or **X-WOPI-SuggestedTarget** (indicates *suggested* mode) request headers is used. We'll describe the expected behavior for each mode in detail below.

Note that a file matching the target name might be locked, and in *specific* mode, the host must respond with a [409 Conflict](#) and include a **X-WOPI-Lock** response header as described below.

Parameters

- **container_id** (*string*) – A string that specifies a container ID of a container managed by host. This string must be URL safe.

Query Parameters

- **access_token** (*string*) – An [access token](#) that the host will use to determine whether the request is authorized.

Request Headers

- **X-WOPI-Override** – The string `CREATE_CHILD_FILE`. Required.
- **X-WOPI-SuggestedTarget** –

A UTF-7 encoded **string** specifying either a file extension or a full file name, including the file extension. Hosts can differentiate between full file names and extensions as follows:

- If the string begins with a period (`.`), it's a file extension.
- Otherwise, it's a full file name.

If only the extension is provided, the name of the initial file without extension should be combined with the extension to create the proposed name.

The response to a request including this header must never result in a [400 Bad Request](#) or [409 Conflict](#). Rather, the host must modify the proposed name as needed to create a new file that is both legally named and does not overwrite any existing file, while preserving the file extension.

This header must be present if **X-WOPI-RelativeTarget** is not present; the two headers are mutually exclusive. If both headers are present, the host should respond with a [501 Not Implemented](#).

- *X-WOPI-RelativeTarget* –

A UTF-7 encoded **string** that specifies a full file name including the file extension. The host must not modify the name to fulfill the request.

If the specified name is illegal, the host must respond with a [400 Bad Request](#).

If a file with the specified name already exists, the host must respond with a [409 Conflict](#), unless the **X-WOPI-OverwriteRelativeTarget** request header is set to `true`. When responding with a [409 Conflict](#) for this reason, the host may include an **X-WOPI-ValidRelativeTarget** specifying a file name that is valid.

If the **X-WOPI-OverwriteRelativeTarget** request header is set to `true` and a file with the specified name already exists and is locked, the host must respond with a [409 Conflict](#) and include an **X-WOPI-Lock** response header containing the value of the current lock on the file.

This header must be present if **X-WOPI-SuggestedTarget** is not present; the two headers are mutually exclusive. If both headers are present, the host should respond with a [501 Not Implemented](#).

- *X-WOPI-OverwriteRelativeTarget* –

A **Boolean** value that specifies whether the host must overwrite the file name if it exists. The default value is `false`. In other words, if **X-WOPI-**

OverwriteRelativeTarget is not explicitly included on the request, hosts must behave as though its value is `false`.

This header is only valid if the **X-WOPI-RelativeTarget** is also included on the request. It must be ignored in all other cases.

If the user is not authorized to overwrite the target file, the host must respond with a [501 Not Implemented](#).

Response Headers

- *X-WOPI-InvalidFileNameError* – A **string** describing the reason the `CreateChildFile` operation could not be completed. This header should only be included when the response code is [400 Bad Request](#). This string is only used for logging purposes.
- *X-WOPI-ValidRelativeTarget* – A UTF-7 encoded **string** that specifies a full file name including the file extension. This header may be used when responding with a [409 Conflict](#), because a file with the requested name already exists, or when responding with a [400 Bad Request](#), because the requested name contained invalid characters. If this response header is included, the WOPI client should automatically retry the `CreateChildFile` operation using the contents of this header as the **X-WOPI-RelativeTarget** value and should not display an error message to the user.

Status Codes

- [200 OK](#) – Success
- [400 Bad Request](#) – Specified name is illegal
- [401 Unauthorized](#) – Invalid [access token](#)
- [404 Not Found](#) – Resource not found/user unauthorized
- [409 Conflict](#) – Target file already exists
- [500 Internal Server Error](#) – Server error
- [501 Not Implemented](#) – Operation not supported

 **Note**

In addition to the request/response headers listed here, this operation may also use the **Standard WOPI request and response headers**.

Response

The response to a `CreateChildFile` call is [JSON](#).

All optional values default to the following values based on their type:

Type	Default value
Boolean	<code>false</code>
String	The empty string
Integer/Long	Varies; see individual properties for details
Array	Empty array

ⓘ Important

No properties should be set to `null`. If you do not wish to set a property, simply omit it from the response; WOPI clients will use the default value in these cases.

Required response properties

The following properties must be present in all `CreateChildFile` responses:

- **Name** - The **string** name of the file, including extension, without a path.
- **Url** - A **string** URI of the form `http://server/<...>/wopi/files/(file_id)?access_token=(access token)`, of the newly created file on the host. This URL is the **WOPISrc** for the new file with an **access token** appended. Or, stated differently, it is the URL to the host's Files endpoint for the new file, along with an **access token**. A **GET** request to this URL will invoke the **CheckFileInfo** operation.

⊗ Caution

This property includes an **access token**, and thus has important security implications. See **Preventing 'token trading'** for more details.

Optional response properties

- **HostViewUrl** - The [HostViewUrl](#), as a **string**, for the newly created file. This should match the value returned in [CheckFileInfo](#).
- **HostEditUrl** A URI to the [host page](#) that loads the `editnew` WOPI action.

The [HostEditUrl](#), as a **string**, for the newly created file. This should match the value returned in [CheckFileInfo](#).

DeleteContainer

Article • 03/04/2023



POST /wopi/containers/(container_id)

The `DeleteContainer` operation deletes a container.

Parameters

- **container_id** (*string*) – A string that specifies a container ID of a container managed by host. This string must be URL safe.

Query Parameters

- **access_token** (*string*) – An [access token](#) that the host will use to determine whether the request is authorized.

Request Headers

- *X-WOPI-Override* – The **string** `DELETE_CONTAINER`. Required.

Status Codes

- [200 OK](#) – Success
- [404 Not Found](#) – Resource not found/user unauthorized
- [409 Conflict](#) – Container has child files/containers
- [500 Internal Server Error](#) – Server error
- [501 Not Implemented](#) – Operation not supported

Note

In addition to the request/response headers listed here, this operation may also use the **Standard WOPI request and response headers**.

EnumerateAncestors (containers)

Article • 03/04/2023



GET /wopi/containers/(container_id)/ancestry

The `EnumerateAncestors` operation enumerates all the parents of a given container, up to and including the root container.

Parameters

- **container_id** (*string*) – A string that specifies a container ID of a container managed by host. This string must be URL safe.

Query Parameters

- **access_token** (*string*) – An [access token](#) that the host will use to determine whether the request is authorized.

Response Headers

- *X-WOPI-EnumerationIncomplete* –

An optional header indicating that the enumeration of the container's ancestry is incomplete. If set, the value of this header must be the string `true`.

A WOPI client may choose to issue additional `EnumerateAncestors` requests to complete the enumeration.

Status Codes

- [200 OK](#) [↗](#) – Success
- [401 Unauthorized](#) [↗](#) – Invalid [access token](#)
- [404 Not Found](#) [↗](#) – Resource not found/user unauthorized
- [500 Internal Server Error](#) [↗](#) – Server error

- [501 Not Implemented](#) – Operation not supported

ⓘ Note

In addition to the request/response headers listed here, this operation may also use the **Standard WOPI request and response headers**.

Response

The response to a `EnumerateAncestors` call is [JSON](#) containing the following required properties:

- **AncestorsWithRootFirst** - An array of JSON-formatted objects containing the following properties:
 - **Name** - The name of the container without a path. This value should match the Name property in a [CheckContainerInfo](#) response. Required.
 - **Url** - A URI to the container, including a valid [access token](#). Required.

⊗ Caution

This property includes an **access token**, and thus has important security implications. See **Preventing 'token trading'** for more details.

The array must always be ordered such that the ancestor closest to the root is the first element.

If there are no ancestor containers, this property should be an empty array.

Consider a file in the following container hierarchy:

`/root/grandparent/parent/mycontainer.`

When called on this container, the `EnumerateAncestors` operation should return the following:

JSON

```
{
  "AncestorsWithRootFirst": [
    {
      "Url": "http://.../wopi*/containers/<containerId>?access_token=
<per_container_token>",
```

```
    "Name": "root"
  },
  {
    "Url": "http://.../wopi*/containers/<containerId2>?access_token=
<per_container_token2>",
    "Name": "grandparent"
  },
  {
    "Url": "http://.../wopi*/containers/<containerId3>?access_token=
<per_container_token3>",
    "Name": "parent"
  }
]
}
```

EnumerateChildren (containers)

Article • 03/04/2023



GET /wopi/containers/(container_id)/children

The EnumerateChildren operation enumerates all the immediate children of a given container. Note that paging is deliberately not supported by this operation. Responses are expected to include all immediate children of the container.

Parameters

- **container_id** (*string*) – A string that specifies a container ID of a container managed by host. This string must be URL safe.

Query Parameters

- **access_token** (*string*) – An [access token](#) that the host will use to determine whether the request is authorized.

Request Headers

- *X-WOPI-FileExtensionFilterList* –

A **string** value that the host must use to filter the returned child files. This header must be a list of comma-separated file extensions with a leading dot (.). There must be no whitespace and no trailing comma in the string. Wildcard characters are not permitted.

If this header is included, the host must only return child files whose file extensions match the filter list, based on a case-insensitive match. For example, if this header is set to the value `.doc,.docx`, the host should include only files with the `doc` or `docx` file extension.

This header value only affects the child files that are returned; it does not have any effect on child containers.

Status Codes

- [200 OK](#) – Success
- [401 Unauthorized](#) – Invalid [access token](#)
- [404 Not Found](#) – Resource not found/user unauthorized
- [500 Internal Server Error](#) – Server error
- [501 Not Implemented](#) – Operation not supported

ⓘ Note

Standard WOPI request and response headers

In addition to the request/response headers listed here, this operation may also use the [Standard WOPI request and response headers](#).

Response

The response to a `EnumerateChildren` call is [JSON](#).

All optional values default to the following values based on their type:

Type	Default value
Boolean	<code>false</code>
String	The empty string
Integer/Long	Varies; see individual properties for details
Array	Empty array

ⓘ Important

No properties should be set to `null`. If you do not wish to set a property, simply omit it from the response; the WOPI clients will use the default value in these cases.

Required response properties

The following properties must be present in all `EnumerateChildren` responses:

- **ChildContainers** - An array of JSON-formatted objects containing the following properties:
 - **Name** - The name of the container without a path. This value should match the *Name* property in a [CheckContainerInfo](#) response. Required.
 - **Url** - A URI to the container, including a valid [access token](#). Required.

⊗ **Caution**

This property includes an **access token**, and thus has important security implications. See **Preventing 'token trading'** for more details.

If there are no child containers, this property should be an empty array.

- **ChildFiles** - An array of JSON-formatted objects containing the following properties:
 - **Name** - Should match the [BaseFileName](#) property in [CheckFileInfo](#). Required.
 - **Size** - Should match the [Size](#) property in [CheckFileInfo](#). Required.
 - **Url** - A URI to the file, including a valid [access token](#). Required.

⊗ **Caution**

This property includes an **access token**, and thus has important security implications. See **Preventing 'token trading'** for more details.

- **Version** - Should match the [Version](#) property in [CheckFileInfo](#). Required.
- **LastModifiedTime** - Should match the [LastModifiedTime](#) property in [CheckFileInfo](#). Optional.

If there are no child files, this property should be an empty array.

Sample response:

JSON

```
{
  "ChildContainers": [
    {
      "Url": "http://.../wopi*/containers/<containerId>?access_token=
<per_file_token>",
```

```
        "Name": "FolderName"
    },
    {
        "Url": "http://.../wopi*/containers/<containerId2>?access_token=
<per_file_token2>",
        "Name": "FolderName2"
    }
],
"ChildFiles": [
    {
        "Url": "http://.../wopi*/files/<fileId>?access_token=
<per_file_token>",
        "Name": "FileName",
        "Version": "version1",
        "Size": 7,
        "LastModifiedTime": "time last modified"
    }
]
}
```

GetShareUrl (containers)

Article • 03/04/2023



ⓘ Note

GetShareUrl (files) returns a Share URL suitable for viewing a shared *file* when launched in a web browser.

POST (/wopi/containers/(container_id))

The GetShareUrl operation returns a [Share URL](#) that is suitable for viewing a shared container when launched in a web browser. A host can support multiple Share URL types, as described by the [SupportedShareUrlTypes](#) property. The **X-WOPI-UrlType** request header contains the Share URL type that should be returned.

If the **X-WOPI-UrlType** header is not present or contains a value that is invalid or not supported by the host, the host should respond with a [501 Not Implemented](#) [↗](#).

Parameters

- **container_id** (*string*) – A string that specifies a container ID of a container managed by host. This string must be URL safe.

Query Parameters

- **access_token** (*string*) – An [access token](#) that the host will use to determine whether the request is authorized.

Request Headers

- **X-WOPI-Override** – The **string** `GET_SHARE_URL`. Required.
- **X-WOPI-UrlType** – A **string** indicating what Share URL type to return. Required.

Status Codes

- [200 OK](#) – Success
- [401 Unauthorized](#) – Invalid [access token](#)
- [404 Not Found](#) – Resource not found/user unauthorized
- [500 Internal Server Error](#) – Server error
- [501 Not Implemented](#) – Operation not supported

ⓘ Note

In addition to the request/response headers listed here, this operation may also use the **Standard WOPI request and response headers**.

Response

The response to a `GetShareUrl` call is [JSON](#) containing the following required properties:

- **ShareUrl** - A URI that points to a webpage that allows the user to access the container. Required.

ⓘ Note

Learn more about **Share Url**.

RenameContainer

Article • 03/04/2023



POST (/wopi/containers/(container_id))

The RenameContainer operation renames a container.

Parameters

- **container_id** (*string*) – A string that specifies a container ID of a container managed by host. This string must be URL safe.

Query Parameters

- **access_token** (*string*) – An [access token](#) that the host will use to determine whether the request is authorized.

Request Headers

- *X-WOPI-Override* – The **string** `RENAME_CONTAINER`. Required.
- *X-WOPI-RequestedName* – A UTF-7 encoded **string** that is a container name. Required.

Response Headers

- *X-WOPI-InvalidContainerNameError* – A **string** describing the reason the RenameContainer operation could not be completed. This header should only be included when the response code is [400 Bad Request](#). This string is only used for logging purposes.

Status Codes

- [200 OK](#) – Success
- [400 Bad Request](#) – Specified name is illegal

- [401 Unauthorized](#) – Invalid [access token](#)
- [404 Not Found](#) – Resource not found/user unauthorized
- [409 Conflict](#) – Target container already exists
- [500 Internal Server Error](#) – Server error
- [501 Not Implemented](#) – Operation not supported

ⓘ Note

In addition to the request/response headers listed here, this operation may also use the **Standard WOPI request and response headers**.

Response

The response to a `RenameContainer` call is [JSON](#) containing the following required property:

- **Name** (*string*) - The name of the renamed container.

CheckEcosystem

Article • 03/04/2023



GET /wopi/ecosystem

The CheckEcosystem operation is similar to the the [CheckFileInfo](#) operation, but does not require a file or container ID.

Query Parameters

- **access_token** (*string*) – An [access token](#) that the host will use to determine whether the request is authorized.

Status Codes

- [200 OK](#) – Success
- [401 Unauthorized](#) – Invalid [access token](#)
- [404 Not Found](#) – Resource not found/user unauthorized
- [500 Internal Server Error](#) – Server error

Response

The response to a CheckEcosystem call is [JSON](#).

All optional values default to the following values based on their type:

Type	Default value
Boolean	<code>false</code>
String	The empty string
Integer/Long	Varies; see individual properties for details
Array	Empty array

Important

No properties should be set to `null`. If you do not wish to set a property, simply omit it from the response; WOPI clients will use the default value in these cases.

Optional response properties

- **SupportsContainers** - Should match the [SupportsContainers](#) property in [CheckFileInfo](#).

Note

Since all properties in the CheckEcosystem response are optional, an empty JSON response is valid.

GetEcosystem (files)

Article • 03/04/2023

Mobile



Desktop



ⓘ Note

GetEcosystem (containers) returns the URI for the WOPI server's Ecosystem endpoint, given a *container* ID.

GET /wopi/files/(file_id)/ecosystem_pointer

The GetEcosystem operation returns the URI for the WOPI server's Ecosystem endpoint, given a file ID.

Parameters

- **file_id** (*string*) – A string that specifies a [file ID](#) of a file managed by host. This string must be URL safe.

Query Parameters

- **access_token** (*string*) – An [access token](#) that the host will use to determine whether the request is authorized.

Status Codes

- [200 OK](#) – Success
- [401 Unauthorized](#) – Invalid [access token](#)
- [404 Not Found](#) – Resource not found/user unauthorized
- [500 Internal Server Error](#) – Server error
- [501 Not Implemented](#) – Operation not supported

ⓘ Note

Standard WOPI request and response headers

In addition to the request/response headers listed here, this operation may also use the [Standard WOPI request and response headers](#).

Response

The response to a GetEcosystem call is [JSON](#) containing the following required properties:

Url

A **string** URI for the WOPI server's Ecosystem endpoint, with an [access token](#) appended. A [GET](#) request to this URL will invoke the [CheckEcosystem](#) operation.

⊗ Caution

This property includes an **access token**, and thus has important security implications. See **Preventing 'token trading'** for more details.

GetEcosystem (containers)

Article • 03/04/2023



GET (/wopi/containers/(container_id)/ecosystem_pointer

The GetEcosystem operation returns the URI for the WOPI server's Ecosystem endpoint, given a container ID.

ⓘ Note

GetEcosystem (files) returns the URI for the WOPI server's Ecosystem endpoint, given a *file* ID.

Parameters

- **container_id** (*string*) – A string that specifies a container ID of a container managed by host. This string must be URL safe.

Query Parameters

- **access_token** (*string*) – An [access token](#) that the host will use to determine whether the request is authorized.

Status Codes

- [200 OK](#) – Success
- [401 Unauthorized](#) – Invalid [access token](#)
- [404 Not Found](#) – Resource not found/user unauthorized
- [500 Internal Server Error](#) – Server error
- [501 Not Implemented](#) – Operation not supported

ⓘ Note

In addition to the request/response headers listed here, this operation may also use the **Standard WOPI request and response headers**.

Response

The response to a `GetEcosystem` call is [JSON](#) containing the following required properties:

- **Url** (*string*)- A URI for the WOPI server's Ecosystem endpoint, with an [access token](#) appended. A [GET](#) request to this URL will invoke the [CheckEcosystem](#) operation.

⊗ Caution

This property includes an **access token**, and thus has important security implications. See **Preventing 'token trading'** for more details.



GetFileWopiSrc (ecosystem)

Article • 03/04/2023

POST /wopi/ecosystem

⊗ Caution

This operation *should not* be called by WOPI clients at this time. It is reserved for future use and subject to change.

The GetFileWopiSrc operation is used to convert a host-specific file identifier into a [WopiSrc](#) value.

The WOPI client passes the host-specific file identifier in the **X-WOPI-HostNativeFileName** header. The host, in turn, returns a WopiSrc URL with an [access token](#) appended.

This operation is useful in cases where a WOPI client receives a file identifier that is not a proper WopiSrc, but can be translated into a valid WopiSrc value by the WOPI host.

For example, iOS applications can open files in Microsoft 365 for mobile using URL schemes. However, it may not be feasible for an application to generate a WopiSrc value itself. As long as it can generate a string value that can later be converted to a WopiSrc value by calling this operation, then the application can pass this value and rely on Microsoft 365 for mobile to call this operation to convert the file identifier into a WopiSrc.

📘 Important

While a WOPI host can accept any string value as input, hosts are strongly recommended to support the following values in **X-WOPI-HostNativeFileName**:

- [HostEditUrl](#)
- [HostViewUrl](#)
- Any value the host returns in response to a [GetShareUrl \(files\)](#) call

📌 Note

This operation is also exposed as a **shortcut operation**.

Query Parameters

- `access_token` (*string*) – An [access token](#) that the host will use to determine whether the request is authorized.

Request Headers

- `X-WOPI-Override` – The **string** `GET_WOPI_SRC_WITH_ACCESS_TOKEN`. Required.
- `X-WOPI-HostNativeFileName` – A **string** representing the host-specific file identifier for a file.

Status Codes

- [200 OK](#)  – Success
- [401 Unauthorized](#)  – Invalid [access token](#)
- [404 Not Found](#)  – Resource not found/user unauthorized
- [500 Internal Server Error](#)  – Server error
- [501 Not Implemented](#)  – Operation not supported

Note

In addition to the request/response headers listed here, this operation may also use the **Standard WOPI request and response headers**.

Response

The response to a `GetFileWopiSrc` call is [JSON](#)  containing the following required property:

- `Url` - A URI that represents a [WopiSrc](#) value with an [access token](#) appended.

Caution

This property includes an **access token**, and thus has important security implications. See **Preventing 'token trading'** for more details.

Sample response:

JSON

```
{
  "Url": "http://.../wopi*/[containers|files]/<id>?access_token=
<file|container_token>&access_token_ttl=<timestamp>"
}
```


GetRootContainer (ecosystem)

Article • 03/04/2023



GET /wopi/ecosystem/root_container_pointer

The GetRootContainer operation returns the root container. A WOPI client can use this operation to get a reference to the root container, from which the client can call [EnumerateChildren \(containers\)](#) to navigate a container hierarchy.

ⓘ Note

This operation is also exposed as a **shortcut operation**.

Query Parameters

- **access_token** (*string*) – An [access token](#) that the host will use to determine whether the request is authorized.

Status Codes

- [200 OK](#) [↗] – Success
- [401 Unauthorized](#) [↗] – Invalid [access token](#)
- [404 Not Found](#) [↗] – Resource not found/user unauthorized
- [500 Internal Server Error](#) [↗] – Server error
- [501 Not Implemented](#) [↗] – Operation not supported

ⓘ Note

In addition to the request/response headers listed here, this operation may also use the **Standard WOPI request and response headers**.

Response

The response to a `GetRootContainer` call is [JSON](#).

All optional values default to the following values based on their type:

Type	Default value
Boolean	<code>false</code>
String	The empty string
Integer/Long	Varies; see individual properties for details
Array	Empty array

Important

No properties should be set to `null`. If you do not wish to set a property, simply omit it from the response; WOPI clients will use the default value in these cases.

Required response properties

The following properties must be present in all `GetRootContainer` responses:

- **ContainerPointer** - A JSON-formatted object containing the following properties:
- **Name** The name of the container without a path. This value should match the *Name* property in a [CheckContainerInfo](#) response. Required.
- **Url** - A URI to the container, including a valid [access token](#). Required.

Caution

This property includes an **access token**, and thus has important security implications. See **Preventing 'token trading'** for more details.

Other response properties

- **ContainerInfo** - Hosts can optionally include the `ContainerInfo` property, which should match the [CheckContainerInfo](#) response for the root container.

If not provided, the WOPI client will call [CheckContainerInfo](#) to retrieve it. We strongly recommend including this property in the response so that the WOPI client does not need to make an additional call to [CheckContainerInfo](#).

Sample response:

JSON

```
{
  "ContainerPointer" : {
    "Url" : "http://.../wopi*/containers/<containerId>?access_token=
<per_container_token>",
    "Name" : "Container Name"
  },
  "ContainerInfo" : {
    "Name" : "Container Name",
    "HostUrl" : "",
    "SharingUrl" : "",
    "UserCanCreateChildContainer" : false,
    "UserCanCreateChildFile" : false,
    "UserCanDelete" : false,
    "UserCanRename" : false
  }
}
```

Bootstrap

Article • 03/04/2023

Mobile



Desktop



GET /wopibootstrapper

The Bootstrap operation is used to 'convert' an OAuth token into appropriate WOPI [access tokens](#) and provides access to WOPI operations for applications using OAuth tokens, like Microsoft 365 for mobile. For more information see [native](#).

Important

Connections to the bootstrapper must be made using TLS.

Request Headers

- [Authorization](#)  – A **string** in the format `Bearer: <TOKEN>` where `<TOKEN>` is a Base64-encoded OAuth 2.0 token. If this header is missing or blank, or if the token provided is invalid, the host must respond with a [401 Unauthorized](#)  response and include the [WWW-Authenticate](#)  header, as described in the next section.

Response Headers

- [WWW-Authenticate](#)  – A **string** value formatted as described in [WWW-Authenticate response header format](#).
- [Content-Type](#)  – Must be `application/json` for authenticated responses.

Status Codes

- [200 OK](#)  – Success
- [401 Unauthorized](#)  – Authorization failure; when responding with this status code, hosts must include a [WWW-Authenticate](#)  response header with values as described in [WWW-Authenticate response header format](#)
- [404 Not Found](#)  – Resource not found/user unauthorized

- [500 Internal Server Error](#) – Server error

Response

The response to a Bootstrap operation differs based on whether it's **authenticated** or **unauthenticated**. When possible, the WOPI client will provide authentication state information, as defined by OAuth 2.0 protocol, in the HTTP header in every request to the Bootstrapper endpoint. This authentication state will be sent in the form of the OAuth 2.0 Access token, and will be contained in the [Authorization](#) header as described previously.

The host must verify that the provided token is valid. If it isn't, the host must respond as described in the [unauthenticated response](#) section of this article. If the provided token is valid, then the host must respond as described in the [authenticated response](#) section of this article.

Authenticated response

When an **authenticated** request (i.e. a valid OAuth 2.0 access token is included in the [Authorization](#) HTTP header) is made to this endpoint, it returns a [200 OK](#) response with a [JSON](#) response body. The response must include a single `Bootstrap` property, with the following properties nested within it as needed.

Required response properties

The following properties must be present in the `Bootstrap` property in all [200 OK](#) Bootstrap responses:

EcosystemUrl

A **string** URI for the WOPI server's Ecosystem endpoint, with a WOPI [access token](#) appended. A [GET](#) request to this URL will invoke the [CheckEcosystem](#) operation.

UserId

A **string** value uniquely identifying the user making the request. This value should match the [UserId](#) value provided in [CheckFileInfo](#). This ID is expected to be unique per user and consistent over time. See [Requirements for user identity properties](#) for more information.

SignInName

A **string** value identifying the user making the request. This value is used to distinguish a user's account in the event a user has multiple accounts with a given host. This value is often an email address, though it isn't required to be.

Optional response properties

UserFriendlyName

A **string** that is the name of the user. This value should match the [UserFriendlyName](#) value provided in [CheckFileInfo](#).

Sample response

JSON

```
{
  "Bootstrap": {
    "EcosystemUrl": "http://.../wopi*/ecosystem?access_token=
<ecosystem_token>",
    "UserId": "User ID",
    "SignInName": "user@contoso.com",
    "UserFriendlyName": "User Name"
  }
}
```

Unauthenticated response

When an **unauthenticated** request (i.e. no access token is attached in the [Authorization](#) HTTP header) is made to this endpoint, it returns a **401 Unauthorized** response containing sufficient information to facilitate user authentication with the host.

WWW-Authenticate response header format

The response must contain sufficient information for a WOPI client to perform the necessary authentication/authorization/token issuance flows with the host's identity provider, and result in an authenticated call to the same Bootstrapper endpoint.

The information for the successful authentication/authorization/token issuance flows must be returned in the [WWW-Authenticate](#) header of the **401 Unauthorized**

response with type "Bearer." The information that must be returned in a [401 Unauthorized](#) response to an unauthenticated request is as follows:

Parameter	Value	Required	Example
Bearer	n/a	Yes	Bearer
authorization_uri	The URL of the OAuth2 Authorization Endpoint to begin authentication against as described at: RFC 6749#section-3.1	Yes	https://contoso.com/api/oauth2/authorize
tokenIssuance_uri	The URL of the OAuth2 Token Endpoint where authentication code can be redeemed for an access and (optional) refresh token. See Token Endpoint at: RFC 6749#section-3.2	Yes	https://contoso.com/api/oauth2/token
providerId	A well-known string (as registered with Microsoft 365) that uniquely identifies the host. Allowed characters: [a-z, A-Z, 0-9]	No	tp_contoso

Parameter	Value	Required	Example
UrlSchemes	URL scheme your app uses. This is an ordered list by platform. Omit any platforms you don't support. Office will attempt to invoke these URL schemes in order before falling back to the webview auth.	No	<pre>{ "iOS" : ["contoso","contoso-EMM"], "Android" : ["contoso","contoso-EMM"] "UWP": ["contoso","contoso-EMM"]} </pre> The value itself must be URL encoded

These parameters and their values must be formatted as follows:

- Values are contained within double-quotes (").
- Contiguous parameters are separated by commas with no comma after the trailing parameter/value pair.
- If no value is known/required for an optional parameter, it may be omitted from the [WWW-Authenticate](#) header.
- Multiple instances of [WWW-Authenticate](#) HTTP headers may exist in the response to an unauthenticated request to the Bootstrapper endpoint. However, there must be exactly one instance of the [WWW-Authenticate](#) header with the `Bearer` qualifier.

Sample unauthenticated response

text

```
HTTP/1.1 401 Unauthorized
<removed for brevity>
WWW-Authenticate: Bearer
authorization_uri="https://contoso.com/api/oauth2/authorize",tokenIssuance_u
ri="https://contoso.com/api/oauth2/token",providerId="tp_contoso",
UrlSchemes="%7B%22iOS%22%20%3A%20%5B%22contoso%22%2C%22contoso-
EMM%22%5D%2C%20%22Android%22%20%3A%20%5B%22contoso%22%2C%22contoso-
EMM%22%5D%2C%20%22UWP%22%3A%20%5B%22contoso%22%2C%22contoso-EMM%22%5D%7D"
Date: Wed, 24 Jun 2015 21:52:44 GMT
```


GetNewAccessToken

Article • 03/04/2023



POST /wopibootstrapper

The GetNewAccessToken operation is used to retrieve a fresh WOPI [access token](#) for a given resource (i.e. a file or container), provided the caller has a valid OAuth 2.0 token.

This operation is called by OAuth-capable WOPI clients, such as Microsoft 365 for mobile, to refresh WOPI access tokens when they expire.

Request Headers

- *X-WOPI-EcosystemOperation* - The string `GET_NEW_ACCESS_TOKEN`. Required.
- *X-WOPI-WopiSrc* - The [WopiSrc](#) for the file or container. Required.

Important

To reduce the likelihood of token spoofing or other unauthorized access, hosts *must* validate the URL provided in the `X-WOPI-WopiSrc` header. The bootstrapper must only provide a WOPI access token if the requested `WopiSrc` exists and the user is authorized to access it. If not, or if the `X-WOPI-WopiSrc` header is not present, the host should return a `404 Not Found` response as described below.

Important

In addition, the Microsoft M365 for Mobile apps on both iOS (version 2.62 and later) and Android (version 16.0.15330 and later) and the Office desktops apps (applicable for CSPP Plus integrations) will also validate that the resource specified by `X-WOPI-WopiSrc` is in a trusted domain (See [Onboarding information](#)). Any WOPI requests for resources outside of trusted domains will fail.

- [Authorization](#) – A string in the format `Bearer: <TOKEN>`, where `<TOKEN>` is a Base64-encoded OAuth 2.0 token. If this header is missing, or the token provided is invalid, the host must respond with a [401 Unauthorized](#) response and include the [WWW-Authenticate](#) header as described in [WWW-Authenticate response header format](#).

Response Headers

- [WWW-Authenticate](#) – A **string** value formatted as described in [WWW-Authenticate response header format](#). This header should only be included when responding with a [401 Unauthorized](#).

Status Codes

- [200 OK](#) – Success
- [401 Unauthorized](#) – Authorization failure; when responding with this status code, hosts must include a [WWW-Authenticate](#) response header with values as described in [WWW-Authenticate response header format](#)
- [404 Not Found](#) – Resource not found/user unauthorized
- [500 Internal Server Error](#) – Server error

Response

The response to a `GetNewAccessToken` call is [JSON](#) containing the following required properties:

- **Bootstrap** - The contents of this property should be the response to a [Bootstrap](#) call.
- **AccessTokenInfo** - The contents of this property should be a the nested JSON-formatted object with the following properties:
- **AccessToken** - A **string** [access](#) token for the file specified in the **X-WOPI-WopiSrc** request header.
- **AccessTokenExpiry** - A **long** value representing the time that the [access token](#) provided in the response will expire. See [access_token_ttl](#) for more information on how this value is defined.

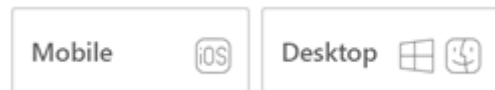
Sample response:

JSON

```
{
  "Bootstrap": {
    "EcosystemUrl": "http://.../wopi*/ecosystem?access_token=
<ecosystem_token>",
    "UserId": "User ID",
    "SignInName": "user@contoso.com",
    "UserFriendlyName": "User Name"
  },
  "AccessTokenInfo": {
    "AccessToken": "1234567890abcdef",
    "AccessTokenExpiry": 1234567890
  }
}
```

Shortcut operations

Article • 03/04/2023



The following operations are defined on the Bootstrapper endpoint in addition to the Ecosystem endpoint. They are provided on the Bootstrapper endpoint as shortcuts to reduce HTTP round trips for native applications that use the Bootstrapper. They are equivalent to calling [Bootstrap](#), then using the [EcosystemUrl](#) to immediately execute an operation on the Ecosystem endpoint.

For example, in order to get the root container, a WOPI client might execute the following operations:

1. Call [Bootstrap](#).
2. Using the [EcosystemUrl](#) from the [Bootstrap](#) response, call the [GetRootContainer \(ecosystem\)](#) operation.

However, using the [GetRootContainer \(bootstrapper\)](#) shortcut operation, the WOPI client can skip calling [Bootstrap](#), and instead get the root container directly using an OAuth 2.0 token.

Important

Implementing shortcut operations is optional. If a host does not implement them, they should simply respond to the request as though it is a standard [Bootstrap](#) request. In other words, a host that doesn't implement shortcut operations can simply ignore the **X-WOPI-EcosystemOperation** header and treat the request as a standard [Bootstrap](#) request.

WOPI clients must thus expect that some shortcut operations will not include the data expected, and should fall back to calling the appropriate operation on the Ecosystem endpoint in such cases.

With this in mind, the responses for each 'shortcut' operation are a combination of the [Bootstrap](#) response and the response for the corresponding operation on the Ecosystem endpoint. The Bootstrapper response is contained in the `Bootstrap` property, and the operation's response is contained in a separate property with a name specific to the operation.

For example, the response to the [GetRootContainer \(bootstrapper\)](#) shortcut operation looks like this:

JSON

```
{
  "Bootstrap": {
    "EcosystemUrl": "http://.../wopi*/ecosystem?access_token=
<ecosystem_token>",
    "UserId": "User ID",
    "SignInName": "user@contoso.com",
    "UserFriendlyName": "User Name"
  },
  "RootContainerInfo": {
    "ContainerPointer" : {
      "Url" : "http://.../wopi*/containers/<containerId>?access_token=
<per_container_token>",
      "Name" : "Container Name"
    },
    "ContainerInfo" : {
      "Name" : "Container Name",
      "HostUrl" : "http://contoso/324332",
      "SharingUrl" : "http://contoso/323432/efewos",
      "UserCanCreateChildContainer" : false,
      "UserCanCreateChildFile" : false,
      "UserCanDelete" : false,
      "UserCanRename" : false
    }
  }
}
```

Note that since these operations are exposed through the Bootstrapper endpoint, they will be called using OAuth 2.0 access tokens, not WOPI access tokens.

-  [GetFileWopiSrc \(bootstrapper\)](#)
- [GetRootContainer \(bootstrapper\)](#)



GetFileWopiSrc (bootstrapper)

Article • 03/04/2023

⚠ Warning

This operation should not be called by WOPI clients at this time. It is reserved for future use and subject to change.

POST /wopibootstrapper

This operation is equivalent to the  [GetFileWopiSrc \(ecosystem\)](#) operation.

📘 Important

The request/response semantics for this operation differ slightly from its counterpart on the Ecosystem endpoint since it is exposed on the Bootstrapper endpoint, and thus will use OAuth 2.0 access tokens for authorization instead of WOPI access tokens.

• Request Headers

- *X-WOPI-EcosystemOperation* – The **string** `GET_WOPI_SRC_WITH_ACCESS_TOKEN`. Required.
- *X-WOPI-HostNativeFileName* – A **string** representing the host-specific file identifier for a file.
- [Authorization](#)  – A **string** in the format `Bearer: <TOKEN>` where `<TOKEN>` is a Base64-encoded OAuth 2.0 token. If this header is missing, or the token provided is invalid, the host must respond with a [401 Unauthorized](#)  response and include the [WWW-Authenticate](#)  header as described in [WWW-Authenticate response header format](#).

• Response Headers

- [WWW-Authenticate](#)  – A **string** value formatted as described in [WWW-Authenticate response header format](#). This header should only be included when responding with a [401 Unauthorized](#) .

• Status Codes

- [200 OK](#)  – Success
- [401 Unauthorized](#)  – Authorization failure; when responding with this status code, hosts must include a [WWW-Authenticate](#)  response header with values

as described in [WWW-Authenticate response header format](#)

- [404 Not Found](#) – Resource not found/user unauthorized
- [500 Internal Server Error](#) – Server error

Response

The response to a `GetFileWopiSrc` call is [JSON](#) containing the following required properties:

- **Bootstrap** - The contents of this property should be the response to a [Bootstrap](#) operation.
- **WopiSrcInfo** - The contents of this property should be the response to a  [GetFileWopiSrc \(ecosystem\)](#) operation.

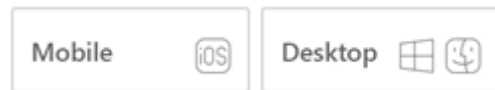
Sample response:

JSON

```
{
  "Bootstrap": {
    "EcosystemUrl": "http://.../wopi*/ecosystem?access_token=
<ecosystem_token>",
    "UserId": "User ID",
    "SignInName": "user@contoso.com",
    "UserFriendlyName": "User Name"
  },
  "WopiSrcInfo": {
    "Url": "http://.../wopi*/[containers|files]/<id>?access_token=
<file|container_token>&access_token_ttl=<timestamp>"
  }
}
```

GetRootContainer (bootstrapper)

Article • 03/04/2023



POST /wopibootstrapper

This operation is equivalent to the [GetRootContainer \(ecosystem\)](#) operation.

📘 Important

The request/response semantics for this operation differ slightly from its counterpart on the Ecosystem endpoint since it is exposed on the Bootstrapper endpoint, and thus will use OAuth 2.0 access tokens for authorization instead of WOPI access tokens.

Request Headers

- *X-WOPI-EcosystemOperation* – The **string** `GET_ROOT_CONTAINER`. Required.
- [Authorization](#) – A **string** in the format `Bearer: <TOKEN>` where `<TOKEN>` is a Base64-encoded OAuth 2.0 token. If this header is missing, or the token provided is invalid, the host must respond with a [401 Unauthorized](#) response and include the [WWW-Authenticate](#) header as described in [WWW-Authenticate response header format](#).

Response Headers

- [WWW-Authenticate](#) – A **string** value formatted as described in [WWW-Authenticate response header format](#). This header should only be included when responding with a [401 Unauthorized](#).

Status Codes

- [200 OK](#) – Success
- [401 Unauthorized](#) – Authorization failure; when responding with this status code, hosts must include a [WWW-Authenticate](#) response header with values as

described in [WWW-Authenticate response header format](#)

- [404 Not Found](#) – Resource not found/user unauthorized
- [500 Internal Server Error](#) – Server error
- [501 Not Implemented](#) – Operation not supported

Response

The response to a `GetRootContainer` call is [JSON](#) containing the following required properties:

- **Bootstrap** - The contents of this property should be the response to a [Bootstrap](#) call.
- **RootContainerInfo** - The contents of this property should be the response to a [GetRootContainer \(ecosystem\)](#) call.

Sample response:

JSON

```
{
  "Bootstrap": {
    "EcosystemUrl": "http://.../wopi*/ecosystem?access_token=
<ecosystem_token>",
    "UserId": "User ID",
    "SignInName": "user@contoso.com",
    "UserFriendlyName": "User Name"
  },
  "RootContainerInfo": {
    "ContainerPointer" : {
      "Url" : "http://.../wopi*/containers/<containerId>?access_token=
<per_container_token>",
      "Name" : "Container Name"
    },
    "ContainerInfo" : {
      "Name" : "Container Name",
      "HostUrl" : "http://contoso/324332",
      "SharingUrl" : "http://contoso/323432/efewos",
      "UserCanCreateChildContainer" : false,
      "UserCanCreateChildFile" : false,
      "UserCanDelete" : false,
      "UserCanRename" : false
    }
  }
}
```

Frequently Asked Questions

Article • 02/10/2025

Frequently Asked Questions for integrating with Microsoft 365 for the web

Technical/implementation questions

- Why does Microsoft 365 for the web pass the access token in both the Authorization HTTP header and as a URL parameter?
- What's the maximum length of a WOPI access token?
- Microsoft 365 for the web sometimes sends the access token in the query string when making requests to the service. Is this a problem?
- How do I find the right URLs for the Microsoft 365 for the web applications?
- What authentication providers does Microsoft 365 for the web support?
- What browsers does Microsoft 365 for the web support?
- Why should I avoid using the CloseButtonClosesWindow property in Microsoft 365 for the web?
- How does Microsoft 365 for the web determine which datacenter to route a given user to?
- Can auto-save be disabled in Microsoft 365 for the web?
- How often should WOPI discovery be run?
- If I make an edit and immediately close the application, occasionally my edit is lost - why?
- How does a WOPI host know when an editing session is finished?
- What are the file sizes supported by Microsoft 365 for the web?
- What are the IP ranges and ports used by Microsoft 365 for the web?
- What languages does Microsoft 365 for the web support?
- Why do I get an error when I try to view PDF documents in Microsoft 365 for the web?
- How should users give feedback to Microsoft regarding Microsoft 365 for the web?

Administrative questions

- Can I add new domains to the WOPI domain allow list after launch?
- Can I add more than one domain to the WOPI domain allow list?
- What happens our company has been bought out by another company?
- How long will the verification or reverification process take?

- What do I need to provide for verification or reverification testing?
- What happens if we don't respond or fix the identified issues?
- What if I need extra time for test reverification, what should I do?
- Can we use the same test accounts?

Frequently Asked Questions for integrating with Microsoft 365 for mobile

- Why does calling the Token Issuance URL return *The request was aborted: Could not create SSL/TLS secure channel*?
- I have already integrated my app with Microsoft 365 for the web. What additional work is there to integrate with Microsoft 365 for mobile?
- What file types are supported in Microsoft 365 for mobile?

Microsoft Cloud Storage Partner Program Integration Terms

Article • 02/27/2025

Updated February 27, 2025

These Microsoft Cloud Storage Partner Program Integration Terms (the “**Terms**”) are an agreement between Microsoft Corporation (“**Microsoft**”) and the entity represented by you, the person affirming agreement to these Terms, including any Affiliates of that entity (collectively, “**Company**”). Company is authorized to agree to these Terms only if after it has been approved by Microsoft through the Microsoft application process for the Program (defined below). **Microsoft has the right to make changes to these Terms as described in Section 3 below, and Company consents to those changes as described in that same section.**

If Company has an existing contract with Microsoft governing its participation in the Program (e.g., a “Microsoft Cloud Storage Partner Program Integration Agreement”) as of the date when Company agrees to these Terms, then as of that date that previous agreement concerning the Program is terminated by mutual agreement.

SECTION 1 - Recitals

Company wants to participate in Microsoft’s Cloud Storage Partner Program (the “**Program**”), which allows for various technology integration scenarios with Microsoft 365 Technologies. Company and Microsoft desire to work together to enable Company users to view and edit documents stored on the Company Service in the Supported Office File Formats, using Microsoft 365 Technology that integrates with a Company Application, and to enable users of Microsoft 365 Technology to utilize the functionality of Company offerings from within Microsoft 365 Technology file experiences, as applicable.

SECTION 2 - Definitions

(a) “**Affiliate**” means each legal entity that owns, is owned by, or is commonly owned with a party. “Own” means having more than 50% ownership or the right to direct the management of the entity.

(b) “**Company Application**” means collectively all language versions of Company’s cloud storage client application identified by Company as part of the process of signing up to the Program, that works with the Company Service and is Company-branded (not co-

branded), and that may integrate with the Microsoft 365 Technology identified in the Schedule, including all documentation that Company provides with its general commercial releases of that application. Where that Company application is a web version, then “client application” above is deemed to refer instead to the application functionality described above that is provided to users through a web browser to allow the user to access the Company Service.

(c) “Company Service” means the Company cloud storage service that is Company-branded (not co-branded), and that is identified by Company as part of the process of signing up to the Program.

(d) “End User” means any person that uses the Microsoft 365 Technology, as integrated with the Company Service and/or Company Application in accordance with the Schedule, to access (1) files stored in the Company Service; or (2) any of the functionality of the Company Application, developed in accordance with the Schedule.

(e) “Microsoft 365 Technology” means a Microsoft software application, online application or service identified in a Schedule. For the avoidance of doubt, Microsoft may substitute a different name for any such application or service from time to time in its sole discretion.

(f) “Program Documentation” means the documentation for the Program, specified by Microsoft and available at <https://learn.microsoft.com/microsoft-365/cloud-storage-partner-program/> (or any successor site Microsoft designates), and updates to that documentation specified by Microsoft.

(g) “Supported Office File Formats” means the file formats specified for the Microsoft 365 Technology in the Program Documentation.

(h) “Term” is defined in Section 5(a).

SECTION 3 Schedules; updates; changes to Terms; incident reporting

(a) Schedule. “Schedule” means the schedule at the end of these terms, and may also refer to added schedules as stated below in this Section 3. That initial Schedule is incorporated into these Terms by this reference. These Terms’ main body provisions will control over any conflicting terms in the Schedule, unless the Schedule expressly states otherwise.

(b) Integration; conformance to Schedule; Editing Options.

(1) Company will use commercially reasonable efforts to make each Company Application, as integrated with the Microsoft 365 Technology pursuant to the Program Documentation, available during the Term. As used in these Terms, making a Company Application or Company Service available means making it available for Company's internal use and/or providing it (or access to it) to a third party for that third party's use. **But, Company will not make any Company Application or Company Service available, as so integrated and/or using Microsoft Trademarks as permitted by these Terms, until both the Company Application and Company Service have been reviewed and approved by Microsoft in writing (email sufficient).** Microsoft will make its decision about whether to approve the Company Application and Company Service based on whether the item in question conforms to the Schedule (including conformance to the Program Documentation), including as to use of the Microsoft Trademarks. Once the Company Application and Company Service (including updates to them) are made available, Company will see that each conforms to the Schedule, including if: (1) Microsoft makes any updated version of a Microsoft 365 Technology available; or (2) Company makes any updated version of a Company Application or Company Service available. Each updated version of a Company Application, Company Service, or Microsoft 365 Technology referred to in this Section 3(b) will become the "Company Application," "Company Service," or "Microsoft 365 Technology," respectively, once it is made available.

(2) From the Company Application, wherever an End User is presented with a choice of methods for editing a document in one of the Supported Office File Formats, Company will ensure that Microsoft 365 for the web (or other applicable Microsoft 365 Technology) will persistently be presented as an option. This will take the form of or be substantially similar to the following flow:

(i) "Open in Microsoft Word in the browser" [button or link to Word for the web]" (where Word is replaced by Excel or PowerPoint depending on the file type the End User is opening).

(ii) Placement of Microsoft 365 Technology. Company will also present the respective Microsoft 365 Technologies as the first listed option or default for editing.

But, if an End User is not presented with a choice or has already selected an option as a default choice for editing a document in a Supported Office File Format, this subsection (2) does not require that the Company Application continue to present options to that End User thereafter with respect to editing that Supported Office File Format.

(c) Added Schedules; changes to these Terms.

(1) **Added Schedule.** When Microsoft introduces a feature set that is new to the Program, then subject to Sections 3(c)(3)-(4) Microsoft may add a new Schedule to these

Terms (or otherwise change these Terms) to govern Company's use of that new feature set. Where these Terms refer to "the Schedule," those references will apply separately and independently to the initial Schedule and to each such added Schedule. Each Schedule's terms do not modify or apply to any other Schedule, unless that other Schedule's terms expressly state otherwise.

(2) **Other Terms changes.** In addition to adding Schedules as described above, Microsoft may change these Terms at any time. Those changes are effective as indicated below.

(3) **Notice; when changes are effective.** Company accepts each added Schedule, and each change Microsoft makes to these Terms as described above, by continuing to make the Company Application and/or Company Service available as integrated with the Microsoft 365 Technology (including by implementing internet services using MS-WOPI or WOPI (defined in the Schedule), more than 60 days after being notified of that added Schedule or that change (through email or other reasonable means). In that event, the respective added Schedule and change to these Terms will become effective and be incorporated into these Terms when that 60-day period ends. If Company *does not* want to agree to the added Schedule or changed terms, Company must both stop making the integrated Company Application and/or Company Service available and provide Microsoft with notice of its rejection of the added Schedule or changed terms, before the end of that 60-day period, and then these Terms (and all of Company's rights under these Terms) will terminate at the end of that 60-day period.

(4) **Security Incident and technical requirements changes have shorter notice period.** Microsoft may also change these Terms to address a Security Incident (defined below), or Microsoft may update technical requirements in the Schedule by providing an updated Schedule to Company. Updates to technical requirements, in this context, include updates to apps included in any Microsoft 365 Technology, updates to Microsoft APIs used to enable the integration of that Microsoft 365 Technology with other parties' applications (such as Company Applications), updates to MS-WOPI or WOPI and online documentation for them, and updates to the Microsoft web domains used for the integration. Security Incident updates and technical requirement updates to these Terms are governed by subsection (3) above, like other changes to these Terms, except that for Security Incident updates and technical requirement updates the period referred to in that subsection will be 30 days.

(d) **Subcontracting.** Either party may subcontract its obligations under this Section 3 to any third party without the other party's prior written consent. Each party will be liable for its subcontractors' actions and omissions relating to these Terms as if they were that party's own actions and omissions.

(e) **Incident reporting**

(1) **Security Incidents.** Within 24 hours after Company becomes aware of any Security Incident during the Term, as a result of its interaction with or support of any Microsoft 365 Technology in relation to these Terms, Company will notify Microsoft. **“Security Incident”** means any attempted or actual unlawful third-party access to a Microsoft 365 Technology (or such third-party access that breaches Microsoft’s terms of service for that Microsoft 365 Technology), including unauthorized access resulting in loss, disclosure, or alteration of customer data used or stored via use of the Microsoft 365 Technology.

(2) **Privacy Incidents.** Within 48 hours after Company becomes aware of any Privacy Incident during the Term, Company will notify Microsoft. **“Privacy Incident”** means any attempted or actual unlawful third-party access to or use or disclosure of End User data stored via use of the Company Service as integrated with the Microsoft 365 Technology pursuant to these Terms, where a reasonable person would conclude that that unlawful access, use, or disclosure creates legal obligations for Microsoft with respect to privacy issues.

(3) **Notices: special procedures apply.** The Program Documentation has instructions for how to give Microsoft the notices required under this Section 3(e), and Company will comply with those instructions. Also, Company’s mere compliance with its obligations to notify Microsoft under this Section 3(e) is not an acknowledgement by Company of any fault or liability with respect to the reported Security Incident or Privacy Incident.

SECTION 4 - Intellectual property

(a) Mutual trademark licenses

(1) **“Microsoft Trademarks”** means the Microsoft trade names, business names, domain names, product or service names, logos, trade dress, and other trademarks identified by Microsoft in writing for use by Program partners in connection with the Program. During the Term, Microsoft grants to Company a non-exclusive, limited, worldwide, non-transferable, non-assignable, royalty free, fully paid up license to use the Microsoft Trademarks as follows: (A) within the user interface of the Company Application developed in accordance with the Schedule in a manner approved by Microsoft prior to availability; and (B) subject to Microsoft’s prior written approval (not to be unreasonably withheld), in Company’s promotional materials related to the Microsoft 365 Technology integration with the Company Application. But, once Company obtains that Microsoft approval of an initial and representative sample use of Microsoft Trademarks in its promotional materials, Company will not be required to get Microsoft’s approval of future promotional materials that are substantially similar to those previously approved by Microsoft. Company may sublicense the rights granted under this Section 4(a)(1) to third parties only to allow them to act on behalf of Company.

(2) ***“Company Trademarks”*** means the Company trade names, business names, domain names, product or service names, logos, trade dress, and other trademarks identified by Company in writing for use by Microsoft under these Terms. During the Term, Company grants to Microsoft a non-exclusive, limited, worldwide, non-transferable, non-assignable, royalty free, fully paid up license to use the Company Trademarks as follows: (A) in connection with any onboarding process related to a specific Microsoft 365 Technology for purposes of Microsoft providing End Users access to Company Service from that Microsoft 365 Technology; and (B) as otherwise may be agreed to by the parties in writing (e.g., in marketing or promotional materials). Microsoft may sublicense the rights granted under this Section 4(a)(2) to third parties only to allow them to act on behalf of Microsoft.

(b) Trademark license conditions

(1) Each party granting license rights under Section 4(a) (***“Licensor”***) may provide reasonable written branding guidelines (the ***“Branding Guidelines”***) to the other party (***“Licensee”***). Licensee will comply with the Branding Guidelines in its use of the ***“Licensed Trademarks”*** (which means the Microsoft Trademarks (where Company is Licensee) or the Company Trademarks (where Microsoft is Licensee)). Microsoft’s Branding Guidelines are provided in the Program Documentation.

(2) The license granted to Licensee under Section 4(a) is conditioned on Licensee complying with Sections 4(b)(1)-(3). Also, the Company Application (where Company is Licensee) and the Microsoft 365 Technology (where Microsoft is Licensee) must (A) meet or exceed quality and performance standards generally accepted in the industry; (B) be at least equal to the quality of applications and services that Licensee distributed and provided before starting to use the Licensed Trademarks under these Terms, and (C) comply with all applicable laws, rules, and regulations. If Licensee fails to comply with Section 4(b)(2)(A)-(C) and does not cure that failure within 15 days after receiving notice, Licensor’s sole and exclusive remedy for that noncompliance is that it may terminate the license granted by Licensor under Section 4(a)(1) or (2).

(3) In its reasonable discretion, Licensor may update the Licensed Trademarks and the Branding Guidelines by sharing them with Company in writing. (For Microsoft, this may include posting a notice of the updates in the Program Documentation.) Licensee will implement any changes required by the updated Licensed Trademarks or Branding Guidelines within a reasonable time (not to exceed 60 days) after notice from Licensor.

(4) All goodwill regarding the Licensed Trademarks will inure to Licensor’s benefit.

(c) No joint development. The parties do not intend to engage in any joint development under these Terms, and will not do so unless expressly agreed otherwise in a separate written agreement signed by both parties. Company will not represent to any

third party that its participation in the Program constitutes any sort of joint development by Microsoft and Company.

(d) Feedback. Company may voluntarily provide suggestions, comments or other feedback to Microsoft with respect to any Microsoft 365 Technology ("**Feedback**"). Microsoft may use Feedback for any purpose without obligation of any kind. Microsoft will not disclose the source of the Feedback without Company's consent. Unless the parties specifically agree otherwise in writing, Feedback will not be subject to any confidentiality obligation.

(e) Excluded license actions. Each party acknowledges and agrees that nothing in these Terms grants it the right to, and that party agrees that it will not, distribute anything or otherwise take any action that would subject any of the other party's software, technology, or other property relating to these Terms to license terms that seek to require that software, technology, or other property to be: (1) disclosed in source code form to third parties; (2) licensed to third parties for the purpose of making derivative works; or (3) redistributable to third parties at no charge.

(f) No implied licenses. Each party reserves all rights not expressly granted under these Terms. Other than those express grants, these Terms do not grant any rights to either party's intellectual property rights, by implication, estoppel, or otherwise.

(g) Territories. Without limiting any other part of these Terms, Company agrees that it will not use its offerings that are the subject of these Terms and the Schedule to make Microsoft 365 Technologies that are the subject of these Terms and the Schedule available to users in or headquartered in the mainland of the People's Republic of China.

(h) Commercial Use Restriction. In some territories, the Microsoft 365 Technologies may only be available for commercial use. Upon acceptance to the program, Microsoft will provide Company with a list of territories in which consumer use is not available.

SECTION 5 - Term and termination

(a) Term. These Terms become effective for both parties when Company agrees to them through the process Microsoft has established for the Program, and these Terms will then continue for a one-year period (the "**Initial Term**"), at which point these Terms will automatically renew for successive, additional one-year periods (each, a "**Renewal Term**"), unless either party gives the other party notice of non-renewal at least 60 days before the expiration of the Initial Term or then-current Renewal Term. The Initial Term and all Renewal Terms (if any) are collectively referred to as the "**Term**." This Section 5(a) does not change either party's right to terminate these Terms as stated elsewhere in these Terms.

(b) Termination for cause. If a party materially breaches these Terms and fails to cure that breach within 30 days after notice of termination by the non-breaching party, then these Terms will terminate automatically upon the expiration of that period, without any additional notice. Also, Microsoft may immediately terminate these Terms upon notice for Company's repetitive and/or persistent breaches of the Branding Guidelines.

(c) Termination without cause. Either party may terminate these Terms in its discretion without cause by giving the other party at least 60 days' prior notice.

(d) Survival. The following provisions of these Terms will survive its termination or expiration: Sections 2 (Definitions), 3(d) (Subcontracting), 4(d) (Feedback), 4(f) (No implied licenses), 5(d) (Survival), 6 (Representations and warranties; indemnification), 7 (Insurance), 8 (Limitations of liability), and 9 (Miscellaneous).

SECTION 6 - Representations and warranties; indemnification

(a) By each party. Each party represents and warrants that:

(1) it has all rights and authority necessary to agree to these Terms and to grant the rights set forth in them; and (2) its entry into and performance under these Terms will not violate any agreement or obligation between that party and any third party.

(b) DISCLAIMER. EXCEPT AS SET FORTH IN SECTION 6(a), ANYTHING PROVIDED UNDER THIS AGREEMENT IS PROVIDED "AS IS." TO THE MAXIMUM EXTENT PERMITTED BY LAW, EACH PARTY DISCLAIMS ANY AND ALL OTHER WARRANTIES, WHETHER EXPRESS OR IMPLIED, INCLUDING ANY IMPLIED WARRANTIES OF MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE, WHETHER ARISING BY A COURSE OF DEALING, USAGE OR TRADE PRACTICE OR COURSE OF PERFORMANCE. THERE IS NO WARRANTY OF TITLE OR NON-INFRINGEMENT.

(c) Indemnification.

(1) Company will defend, hold harmless, and indemnify Microsoft, its Affiliates, directors, officers, employees, and agents from and against all claims or actions of any unaffiliated third party (and all resulting costs, damages, and fees reasonably incurred by them (including fees of attorneys and other professionals)) that, if proven, would establish:

(i) Company's infringement or misappropriation of any third-party intellectual property rights with respect to a Company Service or a Company Application; or

(ii) Company's failure to comply with applicable laws, rules or regulations with respect to a Company Service or a Company Application.

However, Company will have no liability under this Section 6(c)(1) to the comparative extent that those third-party claims result from Company's compliance with the express instructions of Microsoft for any Company Application as set forth in the Schedule.

(2) Microsoft will defend, hold harmless, and indemnify Company, its Affiliates, directors, officers, employees, and agents harmless from and against all claims or actions of an unaffiliated third party (and all resulting costs, damages, and fees reasonably incurred by them (including fees of attorneys and other professionals)) that, if proven, would establish:

(i) Microsoft's infringement or misappropriation of any third-party intellectual property rights with respect to a Microsoft 365 Technology or other Microsoft technologies expressly licensed by Microsoft to Company hereunder that are required for a Company Application or Company Service to integrate and interoperate with a Microsoft 365 Technology pursuant to the Schedule; or

(ii) Microsoft's failure to comply with applicable laws, rules or regulations with respect to a Microsoft 365 Technology.

(3) **Indemnification procedures.** As to each claim or action for which it seeks protection under this Section 6, the indemnified party will:

(i) Provide the indemnifying party with reasonably prompt notice of any claims;

(ii) Permit the indemnifying party to answer and defend claims through mutually acceptable counsel; and

(iii) Provide the indemnifying party with reasonable information and assistance to help the indemnifying party defend claims, at the indemnifying party's expense.

Any indemnified party may employ separate counsel and participate in the defense of any claim at its own expense. The indemnifying party will not settle any claim on the indemnified party's behalf, or publicize the settlement, without the prior written consent of the indemnified party, not to be unreasonably withheld or delayed.

SECTION 7 - Insurance

During the Term, Company will maintain the following insurance coverage

(a) **General.** Company will maintain sufficient insurance coverage to meet obligations created by these Terms and by law. Company's insurance must include the following coverage to the extent these Terms creates risks generally covered by these insurance policies: (1) Commercial General Liability (occurrence form) including contractual, and

product liability with limits of at least \$1,000,000 per occurrence; (2) Privacy and cybersecurity liability (including costs arising from data destruction, hacking or intentional breaches, crisis management activity related to data breaches, and legal claims for security breach, privacy violations, and notification costs) of at least \$1,000,000 per claim; (3) Workers' compensation satisfying all statutory limits; and (4) Employer's liability with limits of at least \$500,000 per occurrence.

Company will name Microsoft, its subsidiaries, and their respective directors, officers and employees as additional insureds in the Commercial General Liability policy, to the extent of contractual liability assumed by Company in Section 6 (Representations and warranties; indemnification). Microsoft must approve any deductible or retention in excess of \$100,000 per occurrence or accident.

(b) Professional liability/errors & omissions liability. Company will purchase and maintain professional liability/errors and omissions insurance if the services it performs create exposures generally covered by such a policy. The policy will: (1) Have limits of at least \$1,000,000 each claim. (2) Cover infringement of third party proprietary rights (including, for example, copyright and trademark rights) if such coverage is reasonably commercially available; and (3) Have a retroactive coverage date no later than the date on which Company enters into these Terms.

Company will maintain either active policy coverage or an extended reporting period providing coverage for claims first made and reported to the insurance company within 12 months after termination or expiration of these Terms.

(c) Proof of coverage. Upon request, Company will provide Microsoft with proof of the insurance coverage required by this Section 7. If Microsoft reasonably determines that Company's coverage is less than that required to meet its obligations, Company will promptly buy additional coverage and notify Microsoft in writing.

SECTION 8 - Limitations of liability

(a) TO THE MAXIMUM EXTENT PERMITTED BY APPLICABLE LAW NEITHER PARTY WILL BE LIABLE TO THE OTHER FOR ANY CONSEQUENTIAL, SPECIAL, EXEMPLARY, OR PUNITIVE DAMAGES (INCLUDING DAMAGES FOR LOSS OF DATA, REVENUE, AND/OR PROFITS), WHETHER FORESEEABLE OR UNFORESEEABLE, ARISING OUT OF THIS AGREEMENT, REGARDLESS OF WHETHER THE LIABILITY IS BASED ON BREACH OF CONTRACT, TORT, STRICT LIABILITY, BREACH OF WARRANTIES OR OTHERWISE, AND EVEN IF THE PARTY HAS BEEN ADVISED OF THE POSSIBILITY OF THOSE DAMAGES.

(b) EACH PARTY'S TOTAL LIABILITY FOR ALL CLAIMS ARISING OUT OF OR RELATED TO THIS AGREEMENT, WHETHER IN CONTRACT, TORT, OR OTHERWISE, WILL NOT EXCEED

\$1,000,000.

(c) SECTIONS 8(a)-(b) DO NOT APPLY TO LIABILITY ARISING FROM: (1) BREACH OF OR CLAIMS UNDER SECTION 6 (REPRESENTATIONS AND WARRANTIES; INDEMNIFICATION); (2) ANY INFRINGEMENT OR MISAPPROPRIATION BY ONE PARTY OF ANY INTELLECTUAL PROPERTY RIGHTS OF THE OTHER (INCLUDING A BREACH OF THE TRADEMARK LICENSE PROVISIONS OF SECTION 4 THAT REMAINS UNCURED); OR (3) FRAUD.

SECTION 9 - Miscellaneous

(a) **Notices.** Notices must be in writing, and may be provided either by electronic or physical mail. Microsoft may provide notice to Company using the reasonable contact information provided by Company as part of the process of signing up to the Program, or through a web site that Microsoft identifies. Company must provide notice to Microsoft at:

(1) **The Microsoft notice address available in the Program Documentation,**

(2) *with a copy to:*

Microsoft Corporation

One Microsoft Way

Redmond, WA 98052

Office Business Development

Email: CSPP_BD@microsoft.com

(3) *and with a copy to:*

Corporate, External, & Legal Affairs (CELA)

Attn: Office CELA

CSPP_LEGAL@microsoft.com

Fax: 425.936.7329

Each party may change the persons to whom notices will be sent by giving notice to the other. Also, this Section 9(a) does not modify the requirements for notices in Sections 3(c)(3) (Notice; when changes are effective) or 3(e) (Incident reporting).

(b) **Relationship.** The parties are independent contractors. These Terms do not create an agency, partnership, joint venture, franchise, or employment relationship. Neither party has the authority to make any statements, representations or commitments on the other's behalf. These Terms do not create an exclusive relationship between the parties. Nothing in these Terms restricts either party from independently, directly or indirectly, acquiring, licensing, developing, manufacturing, or distributing new or existing products

or services with the same or similar functions as the products and offerings that are the subject of these Terms.

(c) Governing law; jurisdiction. The laws of the State of Washington govern these Terms. If federal jurisdiction exists, the parties consent to exclusive jurisdiction and venue in the federal courts in King County, Washington. If not, the parties consent to exclusive jurisdiction and venue in the Superior Court of King County, Washington. If either Microsoft or Company employs attorneys to enforce any rights arising out of or relating to these Terms, the prevailing party will be entitled to recover its reasonable attorneys' fees, costs, and other expenses, including the costs and fees incurred on appeal or in a bankruptcy or similar action.

(d) Compliance with law; restricted geographies. With regard to its offerings that are the subject of these Terms and the Schedule, each party will comply with all applicable laws, including (1) data privacy laws such as the EU General Data Protection Regulation; and (2) all laws and regulations applicable to the import or exports of services or technology that may be subject to U.S. and other countries' export jurisdictions, including trade laws such as the U.S. Export Administration Regulations and International Traffic in Arms Regulations, and sanctions regulations administered by the U.S. Office of Foreign Assets Control ("OFAC"). For additional information, see [Exporting Microsoft Products](#) . Also, without limiting the previous parts of this subsection (d), Company agrees that it will not use its offerings that are the subject of these Terms and the Schedule to make Microsoft 365 Technologies that are the subject of these Terms and the Schedule available to users in or headquartered in Cuba, Iran, North Korea, Sudan, Syria, Region of Crimea, or Russia.

(e) No waiver. A party's delay or failure to exercise any right or remedy will not result in a waiver of that or any other right or remedy.

(f) Assignment. Except for an assignment to an Affiliate as provided below, neither party will sell, assign, delegate, or transfer these Terms, in whole or in part, by assignment or operation of law, without giving the other party prior notice. These Terms will inure to the benefit of and bind all successors, assigns, receivers and trustees of each party.

(g) Severability. If any court of competent jurisdiction determines that any provision of these Terms is illegal, invalid or unenforceable, the remaining provisions will remain in full force and effect.

(h) Entire agreement. These Terms form the entire agreement between the parties regarding participation in the Program and its other subject matter, and supersede all prior and contemporaneous communications regarding all such subject matter, whether written or oral. Except as stated in Section 3, these Terms may be modified only by a written agreement signed by duly authorized representatives of both parties. These

Terms do not supersede any separate written license agreement between Microsoft and Company.

(i) Construction. Unless otherwise expressly stated, when used in these Terms the words “include,” “includes,” and “including” will be deemed in each case to be followed by the words “without limitation.” Unless otherwise specified, all references to “days” mean “calendar days,” and all references to “\$” or “dollars” mean U.S. dollars. Neither party has entered these Terms in reliance on any promise, representation, or warranty not expressly set forth in these Terms. These Terms will be construed according to the fair intent of the language as a whole, and not for or against either party.

Schedule—Microsoft 365 Technology: Microsoft 365 for the web

SECTION A: Application; Defined Terms

This Schedule applies to:

- ***“Microsoft 365 for the web,”*** which for purposes of this Schedule means collectively the Microsoft Word, Excel, and PowerPoint applications for web browsers. This does not include the “Office” app for desktop/laptop devices (i.e., progressive web app, also known as “PWA”, and hybrid web app, also known as “HWA”) or Office web files accessed within Microsoft Teams and Outlook applications. For the avoidance of doubt, this also does not include any other “for the web” applications (e.g., Visio for the web).

SECTION B: Requirements

1. General Compliance Requirements; Administrator and Other Exceptions

The Company Application and Company Service must comply with the requirements stated in these Terms (including this Schedule) and the requirements stated in the Program Documentation. Company will not initially make the Company Application or Company Service (or any update to them) available until each is compliant. The following requirements do not apply to End Users in instances where the Microsoft 365 for the web integration is either disabled or otherwise not enabled by an administrator (if any) of the End User.

2. WOPI License

From the Company Application, End Users will be able to open, view, edit, and save Office documents in Microsoft 365 for the web in the Supported Office File Formats and

store them back to the Company Service. All of these operations will require that Company implement internet services using the Microsoft Web Application Open Platform Interface Protocol ("**MS-WOPI**" or "**WOPI**"). Subject to Section 3(b) (Integration; conformance to Schedule) of the Terms, Microsoft grants Company a non-exclusive, personal, royalty free license under the **Necessary Claims** to WOPI solely for purposes of implementing WOPI in accordance with the technical documentation provided in the Program Documentation in order to integrate and interoperate with the Microsoft 365 Technology as contemplated in these Terms. As used in this paragraph, "Necessary Claims" means the claims of a patent or patent application owned (or that Microsoft has the right to sublicense without a fee) and that are necessarily infringed by implementing WOPI in accordance with that technical documentation. Necessary Claims do not include any claims directed to any technologies other than WOPI or the implementation contemplated in this Schedule. Without limiting the previous sentence, Necessary Claims do not include any claims directed to (a) underlying or enabling technologies that may be provided in connection with WOPI or in connection with any implementation of WOPI, where those technologies are not required to implement WOPI in accordance with that technical documentation; (b) any portions of the Company Application or Company Service other than the implementation required to integrate them with Microsoft 365 Technology as contemplated by these Terms; or (c) any implementation of any other technical documentation, specifications, or technologies that are merely referred to in the body of the WOPI technical documentation. Company may sublicense the rights granted under this Section B(2) to third parties only to allow them to act on behalf of Company with respect to the Company Application and Company Service.

3. Company Application

a. When an End User uses Microsoft 365 for the web to view and/or edit, Microsoft 365 for the web keeps a temporary copy of the file being viewed and edited for the purposes of rendering and making changes to the file. Company will inform its End Users that Microsoft 365 for the web is a Microsoft service and use of Microsoft 365 for the web is subject to Microsoft's terms of use and privacy policy.

b. Company acknowledges that use of Microsoft 365 for the web through this integration with the Company Service and/or Company Application is intended for consumers, commercial customers, and government customers, but it does not support Microsoft's Government Community Cloud (GCC) environment, and is not intended to be used by end customers who need to comply with FedRAMP, export control, IRS 1075 or CJIS obligations (which could also include, e.g., various contractors holding or processing data on behalf of the US Government). Moreover, the phrase "commercial customers" here requires Company to have multiple customers for that integration, and excludes Company providing that integration to only one commercial customer.

Company agrees to ensure that it offers the Company Service and Company Application consistent with these restrictions, and the parties agree that Company's breach of that obligation will be a material breach of these Terms.

c. This subsection (c) addresses customers that have data residency requirements. Company acknowledges that, unless stated otherwise in the Program Documentation, the Microsoft 365 Technology is not intended to support or maintain limits in the presence or storage of End User data files to only servers located in particular geographic locations or to sovereign cloud facilities, when that Microsoft 365 Technology is used to view and edit documents stored in a storage solution that is not provided by Microsoft and is managed entirely by Company or a third party. (For the avoidance of doubt, such a storage solution is not "provided by Microsoft" merely because it runs on Microsoft Azure.) Company will not represent to the contrary to any third party, including End Users.

d. Microsoft has no obligation under these Terms or this Schedule to provide support to Company with respect to this integration.

e. End User Support. Each party will provide support to its customers in accordance with its subscription agreements or terms of service for its respective offering that is part of this integration.

f. Company will not route or process production-level files, storage, data, or other materials or traffic through or using its test environment for the Company Application or Company Service, nor will it assist End Users to do so or promote End Users taking that approach.

4. Privacy Policy *[Intentionally omitted]*

Microsoft Cloud Storage Partner Program Terms FAQ

Article • 05/02/2023

Do you have questions about the Terms and obligations of being a member of the Microsoft Cloud Service Partner Program? We have prepared this FAQ just for you.

[Reviewing the CSPP Terms](#)

[Agreeing to the CSPP Terms](#)

[Changes to the Cloud Storage Partner Program](#)

[Legal questions](#)

[CSPP Integration Agreement \(contract\)](#)

[Maintaining Compliance with the CSPP Terms](#)

[Additional questions and concerns](#)

Reviewing the CSPP Terms

Where can I see the Microsoft Cloud Storage Partner Program Integration Terms?

[Microsoft Cloud Storage Partner Program Terms](#) 

When did the initial email notification explaining the Microsoft Cloud Storage Partner Program Integration Terms go out?

The initial email notification went out on February 23rd, 2023.

What if the emails did not reach any of our contacts?

Per the original agreement, it is the responsibility of the partner to keep the CSPP Admin team updated on who the points of contacts are. The CSPP Team sent a number of emails explaining the changes to the program Terms to those contacts.

Please provide us with the most current point of contact and we can provide them with the Microsoft Cloud Storage Partner Program Integration Terms for review and

acceptance. Once accepted, the WOPI solution will be reactivated.

Agreeing to the CSPP Terms

Where can I agree to the Terms?

Use the [Microsoft Cloud Storage Partner Program Terms Agreement](#) site to accept the new CSPP Terms.

Who in my company should sign the terms? The email address of the individual agreeing to the terms must:

- Have the same domain as the partner URLs in our [CSPP WOPI Allow List](#).
- Must match the first and last name entered in the agreement form

How long do I have to agree to the Terms?

You must agree to the Microsoft Cloud Storage Partner Program Terms by April 30th, 2023.

What if I don't sign before the deadline?

If you have not agreed to the Microsoft Cloud Storage Partner Program Integration Terms by April 30th, 2023, your current Microsoft Cloud Storage Partner Program Integration Agreement will be terminated and your CSPP WOPI connection will be removed. You will then need to reapply to join CSPP as a new partner.

How much time do I have to review this with my legal team?

You must agree to the Microsoft Cloud Storage Partner Program Integration Terms by April 30th, 2023. If additional time is needed, please contact csppteam@microsoft.com to discuss timing for reviewing the program Terms.

What if we need more time to review the Microsoft Cloud Storage Partner Program Integration Terms?

If you need more time to review the program Terms, please contact csppteam@microsoft.com so we can assist.

Who within my company should agree to the program Terms?

Anyone within the company who is responsible for legal approval.

What if my company requires more than one agreeing authority?

Though at least one individual from each partner company must indicate agreement to the new terms, it is acceptable to have multiple individuals from a single company submit an agreement.

If I am allowed extra time to review the Microsoft Cloud Storage Partner Program Integration Terms, what happens with my WOPI solution?

When you reach out to the CSPP Team at csppteam@microsoft.com, they will advise you how much extra time (if any) you will be granted or stipulations that must be met before you need to agree to the CSPP Terms.

Do I need to enter my name or my company's name in the signature field of the CSPP Terms Agreement form?

The signature field must be filled out with the name of an individual. Entering a company or department name in the signature field does not constitute agreement to the CSPP Terms.

Changes to the Cloud Storage Partner Program

Does this mean that Microsoft can/will charge for the Cloud Storage Partner Program?

There are no current plans to charge for participation in the CSPP basic program. Microsoft reserves the right to make changes to this policy at any time. You will be notified in advance if Microsoft decides to start charging for CSPP membership.

Do the Microsoft Cloud Storage Partner Program Integration Terms affect CSPP Mobile?

Changes have been made for admission to the Mobile Program. These changes do not affect partners already in the Mobile Program. Contact csppteam@microsoft.com for

more information.

Will the Microsoft Cloud Storage Partner Program Integration Terms require updates to our WOPI Solution?

You will not need to make any changes to your WOPI solution.

Do the Microsoft Cloud Storage Partner Program Integration Terms affect Government Licensing?

The Microsoft Cloud Storage Partner Program Integration Terms has an update regarding Government Licensing. Please reference Terms for further details.

Do the Microsoft Cloud Storage Partner Program Integration Terms affect data residency requirements?

The Microsoft Cloud Storage Partner Program Integration Terms has an update regarding Data Residency. Please reference Terms for further details.

Legal questions

What if I need changes/edits to the Microsoft Cloud Storage Partner Program Integration Terms?

Microsoft does not accept changes or addendums to the Microsoft Cloud Storage Partner Program Integration Terms.

How long is my Microsoft Cloud Storage Partner Program Integration Terms binding when I agree to Terms?

Please refer to the appropriate section of the Microsoft Cloud Storage Partner Program Integration Terms for details.

Who can I talk to about the insurance requirements for CSPP participation?

We require that every CSPP partner carry insurance to cover financial liabilities that may arise resulting from the implementation and/or use of the partner's CSPP solution. If you have questions regarding the insurance requirements, reach out to

csppteam@microsoft.com for assistance.

What happens if I break these Microsoft Cloud Storage Partner Program Integration Terms?

Please refer to the Termination for cause section within the Terms agreement.

What happens if I change my mind after agreeing to the Microsoft Cloud Storage Partner Program Integration Terms?

Please reference the Termination without clause in the program Terms.

Can I obtain a copy of the Microsoft Cloud Storage Partner Program Integration Terms in my local language?

We are unable to provide localized versions of the CSPP Terms.

What happens if I don't agree to the Terms?

If the Microsoft Cloud Storage Partner Program Terms are not accepted by April 30th, 2023, the current Microsoft Cloud Storage Partner Program Integration Agreement will be terminated, and the CSPP WOPI connection will be removed.

How can my legal team review the Microsoft Cloud Storage Partner Program Integration Terms?

Please share the link to the [Microsoft Cloud Storage Partner Program Terms](#) as needed within your organization.

CSPP Integration Agreement (contract)

Where is my original CSPP Integration Agreement and can I have a copy?

To obtain a copy of your signed Microsoft Cloud Storage Partner Program Integration Agreement, please contact csppteam@microsoft.com for assistance.

What if I do not currently have an existing contract but have my WOPI solution in production?

It is a requirement that you agree these Terms to be enrolled in the Cloud Storage Partner Program (CSPP). If the Cloud Storage Partner Program Terms are not agreed to by April 30th, 2023, your organization will be removed from CSPP.

What is the termination policy with my current agreement?

Please reference your Microsoft Cloud Storage Partner Program Integration Agreement for details.

Can I stay on my existing contract?

Unfortunately, no. Per the email sent on February 23rd, 2023, whether or not you agree to the new CSPP Terms, any existing legal agreement between your organization and Microsoft for the Cloud Storage Partner Program will be terminated on April 30th, 2023.

I recently agreed the NDA and CSPP Integration Agreement. Why do I need to accept the Microsoft Cloud Storage Partner Program Integration Terms?

We are changing our approach to contracts for the CSPP Program.

What are the differences between the CSPP Terms and my CSPP Integration Agreement?

In the time since the Cloud Storage Partner Program was first implemented, several versions of the CSPP Integration Agreement have been used with different partners. Please reference your existing Microsoft Cloud Storage Partner Program Integration Agreement and the Microsoft Cloud Storage Partner Program Terms to identify the differences between your existing agreement and the new terms. Contact csppteam@microsoft.com if you need a copy of your CSPP Integration Agreement.

Maintaining compliance with the CSPP Terms

Will the Terms be updated on a regular basis?

We do not have a schedule for updating the Terms.

How often will you update the CSPP Terms?

We reserve the right to update the terms at any time.

The Microsoft Cloud Storage Partner Program Integration Terms do not contain Microsoft Trademarks and Branding Guidelines, where can this information be found?

CSPP UI and Microsoft Trademarks and UI Guidelines can be found in the [CSPP documentation](#).

How long do we have to make updates that are required by CSPP?

The length of time partners will have to update their solution will depend on the update required. When we send a notice that an update is required, we will also send a date by which time that update must be completed.

What is the review process and how will it be conducted?

The CSPP team periodically reviews a random selection of partners' CSPP WOPI production implementations. We use the same testing process that was used when you went through the initial verification process prior to being added to the CSPP WOPI production environment.

We do perform reviews of partners' development implementations after a partner has been launched into production.

We do not have a set schedule for the review process.

If we make changes to our application after we are approved for production, do we need to go through CSPP verification testing again?

You **do** need to request reverification testing if:

- You are adding or removing core WOPI functionality
- Making significant changes to your user experience

You **do not** need to request reverification testing:

- If you are making bug fixes
- Changing your solution to comply with CSPP required updates
- Minor changes to your user experience - including, but not limited to, your user interface

If you have any doubt regarding whether or not your application requires reverification testing, contact csppteam@microsoft.com for guidance.

Additional questions and concerns

What to do if I have questions/concerns with the Microsoft Cloud Storage Partner Program Integration Terms?

If you have any questions or concerns, please contact csppteam@microsoft.com for assistance.

Legal

Article • 02/24/2023

Export Restrictions

Microsoft Technologies are subject to U.S. export jurisdiction. Company must comply with all applicable international and national laws, including the U.S. Export Administration Regulations, the International Traffic in Arms Regulations, Office of Foreign Assets Control sanctions programs, and end-user, end use and destination restrictions by U.S. and other governments related to Microsoft Technologies. For additional information, see [Exporting Microsoft Products](#) .